TREKLINK

TrekLink Developer Handbook

Conventions, Workflow & the AI-First Doctrine

Version 1.0 · 2026-10-11

MEMBER	CODE	ROLE
Đỗ Đăng Khoa	`KhoaDD`	Leader / Product Owner
Trần Khải Hoàng	`HoangTK`	Member: Frontend
Lâm Phi Long	`LongLP`	Member: Gateway & Data
Nguyễn Bá Tân	`TanNB`	Member: Reporting & Monitoring
Nguyễn Ngọc Long	`LongNN`	Member: Backend

Capstone FA26SE159 · TrekLink Team
Supervisor: Đặng Ngọc Minh Đức

> Generated from the markdown in `_docs/`. Do not edit this PDF — edit the source and rebuild.

Contents

Orientation

TrekLink: Project Charter

1. Problem
2. What we're building
3. Actors & role summary
4. Research questions (drive the experiment plan: see roadmap)
5. Non-functional targets (binding: used as PR/Definition-of-Done gates)
6. Tech stack (as registered: see D-001 in decisions log, resolved)
7. Deliverables (map to Jira epics and the five Main Flows: see roadmap §4)
8. Legal & compliance
9. Related work / gap statement (condensed)

TrekLink: The Five Main Flows

MF-01: Organization onboarding → Rental contract → Handover

MF-02: Field data → Field Station → Cloud synchronization

MF-03: SOS → Tiered alert → Escalation

MF-04: Live telemetry: web map and organization API

MF-05: Term end → Return → Inspection → Billing → Maintenance

Coverage tracking

TrekLink: Requirements Foundation

1. Context Diagram
2. Actors
3. Use Cases
4. Feature Tree
5. Business Rule Matrix
6. Exception Scenario Matrix
7. Honest status

TrekLink Engineering Conventions: Index

Read in this order

What changed in v2 (2026-09-13)

Adopting these conventions in a repo

Authority

How We Work

Core Engineering Philosophy: Session-Based Development & Living SSOT

1. The Core Philosophy
2. Pillar I: Living Documentation as SSOT
3. Pillar II: Session-Based Development (SBD)
4. Crash-Safe Memory Checkpoint Protocol
5. Context Window & Token Budget Management (AI Agents)
6. Tenets of Iterative Refinement

Spec-Driven Development Workflow (SDSDW)

1. Executive Overview
2. Directory Structure (specs/{module_name}/)
3. Phase-by-Phase Execution Protocol
4. Universal Phase Exit Gate Matrix

Engineering Lifecycle & Operational Workflows

1. Executive Workflow Matrix
2. Lifecycle 1: Normal Feature Development Flow (Spec-First)
3. Lifecycle 2: Diagnostic & Debugging Flow (Hypothesis-Driven)
4. Lifecycle 3: Pull Request & Code Review Flow
5. Lifecycle 4: Review Issue Resolution Flow

Delivery

GitHub Workflow, Branching, Commits & Pull Requests

1. Repositories
2. Branch Model
3. Commit Conventions
4. Rebase Discipline
5. Pull Requests
6. Language Rule (Hard Requirement)
7. Branch Protection Settings
8. Where Work Is Tracked
9. Quick Reference
10. GitLab → GitHub Concept Map

Jira Tracking, Board Workflow & the Development Loop

1. The Three Places Work Lives
2. Work Item Hierarchy
3. The Board Workflow
4. BUG vs NEEDS HELP: they are not the same thing
5. Where Bugs Actually Go
6. Jira ↔ GitHub Integration
7. Sprints

8. The Development Loop (End to End)

9. Backlog ↔ Jira Key Mapping

10. Quick Reference

Team Communication, Daily Reports & Schedule Awareness

1. Channels: What Goes Where

2. The Two Mandatory Announcements

3. Daily Reports

4. Raising Problems

5. Schedule Awareness: Automated Countdown

6. Weekly Rhythm

7. Quick Reference

AI-First Development

The AI-First Doctrine, Toolchain & Prompting Standard

1. The Doctrine in One Page

2. Model Policy (Hard Requirement)

3. Required Toolchain

4. Agent Auto-Steering

5. The Mandatory Session Workflow

6. Token & Context Discipline

7. How to Prompt Well

8. Starter Prompts

9. Common Failure Modes

10. Checklist: New Member Setup

AI Agent Steering, Governance & Thinking Discipline

1. Operating Principles & Decision Hierarchy

2. The Universal Pre-Action Protocol

3. Power Modes & Token Budget

4. Hallucination & Loop Circuit Breakers

5. Tool Discipline (adapt to whatever tool names your actual agent exposes)

5.1 The Mandatory Session Shape

6. Multi-Agent / Subagent Dispatch (if your tool supports it)

Doc-Driven Scaffold & SSOT Conventions

1. TrekLink Application Note (read this first)

2. Universal Documentation-Driven Scaffold & SSOT Architecture

Engineering Standards

Architectural Conventions: NestJS Backend / Node Gateway / React Frontend

0. Decision criteria
1. Module Topology
2. Entity & Audit Conventions
3. Idempotency & Priority Queue (gateway-sync: the module most different from a normal CRUD app)
4. Module Isolation & Parallel Team Scaling
5. Frontend Architecture: Feature-Sliced Design (FSD)

Business parameters are configuration, never constants (D-015)

Backend Code & REST API Conventions (NestJS)

1. The Standard Module Structure
2. Input Validation
3. The Canonical Response Envelope (binding: from API_Design_Template.md)
4. Status Code & Error Taxonomy
5. Controller Standards
6. Async, Transactional & Concurrency Standards

Frontend Code & UI/UX Conventions (React + TypeScript)

1. Architectural Layout (Feature-Sliced Design)
2. State Management: The Triad Standard
3. The Core UX Rule: 4 Steps, 6 Fields
4. Reusable Screen Patterns
5. Keyboard Navigation & Accessibility (WCAG 2.1 AA)
6. Client API & Validation Contract Mirroring

13: Diagram & Figure Conventions

1. Mermaid only
2. Process flows are swimlanes
3. Numbering and captions
4. Figure placement
5. Hard rules for every figure
6. The measurement harness
7. Practical guidance

Appendices

TrekLink: Roadmap & Milestone Calendar

1. The five things to know before reading anything else
2. Gate schedule
3. Week-by-week calendar
4. The five Main Flows
5. Deliverable ledger and ownership
6. ⚠️ The Week-6 scope lock

7. Gate readiness checklists

8. Known failure modes to design against

Appendix A: TP1–TP6 (historical, no longer the planning unit)

Appendix B: Superseded framings

TrekLink: Decisions Log & Risk Register

Decisions

Risk register (carried from FA26SE159, kept live)

Contributing to TrekLink

1. Orient yourself

2. Where work lives

3. Before writing code

4. Branch, commit, PR

5. Merging

6. Backend response contract (non-negotiable)

7. Language

8. Using an AI coding agent

9. Definition of Done, every PR

Orientation

TrekLink: Project Charter

> **Superseded in scope by D-033 (2026-10-03).** This charter describes the booking scope registered in September. TrekLink is now an enterprise device-rental platform built on asset management; read `03-decisions-and-risk-register.md` D-033 to D-035 first. The team, the research questions and the technical foundation below still hold.

Project code: FA26SE159 · **Group code**: GFA26SE55 · **Class/Specialty**: SE **Duration**: 09/2026 – 03/2027 *(as registered, includes the retake window; the working plan ends Week 15, Dec 20 2026)* · **Supervisor**: Đặng Ngọc Minh Đức (Assoc. Prof)

Team & skill matrix (used for backlog and Main Flow assignment, see `03-backlog/` and `02-roadmap-and-milestones.md` (`02-roadmap-and-milestones.md`) §5.1):

Name	Code	MSSV	Framework track	Tech stack	Lane	Main Flow
Đỗ Đăng Khoa	KhoaDD	SE192357	BIT_SE_IC_19A	Embedded C, **NestJS**, Spring Boot, Java, ReactJS, TypeScript, PostgreSQL	Leader, Prisma schema, `devices`, firmware, `gateway-sync`, UI layout, architecture	**MF-02**
Lâm Phi Long	LongLP	SE182769	**BIT_SE_NJS_18D**	Node.js, PostgreSQL, ReactJS, TypeScript	Delegate, `auth`, `rentals`, `billing`	**MF-05**
Trần Khải Hoàng	HoangTK	SE182767	**BIT_SE_NJS_18D**	Node.js, ReactJS	`incidents`, `monitoring` ingestion, WebSocket	**MF-03**
Nguyễn Ngọc Long	LongNN	SE193490	BIT_SE_IC_19A	Java Spring Boot, ReactJS, TypeScript, PostgreSQL	`frontend` (FSD), map and monitoring UI	**MF-04**
Nguyễn Bá Tân	TanNB	SE183860	BIT_SE_IC_18D	PostgreSQL, MySQL, ReactJS, TypeScript	Delegate, `trips`, booking, QA & test lead	**MF-01**

> **Skill-coverage note**, *corrected Session 6.* An earlier version of this matrix recorded NestJS as a skill held only by the leader, and that was wrong. **`BIT_SE_NJS_18D` is the Node.js enrolment track, and NestJS is a framework over Node**; LongLP and HoangTK are both on it, so backend capability is **3 of 5**, not 1 of 5. Core backend work is routed to them by lane. The residual exposure is the two IC-track members, LongNN and TanNB, picking up backend stories solo. Budgeted mitigation: ramp-up and pairing on `01-conventions/05-backend-conventions.md` (`05-backend-conventions.md`) across W2–W5, mentored by Khoa and LongLP. This is the substance of the Report 2 §2.3 Training Plan and is a downgraded risk in `03-decisions-and-risk-register.md` (`03-decisions-and-risk-register.md`).

> **Supervisor mandate** (2026-09-13): the team sets its own scope, flows and technology stack, and pitches and defends the result. The supervisor's outline is guidance, not a restriction. Decisions that depart from the registration form are logged in the decisions register and amended into the form before the Week-6 scope lock.

> This charter reconciles `Phieu_FA26SE159.docx` (primary, authoritative, has formal RQs/NFRs/experiments) with `Phieu_dang_ky_de_tai_TrekLink_FA26_FINAL.docx` (earlier draft, same project). Differences are called out inline and logged as Decision D-000 in `03-decisions-and-risk-register.md` (`03-decisions-and-risk-register.md`).

1. Problem

Market context. Trekking in Vietnam often takes place in remote mountainous areas (Tà Năng–Phan Dũng, Tả Liên Sơn, Bạch Mộc Lương Tử) where cellular connectivity is limited or unavailable for long stretches.

Identified gap. Integrated platforms combining off-grid IoT communication, monitoring, device management and structured incident coordination are not yet widely adopted in Vietnam's trekking service market.

Project opportunity. SU26 produced **TrekLink firmware**: a LoRa mesh (ESP32/ESP32-S3, based on the open-source Meshtastic v2.7.19, ~636 files / ~149k LOC, 4 hardware variants v1–v4) with mesh messaging, GPS tracking, IMU-based fall detection and device-level SOS, already working and **not being rebuilt**. This Capstone extends it into an integrated operational platform for the Vietnamese trekking environment (scope: D-033). Two technical requirements follow:

1. **Store-and-forward to the cloud.** Field events must survive long outages; the node's uplink queue holds 16 events in RAM, so a durable priority queue is added on the device and at the Field Station.
2. **Structured incident operations.** Each SOS becomes one incident with an owner, tiered alerts and an audit trail (D-034).

> Wording follows the Review 1 minutes (2026-10-01). The panel rejected the earlier framing that a device SOS reaches nobody and asked for comparison within Vietnam only.

2. What we're building

TrekLink Operations Platform, extends the inherited firmware with:

- **Gateway Bridge** (Node.js/TypeScript): LoRa-serial → SQLite priority queue (P0 SOS / P1 incident-location / P2 GPS / P3 telemetry) → MQTT → cloud, with store-and-forward and priority-ordered reconnection flush. Delivered in two stages, **both in permanent scope** (D-005): Stage A uses the node's own MQTT uplink; Stage B adds the basecamp bridge that holds the offline buffer. This is **MF-02**.
- **Idempotent event ingestion**: `GatewayEvent.eventId = sha256(nodeNum : packetId)` as the packet-level dedup key, plus open-Incident correlation for episode-level grouping, N deliveries of the same event ⇒ exactly 1 Incident + 1 notification. *The register's `DeviceID + SessionID + SequenceNumber` formula is not constructible from what the firmware transmits; see **D-006** and `04-firmware-ground-truth.md` (`04-firmware-ground-truth.md`) §3. The intent, a natural, device-derived idempotency key, is unchanged.*
- **Device Fleet & Rental Management**: 7-state device lifecycle `Available → Reserved → Rented → In-Field → Returned → Maintenance → Retired`.
- **SOS-to-Incident pipeline**: 5-state FSM `Detected → Acknowledged → In Progress → Resolved → Closed`, actor+timestamp+note on every transition, WebSocket push to Staff/Guide. This is **MF-03**.
- **Real-time monitoring**: live map of trip positions, device telemetry and gateway connectivity, rendered with **Leaflet over OpenStreetMap** and a Vietnamese sovereignty overlay (D-031). This is **MF-04**.
- **Trip/Booking/Billing**: packages, bookings, device reservation, check-out/check-in, deposits, damage/late fees, mock/sandbox payment. **MF-01** and **MF-05**.

Explicitly out of scope: rearchitecting the Meshtastic mesh stack, native mobile apps (responsive web only), production payment gateway, route/trail recommendation, localization, hardware certification (FCC/CE).

> **On firmware scope**, the register's "firmware redesign is out of scope" is read narrowly (**D-008**, supervisor-confirmed 2026-09-13). Firmware **enhancement and integration** is in scope and earns graded credit: a custom PortNum, an explicit SOS discriminator, a boot `sessionId` and per-packet `sequenceNumber`, and beacon priority are all legitimate work. What remains excluded is a rearchitecture of the mesh stack itself.

3. Actors & role summary

Role	Core capability
Admin	Orgs/users/roles, device types & firmware versions, pricing config, system health, audit logs
Staff	Bookings, device allocation, rental agreements, check-out/in, fees, incident acknowledgment/coordination
Guide	Assigned trips/devices, device handover confirmation, live group view, incident ack + response notes
Customer	Browse/book packages, reserve devices, view booking/rental/payment history

4. Research questions (drive the experiment plan: see roadmap)

- **RQ1**: Gateway reliability, delivery rate, data-loss rate, duplicate rate under intermittent connectivity.
- **RQ2**: Effect of connectivity-loss duration/reconnection pattern on delivery rate, sync latency, priority-ordering compliance.
- **RQ3**: SOS-to-Incident pipeline vs. uncoordinated baseline, MTTA, MTTR, traceability score, completion rate, via controlled drills.

5. Non-functional targets (binding: used as PR/Definition-of-Done gates)

Property	Target
API latency	≤300ms for 95% of requests @ 50 concurrent users
Gateway→cloud sync	≤5s when uplink available
Offline recovery delivery rate	≥99% after reconnection (30s–30min loss, ≥20 trials/condition)
Duplicate prevention	0 duplicate Incidents for repeated `eventId` delivery (10× replay test)
Priority-ordering compliance	≥99%, all P0 flushed before P2/P3 on reconnect
Concurrency	≥20 simultaneous gateway submissions, 0 loss/duplication
Fleet scale	≥50 devices, multiple concurrent trips, no architecture change
Security	JWT + RBAC + TLS + bcrypt; immutable/append-only incident & device audit logs

6. Tech stack (as registered: see D-001 in decisions log, resolved)

Layer	Technology
Embedded firmware	ESP32/ESP32-S3, C++/PlatformIO, RadioLib, Meshtastic base, **inherited, and editable this term**. Enhancement and integration are in scope and credited (D-008); mesh-stack rearchitecture is not
Gateway bridge	Node.js + TypeScript, SQLite (local queue), MQTT client
Backend	NestJS (TypeScript), REST + WebSocket (Socket.io)
Database	PostgreSQL. **ORM: Prisma** (resolved via D-001, FA26SE159 form originally specified TypeORM, team's FINAL draft specified Prisma; Prisma locked as team mandate)
Messaging	MQTT (Mosquitto, gateway↔backend) + Socket.io (backend↔browser)
Frontend	React + TypeScript, role-based views, **Leaflet** map over **OpenStreetMap** tiles with a Vietnamese sovereignty overlay (D-031, provider held in configuration, not hardcoded)
Auth/AuthZ	JWT + bcrypt; RBAC, CASL (`@casl/ability`) per the FINAL draft's more specific tech table
DevOps	Docker / Docker Compose, GitHub Actions CI/CD

7. Deliverables (map to Jira epics and the five Main Flows: see roadmap §4)

- TrekLink Operations Platform (web app, 4 role-based views)
- Gateway Bridge module (priority SQLite queue, idempotent MQTT sync, health reporting)
- Event Reliability Layer (eventId scheme, idempotency enforcement, sync audit log, dedup test suite)
- Device Fleet & Rental Management module
- SOS Incident Management module (FSM + WebSocket + audit trail)
- Real-Time Field Monitoring module
- Trip/Booking/Billing module
- Backend API (OpenAPI/Swagger) + WebSocket services
- Technical docs: SRS, 2 UML State Machine diagrams (Device 7-state, Incident 5-state), architecture, DB schema, API reference, test plan, deployment guide, user guide
- Evaluation report: RQ1/RQ2/RQ3 results
- Source + deployment package: gateway/backend/frontend repos, Docker Compose, CI/CD, demo environment

8. Legal & compliance

Short by design, but not optional, the faculty fault handbook §10 treats data, security and legal handling as a graded topic that groups routinely arrive unprepared for.

Area	Position
Map sovereignty	Vietnamese law penalises publishing a map of Vietnam that fails to fully or correctly depict national sovereignty, **Nghị định 174/2026/NĐ-CP Art. 93(3)(a)**, in force 1 Jul 2026, 30–40 M VND, with forced takedown or removal of the application. OpenStreetMap base layers label Hoàng Sa and Trường Sa with foreign toponyms. TrekLink uses OpenStreetMap under the supervisor's acceptance for the course, with an overlay that labels both archipelagos in Vietnamese, and keeps Goong Maps in configuration for any deployment beyond it (**D-031**). The overlay is checked over both archipelagos with filed screenshots (TC-24).
Personal data	Minimum-necessary collection. We store what operations require, name, contact, booking, rental and position history, and nothing else. No national ID, no document verification. Position data is operational telemetry tied to a rental, retained with the trip record.
Secrets	No API key or secret in source. The map key is unavoidably visible in the browser and is mitigated with an HTTP-referer allowlist and a per-IP rate limit; every other credential lives in environment configuration outside version control.
Third-party sources	Every third-party data source is cited: map tiles from OpenStreetMap (© OpenStreetMap contributors, ODbL); mesh firmware forked from Meshtastic (GPL-3.0) with attribution retained.
Payments	Sandbox/mock only. No real funds move, and no card data is collected, transmitted or stored.
Radio	LoRa operation on the Vietnamese licence-exempt band as configured in the inherited firmware. Hardware certification (FCC/CE) is out of scope and is not claimed.
Emergency-response disclaimer	TrekLink is an operations and coordination tool, not a certified safety-of-life system. It does not replace official search-and-rescue channels, and the user guide must say so plainly.

9. Related work / gap statement (condensed)

Meshtastic, which TrekLink builds on, is designed to work independently of infrastructure; TrekLink adds the organization-side layer. Generic IoT fleet platforms (AWS IoT Core, Azure IoT Hub, Balena) have no LoRa integration or store-and-forward for intermittent connectivity. DTN/store-and-forward research (RFC 4838, MQTT-SN) is protocol-level, not tied to a business workflow. IT incident tools (JIRA SM, PagerDuty, Opsgenie) have no IoT field-event ingestion. TrekLink combines the five dimensions in one domain-specific platform. Satellite messengers are deliberately not compared (Review 1: compare within Vietnam only).

TrekLink: The Five Main Flows

> **Source of the flow set**: `capstone/Documents/course-material/TrekLink-proposed-mainflow-ducndm.png`, supplied by the supervisor. These five are **binding**. The set was redrawn after Review 1 by **D-033**, which supersedes D-016; identifiers and owners are unchanged. They are the units the Mainflow Coverage Matrix tracks, the units demoed at each Iteration review, and the units the Faculty Council evaluates.
>
> **What this file is for.** It is the single source for the Main Flow material that appears in Report 2 §3, Report 3 (SRS), the Review 1 slide deck, and the Mainflow Coverage Matrix. Diagrams are Mermaid so they regenerate rather than rot, never paste a screenshot of one back into a doc.
>
> **Layout convention**: each diagram is a true Mermaid `swimlane-beta` diagram, one `subgraph` per actor lane, matching `Documents/templates/Main flows_ex02.jpg`, the actor-partitioned example the course supplies. `swimlane-beta` requires **Mermaid ≥ 11.16.0**; the handbook build is pinned to 12.0.0. Orientation is `TB` so lanes run as columns and the flow runs down the page, which is what fits A4 portrait. See `01-conventions/13-diagram-and-figure-conventions.md` (`13-diagram-and-figure-conventions.md`).
>
> **Every flow carries at least one exception scenario.** That is a W3 Definition-of-Done item and a named anti-fail check: *"Mỗi mainflow đều có ít nhất một exception scenario."* The scenarios below populate the Exception Scenario Matrix directly.

Actors across all flows: Guest · TrekLink Admin · TrekLink Staff · Organization Manager · Organization Operator · TrekLink Device · Field Station · Cloud Backend · SePay. Device holders carry devices and use the stock Meshtastic app; they are not system users.

MF	Name	Owner	Priority
MF-01	Organization onboarding → Rental contract → Handover	TanNB	High
MF-02	Field data → Field Station → Cloud synchronization	KhoaDD	**Highest**
MF-03	SOS → Tiered alert → Escalation	HoangTK	**Highest**
MF-04	Live telemetry: web map and organization API	LongNN	High
MF-05	Term end → Return → Inspection → Billing → Maintenance	LongLP	Medium

The device's life as a rented asset is the spine: MF-01 takes it out, MF-02 to MF-04 run while it is out, MF-05 brings it back and restocks it. Device intake and provisioning are supporting use cases that feed MF-01. **E7 (DevOps/CI-CD) and E8 (Research & Experimental Evaluation) are cross-cutting and deliberately are not Main Flows.** Say so whenever the Coverage Matrix is presented.

MF-01: Organization onboarding → Rental contract → Handover

Goal: take a trekking company from self-registration to holding provisioned, monitored devices under a signed rental contract. See **Figure 1**.

Actors: Organization Manager, Cloud Backend, TrekLink Staff, TrekLink Admin, SePay **Precondition**: devices of the requested variant in `AVAILABLE` **Postcondition**: organization `ACTIVE`; contract `ACTIVE`; devices `RENTED`, provisioned with the organization's channel key and monitored

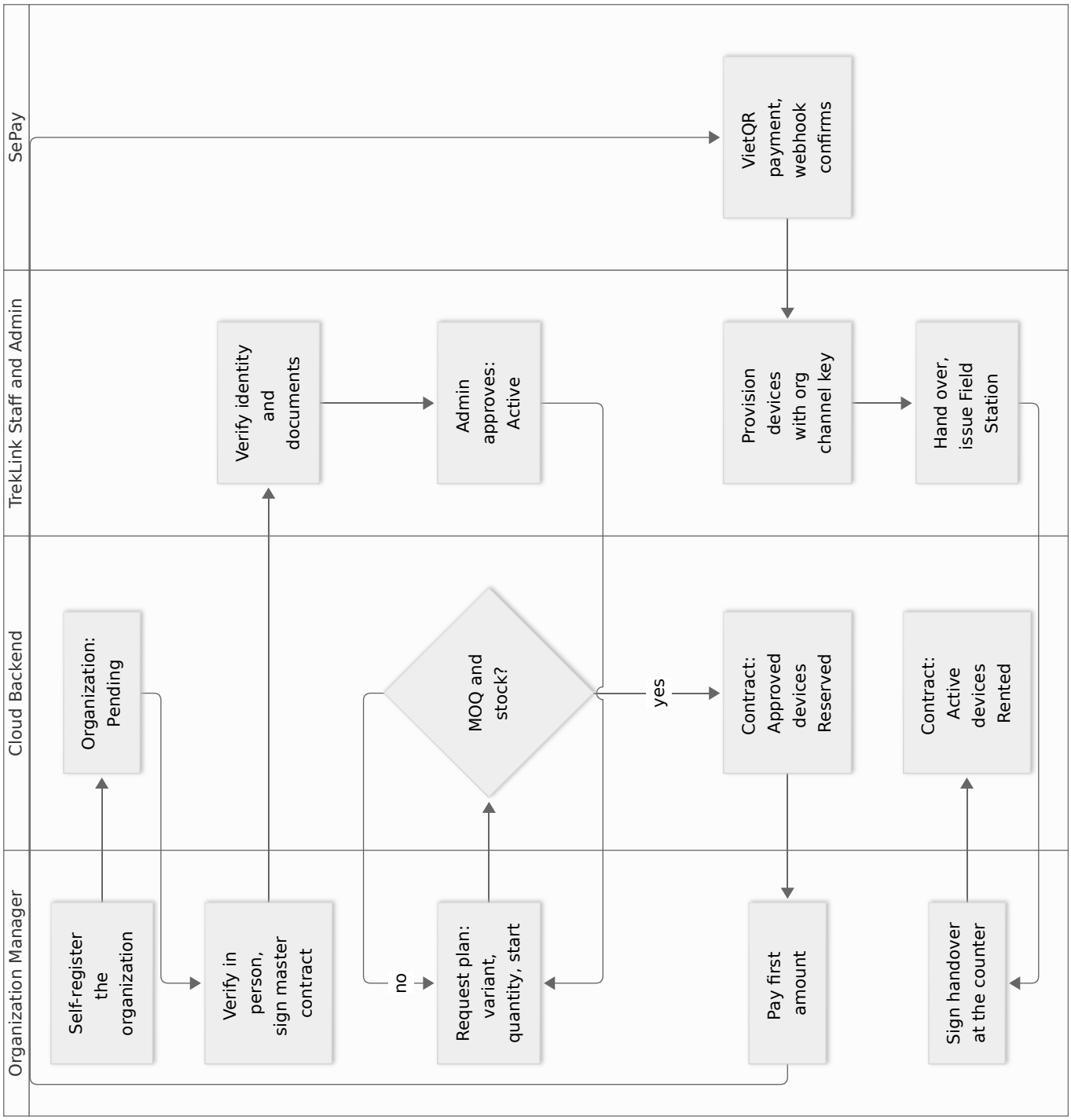


Figure 1: MF-01 Organization onboarding to Rental contract to Handover. Lanes are actors; the flow runs top to bottom. Placement: rotated plate, 182.0 x 194.6 mm, labels at 8.25 pt.

Main path

1. The Manager self-registers the organization (company name, tax code, Manager, contacts); the organization is `PENDING`.
2. The Manager comes to the TrekLink counter or calls; Staff verify identity and documents and the master contract is signed.
3. An Admin approves; the organization is `ACTIVE` and the Manager can invite members and set the on-duty roster.
4. The Manager requests a plan: monthly or day plan, hardware variant, quantity, start date.
5. The backend checks the minimum order quantity and stock; Staff approve and the devices become `RESERVED`.
6. The first amount is paid: the holding fee (half the term fee) for a monthly plan, the full amount for a day plan, through SePay or recorded at the counter.
7. Staff provision each device with the organization's channel key, hand them over with the Field Station package, and the Manager signs the handover note on the spot.
8. The contract becomes `ACTIVE`, the devices `RENTED`, and monitoring starts.

Exception scenarios

#	Scenario	Expected behaviour
E01-1	Two contracts claim the last devices of a variant at once	Allocation is transactional with row locks on the devices; the second approval sees the shortfall and is not approved. Never over-allocate.
E01-2	Requested quantity below the minimum order quantity	Rejected at request time with the current minimum shown.
E01-3	Organization still `PENDING`, `REJECTED` or `SUSPENDED` requests a plan	Rejected; only an `ACTIVE` organization can open a contract.
E01-4	Payment not confirmed by handover time	Handover cannot be signed; the contract stays `APPROVED` until the payment is confirmed or recorded, or Staff cancel it and the devices return to `AVAILABLE`.
E01-5	A reserved device fails the provisioning check at handover	Device to `MAINTENANCE`; Staff swap in another `AVAILABLE` unit of the same variant inside the same contract; if none exists the handover is held.
E01-6	SePay webhook arrives twice, or after a counter payment was recorded	The payment is matched by its reference and recorded once; a duplicate is a logged no-op.
E01-7	The organization cancels after approval, before handover	Contract `CANCELLED`; reserved devices return to `AVAILABLE`; a holding fee already paid is retained.

Business rules touched: organization approval, minimum order quantity, no over-allocation, first-payment rule per plan, on-the-spot handover with signature, channel-key provisioning.

Configurable parameters (D-015): minimum order quantity, monthly price per variant, day-plan lengths and premium, holding-fee ratio, reservation expiry before handover.

MF-02: Field data → Field Station → Cloud synchronization

Goal: deliver every field event from the mesh to the cloud exactly once, including events generated while the uplink is down. This is the flow that carries the project's research claim. See **Figure 2**.

Actors: TrekLink Device, Field Station (Gateway Bridge), Cloud Backend **Precondition**: device `RENTED` on a running contract; a node uplinks over its own Wi-Fi (Stage A) or the organization runs the Field Station (Stage C) **Postcondition**: every event persisted exactly once, in priority order, regardless of uplink state

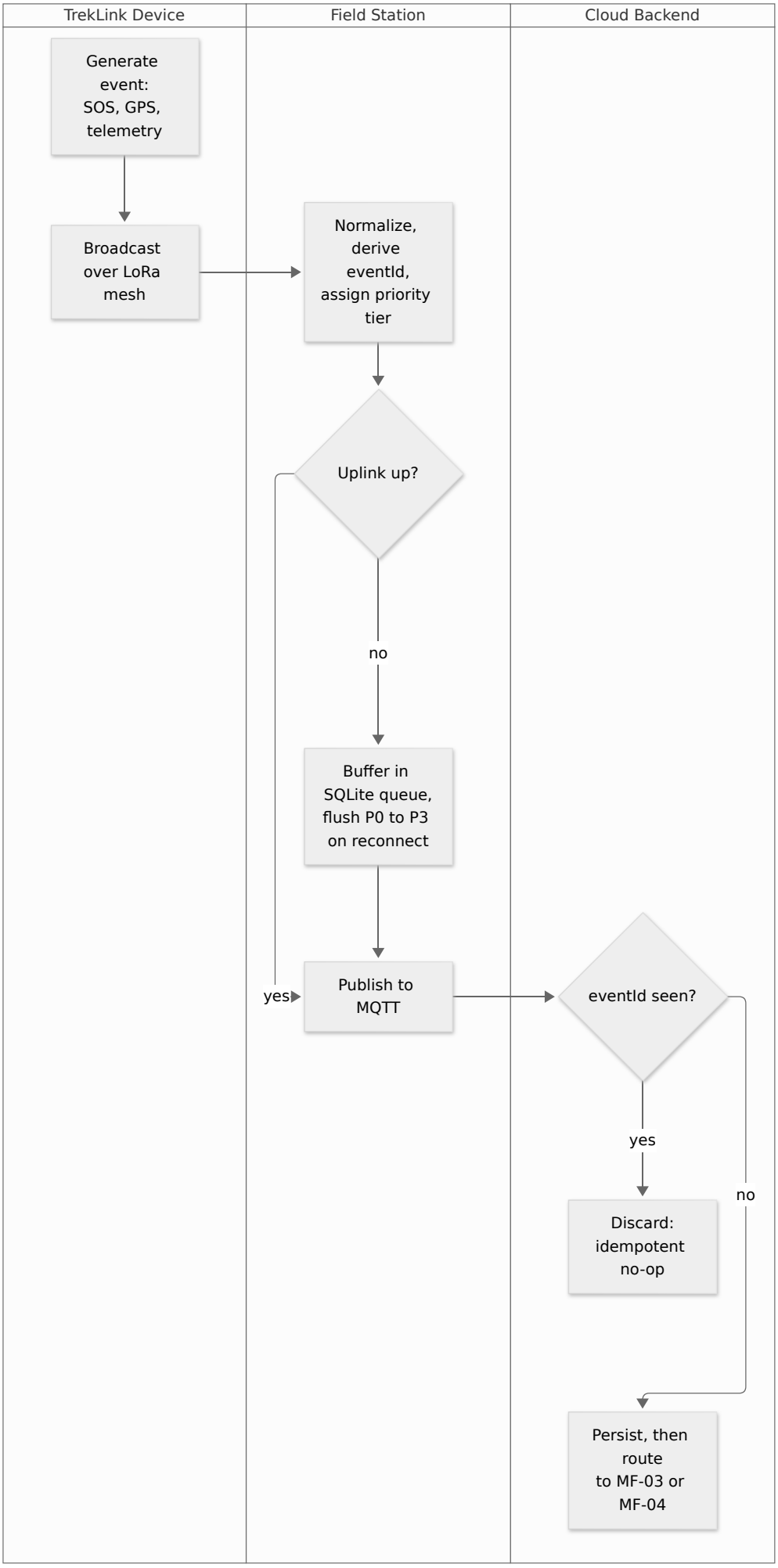

TrekLink Device
Field Station
Cloud Backend
Generate event: SOS, GPS, telemetry
Broadcast over LoRa mesh
Normalize, derive eventId, assign priority tier
Uplink up?
no
Buffer in SQLite queue, flush P0 to P3 on reconnect
yes
Publish to MQTT
eventId seen?
yes
Discard: idempotent no-op
no
Persist, then route to MF-03 or MF-04

Figure 2: MF-02 Field data to Field Station to Cloud synchronization. The SQLite buffer is on the no branch: events generated while the uplink is down are held and flushed in priority order. Placement: inline, 135.0 x 266.0 mm, labels at 7.50 pt.

Priority tiers: `P0` SOS · `P1` incident location · `P2` GPS · `P3` telemetry. All P0 events flush before any P2 or P3 event, the ordering-compliance NFR is measured on exactly this.

Three delivery stages, all in permanent scope (D-005, refined by **D-018**): **Stage A** uses the node's own MQTT uplink and has no durable buffer; **Stage B** is the on-device durable priority queue in the firmware, built first; **Stage C** is the **Field Station**, a bundled executable on the organization's own laptop with one of its rented nodes on a USB serial port; it holds the large SQLite queue for the whole local mesh and shows the field picture locally while offline (form per **D-033**, superseding D-020). Stage A is the first increment, never a substitute. Stage B buffers only its own node's events, so it does not replace Stage C. Documents written before 2026-09-17 call the basecamp bridge "Stage B"; that is Stage C.

Main path

1. Device generates an event and broadcasts it over the LoRa mesh.
2. Gateway receives the packet and normalizes it to the internal `TrekLinkEvent` shape.
3. Gateway derives `eventId = sha256(nodeNum : packetId)` and assigns a priority tier.
4. Uplink available → publish to MQTT. Uplink down → enqueue in SQLite by priority.
5. On reconnection, the queue flushes in strict priority order, oldest first within a tier.
6. Backend consumes, checks `eventId` against a unique index, discards duplicates as a no-op.
7. New events are persisted and routed by kind, SOS into MF-03, positions and telemetry into MF-04.

Exception scenarios

#	Scenario	Expected behaviour
E02-1	Uplink drops mid-flush	Partially flushed batch is not lost; only rows confirmed published are deleted. Flush resumes from the queue head on reconnect.
E02-2	The same packet arrives twice, mesh rebroadcast or replay	Unique index on `eventId` makes the second insert a no-op. **0 duplicate Incidents across a 10× replay is a binding NFR and a demo obligation.**
E02-3	Gateway process restarts with a non-empty queue	SQLite is on disk; the queue survives and flushes on next connect. Restart must not mint new `eventId`s.
E02-4	Malformed or undecodable packet	Logged with the raw payload, counted, and dropped, never crashes the ingest loop, never blocks the queue head.
E02-5	Queue grows beyond its configured bound during a long outage	P3 telemetry is shed first, P0 never. The shedding policy is configuration and the event is logged.
E02-6	Clock skew between gateway and backend	The gateway flush orders by queue sequence and priority, not wall-clock comparison across hosts. The backend orders display, trails and episode correlation by the event's own timestamp when it is valid, because stock firmware drains a queued backlog one entry per reconnect.

Business rules touched: priority-tier assignment, idempotency, flush ordering, queue retention and shedding, reconnect backoff.

Configurable parameters (D-015): MQTT broker URL and topic prefix, reconnect period, flush batch size, queue size bound, shedding policy, priority-tier mapping, sync-latency target.

Research questions: **RQ1** (delivery, loss and duplicate rates under intermittent connectivity) and **RQ2** (effect of outage duration on delivery rate, sync latency, ordering compliance) are both measured on this flow.

MF-03: SOS → Tiered alert → Escalation

Goal: turn a device SOS into one incident that a named organization member owns, and make sure that somebody is always responsible for it. See **Figure 3**. The state machine is **D-034**.

Actors: TrekLink Device, Field Station, Cloud Backend, Organization Operator (on duty and backup), Organization Manager, TrekLink Staff **Precondition**: device `RENTED`, or any device that raises an SOS **Postcondition**: exactly one incident per SOS episode, owned at every moment, `CLOSED` with a complete audit trail

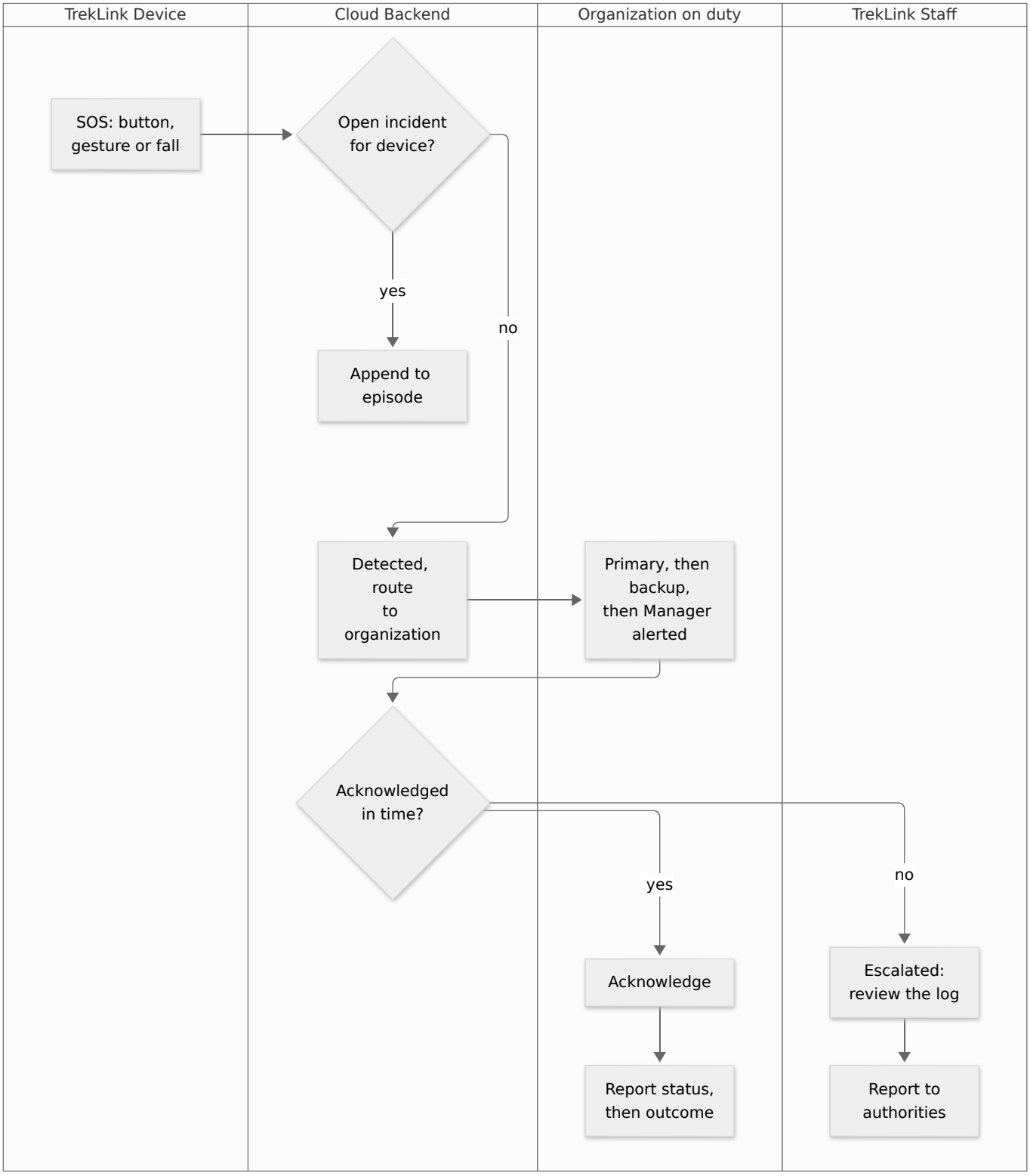


Figure 3: MF-03 SOS to Tiered alert to Escalation. Each tier has its own timeout; an unacknowledged alert always ends with TrekLink Staff. Placement: inline, 182.0 x 207.2 mm, labels at 7.71 pt.

Main path

1. The device raises an SOS; MF-02 delivers it first (P0).
2. The backend correlates it to an open episode or creates the incident and routes it, in one transaction, to the organization holding the device.
3. The on-duty member is alerted on the web, over WebSocket, by email and as an API event. Without acknowledgement within the timeout the backup is alerted, then the Manager.
4. A member acknowledges and owns the response; the organization coordinates the rescue itself, outside TrekLink.
5. The owner reports status on a cadence, then the outcome; the incident is `RESOLVED`, or `FALSE_ALARM` with a reason.
6. After the reopen window the incident is `CLOSED`.
7. If no tier acknowledges, or the response goes silent, the incident is `ESCALATED`; TrekLink Staff review the audit log and report to the authorities, recording the agency and reference.

Exception scenarios

#	Scenario	Expected behaviour
E03-1	The single SOS text frame is lost over RF	Cadence-anomaly detection raises a suspected episode from beacon density, flagged as lower confidence.
E03-2	One episode produces dozens of beacons	Correlation appends to the open incident. One fall produces exactly one incident.
E03-3	Two members acknowledge at once	Compare-and-set on state and version; the first wins, the second sees who owns it.
E03-4	SOS from a device on no running contract, or the organization has no reachable member	`UNROUTED`; TrekLink Staff own it. Safety events are never dropped.
E03-5	The owner stops reporting during a response	After the stale limit, `ESCALATED` with the owner kept and the Manager notified.
E03-6	New SOS from the same device after `RESOLVED`, inside the reopen window	Reopens to `NOTIFY_PRIMARY`; the reopen is in the audit trail.
E03-7	Alert delivery fails (WebSocket down, email bounce)	The incident and its timers proceed; delivery is retried from the outbox and logged.
~~E03-8~~	Withdrawn by D-038: a cancel pressed on the device changes nothing, and the firmware sends none.	Only a person declares a false alarm.
E03-9	The contract is overdue or the organization suspended	The incident is still created and routed. Safety first.

Business rules touched: one episode one incident, tier order and timeouts, acknowledgement authority, stale-response escalation, authority reporting by TrekLink only on escalation, audit immutability.

Configurable parameters (D-015): episode window, cadence-anomaly threshold, tier timeouts, stale-response limit, reopen window, notification channels.

Research question: **RQ3** (time to acknowledge, time to resolve, traceability) is measured on this flow.

MF-04: Live telemetry: web map and organization API

Goal: give each organization a live picture of its own rented devices, on the TrekLink web platform or inside its own system through the API, and give TrekLink a fleet-wide view. See **Figure 4**.

Actors: TrekLink Device, Field Station, Cloud Backend, Organization Operator, Organization Manager, organization's own system (API client), TrekLink Staff **Precondition**: at least one device `RENTED` **Postcondition**: each viewer sees only what its organization rents, within the sync-latency target

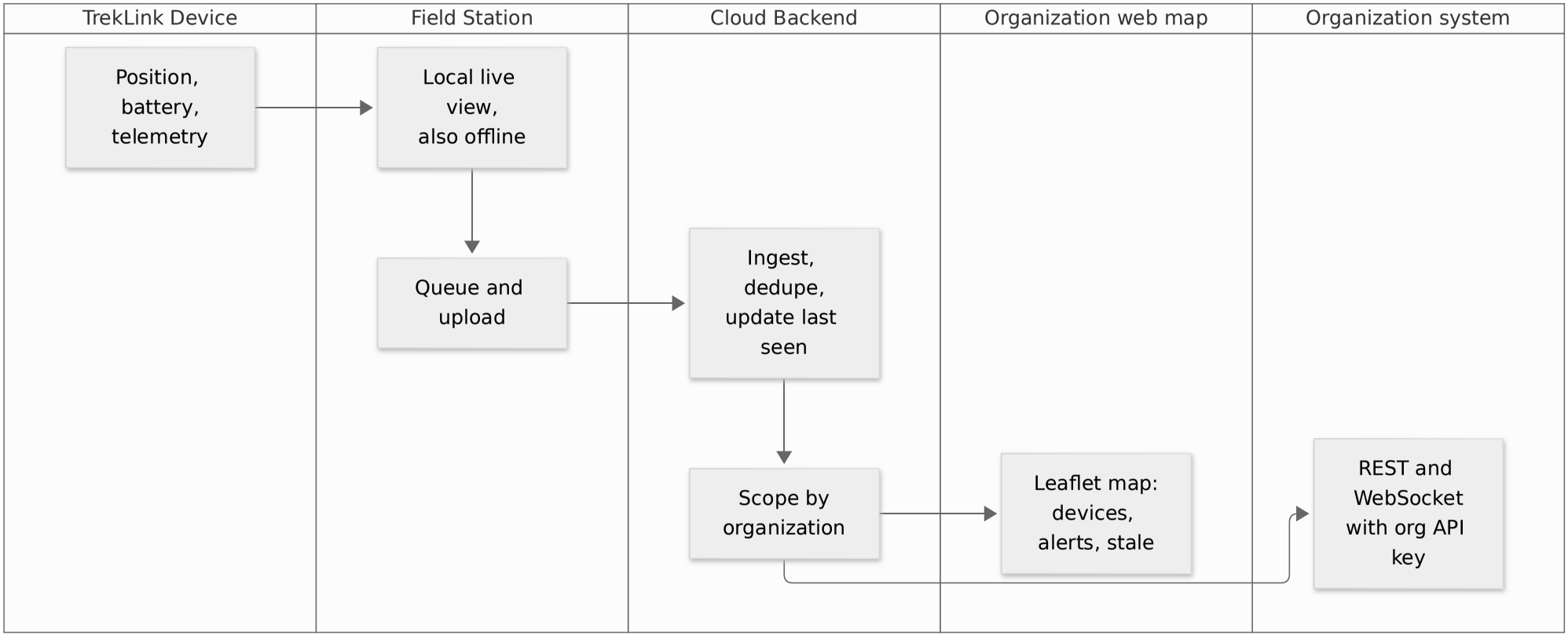

TrekLink Device
Field Station
Cloud Backend
Organization web map
Organization system
Position, battery, telemetry
Local live view, also offline
Queue and upload
Ingest, dedupe, update last seen
Scope by organization
Leaflet map: devices, alerts, stale
REST and WebSocket with org API key

Figure 4: MF-04 Live telemetry to the web map and the organization API. Scoping by organization is applied on the server, at the query and at the WebSocket emit. Placement: rotated plate, 104.2 x 266.0 mm, labels at 9.45 pt.

Main path

1. Devices report position, battery and telemetry; MF-02 delivers them.
2. The backend updates last seen and last position and scopes every read and push to the organization holding the device.
3. Organization operators see their devices and alerts on the Leaflet map; TrekLink Staff see the whole fleet.
4. An organization's own system reads the same data over REST and subscribes over WebSocket with its organization API key.
5. Offline, the Field Station shows the local mesh directly to the operators on site.

Exception scenarios

#	Scenario	Expected behaviour
E04-1	Device silent: out of range or flat battery	Marker ages to stale after the threshold, last seen shown. Never a frozen marker without indication.
E04-2	Field Station or uplink offline	Per-station connectivity indicator turns stale; the local view keeps working on site.
E04-3	Browser or API client loses the WebSocket	Reconnect with resync from the last event cursor; a visible reconnecting indicator on the web.
E04-4	An API key or user asks for another organization's device	Denied on the server; the data is never sent.
E04-5	API key revoked or rate limit exceeded	401 or 429 in the standard envelope; the key's last use is logged.
E04-6	Implausible position jump	Rejected at validation, logged, not plotted.
E04-7	Device returned: no longer rented by the organization	Its live data stops reaching that organization from the check-in time; history stays with TrekLink.

Business rules touched: organization scoping, API key per organization, staleness thresholds, position validation, map sovereignty overlay (D-031).

Configurable parameters (D-015): stale-device and stale-station thresholds, API rate limit, map provider and viewport, battery warning levels, position sanity bounds.

MF-05: Term end → Return → Inspection → Billing → Maintenance

Goal: end the contract, take every device back at the counter, assess it, settle the money and restock or service the device. See **Figure 5**.

Actors: Organization Manager, TrekLink Staff, TrekLink Admin, Cloud Backend, SePay **Precondition**: contract `RETURN_DUE` (day plan ended, or monthly notice given and the term ended) **Postcondition**: contract `CLOSED`; devices `AVAILABLE`, `MAINTENANCE`, `LOST` or `RETIRED`

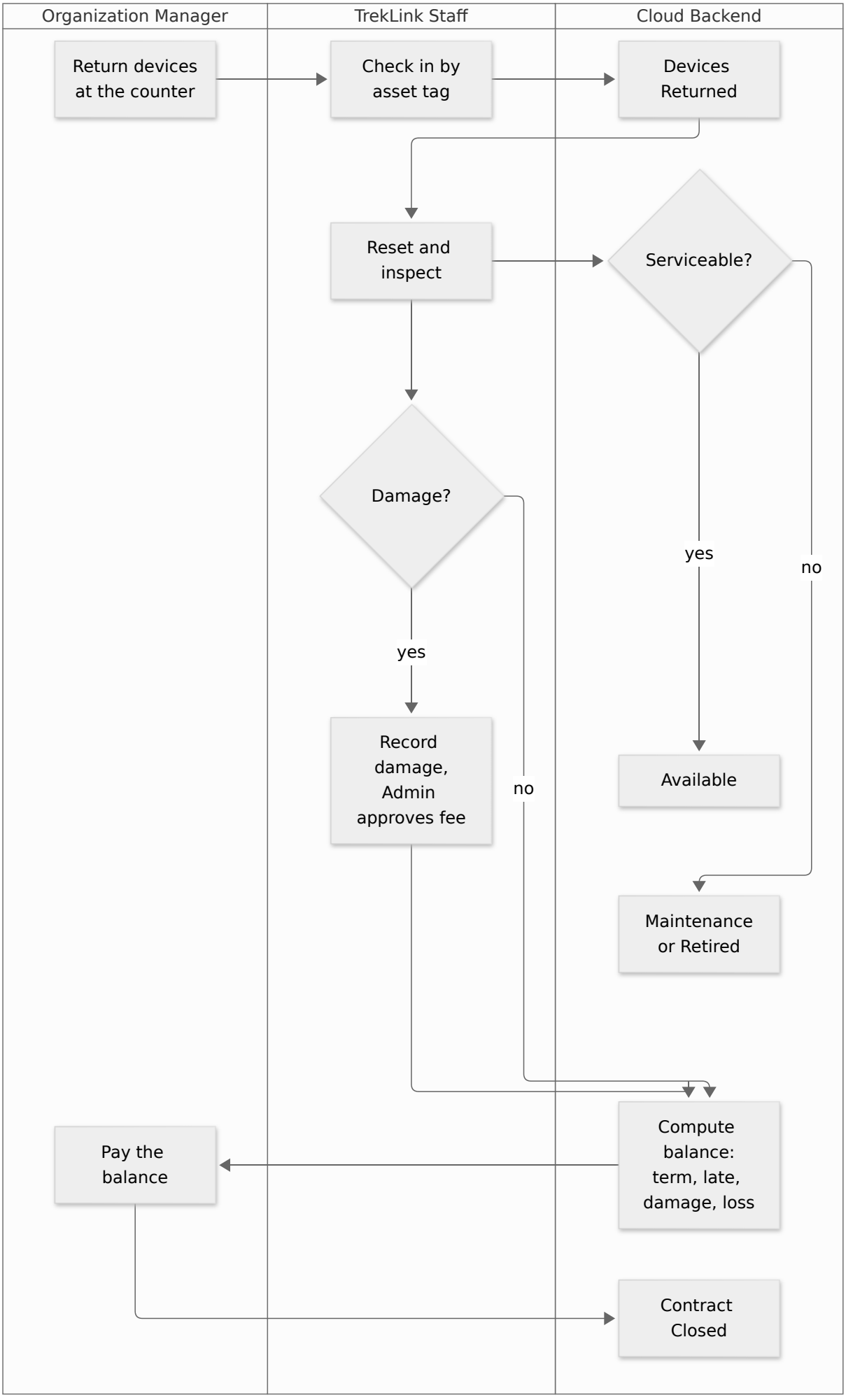

Organization Manager
TrekLink Staff
Cloud Backend
Return devices at the counter
Check in by asset tag
Devices Returned
Reset and inspect
Serviceable?
Damage?
yes
no
yes
no
Record damage, Admin approves fee
Available
Maintenance or Retired
Compute balance: term, late, damage, loss
Pay the balance
Contract Closed

Figure 5: MF-05 Term end to Return to Inspection to Billing to Maintenance. Placement: inline, 162.0 x 266.0 mm, labels at 9.00 pt.

Main path

1. A day plan reaches its end date, or a monthly contract reaches the end of the term after notice; the contract is `RETURN_DUE`.
2. The Manager returns the devices at the counter; Staff check each in by asset tag; devices `RETURNED`.
3. Staff reset each device (channel key, node database, owner name) and inspect it.
4. The backend computes the balance: the second half of the term fee for a monthly plan, plus late, damage and loss charges; damage above the threshold waits for Admin approval.
5. The Manager pays through SePay or at the counter; the contract is `CLOSED` when every charge is paid and no incident is open.
6. Serviceable devices return to `AVAILABLE`; others go to `MAINTENANCE`, and to `RETIRED` if beyond repair.

Exception scenarios

#	Scenario	Expected behaviour
E05-1	Devices returned late	Contract `OVERDUE`; late fee per device per day after the grace period, itemized on the invoice.
E05-2	Devices not returned after the grace period, or the organization stops responding	Contract `DEFAULTED`; the organization is `SUSPENDED`; the devices' last positions are tracked; Staff report to the authorities with the audit log as evidence.
E05-3	Device recorded lost	Device `LOST`; loss charge at its remaining value; `RETIRED` on write-off, back to `RETURNED` if recovered.
E05-4	Payment fails part-way	The contract does not close; the balance stays outstanding and visible to both sides.
E05-5	Monthly term ends without notice	No return: the term's balance and the next term's holding fee fall due together, and the contract runs on.
E05-6	The inspector also approves a damage waiver above the threshold	Refused: the approver must differ from the inspector.
E05-7	An incident on a returned device is still open	The contract cannot close until the incident is `CLOSED` (D-034).

Business rules touched: term balance, late fee, damage fee and approval, loss at remaining value, reset before restock, separation of duty, closing conditions.

Configurable parameters (D-015): late fee and grace period, damage-fee schedule and approval threshold, remaining-value schedule per variant, default threshold, maintenance turnaround target.

Coverage tracking

The Mainflow Coverage Matrix is the single instrument the supervisor uses to confirm sprint progress at the weekly Group Meeting. It is authoritative over any other progress view.

Mainflow	Requirement	Implementation	Tested	Demo Ready	Owner
MF-01	☐	☐	☐	☐	TanNB
MF-02	☐	☐	☐	☐	KhoaDD
MF-03	☐	☐	☐	☐	HoangTK
MF-04	☐	☐	☐	☐	LongNN
MF-05	☐	☐	☐	☐	LongLP

Milestones: MF-01 and MF-02 complete by **W7**; MF-03 in progress with MF-04 and MF-05 holding Figma plus detailed spec by **W7**; feature complete by **W11**; 100 % Implemented / Tested / Demo Ready by **W12**, before the Faculty Council.

Demo discipline, the faculty handbook ranks a Main Flow breaking during the demo as the single most common cause of failure. Every flow gets a rehearsed demo script, realistic seed data, and a backup screen recording. Demo the real product; never build a special build to demo.

TrekLink: Requirements Foundation

> **What this is.** The five requirement artifacts SEP490 requires by the end of W3 and presents at Review 1 (W4): Context Diagram, Actors, Use Cases, Feature Tree, Business Rule Matrix, and the Exception Scenario Matrix. Report 3 (SRS) is written **from** this file; it is not a second copy.
>
> **Source of the Main Flows** these decompose: `05-main-flows.md` (`05-main-flows.md`) (**D-033**, superseding D-016).
>
> **Rewritten 2026-10-03** for the enterprise rental platform (D-033, D-034, D-035). The full use case catalogue, the specifications and the functional requirements live in the Report 3 generator (`capstone/scripts/srs/srs_data.py`); this file holds the diagrams and matrices it draws from.
>
> **Anti-fail checks this file exists to pass** (SEP490 W3, and the faculty fault handbook §2): no requirement of the form "Admin manages everything" · use cases are functions, not workflows · `include` / `extend` / generalization used correctly · every actor present · use case names in verb-object form · every Main Flow carries at least one exception scenario · every business rule traceable to a requirement, an implementation and a test case.

---

1. Context Diagram

The system boundary, the external actors, and the external systems it exchanges data with. Nothing about internal structure appears here, that is the architecture diagram's job. See **Figure 6**.

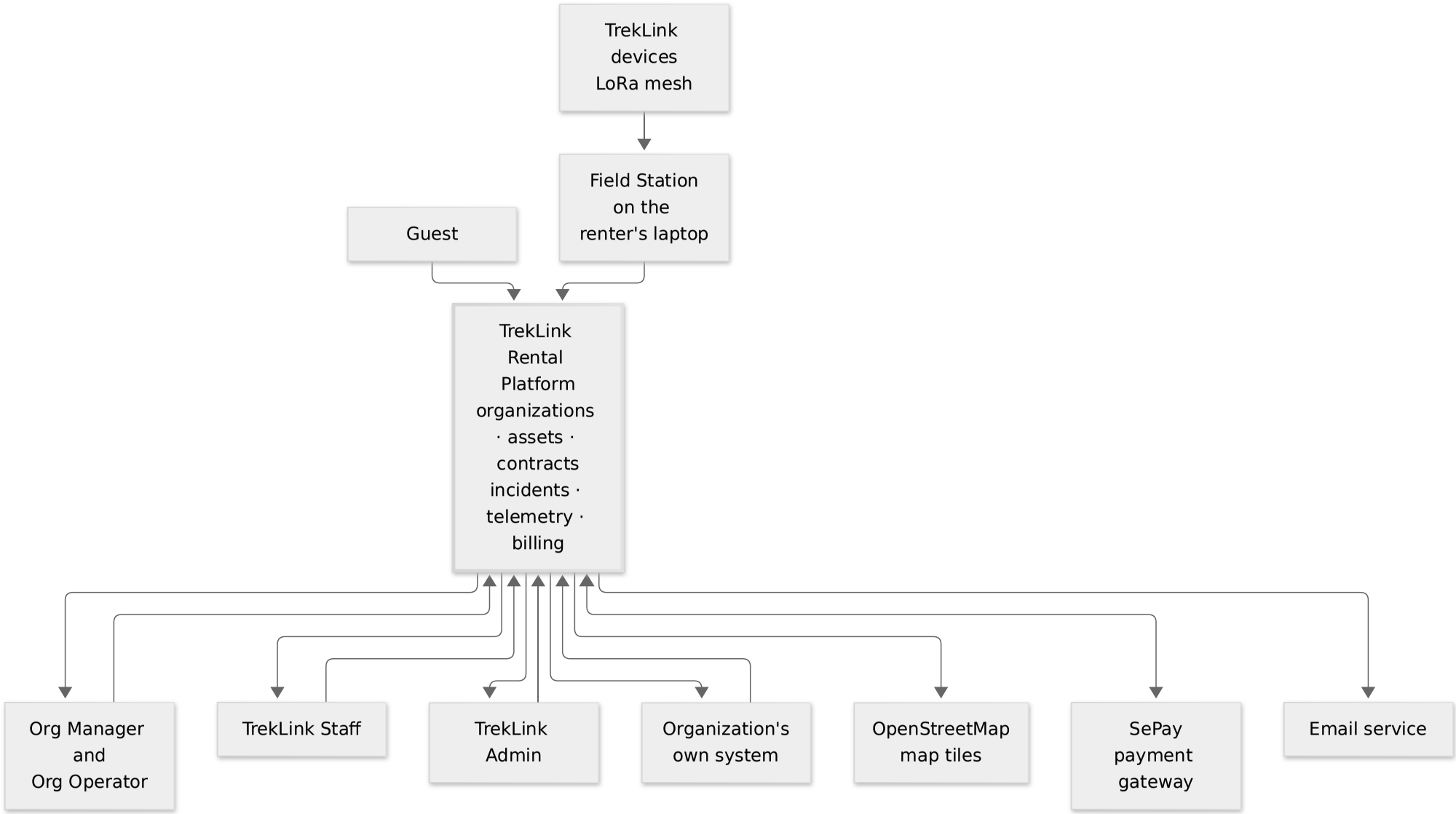

TrekLink devices LoRa mesh
Field Station on the renter's laptop
Guest
TrekLink Rental Platform organizations · assets · contracts incidents · telemetry · billing
Org Manager and Org Operator
TrekLink Staff
TrekLink Admin
Organization's own system
OpenStreetMap map tiles
SePay payment gateway
Email service

Figure 6: Context Diagram. The system boundary, its human actors, the organization's own system, and the external systems it exchanges data with. Placement: rotated plate, 145.1 x 266.0 mm, labels at 9.14 pt.

2. Actors

Actor	Type	Description	Main Flows
Guest	Primary, human	Registers an organization	MF-01
Org Manager	Primary, human	Signs contracts and handover notes, requests plans, pays, manages members, roster and API keys; last alert tier	MF-01, MF-03, MF-04, MF-05
Org Operator	Primary, human	On-duty member: receives alerts, acknowledges, reports status, watches the map, runs the Field Station	MF-02, MF-03, MF-04
TrekLink Staff	Primary, human	Verification, contract approval, device intake, handover, check-in, reset and inspection, payments at the counter, authority reports	MF-01, MF-03, MF-05
TrekLink Admin	Primary, human	Organization approval, plans and prices, parameters, damage approvals, retirement, platform accounts	all
Organization System	External system	The organization's own software reading telemetry with its API key	MF-04
TrekLink Device	Secondary, system	ESP32 LoRa node; emits SOS, position, telemetry	MF-02, MF-03, MF-04
Field Station	Secondary, system	TrekLink's executable on the renter's laptop: queue, upload, local page	MF-02, MF-04
Scheduler	Secondary, system	Alert tiers, staleness, term rollover, overdue and default	MF-03, MF-04, MF-05
SePay	External system	VietQR payment requests and confirming webhooks, sandbox	MF-01, MF-05
OpenStreetMap	External system	Raster map tiles (**D-031**)	MF-04

Role generalization. A TrekLink Admin holds every TrekLink Staff permission; an Org Manager holds every Org Operator permission inside the same organization. Device holders are not actors: they use the stock Meshtastic app.

3. Use Cases

Named verb-object. Each is a **function a user invokes**, not a multi-step process; the Main Flow diagrams in `05-main-flows.md` (`05-main-flows.md`) are where workflows belong. The full catalogue (UC-01 to UC-59) with specifications is generated into Report 3 §2.2 from `srs_data.py` .

3.1 Use case diagram

The headline use cases, split across three figures for legibility. See **Figure 7**, **Figure 8** and **Figure 9**.

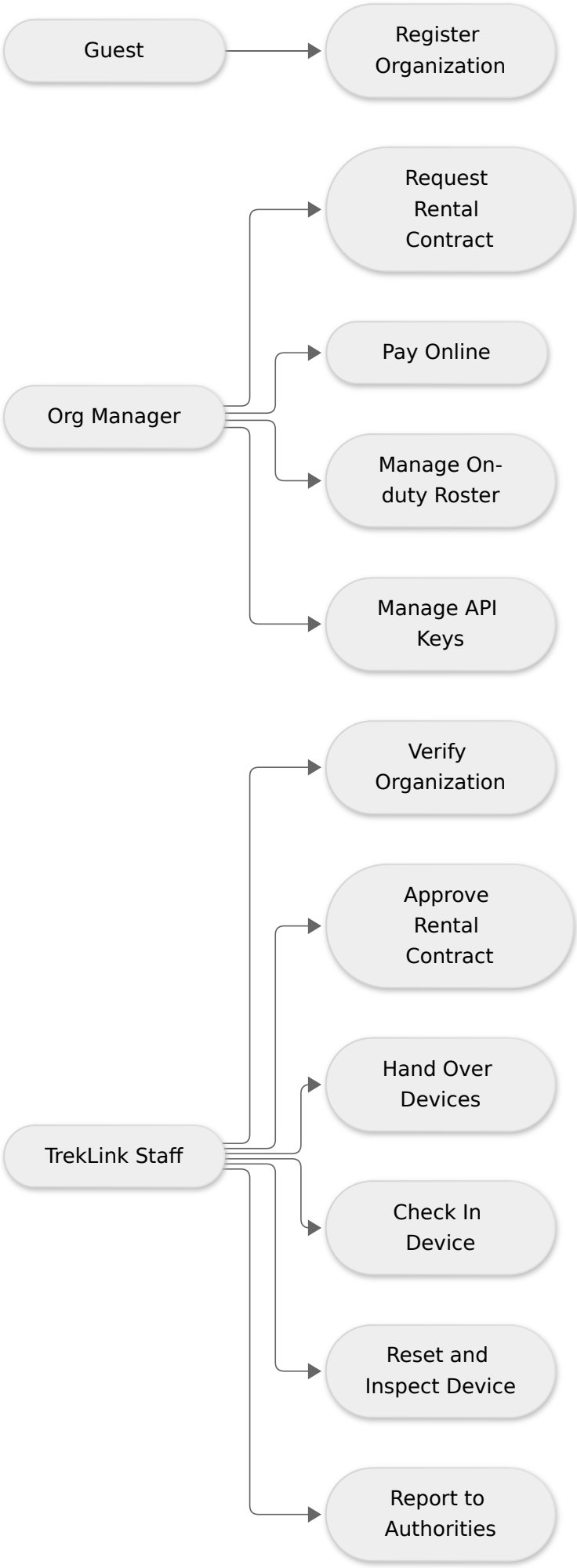

Guest
Register Organization
Org Manager
Request Rental Contract
Pay Online
Manage On-duty Roster
Manage API Keys
TrekLink Staff
Verify Organization
Approve Rental Contract
Hand Over Devices
Check In Device
Reset and Inspect Device
Report to Authorities

Figure 7: Use cases invoked by the Guest, the Org Manager and TrekLink Staff. Placement: inline, 100.3 x 266.0 mm, labels at 10.19 pt.

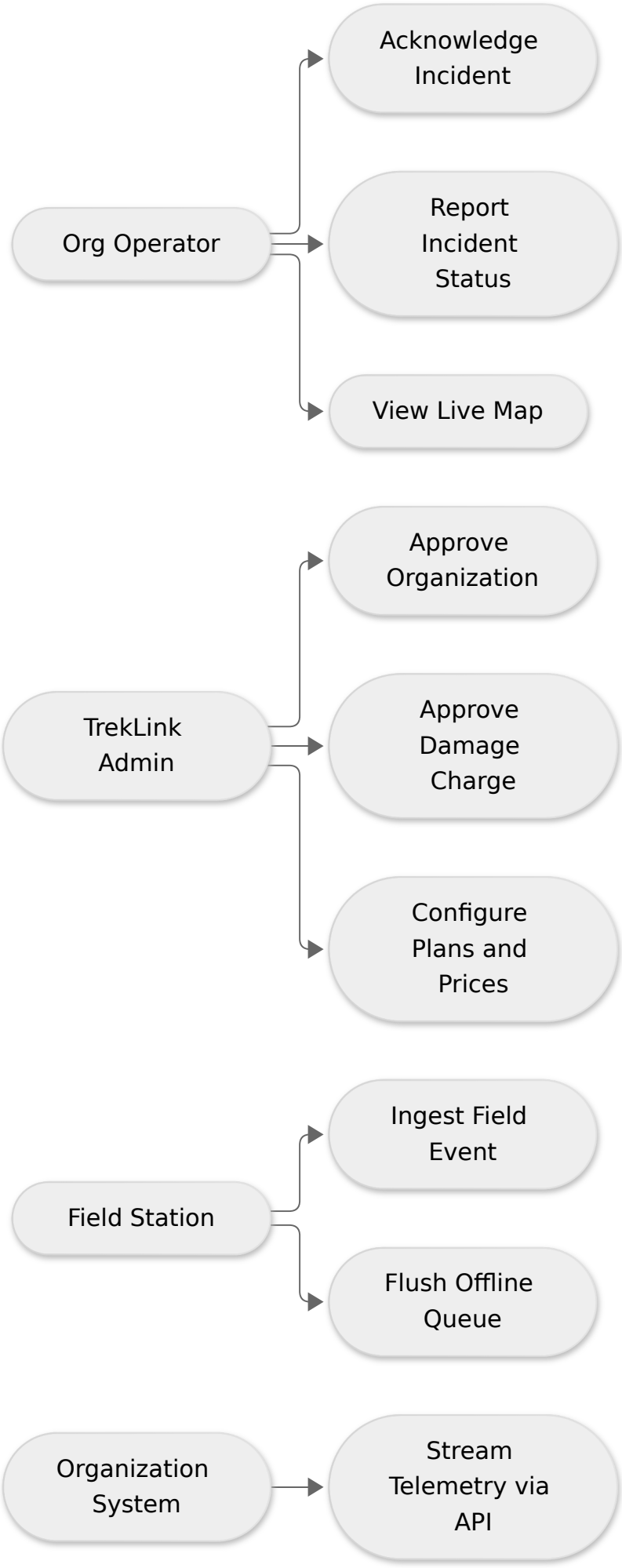

Org Operator
Acknowledge Incident
Report Incident Status
View Live Map
TrekLink Admin
Approve Organization
Approve Damage Charge
Configure Plans and Prices
Field Station
Ingest Field Event
Flush Offline Queue
Organization System
Stream Telemetry via API

Figure 8: Use cases invoked by the Org Operator, the TrekLink Admin, the Field Station and the organization's own system. Placement: inline, 110.0 x 266.0 mm, labels at 11.69 pt.

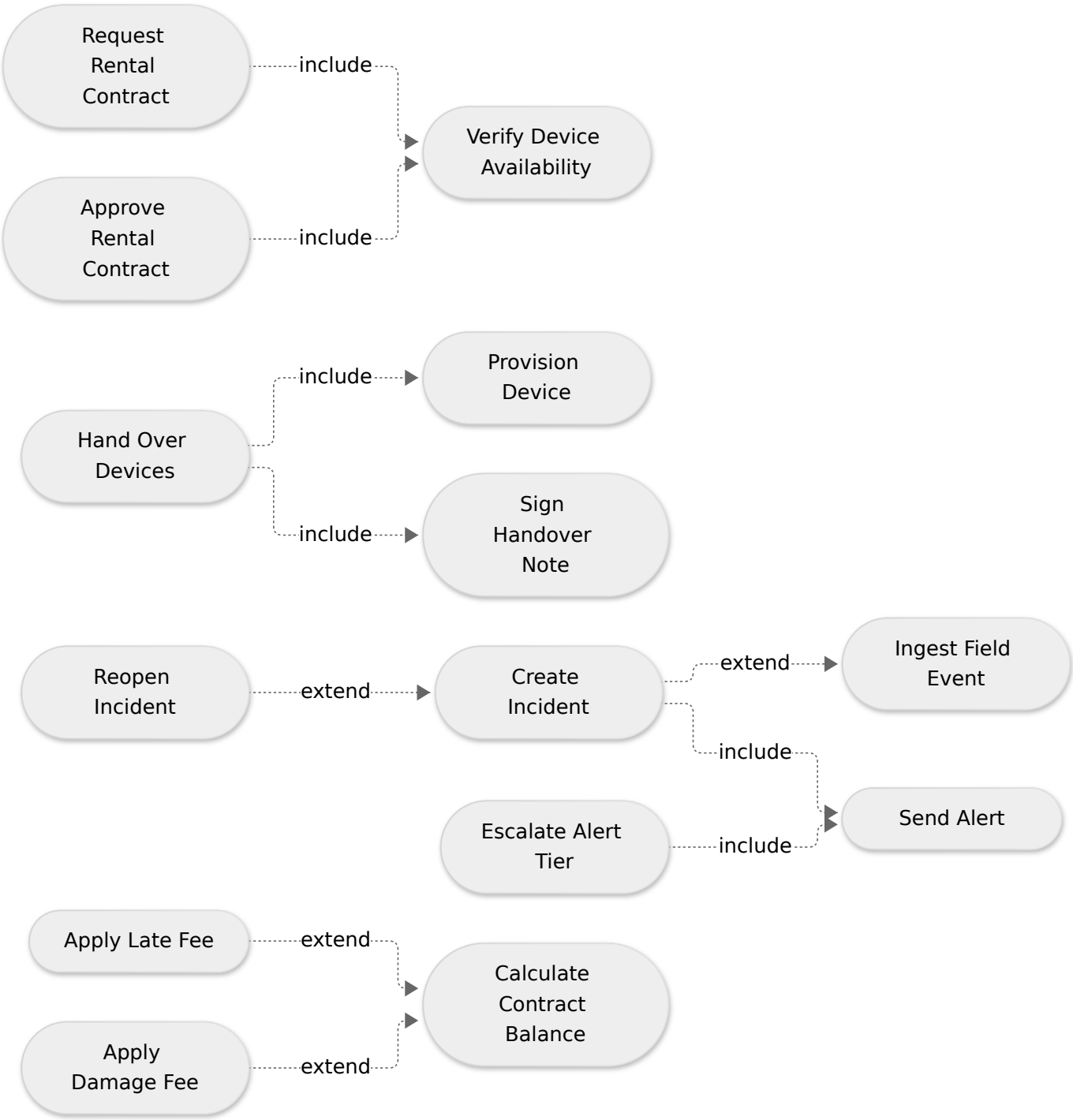


Figure 9: `include` and `extend` relationships. An included case always runs; an extending case runs only when its condition holds. Placement: rotated plate, 182.0 x 187.7 mm, labels at 10.21 pt.

3.2 `include` and `extend`: how they are used here

- `include`: the included use case **always** runs as part of the base. Request and Approve Rental Contract always verify availability; Hand Over Devices always provisions every device and captures the signed handover note; Create Incident and Escalate Alert Tier always send an alert.
- `extend`: the extending use case runs **only when a condition holds**. Create Incident extends Ingest Field Event when the event is an SOS or crosses the cadence threshold; Reopen Incident extends Create Incident when the same device raises a new SOS inside the reopen window; Apply Late Fee and Apply Damage Fee extend Calculate Contract Balance when the return is late or damaged.

- **Generalization**: only between actors (§2).

4. Feature Tree

Split across three figures for legibility. See **Figure 10**, **Figure 11** and **Figure 12**.

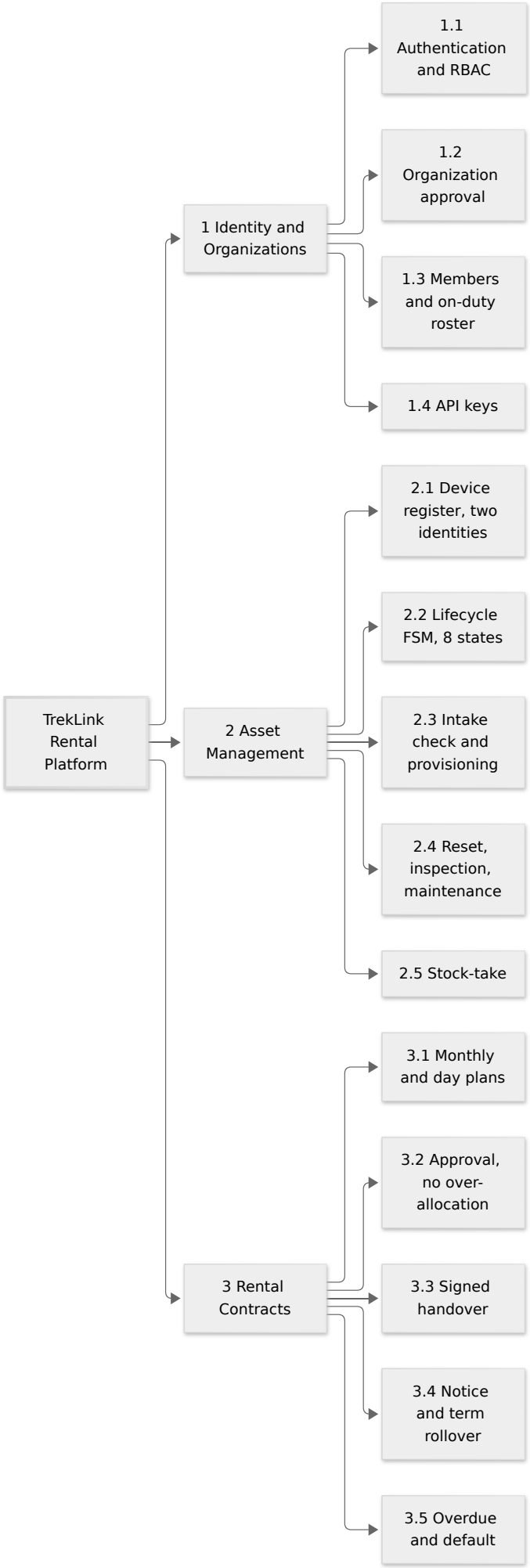

TrekLink Rental Platform
1 Identity and Organizations
1.1 Authentication and RBAC
1.2 Organization approval
1.3 Members and on-duty roster
1.4 API keys
2 Asset Management
2.1 Device register, two identities
2.2 Lifecycle FSM, 8 states
2.3 Intake check and provisioning
2.4 Reset, inspection, maintenance
2.5 Stock-take
3 Rental Contracts
3.1 Monthly and day plans
3.2 Approval, no over-allocation
3.3 Signed handover
3.4 Notice and term rollover
3.5 Overdue and default

Figure 10: Feature Tree, part 1 of 3, Identity and Organizations, Asset Management, Rental Contracts. Placement: inline, 91.4 x 266.0 mm, labels at 7.25 pt.

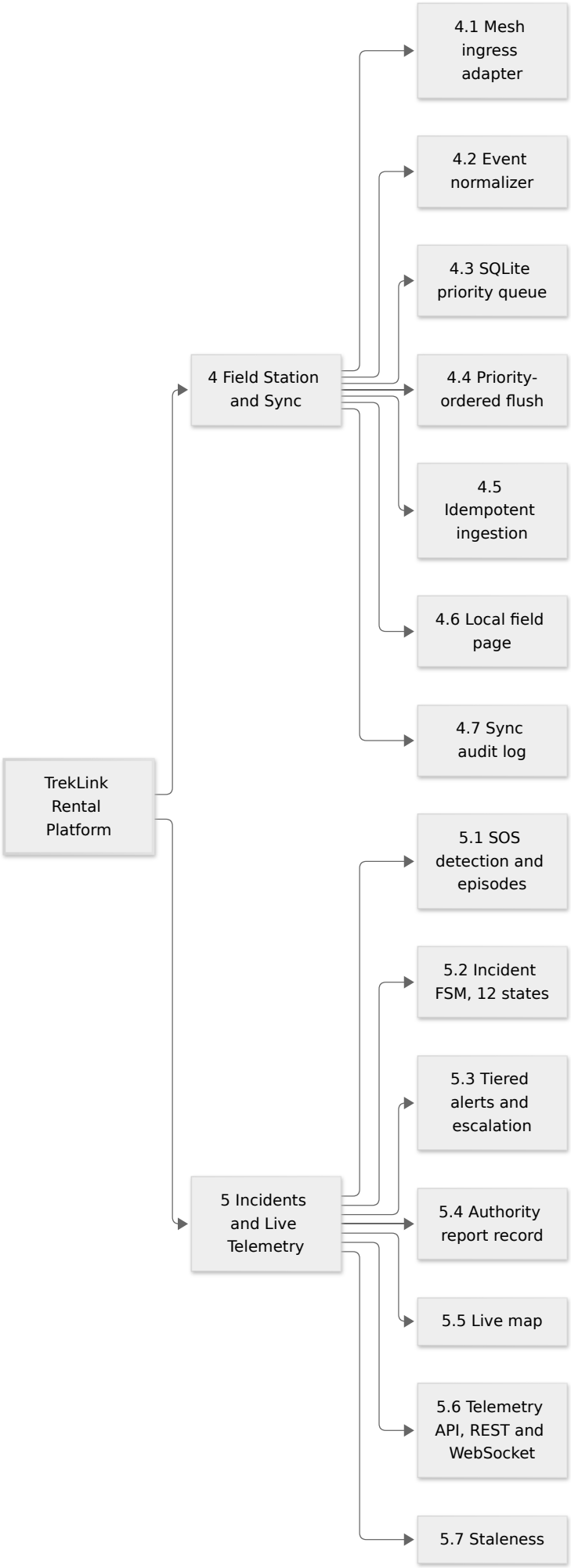

TrekLink Rental Platform
4 Field Station and Sync
4.1 Mesh ingress adapter
4.2 Event normalizer
4.3 SQLite priority queue
4.4 Priority-ordered flush
4.5 Idempotent ingestion
4.6 Local field page
4.7 Sync audit log
5 Incidents and Live Telemetry
5.1 SOS detection and episodes
5.2 Incident FSM, 12 states
5.3 Tiered alerts and escalation
5.4 Authority report record
5.5 Live map
5.6 Telemetry API, REST and WebSocket
5.7 Staleness

Figure 11: Feature Tree, part 2 of 3, Field Station and Sync, Incidents and Live Telemetry. Placement: inline, 100.4 x 266.0 mm, labels at 7.69 pt.

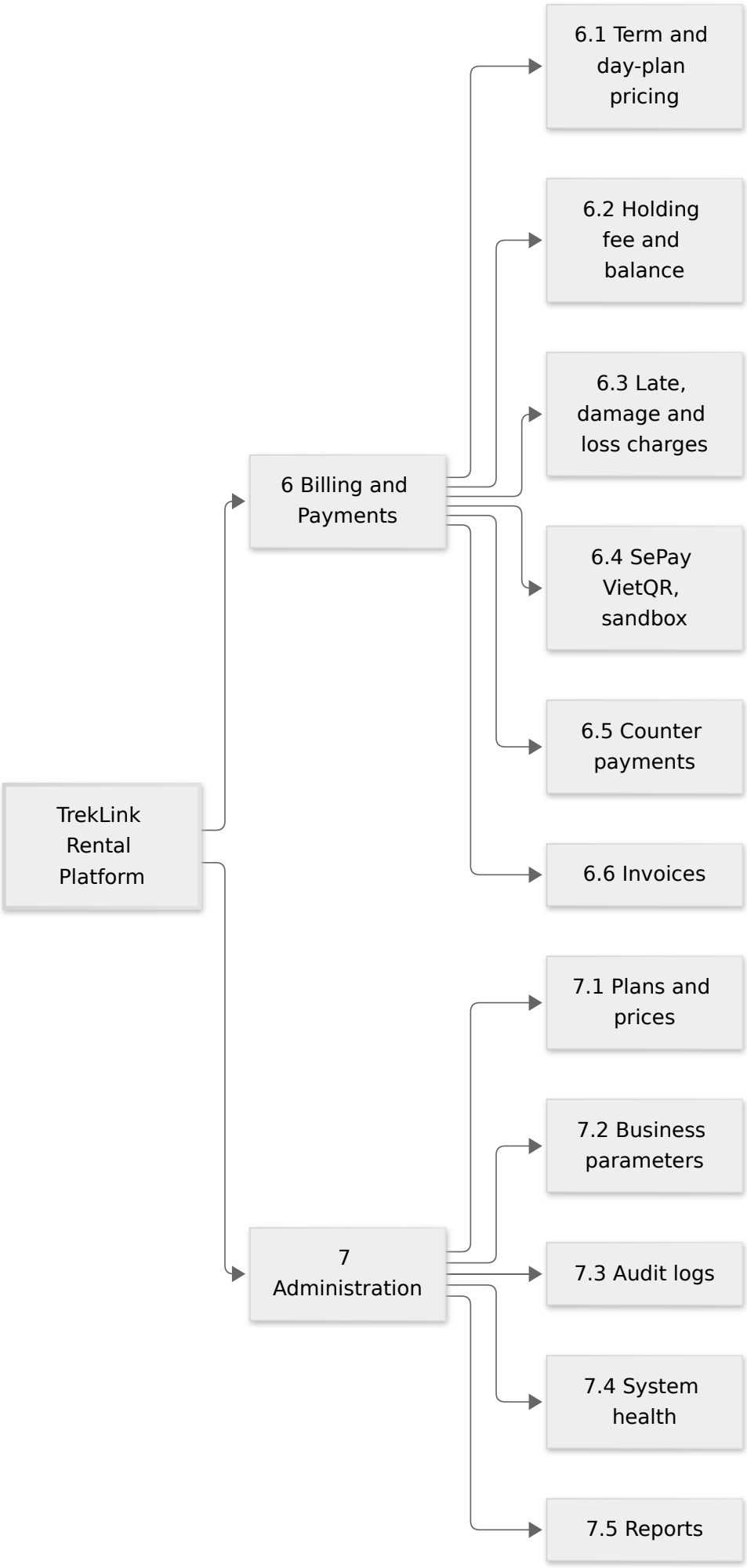

TrekLink Rental Platform
6 Billing and Payments
6.1 Term and day-plan pricing
6.2 Holding fee and balance
6.3 Late, damage and loss charges
6.4 SePay VietQR, sandbox
6.5 Counter payments
6.6 Invoices
7 Administration
7.1 Plans and prices
7.2 Business parameters
7.3 Audit logs
7.4 System health
7.5 Reports

Figure 12: Feature Tree, part 3 of 3, Billing and Payments, Administration. Placement: inline, 130.4 x 266.0 mm, labels at 9.99 pt.

5. Business Rule Matrix

The business rules (BR-01 to BR-35), each with the requirement that states it, the module that implements it and its test case, are generated into Report 3 §5.1 from the SRS builder, so the identifiers cannot drift between this file and the report. Every Implementation and Tested cell is empty until the code and the test exist.

6. Exception Scenario Matrix

The per-flow tables in `05-main-flows.md` (`05-main-flows.md`) are the source: E01-1 to E01-7, E02-1 to E02-6, E03-1 to E03-9, E04-1 to E04-7 and E05-1 to E05-7. Report 3 §5.2 consolidates them with the requirement each one tests. The faculty handbook's five generic probes are covered: payment failing part-way (E01-4, E05-4), two users acting at once (E01-1, E03-3), cancellation after approval (E01-7), data outside plausible range (E04-6), and one organization reaching another's data (E04-4).

7. Honest status

Layer	Status as of 2026-10-03
Specs	The `treklink-web` module specs were written for the booking scope and are being rewritten against D-033 to D-035
Backend	The `platform` module (health, business parameters, audit log) is implemented; the other modules are empty
Database	The Prisma schema still carries the booking scope (`TrekPackage`, `Booking`); its migration follows the spec rewrite
Implementation of any Main Flow	**None.** MF-01 to MF-05 are all at 0 %

TrekLink Engineering Conventions: Index

> **This folder is the TrekLink Developer Handbook.** It is the authoritative, machine-readable source. `TrekLink_Developer_Handbook_v1.0.pdf` is a **generated render** of these files, never hand-edited, rebuilt automatically by CI whenever anything here changes. If the PDF and these files ever disagree, **these files win** and the PDF is stale; rebuild it.

Conventions v2 (2026-09-13). Originally ported from a session-driven + spec-driven (Kiro-style) framework used on a .NET project, then adapted to TrekLink's real stack: **NestJS/TypeScript backend, Node.js/TypeScript gateway, React/TypeScript frontend, PostgreSQL, GitHub, Jira**.

Read in this order

#	File	Domain
01	`01-session-based-development-and-ssot.md` (`01-session-based-development-and-ssot.md`)	Core philosophy: docs as SSOT, session-based development, crash-safe checkpointing
02	`02-spec-driven-development-workflow.md` (`02-spec-driven-development-workflow.md`)	EARS requirements → design → tasks. The spec-before-code gate.
03	`03-operational-workflows.md` (`03-operational-workflows.md`)	The four lifecycles: feature dev, debugging, code review, review resolution
04	`04-architecture-conventions.md` (`04-architecture-conventions.md`)	Module boundaries, entity conventions, Feature-Sliced frontend
05	`05-backend-conventions.md` (`05-backend-conventions.md`)	NestJS modules, DTOs, the response envelope, error handling
06	`06-frontend-conventions.md` (`06-frontend-conventions.md`)	FSD, state management, forms/UX, WCAG, Leaflet + OpenStreetMap with the sovereignty overlay, WebSocket
07	`07-github-workflow-git-conventions.md` (`07-github-workflow-git-conventions.md`)	**Branching, commits, PRs, review, merge.** The most-referenced chapter.
08	`08-ai-agent-steering-and-discipline.md` (`08-ai-agent-steering-and-discipline.md`)	AI agent behaviour: thinking discipline, blast radius, circuit breakers, doc-sync
09	`09-doc-driven-scaffold-and-ssot-conventions.md` (`09-doc-driven-scaffold-and-ssot-conventions.md`)	The `ignore/` personal garden scaffold and how it reconciles with tracked `specs/`
10	`10-jira-tracking-and-workflow.md` (`10-jira-tracking-and-workflow.md`)	**Jira board, statuses, the development loop, backlog↔Jira mapping**
11	`11-ai-first-doctrine-and-toolchain.md` (`11-ai-first-doctrine-and-toolchain.md`)	**Model policy, required toolchain, session workflow, prompting standard**
12	`12-communication-and-daily-reports.md` (`12-communication-and-daily-reports.md`)	Channels, daily reports, merge announcements, schedule countdown
13	`13-diagram-and-figure-conventions.md` (`13-diagram-and-figure-conventions.md`)	Mermaid only, swimlanes for multi-actor flows, computed figure placement, the 7 pt legibility floor
14	`14-prose-and-wording.md` (`14-prose-and-wording.md`)	**No em dash. Banned openers, aphorism formulas, unverified claims, fabricated precision. The always-on AI prose and caveman skills.**
15	`15-graded-report-conventions.md` (`15-graded-report-conventions.md`)	**Graded reports: school template and regulations, the common thesis flaws, no verbose or meta text, upright figures**

If you read only three: 07 (git), 10 (Jira), 11 (AI-first). Those three cover everything you touch daily. **14 applies to every line you write**, including the ones in those three.

What changed in v2 (2026-09-13)

Logged as **Decision D-009**, which supersedes D-003. Every one of these was a live contradiction in v1, the documented commands did not work.

Was	Now
`develop` integration branch	**`dev`**, no repo ever had a `develop` branch
`[Feature]` bracket-tag commits	**Conventional Commits**, `feat(TK-45): add device FSM guard`
Dual `features/Design_X` + `features/Implementation_X` branches	**One branch per unit of work**, spec commits first
`release/sprint_x` branches	**Dropped**, `main` is the release
Squash-merge always	**Rebase & merge** default; squash only when multi-commit; merge commits **prohibited**
GitHub Issues + Milestones as the tracker	**Jira** is the single tracker; GitHub Issues = daily reports + bugs
`points:*` labels	Jira story points, **Fibonacci** scale (1/2/3/5/8/13/21)
Two PR templates	**One** `pull_request_template.md`, Design DoD optional
AI agent guidance scattered	Chapters **11** and **12**, plus a mandatory session workflow

Adopting these conventions in a repo

All three TrekLink repos use **identical** conventions. There is no per-repo exception.

1. Point the repo's `AGENTS.md` and `CLAUDE.md` at this folder, one canonical file, copied verbatim into `capstone/`, `treklink-docs/`, `treklink-web/`, and `treklink-firmware/`.
2. Copy `../.github/` into the repo root as-is (PR template, issue templates, labels).
3. Run the label setup script in `../.github/labels-and-milestones.md` (`labels-and-milestones.md`) §3.
4. Apply branch protection per `07-github-workflow-git-conventions.md` (`07-github-workflow-git-conventions.md`) §7.
5. Create a tracked `specs/{module}/` folder per module, layout in `02-spec-driven-development-workflow.md` (`02-spec-driven-development-workflow.md`) §2.
6. **Never write production code before the requirements → design → tasks gate has been passed.**

> Do **not** vendor a second copy of these files into another repo. A duplicated convention set drifts and then actively teaches the wrong rules, that is exactly what happened with `treklink-web/docs/conventions/`, which has been deleted for this reason. Link to this folder; don't copy it.

Authority

When two documents conflict, the higher tier wins until an ADR in `../00-project-context/03-decisions-and-risk-register.md` (`03-decisions-and-risk-register.md`) supersedes it:

1. **Tier 1**, Project Charter & capstone registration form
2. **Tier 2**, Module specs (`treklink-web/specs/{module}/`) and the backlog (`../03-backlog/`)
3. **Tier 3**, These conventions
4. **Tier 4**, Templates (`../02-templates/`), starting points, not binding once filled in
5. **Tier 5**, Source code & tests

How We Work

Core Engineering Philosophy: Session-Based Development & Living SSOT

> **Fundamental Law of Engineering Continuity**: In modern software engineering with human-AI collaboration, conversational memory is volatile, ephemeral, and lossy. **The codebase and its living documents are the only durable sources of truth.** Every decision, state transition, and requirement MUST be grounded in structured documents, and every work stream MUST execute in bounded, crash-resilient sessions.

This applies with extra force on TrekLink: a 5-person team working 2-week sprints across 4 codebases (firmware [frozen], gateway, backend, frontend), reviewed by lecturers at fixed checkpoints (Week 4/8/14) who were not in the room for any implementation decision. If it isn't written down, the reviewer can't grade it and the next session can't resume it.

---

1. The Core Philosophy

Software projects fail not from lack of code, but from **context drift, ungrounded assumptions, and fragmented memory**. To eliminate this entropy, this framework rests on two pillars:

1. **Documents as the Single Source of Truth (SSOT)**: If a requirement, architectural choice, or API contract is not written down in an official document, **it does not exist**.
2. **Session-Based Development (SBD)**: All engineering work is structured into deterministic, self-contained sessions with rigorous pre-flight discovery, milestone checkpoints, and explicit handoffs.

See **Figure 13**.

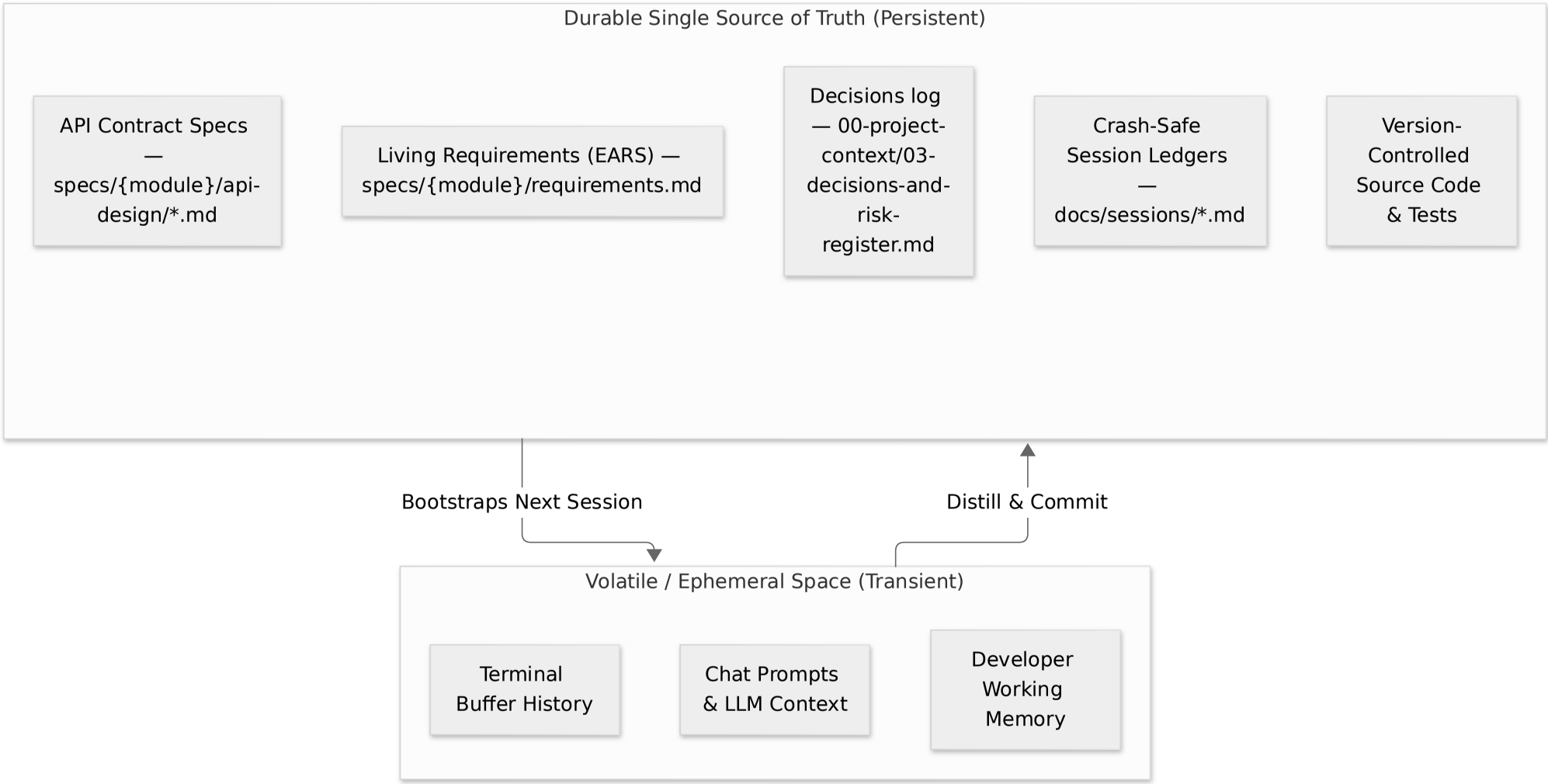

Durable Single Source of Truth (Persistent)
API Contract Specs — specs/{module}/api-design/*.md
Living Requirements (EARS) — specs/{module}/requirements.md
Decisions log — 00-project-context/03-decisions-and-risk-register.md
Crash-Safe Session Ledgers — docs/sessions/*.md
Version-Controlled Source Code & Tests
Bootstraps Next Session
Distill & Commit
Volatile / Ephemeral Space (Transient)
Terminal Buffer History
Chat Prompts & LLM Context
Developer Working Memory

Figure 13: Volatile context versus durable SSOT. Anything that exists only in a chat window is lost at session end; anything that matters is written to a document under version control. Placement: rotated plate, 136.6 x 266.0 mm, labels at 9.92 pt.

2. Pillar I: Living Documentation as SSOT

2.1 The SSOT Hierarchy

Level	Document Type	Purpose	Mutability
Tier 1	Project Charter, System Architecture & Data Schema	Module boundaries, domain entities, invariants	High ceremony; requires team + supervisor sign-off (esp. at Review 1/2)
Tier 2	Feature Specs (`specs/{module}/`)	EARS requirements, API contracts, task breakdowns	Updated per module/user story; locked during implementation
Tier 3	Engineering Conventions (`docs/conventions/` , this folder)	Architecture rules, code style, Git flow, agent discipline	Living rules; updated on retrospective consensus
Tier 4	Session Ledgers & Worklogs (`docs/sessions/`)	Current milestone progress, test metrics, open bugs	Frequently updated during active sessions
Tier 5	Source Code & Test Suites	Concrete implementation fulfilling Tiers 1–3	Changes only to satisfy Tiers 1–4

2.2 In-Repo Living Documents

- **Zero third-party dependency for context**: docs live inside the repo, version-controlled, next to the code they describe.
- **Atomic spec-code merges**: when an endpoint changes, its `specs/{module}/api-design/*.md` file is updated **in the same PR**. This generalizes to every tier above it, not just API endpoints: a finding that touches the charter, a decision, a spec, or a convention gets that document updated **in the same working pass it was found in**, see `08-ai-agent-steering-and-discipline.md` Stage 2.5b. A doc update queued for "later in the session" is functionally the same failure mode as an endpoint shipped without its spec update, just easier to miss because nothing blocks the PR on it.
- **No stale documentation**: specs gate coding, they aren't written after the fact.

3. Pillar II: Session-Based Development (SBD)

Break open-ended work into bounded execution cycles, 1–3 hours for humans, or one AI-agent context window.

3.1 The 4-Phase Session Lifecycle

See **Figure 14**.

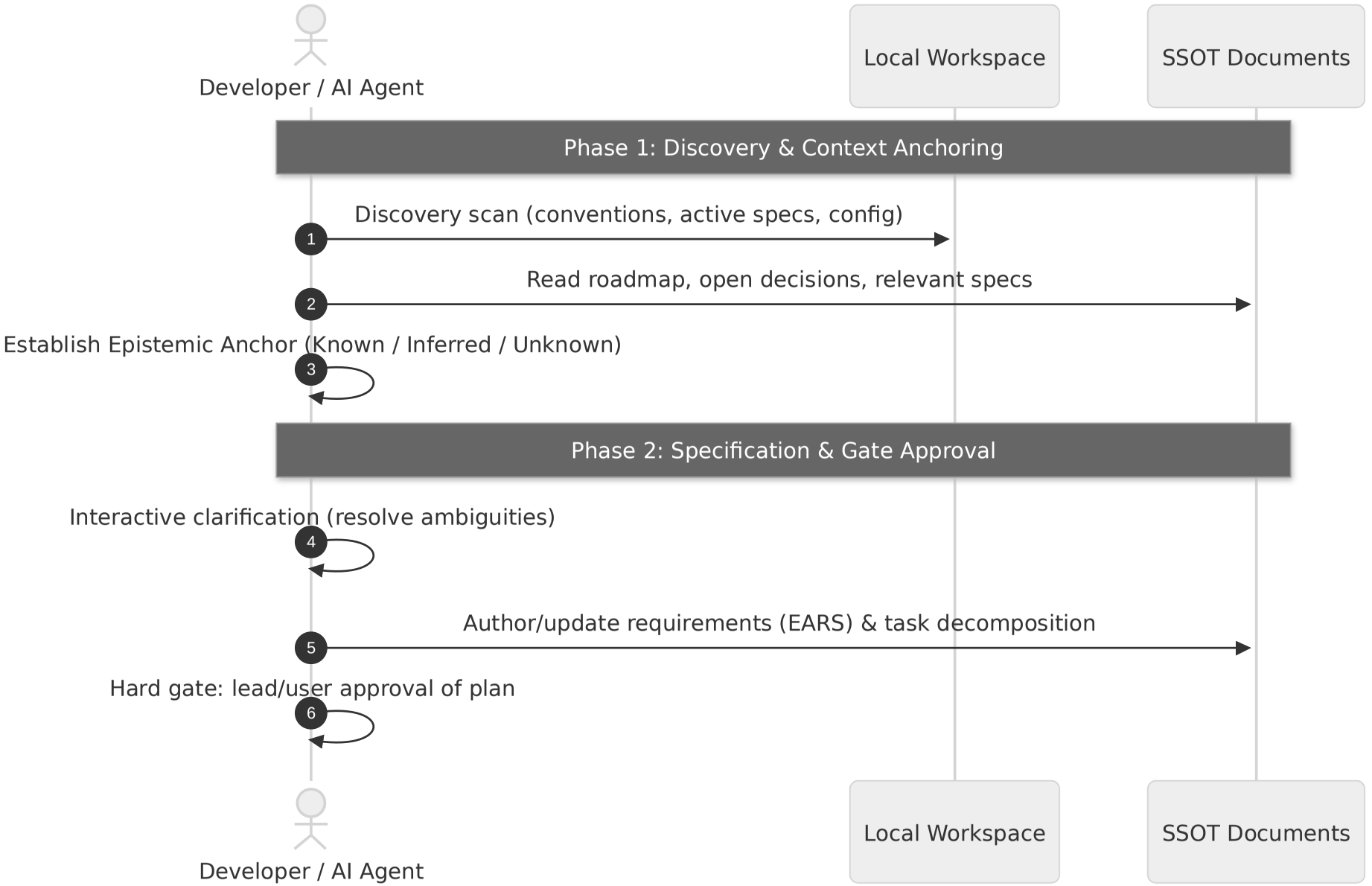

Developer / AI Agent
Local Workspace
SSOT Documents
Phase 1: Discovery & Context Anchoring
Discovery scan (conventions, active specs, config)
1
Read roadmap, open decisions, relevant specs
2
Establish Epistemic Anchor (Known / Inferred / Unknown)
3
Phase 2: Specification & Gate Approval
Interactive clarification (resolve ambiguities)
4
Author/update requirements (EARS) & task decomposition
5
Hard gate: lead/user approval of plan
6
Developer / AI Agent
Local Workspace
SSOT Documents

Figure 14: The session lifecycle, phases 1 and 2, discovery and context anchoring, then specification and the approval gate. No code is written before the gate. Placement: rotated plate, 179.9 x 266.0 mm, labels at 8.58 pt.

See **Figure 15**.

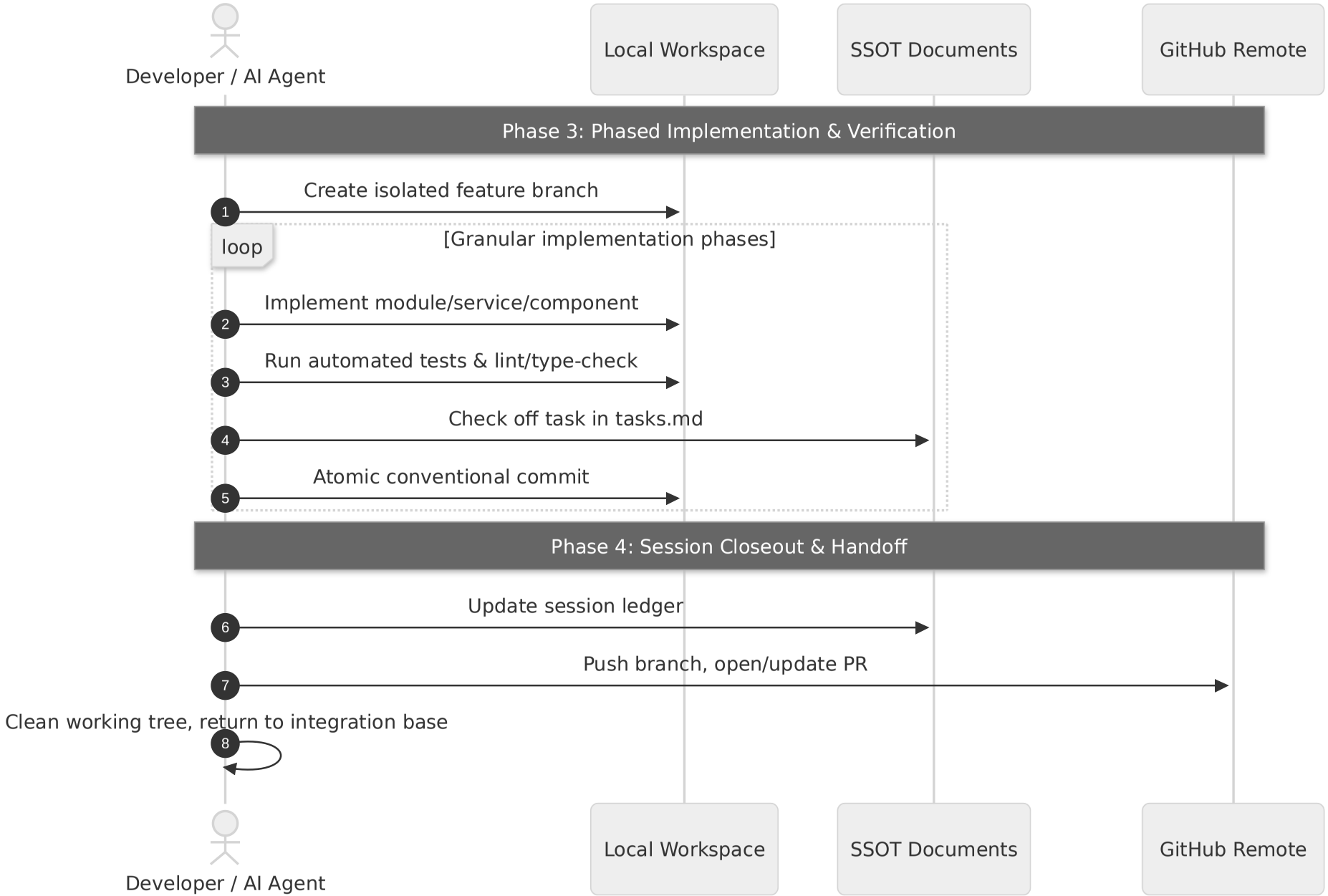

Developer / AI Agent
Local Workspace
SSOT Documents
GitHub Remote
Phase 3: Phased Implementation & Verification
Create isolated feature branch
1
loop
[Granular implementation phases]
Implement module/service/component
2
Run automated tests & lint/type-check
3
Check off task in tasks.md
4
Atomic conventional commit
5
Phase 4: Session Closeout & Handoff
Update session ledger
6
Push branch, open/update PR
7
Clean working tree, return to integration base
8
Developer / AI Agent
Local Workspace
SSOT Documents
GitHub Remote

Figure 15: The session lifecycle, phases 3 and 4, phased implementation with verification, then closeout and handoff. Placement: rotated plate, 182.0 x 257.9 mm, labels at 7.66 pt.

Phase 1: Discovery & Context Anchoring

1. **Discovery scan**: conventions, active specs, roadmap position (which sprint/week are we in).
2. **Epistemic Reality Anchor**:
   - **KNOWN**: verified from files read this session.
   - **INFERRED**: derived pattern (state confidence).
   - **UNKNOWN**: missing context, flag **before** proceeding, check `03-decisions-and-risk-register.md` for anything OPEN that blocks you.
3. **Kalama Empirical Proof Standard**: reject "seems reasonable", accept only what's backed by file evidence or test output.

Phase 2: Specification & Gate Approval

1. Never write production code from a bare chat prompt or vague issue.
2. Author/review the 3-file spec suite in `specs/{module}/`: `requirements.md` → `design.md` → `tasks.md`.
3. Pause and clarify edge cases before coding.

Phase 3: Phased Implementation & Verification

1. Branch off `dev` (or `main` for hotfixes).
2. Work `tasks.md` phase-by-phase.
3. **Read-Before-Write**: never edit a file not opened and inspected this session.
4. Run `npm test` / `npm run lint` / `npm run typecheck` (per-package, see `05-backend-conventions.md`, `06-frontend-conventions.md`) after every atomic phase.

Phase 4: Session Closeout & Crash-Safe Handoff

1. Update the session ledger.
2. Commit with bracket tags (`feat:`, `fix:`, `refactor:`, `docs:`, full list in `07-github-workflow-git-conventions.md`).
3. Rebase on the upstream integration branch.
4. Push and open/update the PR; output a concise handoff so the next session (human or AI) resumes with zero overhead.

---

4. Crash-Safe Memory Checkpoint Protocol

> **(Session 4 revision, namespacing for concurrency)** The original version of this section allowed a single rolling `docs/sessions/current.md` as an alternative to per-date files. That's unsafe once more than one session (human or AI-agent) can be active on the repo at the same time, a five-person team plus their own AI agents, each possibly running more than one session window, is exactly that case. **Per-session-instance files are now mandatory, not optional.** This isn't paperwork: this exact gap caused a real loss this term, a Session 4 audit produced several non-obvious findings (firmware wire-format facts, scaffold bugs) that sat only in chat output, in a repo that already had this protocol on paper, because nothing forced them to disk mid-session. See `08-ai-agent-steering-and-discipline.md` Stage 2.5 for the enforcement rule.

Tracked tier, `docs/sessions/<YYYY-MM-DD>-<HHMM>-<topic-slug>.md`, one file per session instance (e.g. `2026-09-13-1830-gateway-sync-audit.md`). `docs/sessions/current.md` is no longer where work-in-progress gets written, it becomes a short **index**: a few lines per active or recent session file, updated whenever a session file is created or closed out, so a new session can see at a glance what else is/was in flight before re-deriving it.

Personal scratch tier, same pattern, one level down: `ignore/[name]/docs/sessions/<YYYY-MM-DD>-<HHMM>-<topic-slug>.md` per session instance, replacing the old single shared `ignore/[name]/docs/current-progress.md`. That file still exists but changes role: a short rolling pointer/summary ("latest: see `sessions/2026-09-13-1830-gateway-sync-audit.md`"), not the place high-frequency scratch writes go, that would recreate the same collision risk one level down.

Maintain a session file at every major milestone, and, per the enforcement rule in `08-ai-agent-steering-and-discipline.md` Stage 2.5, immediately on any non-obvious finding, not only at milestones:

```
# Session Checkpoint: 2026-09-14 — gateway-sync

## 1. Active Focus & Objectives
- Module: Gateway Bridge — SQLite priority queue (TP2)
- Branch: feat/TK-45-gateway-priority-queue

## 2. Completed Milestones
- [x] Phase 1: SQLite schema for P0-P3 queue tiers
- [x] Phase 2: Enqueue/dequeue with priority ordering + unit tests

## 3. Immediate Next Steps (for resuming session)
1. Implement MQTT publish-on-reconnect flush routine
2. Add integration test: 30s / 2min / 5min connectivity-loss simulation
3. Wire health-reporting endpoint (queue depth per tier)

## 4. Uncommitted State / Known Blockers
- Working tree: clean (commit `a1b2c3d`)
- Test status: 18/18 passing
- Open questions: none (D-001 ORM decision doesn't block gateway work)
```

---

5. Context Window & Token Budget Management (AI Agents)

1. **Context decay**: past ~70–80% capacity, reasoning fidelity drops. Compact proactively.
2. **Proactive compaction**: grep for the specific error, don't dump whole logs; read targeted line ranges, not whole large files; summarize findings into the session ledger instead of re-reading the repo every turn.
3. **Handoff over compounding**: for multi-day work (e.g. TP2's whole Gateway Bridge), split into clean sessions, Session N ends by committing + updating `tasks.md`; Session N+1 boots by reading the spec, with a full context budget for implementation.

---

6. Tenets of Iterative Refinement

1. **Working software at every step**: never leave `dev` in a broken/uncompilable state. Incomplete work stays on its feature branch.
2. **Minimal sufficient change**: solve the registered requirement, no speculative abstractions ahead of the TP schedule.
3. **Continuous verification**: a feature is done when tests prove it satisfies the EARS requirements, not when code is typed.

Spec-Driven Development Workflow (SDSDW)

> **Core Axiom**: Specification is the architectural blueprint of software. Writing code without an approved specification is constructing a building without an engineering schematic. This workflow guarantees requirements, interactions, contracts, and tasks are formally verified before a single line of production code is written, this is the "Kiro spec-driven development" pattern (`requirements.md` → `design.md` → `tasks.md`) combined with session-based development from `01-session-based-development-and-ssot.md` .

---

1. Executive Overview

See **Figure 16**.

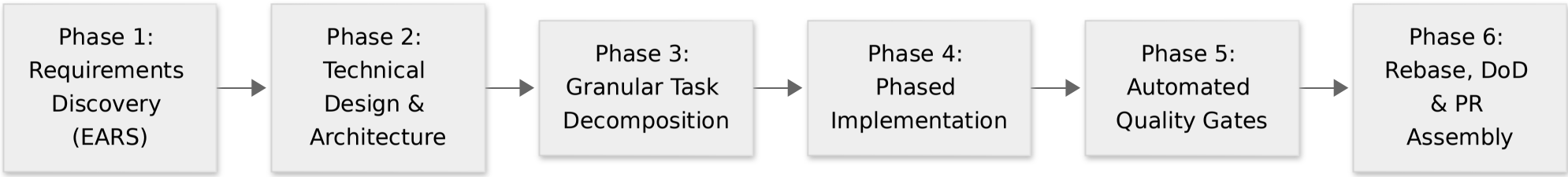

Phase 1: Requirements Discovery (EARS)
Phase 2: Technical Design & Architecture
Phase 3: Granular Task Decomposition
Phase 4: Phased Implementation
Phase 5: Automated Quality Gates
Phase 6: Rebase, DoD & PR Assembly

Figure 16: The spec-before-code gate: EARS requirements, then design, then granular task decomposition. Each phase is approved before the next begins. Placement: rotated plate, 32.1 x 266.0 mm, labels at 10.70 pt.

Universal tenets

1. **Spec before code** (Read → Clarify → Design → Code). Never guess business rules, RBAC edge cases, or error conditions, TrekLink's grading depends on traceable requirements (RQ1–RQ3 must map back to specific FRs).
2. **Standardized directory topology**: specs live in `specs/{module_name}/` inside each repo, next to the code.
3. **EARS-compliant criteria** for every functional requirement.
4. **Mermaid** for all architecture diagrams, state machines (Device 7-state, Incident 5-state, these are graded deliverables), and sequence flows.
5. **In-codebase tracked deliverables**: specs, migrations, and docs are committed, never left in an uncommitted scratchpad.
6. **Docs sync at the moment of discovery, not at phase-exit cleanup** (`08-ai-agent-steering-and-discipline.md` Stage 2.5b). If implementation, testing, or a code-level investigation reveals that a requirement, design assumption, or task in this module's spec was wrong or incomplete, fix `requirements.md` / `design.md` / `tasks.md` in that same working pass, immediately, not queued for a later "update the docs" step. Phase 6's `6.3` below is a final **audit** that sync already happened throughout, not the first time specs get touched.

---

2. Directory Structure (`specs/{module_name}/`)

TrekLink modules (per the roadmap's TP mapping and `07-github-workflow-git-conventions.md` 's `module:*` labels): `auth` , `devices` , `rentals` , `trips` , `gateway-sync` , `incidents` , `monitoring` , `billing` , `frontend` .

```
specs/{module_name}/
├── requirements.md                 # EARS-format business rules & criteria
├── design.md                       # Architecture, domain models, sequence flows, state machines
├── tasks.md                        # Phased, granular implementation checklist
└── api-design/                     # One file per endpoint, using 02-templates/04-api-endpoint-
template.md
    ├── 00-api-testing-guide.md     # Swagger / Postman / curl walkthrough
    ├── 01-post-devices-register.md
    ├── 02-get-devices-list.md
    └── ...

# Example for the gateway-sync module:
specs/gateway-sync/
├── requirements.md    # eventId scheme, P0-P3 priority rules, idempotency criteria
├── design.md           # SQLite queue schema, MQTT topic design, reconnect/flush sequence diagram
├── tasks.md
└── api-design/
    └── 01-post-events-ingest.md
```

Frontend-only concerns (screens, component architecture) live under `specs/frontend/` with the same 3-file pattern, referencing backend `api-design/` files rather than duplicating them.

---

3. Phase-by-Phase Execution Protocol

Phase 1: Requirements Discovery & EARS Specification

1. Context ingestion

Review the relevant section of `00-project-context/01-project-charter.md` , existing entities in `design.md` of dependent modules, and upstream/downstream module boundaries (e.g. `incidents` depends on `devices` + `rentals` existing first).

2. Interactive clarification interview (mandatory quality gate)

Before writing specs or code, formulate 3–5 high-value clarifying questions targeting:

- **Domain edge cases**: e.g. what happens to an in-field device's rental record if a trip's Guide reports it lost?
- **Authorization**: which of Admin/Staff/Guide/Customer can trigger this action; ownership checks (a Guide can only see their own assigned trips).
- **Reliability**: for gateway-sync/incidents, does this requirement interact with idempotency or priority ordering?

> **MANDATORY HARD STOP**: stop after presenting the questions and wait for an explicit answer. No specs, designs, tasks, or code before that.

3. Authoring `requirements.md` in EARS syntax

Pattern	Template	TrekLink example
Ubiquitous	*The system SHALL [response]*	The system SHALL record actor (user ID + role) and UTC timestamp on every incident state transition.
Event-driven	*WHEN [trigger], the system SHALL [response]*	WHEN a valid SOS event passes the idempotency check, the system SHALL create exactly one Incident and notify Staff + Guide via WebSocket within 2 seconds.
State-driven	*WHILE [state], the system SHALL [response]*	WHILE a device is in `Maintenance` status, the system SHALL reject any rental allocation attempt referencing it.
Unwanted behavior	*IF [condition/error], THEN the system SHALL [response]*	IF the same `eventId` is delivered more than once, THEN the system SHALL discard the duplicate without creating a second Incident.
Optional feature	*WHERE [feature/flag], the system SHALL [response]*	WHERE sandbox payment mode is enabled, the system SHALL mark all transactions with a `sandbox: true` flag.

Phase 2: Technical Architecture & Design (`design.md`)

Backend/gateway modules:

1. Domain & data modeling, entities, enums, FK constraints, state machines (render as Mermaid `stateDiagram-v2` for Device/Incident FSMs).
2. Service & queue architecture, NestJS providers/services, MQTT topics, SQLite queue schema for gateway-sync.
3. Endpoint spec (`api-design/*.md`) using `02-templates/04-api-endpoint-template.md` .

Frontend:

1. Component architecture (Feature-Sliced Design, see `06-frontend-conventions.md`).
2. Server vs. client vs. form state lifecycle.
3. Validation schemas (Zod) mirroring backend `class-validator` DTOs 1:1.

Phase 3: Granular Task Decomposition (`tasks.md`)

```
# Implementation Tasks: [Module Name]

## Phase 1: Foundation & Domain Modeling
- [ ] 1.1 Define entities/enums (Prisma, per D-001)
- [ ] 1.2 Write migration(s)
- [ ] 1.3 Add DTOs and error codes

## Phase 2: Core Service / Mutation Logic
- [ ] 2.1 Implement service method(s) with class-validator DTO input
- [ ] 2.2 Wrap multi-table writes in a transaction
- [ ] 2.3 Unit test success + failure paths

## Phase 3: Query / Retrieval
- [ ] 3.1 Implement paginated list/query endpoints
- [ ] 3.2 Unit test filter/sort/pagination boundaries

## Phase 4: API Presentation Layer
- [ ] 4.1 Controller + Guards (JWT + RBAC/CASL)
- [ ] 4.2 Swagger/OpenAPI annotations
- [ ] 4.3 Integration tests for the HTTP contract (match api-design/*.md exactly)

## Phase 5: Frontend Integration (if applicable)
- [ ] 5.1 API client + TypeScript types
- [ ] 5.2 Form (RHF + Zod) or view wired to TanStack Query
- [ ] 5.3 Loading/empty/error states

## Phase 6: End-to-End Verification & DoD Audit
- [ ] 6.1 Full test suite green
- [ ] 6.2 Lint + typecheck clean
- [ ] 6.3 Audit: confirm `specs/{module}/api-design/*.md` already matches actual behavior — it
      should have been updated live in Phases 1-5 per tenet #6 above, so this is a check, not
      the first edit. If it's out of sync here, that's a process miss to flag, not routine cleanup.
```

Phase 4: Phased Implementation

Never skip ahead. Atomic commits per subtask with bracket tags (`07-github-workflow-git-conventions.md`). Zero magic strings, centralize error codes and route constants.

Phase 5: Automated Testing & Verification Gates

- **Unit tests**: every domain rule, FSM transition, validator branch (esp. idempotency and priority-ordering logic, these are graded NFRs).
- **Integration tests**: DB persistence, unique constraints, transaction rollback.
- **Static analysis**: 0 build warnings, 0 lint errors, strict TypeScript.

Phase 6: GitHub Flow, Rebase & PR Assembly

1. One branch: `feat/TK-nn-short-desc` off `dev` , carrying spec commits first, then code+tests (see `07-github-workflow-git-conventions.md` §2.3).
2. `git fetch origin && git rebase origin/dev` .
3. Open the PR, `.github/pull_request_template.md` loads automatically. DoD checked, test output pasted, specs referenced, `Model used:` declared. Fill the optional Design DoD block if this branch introduced or changed a spec. Then move the Jira card to `IN REVIEW` and ping the reviewer in Zalo.

---

4. Universal Phase Exit Gate Matrix

Phase	Core Deliverable	Mandatory Exit Gate
1A: Clarification	Interview	**STOP & WAIT** for answers
1B: Requirements	`requirements.md`	All rules in unambiguous EARS syntax
2: Technical Design	`design.md` + `api-design/`	Architecture/schema/Mermaid diagrams reviewed
3: Task Decomposition	`tasks.md`	Granular checklist approved by lead
4: Implementation	Source code	Matches design, minimal sufficient change
5: Verification	Test suite	100% pass, 0 lint/type errors
6: Delivery & Review	Rebased branch & PR	Linear history, DoD verified, PR description compiled

Engineering Lifecycle & Operational Workflows

> Four canonical, repeatable workflows for feature development, defect diagnosis, code review, and review-comment resolution. Applies equally whether the acting party is a teammate or an AI coding agent.

1. Executive Workflow Matrix

See **Figure 17**.

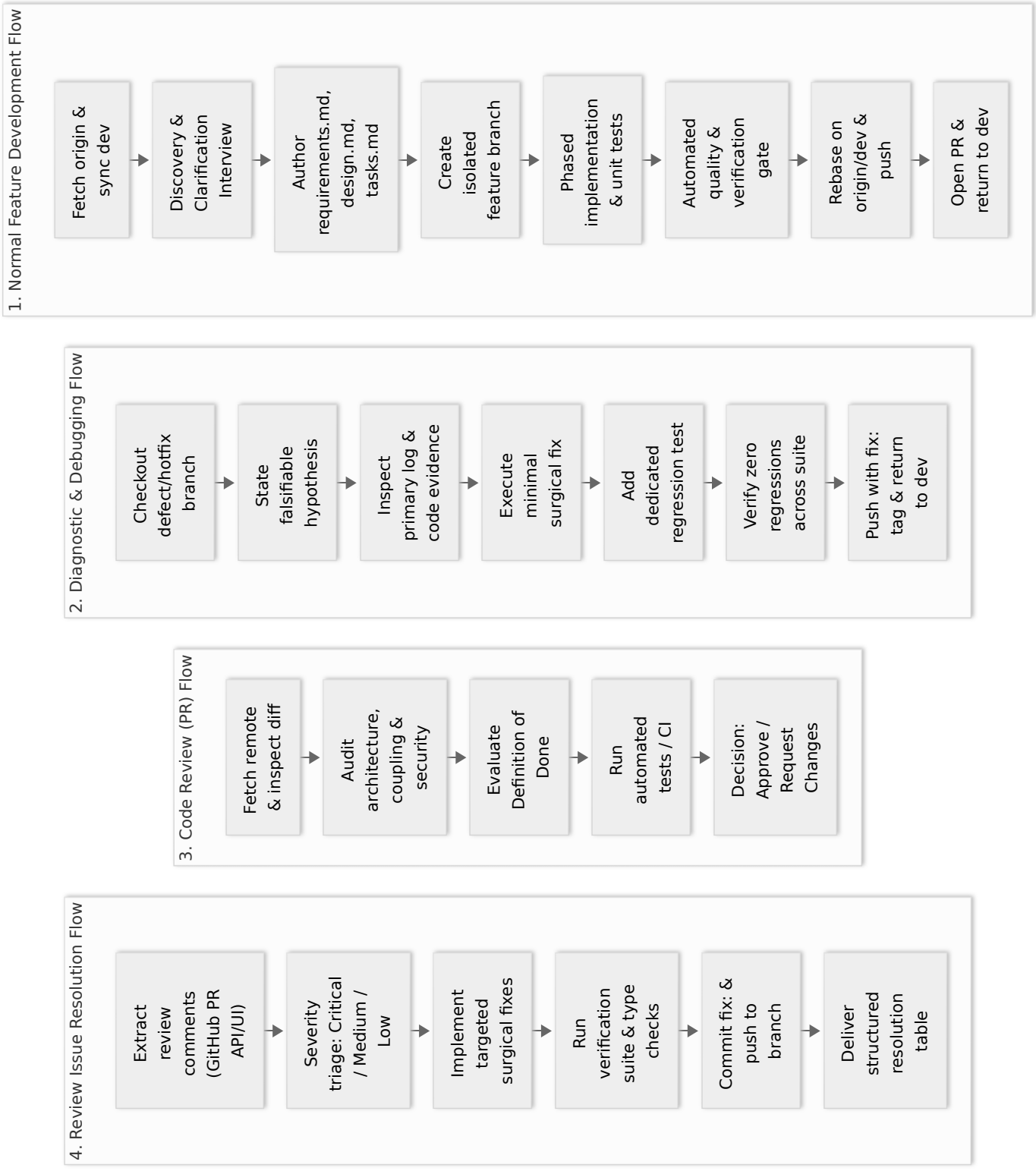


Figure 17: The four operational workflows, normal feature development, defect handling, spec revision, and session handoff, and the points at which they hand off to one another. Placement: rotated plate, 182.0 x 199.7 mm, labels at 8.23 pt.

2. Lifecycle 1: Normal Feature Development Flow (Spec-First)

Step 1: Baseline Synchronization

```
git fetch origin
git checkout dev
git pull origin dev
```

Step 2: Discovery, Clarification & Spec Authoring

Review the target module's charter section, existing entities, and dependent modules. Run the **Phase 1A Mandatory Gate** clarification interview (see `02-spec-driven-development-workflow.md`), hard stop until answered. Author `specs/{module}/requirements.md` → `design.md` → `tasks.md` → `api-design/`.

Step 3: Branch Creation & Isolation

```
git checkout -b feat/TK-45-device-registration
```

One branch per unit of work. Spec commits and implementation commits live on the **same** branch, spec first, see `07-github-workflow-git-conventions.md` §2.3. Branch type is `feat/`, `fix/`, `docs/`, or `chore/` as appropriate.

Step 4: Phased Clean Implementation

Follow `tasks.md` phase-by-phase. Commit small, atomic units using Conventional Commits with the Jira key as scope, `feat(TK-45): ...`, `fix(TK-45): ...`, `test(TK-45): ...` (see `07-github-workflow-git-conventions.md` §3).

Step 5: Quality Gate & Verification

```
# Backend (NestJS) — from the backend package root
npm run lint && npm run typecheck && npm test && npm run test:e2e

# Gateway (Node/TS)
npm run lint && npm test

# Frontend (React)
npm run lint && npm run build && npm test
```

Confirm **100% test pass rate**, 0 lint errors, clean build. Backend endpoint behavior must match `specs/{module}/api-design/*.md` exactly (same response envelope, see `05-backend-conventions.md`).

Step 6: Upstream Rebase & Push

```
git fetch origin
git rebase origin/dev
# re-run Step 5 after resolving any conflicts
git push --force-with-lease origin feat/TK-45-device-registration
```

Step 7: PR Assembly & Return to Base

Open the PR with `.github/pull_request_template.md`, fill the DoD, and **ping the reviewer in Zalo**. Move the Jira card to `IN REVIEW`. Checkout back to `dev`.

3. Lifecycle 2: Diagnostic & Debugging Flow (Hypothesis-Driven)

Step 1: Checkout Target Branch

```
git checkout feat/TK-45-device-registration
# OR for an urgent defect — see 07-github-workflow-git-conventions.md §2.4
# for which branch to cut from (dev vs main):
git checkout -b hotfix/TK-99-jwt-expiry-crash
```

Step 2: Hypothesis Formulation & Investigation

State a single falsifiable hypothesis. Inspect logs/stack traces/source lines with real tool calls, not guesses. Assess blast radius, e.g. a fix to the idempotency key logic touches every consumer of `eventId` across gateway + backend.

Step 3: Minimal Surgical Fix

Address the root cause only. No unrelated refactors or formatting changes bundled in.

Step 4: Dedicated Regression Test

Add a test that reproduces the failure without the fix and passes with it. Run the full suite, zero downstream regressions.

Step 5: Push & Return to Base

```
git commit -m "fix(TK-99): precise description of the resolved defect"
git push --force-with-lease origin hotfix/TK-99-jwt-expiry-crash
git checkout dev
```

Output a diagnostic summary: root cause, fix mechanics, regression-test proof.

---

4. Lifecycle 3: Pull Request & Code Review Flow

Step 1: Branch Fetch & Diff Inspection

```
git fetch origin
git log --oneline origin/dev..origin/{target_branch}
git diff origin/dev...origin/{target_branch}
```

Or via GitHub CLI: `gh pr diff {number}`, `gh pr checks {number}`.

Step 2: Architectural & Code Quality Audit

- [ ] Module boundaries respected (no cross-module repository access, see `04-architecture-conventions.md` §4)
- [ ] DTOs strictly typed with `class-validator`, matching `api-design/*.md`
- [ ] Errors use the standard envelope (`result` / `isSuccess` / `statusCode` / `message`), not ad-hoc shapes
- [ ] Security: mutating endpoints have both a JWT guard and a role/CASL policy check, e.g. `@UseGuards(JwtAuthGuard, PoliciesGuard)` on any endpoint that mutates device/rental/incident state
- [ ] Performance: no N+1 queries, indexed lookups on `eventId` /device ID/trip ID
- [ ] Hygiene: no debug logs, dead code, hardcoded secrets, or `.env` values committed

Step 3: Definition of Done Verification

Confirm DoD checklist in the PR description is honestly checked and CI is green.

Step 4: Structured Review Output

- **Decision**: `[APPROVED]` | `[CHANGES REQUESTED]` | `[NEEDS FIXES]`
- **Findings**: exact file + line references, drop-in diff suggestions.

---

5. Lifecycle 4: Review Issue Resolution Flow

Step 1: Retrieve Review Notes

Pull comments from the GitHub PR conversation thread (`gh pr view {number} --comments` or the UI).

Step 2: Severity Triage

- 🔴 **Critical**: missing auth/RBAC guard, data corruption risk, broken CI, idempotency/priority-ordering violation.
- 🟡 **Medium**: performance issue, missing transaction rollback, unhandled edge case.
- 🟠 **Low/Hygiene**: formatting, naming, doc updates.

Step 3: Implement Targeted Fixes

Address each comment directly, on the same branch, without unrelated scope changes.

Step 4: Verification & Handoff

```
npm test && npm run lint
git commit -m "fix: Address review feedback on {topic}"
git push
```

Output a resolution table:

#	Reviewer Comment	Severity	Applied Fix	Status
1	Missing role guard on device retire endpoint	🔴 Critical	Added `@Roles('Admin')` + CASL `can('retire','Device')` check	Resolved
2	Duplicate-SOS test only covers 2 retries, NFR needs 10	🟡 Medium	Extended test to 10× replay of same `eventId`	Resolved

Delivery

GitHub Workflow, Branching, Commits & Pull Requests

> **Conventions v2, supersedes everything written before 2026-09-13.** The previous version of this file described a `develop` branch and a dual `Design` / `Implementation` branch model inherited from the school's `Git_Lab_Guide.pdf`. Neither matches reality: no repository has ever had a `develop` branch, and the dual-branch model doubles PR overhead for a 5-person team on a 13-week term. Decision **D-009** records the change and supersedes **D-003**.

> **PR = MR.** "Pull Request" (GitHub) and "Merge Request" (GitLab) are used interchangeably across this documentation and in team chat. They mean the same thing and follow the same workflow. The school's guide says MR; GitHub says PR; we say either.

---

1. Repositories

Three repositories under `github.com/TrekLink-Team`, cloned as siblings inside one `capstone/` parent folder. See the root `README.md` §1 for the full layout and why the parent folder matters for AI agents.

Repo	Contents	Branch model
`treklink-docs`	This documentation set (SSOT). No application code.	Full model below
`treklink-web`	The active build: `gateway/`, `backend/`, `frontend/` as workspace packages	Full model below
`treklink-firmware`	Inherited SU26 firmware, editable per D-008, surgical fixes only	Full model below

All three repos use identical conventions. Same branches, same commit format, same PR process, same AI agent rules. There is no "this repo is different" exception. A teammate who learns the workflow in one repo knows it in all three.

The `capstone/` parent folder is also git-versioned but **never published**, it exists so an AI agent opened on `capstone/` can read all three repos plus `Documents/` in one context window.

---

2. Branch Model

See **Figure 18**.

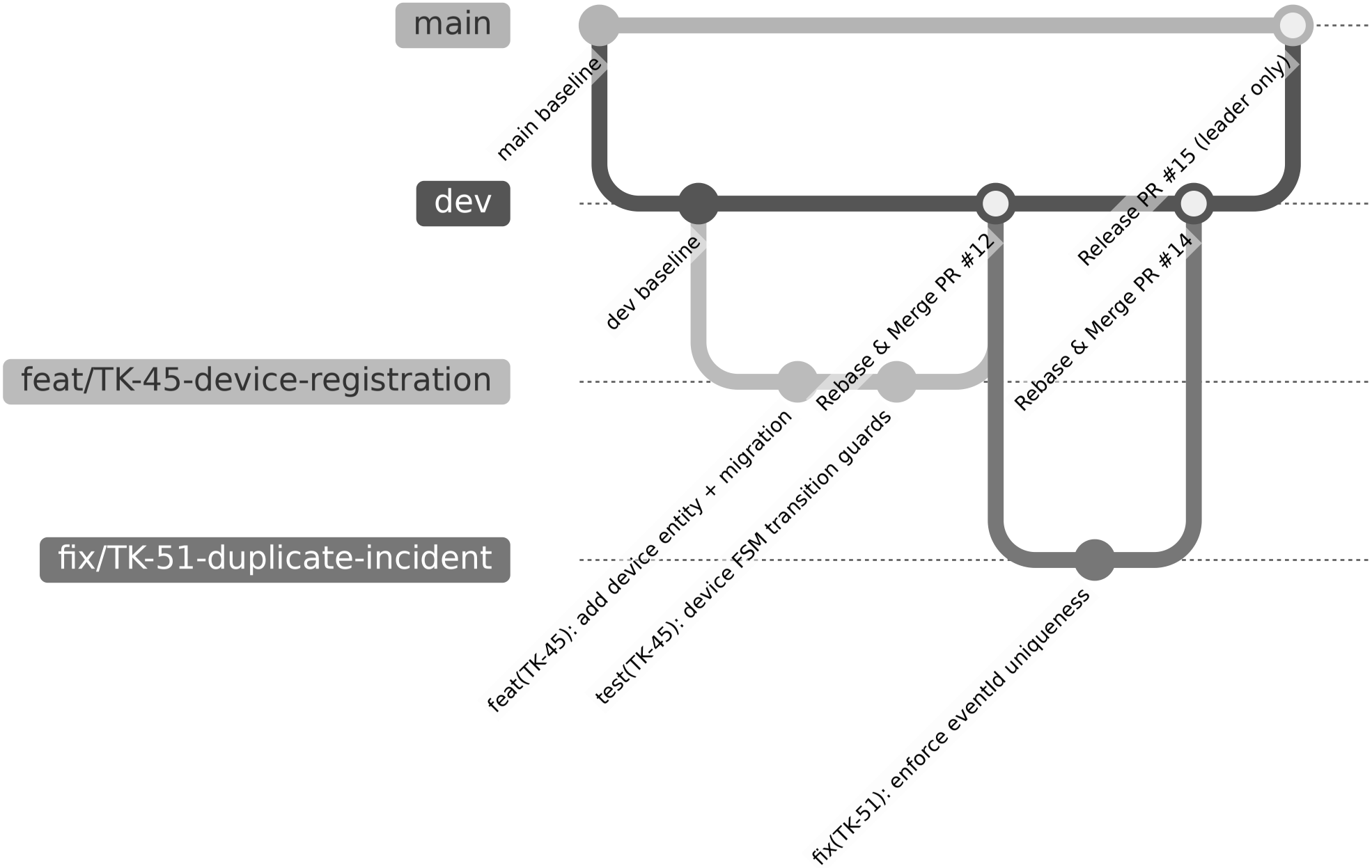

main
dev
feat/TK-45-device-registration
fix/TK-51-duplicate-incident
main baseline
dev baseline
feat(TK-45): add device entity + migration
test(TK-45): device FSM transition guards
Rebase & Merge PR #12
fix(TK-51): enforce eventId uniqueness
Release PR #15 (leader only)
Rebase & Merge PR #14

Figure 18: Branch topology. `dev` is the integration branch; feature branches rebase onto it and are deleted after merge. There is no `develop`, and merge commits are prohibited. Placement: rotated plate, 170.5 x 266.0 mm, labels at 10.75 pt.

2.1 The branches

Branch	Branches from	Merges into	Purpose
`main`	—	—	**Protected.** Production / demo-ready state. This *is* the release, there are no `release/*` branches.
`dev`	`main`	`main`	**Protected.** Active integration branch. Everything lands here first.
`feat/*`	`dev`	`dev`	New capability. One per Jira story or task.
`fix/*`	`dev`	`dev`	Defect in unreleased work (found on `dev`, or a GitHub Issue bug).
`hotfix/*`	`dev` **or** `main`	see §2.4	Urgent defect. Branch point depends on where the fault is.
`docs/*`	`dev`	`dev`	Documentation-only changes. Primary branch type in `treklink-docs`.
`chore/*`	`dev`	`dev`	Dependencies, tooling, config, cleanup. No behaviour change.

> **No direct pushes. Ever. To any branch.** Every change, including a one-line typo fix, including the leader's own changes, is branched, pushed, and merged through a PR. There is no exception for "it's tiny" or "it's urgent". Urgency is what `hotfix/*` is for, and a hotfix is still a PR.

2.2 Branch naming

```
{type}/{JIRA-KEY}-{short-kebab-description}
```

Example	Notes
`feat/TK-45-device-registration`	Standard case. Jira key, then 2–4 words.
`fix/TK-51-duplicate-incident-on-replay`	
`docs/TK-88-handbook-v1`	
`chore/TK-12-bump-nestjs-10`	
`hotfix/TK-99-jwt-expiry-crash`	
`chore/repo-gitignore-cleanup`	No Jira key. Housekeeping has no card.
`fix/daily-report-reported-mentions`	No Jira key. Automation and CI work has no card.

Rules:

- **Lower-case, kebab-case** after the type. No underscores, no `CamelCase`, no spaces.
- **Jira key uppercase**, exactly as Jira issued it (`TK-45`, not `tk-45`).

> **Jira keys exist for Epics and User Stories only** (**D-024**). A branch carries a key when its unit of work is an Epic or a User Story card. Everything else carries no key and uses a plain descriptive branch name: repository housekeeping, CI and automation, tooling, conventions and documentation that is not a story deliverable, and the Task or Subtask breakdown under a story, which is tracked inside the parent card rather than as its own branch.
>
> Do not invent a card so a branch can have a key. An invented card pollutes the backlog, the Coverage Matrix and the burndown, all three of which the supervisor reads.

- Keep it under ~50 characters total. The description is a reminder, not a summary.
- One branch per unit of work. Do not accumulate unrelated changes on a long-lived personal branch.

Why the Jira key is in the branch name: the org-level Jira↔GitHub integration scans branch names, commit messages, and PR titles for issue keys. With the key present, the Jira card's Development panel auto-populates with the branch, its commits, and its PR, for free, with zero extra effort from you. That is the traceability the supervisor and the council look for, and it costs nothing but a naming habit.

2.3 Design and implementation live on the same branch

The old dual-branch model (`features/Design_X` + `features/Implementation_X`) is **retired**. A single `feat/*` branch carries the spec commits *and* the implementation commits, in that order.

You still cannot write code before the spec exists, the spec-before-code gate in `02-spec-driven-development-workflow.md` (`02-spec-driven-development-workflow.md`) is unchanged and non-negotiable. The gate is enforced by *commit order within the branch*, not by a second branch: the reviewer reads `git log` and sees `docs(TK-45): requirements + design for devices` landing before `feat(TK-45): implement device entity` . That gives the same review independence at half the PR overhead.

The Design DoD section in the PR template is therefore **optional**, fill it in when the branch introduced or changed a spec, skip it when it didn't.

2.4 Hotfix protocol

Hotfixes are branch-specific. Which branch you cut from depends on where the fault actually is.

See **Figure 19**.

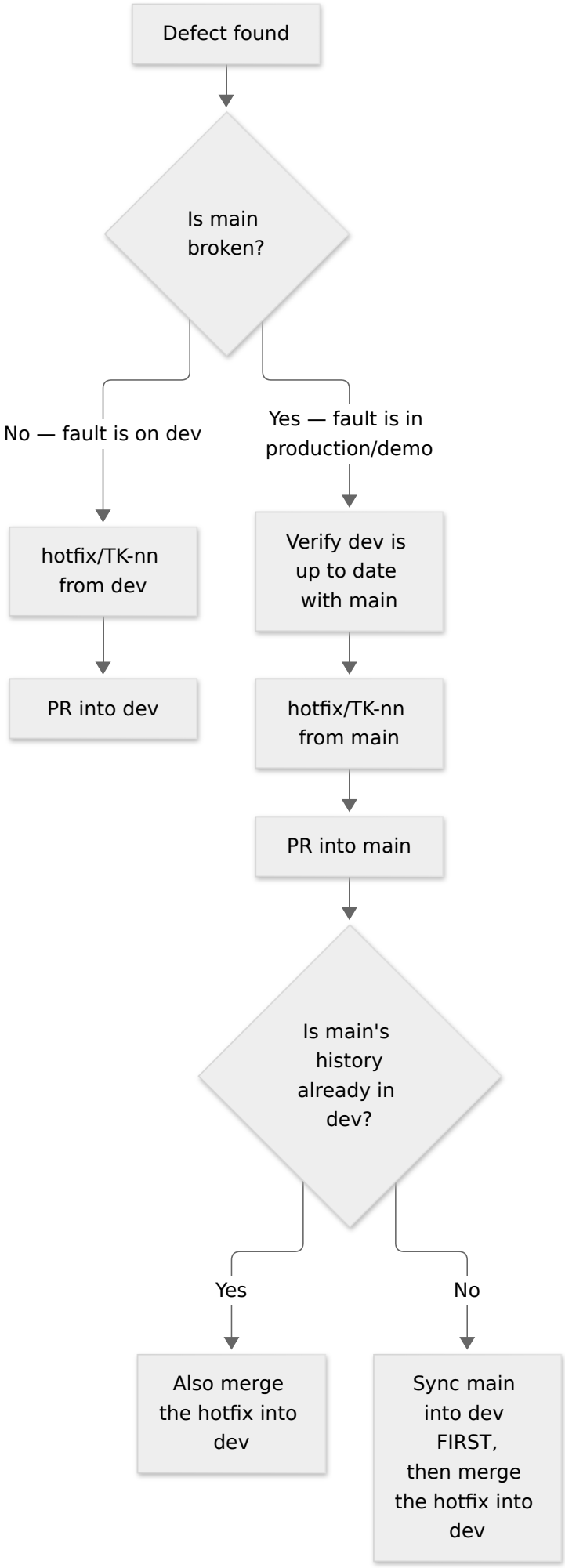

Defect found
Is main broken?
No — fault is on dev
Yes — fault is in production/demo
hotfix/TK-nn from dev
PR into dev
Verify dev is up to date with main
hotfix/TK-nn from main
PR into main
Is main's history already in dev?
Yes
No
Also merge the hotfix into dev
Sync main into dev FIRST, then merge the hotfix into dev

Figure 19: Defect routing: whether a fault becomes a hotfix off `main` or an ordinary fix off `dev` depends on whether `main` is broken. Placement: inline, 101.1 x 266.0 mm, labels at 9.95 pt.

Before cutting any hotfix: bring every branch up to date. A hotfix applied to a stale base is how a fix gets silently reverted by the next merge. Concretely:

```
git fetch origin
git checkout main && git pull origin main
git checkout dev  && git pull origin dev
# confirm dev contains main's history:
git log --oneline main ^dev        # must print nothing
```

If that last command prints commits, `dev` is behind `main`, sync it before hotfixing, or the fix will only exist on one side.

A hotfix landed on `main` **must also reach** `dev`, always. A fix that lives only on `main` will be undone the next time `dev` is promoted.

---

3. Commit Conventions

3.1 Format

```
{type}({scope}): {imperative description}
```

The scope is the **Jira key** when the unit of work is an Epic or a User Story card (**D-024**). Housekeeping, CI, automation, tooling and conventions work has no key, so it has no scope.

```
git commit -m "feat(TK-45): add device FSM transition guard"
git commit -m "fix(TK-51): reject duplicate eventId before incident creation"
git commit -m "docs(TK-88): add Jira workflow chapter to handbook"
git commit -m "chore: refactor conventions files"          # no card — plain, unscoped
```

3.2 Types

All standard Conventional Commit types are available:

Type	Use for
`feat`	New capability
`fix`	Bug fix
`docs`	Documentation only
`test`	Adding or fixing tests
`refactor`	Restructuring with no behaviour change
`perf`	Performance work
`style`	Formatting, whitespace, lint fixes, no logic
`chore`	Dependencies, tooling, config, housekeeping
`ci`	CI/CD pipeline changes
`build`	Build system, bundler, compiler config
`revert`	Reverting a previous commit

3.3 Scope

Mandatory **where meaningful**. Use the Jira key. When the change is genuinely general, cross-cutting, or has no card, repository housekeeping, a sweep across every file, a `.gitignore` tweak, **omit the scope entirely** rather than inventing one:

```
chore: refactor conventions files          # correct — general, no card
chore(misc): refactor conventions files    # wrong — "misc" is noise
```

3.4 Quality rules

1. Understandable from the subject line alone.
2. No vague messages. `"update"`, `"fix bug"`, `"wip"`, `"asdf"` tell the next reader nothing.
3. One logical change per commit.
4. Never bundle formatting churn with functional changes, that is what `style:` is for.
5. Imperative mood: "add", not "added" or "adds".
6. English only. See §6.

3.5 Commit messages are a practice, not a gate

> **Commit hygiene must never block a delivery.** There is no commitlint hook and no CI check on commit messages, deliberately. When you are shipping at 23:50 the night before a review, a commit that says `fix: login` is acceptable and nobody will chase you for it.
>
> Adhere to the format whenever you reasonably can, it is what makes `git log --oneline` readable during Review 2 prep and what feeds the Jira Development panel. But a perfect commit history on an unshipped feature is worth nothing. Ship, then tidy if there's time.

---

4. Rebase Discipline

We keep linear history. No merge commits anywhere, ever.

4.1 Before opening a PR

```
git fetch origin
git rebase origin/dev
# resolve conflicts, preserving business rules — re-read the spec if unsure
npm test                                   # must pass before you push
git push --force-with-lease origin feat/TK-45-device-registration
```

Always `--force-with-lease`, never `--force`. It refuses to overwrite work you haven't seen, which matters when an AI agent or a second machine has pushed to the same branch.

4.2 After *any* PR merges into `dev`: everybody rebases

This is a team-wide obligation, not a courtesy:

- **If you have no open PR**: `git checkout dev && git pull origin dev` before you branch again.
- **If you have an open PR**: rebase it onto the new `dev` and force-push. Your PR now reviews against current reality.

```
git fetch origin
git checkout feat/TK-45-device-registration
git rebase origin/dev
npm test
git push --force-with-lease
```

This is why **a merge into `dev` must be announced in the Zalo group** (see `12-communication-and-daily-reports.md` (`12-communication-and-daily-reports.md`)). The announcement is the signal for everyone to stop, pull, and rebase. Merging silently and letting four teammates discover it through conflicts three hours later is the single most expensive avoidable mistake on this project.

---

5. Pull Requests

5.1 Opening a PR

```
git push -u origin feat/TK-45-device-registration
gh pr create \
  --base dev \
  --title "feat(TK-45): device registration" \
  --body-file .github/pull_request_template.md \
  --assignee @me \
  --reviewer {reviewer-github-handle}
```

Then **ping the reviewer in Zalo** with the PR link. GitHub notifications are not reliably read; the Zalo ping is what actually starts the review clock.

Draft PRs are allowed as a work-in-progress signal, and reviewers will not look at them. When you want a review, mark it Ready for Review, that is the explicit handoff. A draft PR sitting for two days is not "waiting for review", it is not in the queue at all.

Stacked PRs (a PR based on another open PR) are allowed. Set the `--base` to the parent branch and say so in the description, so the reviewer knows the diff includes inherited commits.

5.2 Who reviews

The **backlog's `Reviewer` field is the default reviewer** and it is binding. It is set per story in `../03-backlog/02-user-stories.md` (`02-user-stories.md`).

- The leader (**KhoaDD**) is the Product Owner and default reviewer for most stories.
- **TanNB** and **LongLP** are delegated reviewers, the leader may assign review to either when capacity requires it.
- **The PR author may re-assign the reviewer** if the assigned reviewer is busy, without asking permission first. Tell the new reviewer; you do not need to tell the busy one.

5.3 Self-approval: the narrow exception

> **A PR author may self-approve and merge ONLY if the change meets every one of these:**
>
> - under **50 lines** changed, and
> - **no behavioural impact**, nothing a test or a user could observe differently, and
> - it is housekeeping: a chore, a cleanup, a file move, formatting, a comment, a doc typo.
>
> **Everything else requires a second party. Including the leader's own PRs.**

The Product Owner is not exempt. When the leader authors a non-trivial PR, the leader **must run a separate AI review pass on it before merging**, a fresh agent session, pointed at the diff, with no memory of having written the code. The output of that pass goes in the PR as a review comment.

The reasoning is not bureaucratic: work is not correct because of who produced it. An author reviewing their own code re-reads their intent, not their output. A fresh reviewer, human or agent, reads what is actually there. That asymmetry is the entire value of review, and it does not disappear when the author happens to be the PO.

5.4 Reviewing a PR

Pull the branch and actually run it. Reading the diff in the browser is not a review.

```
gh pr checkout 42
npm ci

# Run the same gates the author was supposed to run:
npm --prefix backend  run lint && npm --prefix backend  run typecheck && npm --prefix backend  test
npm --prefix gateway  run lint && npm --prefix gateway  run typecheck && npm --prefix gateway  test
npm --prefix frontend run lint && npm --prefix frontend run typecheck && npm --prefix frontend run build

# Bring up the local stack if the change touches runtime behaviour:
docker compose up -d
```

Unit tests are not optional and not negotiable.

- The author runs them and they pass **before** the PR is opened.
- The reviewer runs them **again** on their own machine.
- A feature with no tests is not done. Shipping untested code by simply not writing a test is cheating the gate, and the reviewer should request changes on that basis alone.
- A **failing test is evidence of a flaw** until the author explains otherwise. "That test is flaky" is a claim that needs proof, not a dismissal.

Then decide, using the severity vocabulary in `03-operational-workflows.md` (`03-operational-workflows.md`) §4: `[APPROVED]` · `[CHANGES REQUESTED]` · `[NEEDS FIXES]` .

> **On "LGTM".** It is a legitimate approval for a genuinely small, genuinely clear change you have actually read and run. It is not a way to clear your review queue. If the diff is 400 lines and your review is four characters, you did not review it, you signed for it. Use it gracefully.

5.5 Review SLA

- A PR opened during working hours (**08:00–17:00**) is reviewed **before 17:00 that day**.
- A PR left unreviewed at 17:00 becomes **top priority the next morning**.
- Absolute ceiling: **1 day**. After that the author escalates in Zalo, or re-assigns per §5.2.

5.6 Mid-review fixes

When changes are requested:

1. Open the PR skill / read the review comments in full before touching anything.
2. `git checkout feat/TK-45-...` , confirm you are on the right branch. Fixing a review on `dev` is a classic and painful mistake.
3. Fix, commit (`fix(TK-45): address review — validate deviceId length`), rebase if `dev` moved.
4. `git push --force-with-lease` .
5. **Comment on the PR** saying what you changed and what you deliberately did not. Then wait. Pushing silently does not re-request review.

If you are stuck on a review comment rather than disagreeing with it, move the Jira card to `NEEDS HELP` and say so in the PR, see `10-jira-tracking-and-workflow.md` (`10-jira-tracking-and-workflow.md`) §4.

5.7 Merging

Situation	Method
Single commit on the branch	**Rebase and merge**
Multiple commits on the branch	**Squash and merge**
Anything at all	**Never "Create a merge commit"**

> **"Create a merge commit" is prohibited in this project.** It produces a non-linear history that makes `git log --oneline`, `git bisect`, and Review-2 traceability materially harder. Disable it in repository settings so the option is not offered.

Branch deletion after merge:

- `feat/*`, `fix/*`, `docs/*`, `chore/*`, `hotfix/*` → **delete the source branch** ("Delete branch after PR"). They are temporary by definition and a stale branch list hides the live work.
- `dev` → `main` → **never delete the source branch.** `dev` is permanent.

Who merges into `main`: the leader only. The supervisor never touches the repository, all review flows through the leader, who opens and merges the `dev` → `main` PR.

How `dev` reaches `main`: fast-forward, never the merge button (D-025). The release PR is still opened and approved as usual, but it is landed by the leader with a fast-forward push:

```
git fetch origin
git merge-base --is-ancestor origin/main origin/dev && git push origin origin/dev:main
```

GitHub's "Rebase and merge" re-creates every commit with a new SHA even when a fast-forward is possible, and "Squash and merge" collapses them into one new commit. Either way `main` ends up with commits `dev` does not have, and the next release PR conflicts on every file both sides touched. A fast-forward push gives `main` exactly `dev`'s commits. It is not a force-push, so branch protection permits it for an admin while `enforce_admins` is off, and GitHub marks the open release PR as merged. If `merge-base --is-ancestor` fails, `main` has diverged: stop and realign first (D-025 records the procedure) rather than pushing a merge.

---

6. Language Rule (Hard Requirement)

> **ENGLISH ONLY** in every artifact this project produces: source code, identifiers, comments, commit messages, branch names, PR titles and descriptions, review comments, specs, conventions, documentation, Jira cards, and diagrams.
>
> **You may prompt an AI agent in Vietnamese.** English is preferred and gets better results, but chatting to your agent in Vietnamese is fine. Regardless of what language you prompted in, **everything the agent writes into the repository must be English.**
>
> The only sanctioned exception is a daily-report entry, where Vietnamese is tolerated when English would cost you clarity, see `12-communication-and-daily-reports.md` (`12-communication-and-daily-reports.md`) §3.

This is not stylistic preference. The council reads the artifacts, the roadmap mandates English (§06 rule 5), and a mixed-language codebase is unreviewable.

---

7. Branch Protection Settings

Configure once per repository, by the leader, in Settings → Branches.

> **Applied on 2026-09-22** to all four code repositories: `treklink-docs`, `treklink-web`, `treklink-firmware` and `TrekLink-Team.github.io`. Both `main` and `dev` in each carry the settings below, plus `delete_branch_on_merge`. `capstone` is deliberately unprotected (D-022).
>
> One deviation, deliberate: `enforce_admins` is **off**. The rule below already exempts the leader from the force-push block, and with no collaborators configured (see the setup gap at the end of this section) a hard admin gate would make every pull request unmergeable. Turn `enforce_admins` on once the org has members who can approve.

`main`

- Require a pull request before merging.
- Require **1 approval**.
- Block force-push **for all members except the leader**.
- Disallow "Create a merge commit"; allow rebase-merge and squash-merge only.
- Do not allow deletions.

`dev`

- Require a pull request before merging.
- Require **1 approval** from the assigned reviewer (see §5.2, §5.3 for the self-approval carve-out).
- Block force-push **for all members except the leader**.
- Disallow "Create a merge commit"; allow rebase-merge and squash-merge only.

CI is an indicator, not a gate

> **A red CI does not block merging.** CI is advisory on this project, deliberately. A 13-week term with a hardware dependency produces plenty of legitimately-red pipelines (missing device, flaky integration, an unconfigured secret) and a hard gate would mean waiting on infrastructure instead of delivering.
>
> **But red CI means merge with caution.** Specifically:
>
> - The **leader may merge** a red-CI PR at their discretion.
> - A **non-leader reviewer may not**, they must get the leader's explicit go-ahead first.
> - Either way, **say in the PR why it's red** before merging. An unexplained red merge is how a real break gets normalised into background noise.

Known setup gap

> **The GitHub organisation does not yet have collaborators configured**, so reviewers cannot currently be assigned in the UI. Until the leader adds all five members to the `TrekLink-Team` org with write access, reviewer assignment happens by Zalo ping and the `--reviewer` flag will fail. This is a one-time setup task owned by the leader.

---

8. Where Work Is Tracked

Jira is the single work tracker. GitHub Issues hold daily reports and real defects only. Full detail in `10-jira-tracking-and-workflow.md` (`10-jira-tracking-and-workflow.md`).

Concern	Lives in
Epics, User Stories, Tasks, Subtasks, sprints, board, timeline	**Jira** (TK project)
Daily reports	**GitHub Issues** (`treklink-docs`)
Standalone bugs & blockers	**GitHub Issues** (`treklink-docs` or the affected repo)
The authoritative backlog with EARS criteria	**`treklink-docs/_docs/03-backlog/`** (read-only to members)
Specs	**`treklink-web/specs/{module}/`**

> GitHub **Milestones are no longer used as sprints**, sprints live in Jira. The `points:*` and `type:*` label taxonomy is retained, but only for labelling daily reports and bug issues. See `../.github/labels-and-milestones.md` (`labels-and-milestones.md`).

---

9. Quick Reference

```
# ── Start work ──────────────────────────────────────────────────────
git fetch origin && git checkout dev && git pull origin dev
git checkout -b feat/TK-45-device-registration

# ── Work ────────────────────────────────────────────────────────────
git add -p
git commit -m "feat(TK-45): add device FSM transition guard"

# ── Before opening the PR ───────────────────────────────────────────
git fetch origin && git rebase origin/dev
npm test                                  # must be green
git push -u origin feat/TK-45-device-registration --force-with-lease
gh pr create --base dev --title "feat(TK-45): device registration"
# → then ping the reviewer in Zalo, and move the Jira card to IN REVIEW

# ── Reviewing someone else's PR ─────────────────────────────────────
gh pr checkout 42 && npm ci && npm test
gh pr review 42 --approve                 # or --request-changes --body "..."

# ── After any merge lands on dev ────────────────────────────────────
git fetch origin && git rebase origin/dev && git push --force-with-lease
```

---

10. GitLab → GitHub Concept Map

For anyone cross-reading the school's `Git_Lab_Guide.pdf` :

GitLab (school guide)	TrekLink on GitHub
Merge Request (MR)	Pull Request (PR), terms used interchangeably
Issue Board	Jira board (Kanban view)
Labels (effort points 1/2/3/5/8/13)	Jira story points, same Fibonacci scale (1/2/3/5/8/13/21)
Milestones (sprints)	Jira sprints
Child tasks	Jira subtasks, and see §8: subtasks are PR-controlled, not backlogged
`develop` branch	**`dev`**
`features/Implementation_X` + `features/Design_X`	**one** `feat/TK-nn-desc` branch
`release/sprint_x`	**dropped**, `main` is the release
`hotfix/Bug_X`	`hotfix/TK-nn-desc`
`[Feature]` bracket-tag commits	**Conventional Commits**, `feat(TK-45): ...`
MR approval + merge	PR review + Rebase-and-merge (or Squash if multi-commit)

Jira Tracking, Board Workflow & the Development Loop

> **Jira is the single work tracker.** Epics, User Stories, Tasks, Subtasks, sprints, the board and the timeline all live in Jira. GitHub Issues hold daily reports and standalone bugs, nothing else. This chapter is the authority on how work is logged, moved, and closed.
>
> **Project**: `TK` , treklink-capstone.atlassian.net **Framework**: Scrum, 2-week sprints aligned to the roadmap calendar.

---

1. The Three Places Work Lives

A recurring source of confusion, settled here once:

Artifact	Where	Who writes it	Mutability
The backlog, 88 stories with EARS acceptance criteria, owners, reviewers, points	`../03-backlog/` in `treklink-docs`	**Leader only**, via `build_backlog.py` + a `docs:` PR	**Read-only to members.** Never hand-edit the generated `.md` files.
The tracker, live cards, sprint state, board position	**Jira**	Leader creates Epics/Stories; anyone creates Tasks/Subtasks	Live, moves constantly
The spec, full EARS requirements, design, tasks	`treklink-web/specs/{module}/`	The story's owner	Per-story, locked during implementation

The backlog is the **source**; Jira is the **execution copy**. A story's EARS criteria in the backlog get expanded into that module's `requirements.md` when its sprint starts. If the two ever disagree after that point, the spec wins, it is the more detailed, reviewed artifact.

---

2. Work Item Hierarchy

```
Epic  (E1–E8, from the backlog — leader creates)
  └── User Story  (US-001 … US-088, from the backlog — leader creates)
        └── Subtask  (invented during implementation — anyone creates)
```

2.1 Epics and User Stories: leader creates

Epic and Story cards mirror the backlog one-for-one. The leader creates them in Jira at sprint planning, carrying over the backlog's summary, acceptance criteria, owner, reviewer, and points.

A member may create an Epic or Story when one is genuinely missing or miscellaneous work appears that nobody anticipated. Tell the leader when you do, so it can be reconciled back into the backlog.

2.2 Subtasks: anyone creates, and they stay out of the backlog

> **Subtasks are never logged in the backlog.** They exist in Jira and in the PR, and nowhere else.
>
> A subtask is how you break "implement device registration" into "write the migration", "add the DTO", "wire the guard". That decomposition is real work-management, but recording it in the authoritative backlog is documentation effort with no delivery attached to it. Put the breakdown in the PR description or the module's `tasks.md` , where the reviewer will actually read it.

2.3 Story points

Manual, set by judgement at sprint planning. The scale is **Fibonacci**, which is the industry standard and what Jira ships by default:

```
1 · 2 · 3 · 5 · 8 · 13 · 21 · …
```

Points estimate **difficulty**, not hours. The widening gaps are the point: they force a decision between "about a 5" and "about an 8" rather than inviting false precision at the top end, where estimates are least reliable.

A 1 is trivial, a 3 is an ordinary story, a 5 is a substantial one, and anything at **13 or above is a signal the story should be split** before it enters a sprint.

> An earlier revision of this chapter published a base-5 scale (1/2/3/5/10/15/20/25/30). That was a local invention, not a standard, and the backlog was never estimated against it, the 88 stories in `03-backlog/` have always used 2/3/5/8/13. Corrected here so the documented scale matches the estimates that exist.

---

3. The Board Workflow

See **Figure 20**.

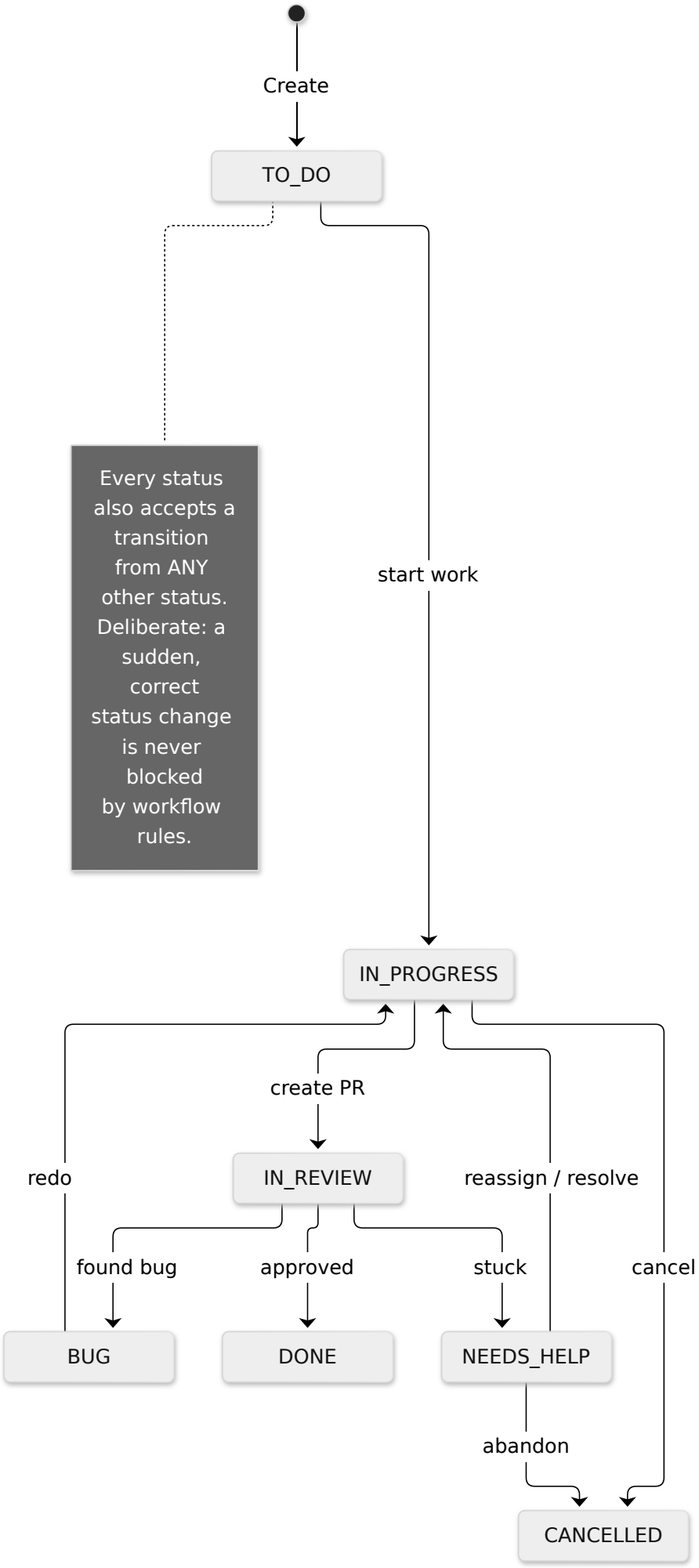

Create
TO_DO
Every status also accepts a transition from ANY other status. Deliberate: a sudden, correct status change is never blocked by workflow rules.
start work
IN_PROGRESS
create PR
redo
IN_REVIEW
reassign / resolve
found bug
approved
stuck
cancel
BUG
DONE
NEEDS_HELP
abandon
CANCELLED

Figure 20: The Jira card state machine. Seven states with `Any` transitions, a card can always be moved to reflect reality, including backwards. Placement: inline, 121.2 x 266.0 mm, labels at 9.69 pt.

Status	Meaning	Who moves it here
TO DO	Card exists, nobody has started	Created by leader at planning
IN PROGRESS	Branch cut, work underway	Owner, when they start
IN REVIEW	**PR is open and awaiting review**	Owner, at PR-open time
DONE	PR merged	Automation on merge; verified by reviewer + author
BUG	Came back from review, **the code is wrong**	Reviewer, on finding a defect
NEEDS HELP	**The person is blocked**	Anyone, the moment they're stuck
CANCELLED	Work abandoned or descoped	Owner or leader

3.1 The "Any" transitions are intentional

Every status accepts a transition from every other status. This is a deliberate design choice, not an oversight. Workflow validation that prevents someone from recording reality, a card that jumps straight from `TO DO` to `DONE` because it turned out to be already implemented, or from `DONE` back to `IN PROGRESS` because a regression surfaced, creates lying boards. The board should be easy to make accurate.

The drawn arrows are the *expected* path. The `Any` transitions are the escape hatch when reality doesn't match.

3.2 `IN REVIEW` means the PR exists

Move the card when you **open the PR**, not when it gets approved. The board should answer "what is waiting on a reviewer right now?" at a glance, and that is the whole point of the column.

3.3 Who moves the card to `DONE`

A Jira automation rule moves it on PR merge (§6). **Humans verify at standup** rather than moving it manually. If automation misses one, the reviewer moves it; if the reviewer forgets, the author does. The author is responsible for their own card being accurate regardless of whose job it nominally was.

---

4. `BUG` vs `NEEDS HELP`: they are not the same thing

This distinction matters enough to be explicit, because conflating them hides who needs rescuing.

> **`BUG` = the code is wrong. `NEEDS HELP` = the person is stuck.**

4.1 `BUG`

Review found a defect. The card returns to the author via `redo` → `IN PROGRESS`. Nothing is blocked, the author knows what to do, they just have to do it. No new card is created; this is still the same story's work.

4.2 `NEEDS HELP`

You are blocked and cannot proceed alone. The protocol:

1. Move the card to `NEEDS HELP`.
2. **Report to the leader**, Zalo, or a GitHub Issue if it needs a written thread, explaining *why* you're blocked. Not "I'm stuck", but "the Prisma migration fails on the FK constraint and I don't understand the cascade rule".
3. The leader delegates: reassigns the card, pairs you with someone, or resolves the blocker.

4. Once cleared: `NEEDS HELP` → `IN PROGRESS`, and the loop repeats until the card is genuinely `DONE`.

Being stuck *while implementing* is the common case, and it does not require opening a fake PR first. Use the `Any` transition: `IN PROGRESS` → `NEEDS HELP` directly.

> Moving a card to `NEEDS HELP` early is a good outcome, not an admission of failure. A blocker raised on day 1 costs an hour of someone's time. The same blocker raised on day 4, at the sprint boundary, costs the sprint.

---

5. Where Bugs Actually Go

Three destinations, three different kinds of defect. Choosing correctly keeps Jira clean without losing anything.

See **Figure 21**.

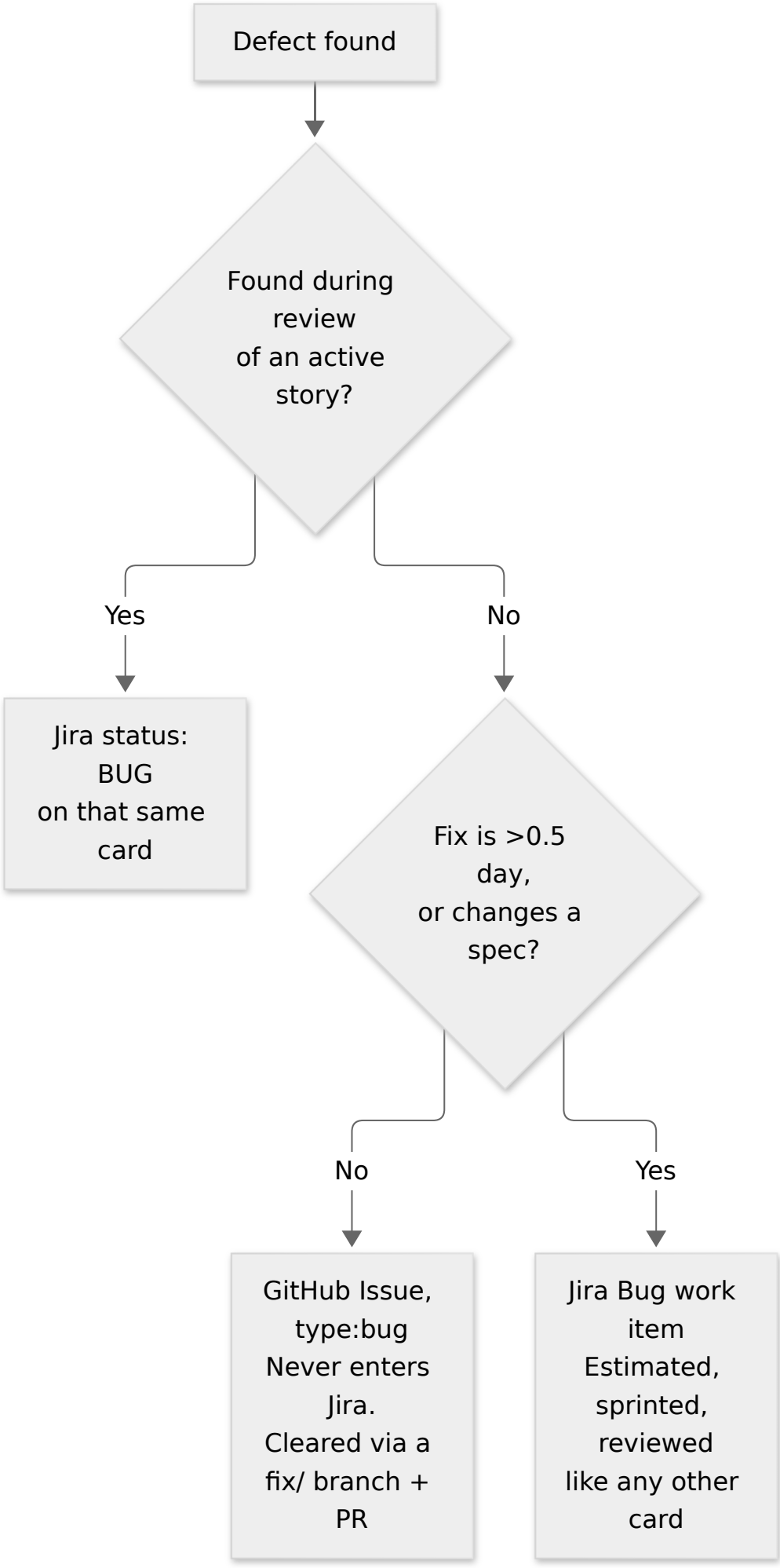

Defect found
Found during review of an active story?
Yes
No
Jira status: BUG on that same card
Fix is >0.5 day, or changes a spec?
No
Yes
GitHub Issue, type:bug Never enters Jira. Cleared via a fix/ branch + PR
Jira Bug work item Estimated, sprinted, reviewed like any other card

Figure 21: Bug routing: the three destinations a defect can take, and the test that decides between them. Placement: inline, 134.4 x 266.0 mm, labels at 12.41 pt.

Destination	When	Why
`BUG` **status** on the existing card	Review found a defect in the story being reviewed	It is that story's work. A second card would double-count the same effort.
GitHub Issue, `type:bug`	Standalone defect found outside any active story, demo, manual testing, someone stumbling on it	Fast. Fixed via a `fix/` branch and a PR, closed by the PR. **Never enters Jira.** GitHub Issues is where the team looks most often, so this is also where it gets seen.
Jira `Bug` **work item**	The fix needs its own estimate, sprint slot, and review cycle	Rule of thumb: **more than half a day, or it changes a spec.**

The escalation trigger is the only rule to remember: *if fixing it costs more than half a day or touches a spec, it earns a card.* Everything smaller is cleared in GitHub Issues without bureaucracy.

---

6. Jira ↔ GitHub Integration

The Jira instance is connected at the **organisation** level to `TrekLink-Team`, which covers all three repositories automatically, no per-repo setup.

6.1 What you get for free

Because branch names are `feat/TK-45-...` and commit scopes are `(TK-45)`, Jira scans and links automatically. Open any card and the **Development panel** shows its branch, every commit, and the PR with current review state. That is the traceability chain, card → branch → commits → PR → merge, that Review 2 and Hội đồng 1.1 look for, built with no extra effort beyond the naming convention you are already following.

This is the entire reason the Jira key is in the branch name. Drop the key, lose the panel.

6.2 Two automation rules (recommended: leader configures once)

Trigger	Action
Pull request **created** referencing `TK-nn`	Transition that card → `IN REVIEW`
Pull request **merged** referencing `TK-nn`	Transition that card → `DONE`

These are a safety net, not the process. You still move your own cards; automation catches the ones you forget. Configure under Project settings → Automation.

6.3 Smart commits: deliberately not used

Jira supports smart commits (`TK-45 #comment ...`, `TK-45 #done`) that transition cards from commit messages. **We do not use them.** They require the git commit author's email to exactly match an Atlassian account, and when it doesn't they fail **silently**, you believe the card moved, it didn't, and the board is quietly wrong. Automation rules (§6.2) key off the PR instead, which is robust.

---

7. Sprints

Two weeks, aligned to the roadmap calendar in `../00-project-context/02-roadmap-and-milestones.md` (`02-roadmap-and-milestones.md`).

View	Use for
Backlog	Create Epics/Stories/Tasks. Groom. Start the sprint from here.
Board (Kanban)	Daily execution. Move cards as work progresses.
Timeline	Overall team schedule, dependencies, deadlines against the roadmap.

7.1 Sprint mechanics

1. At sprint planning the leader pulls stories from the backlog into the Jira Backlog view.
2. Starting the sprint moves them onto the Board.
3. Work the board. Cards move left to right; see §3.
4. At sprint close, anything not `DONE` returns to the backlog or carries over explicitly.

7.1a Creating the backlog in Jira

The leader creates Epics, Stories and sprints from the backlog with one command, never by hand, so Jira and `_docs/03-backlog/` cannot drift. One-time setup:

1. Create a Jira Cloud site (free plan, up to 10 users) and a **Scrum** project with key `TK`. Leave it empty: cards are created in backlog order so their keys match the backlog's `TK-nn`.
2. Invite the four members. Install **GitHub for Jira** from the Atlassian Marketplace and connect the `TrekLink-Team` organisation, so branches and PRs named with `TK-nn` link themselves (§6).
3. Create an API token at `id.atlassian.com`, Security, API tokens. Keep it in your shell, never in a file in any repository.
4. Find the board ID: open the board; it is the number after `/boards/` in the URL.

Then, from `treklink-docs`:

```
export JIRA_BASE_URL=https://<site>.atlassian.net JIRA_EMAIL=<your Atlassian email> JIRA_BOARD_ID=<n>
read -rs JIRA_API_TOKEN && export JIRA_API_TOKEN
python3 _docs/03-backlog/jira_sync.py            # dry run: the plan
python3 _docs/03-backlog/jira_sync.py --apply    # create epics, stories, sprints
```

The script is safe to re-run: each card carries a `backlog-<ID>` label and existing cards are skipped. It creates two-week sprints named `TK Sprint n` from the calendar start in `.github/schedule.yml`; stories planned for a sprint that has already ended go into the current sprint. Start the current sprint from the Backlog view, then add the §6.2 automation rules.

7.2 Two clocks, and they are not the same clock

> The **team sprint** and the **capstone roadmap milestone** are independent timelines. A sprint ending is a team-internal event. A Review 1 dossier deadline is an external, graded, immovable one. **Never assume one implies the other.**

`treklink-docs` CI posts a countdown for both, and flags loudly when they land in the same week, the weeks where the team is simultaneously closing a sprint and preparing a graded submission are exactly the weeks things get dropped. See `12-communication-and-daily-reports.md` (`12-communication-and-daily-reports.md`) §5.

---

8. The Development Loop (End to End)

This is the canonical sequence. Every piece of work follows it.

See **Figure 22**.

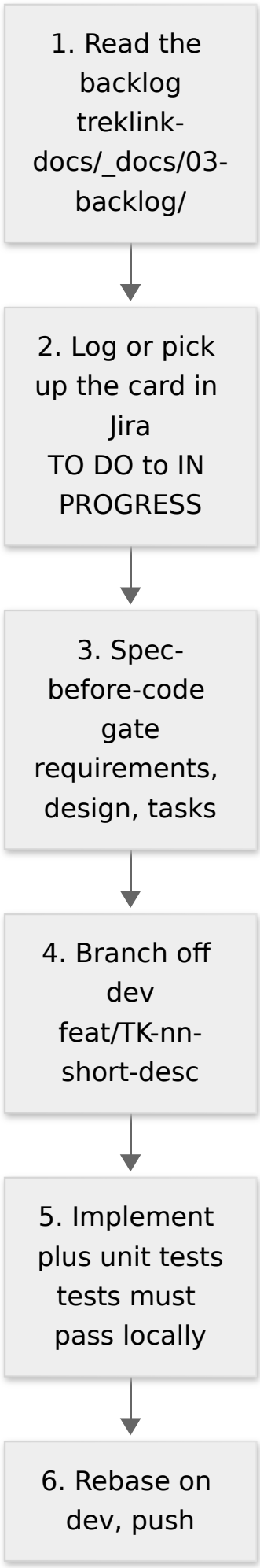

1. Read the backlog treklink-docs/_docs/03-backlog/
2. Log or pick up the card in Jira TO DO to IN PROGRESS
3. Spec-before-code gate requirements, design, tasks
4. Branch off dev feat/TK-nn-short-desc
5. Implement plus unit tests tests must pass locally
6. Rebase on dev, push

Figure 22: The delivery loop, steps 1 to 6, from reading the backlog to pushing a rebased branch. Placement: inline, 46.5 x 266.0 mm, labels at 12.57 pt.

See **Figure 23**.

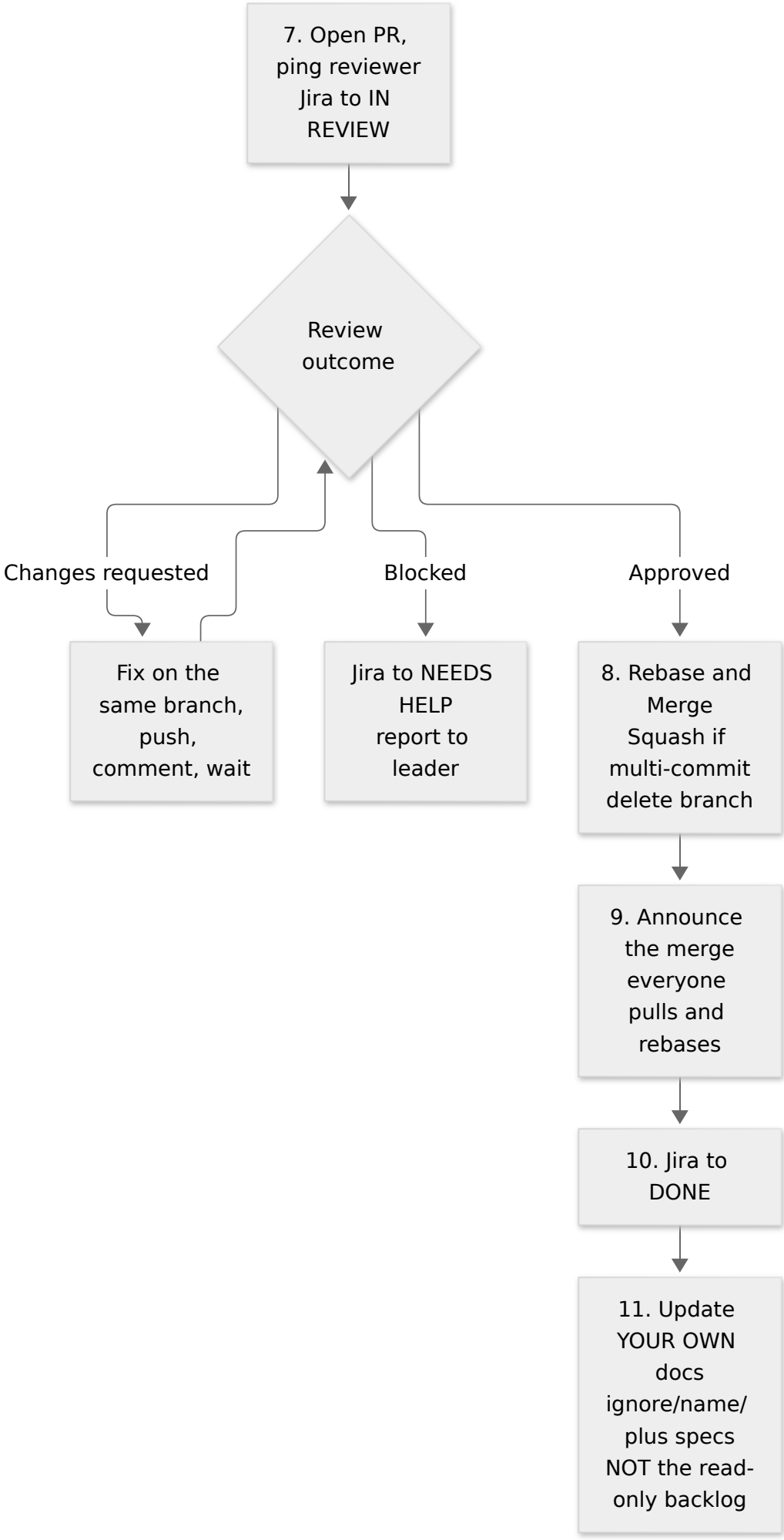

7. Open PR, ping reviewer Jira to IN REVIEW
Review outcome
Changes requested
Blocked
Approved
Fix on the same branch, push, comment, wait
Jira to NEEDS HELP report to leader
8. Rebase and Merge Squash if multi-commit delete branch
9. Announce the merge everyone pulls and rebases
10. Jira to DONE
11. Update YOUR OWN docs ignore/name/ plus specs NOT the read-only backlog

Figure 23: The delivery loop, steps 7 to 11, review outcomes, merge, and the documentation update that closes the card. Placement: inline, 138.6 x 266.0 mm, labels at 10.53 pt.

Step notes

Step 1, read the backlog. Not Jira. The backlog holds the EARS acceptance criteria; the Jira card holds a summary. You implement against the criteria.

Step 3, the spec gate is real. No production code, schema migration, or view before `requirements.md` → `design.md` → `tasks.md` exist and are approved. See `02-spec-driven-development-workflow.md` (`02-spec-driven-development-workflow.md`).

Step 11, update your own docs, not the backlog.

> **The backlog in `treklink-docs/_docs/03-backlog/` is READ-ONLY to members.** It is generated by `build_backlog.py` ; hand-edits are destroyed on the next regeneration. Post-merge updates go to your personal garden (`ignore/{your_name}/`) and to the module's `specs/` . Only the leader regenerates the backlog, via a `docs:` PR.

---

9. Backlog ↔ Jira Key Mapping

Both numbering schemes exist and both matter, the backlog's `US-nnn` is the stable documentation ID that the specs and reports cite; Jira's `TK-nn` is the live tracker ID that branches and commits carry. They must stay correlated.

9.1 How the mapping works

`build_backlog.py` computes Jira keys from a **base offset** plus creation order, rather than requiring 88 keys to be typed by hand:

```
JIRA_PROJECT_KEY = "TK"
JIRA_EPIC_BASE   = 6     # first epic created in Jira is TK-6 (D-029)
JIRA_STORY_BASE  = 14    # first story created in Jira is TK-14 (after 8 epics)
JIRA_OVERRIDES   = {     # manual escape hatch when reality drifts
    "US-042": "TK-137",
}
```

Epics and stories are then numbered in the order the leader creates them in Jira, and the generated backlog, the `.md` files, and the `.xlsx` all carry a `Jira Key` column.

9.2 When it drifts: and it will

Jira issue numbers only ever increase, and a deleted or manually-created card leaves a gap. When the computed key stops matching reality, add an entry to `JIRA_OVERRIDES` and regenerate. The override always wins over the computed value.

Drift is non-blocking. A wrong key in the backlog is a documentation inconvenience, not a broken build. Fix it in the next `docs:` PR; do not stop delivery over it.

9.3 Who maintains it

The **leader**, via a `docs:` PR to `treklink-docs` editing `build_backlog.py` and regenerating. Never by hand-editing `01-epics.md` or `02-user-stories.md` .

```
cd _docs/03-backlog
python3 build_backlog.py
python3 build_backlog.py ../00-project-context/User_Story_Agile_TrekLink.xlsx
```

---

10. Quick Reference

I want to…	Do this
Start a story	Jira `TO DO` → `IN PROGRESS`, read backlog criteria, check the spec exists
Break work down	Create Jira **subtasks**, do not add them to the backlog
Open a PR	Jira → `IN REVIEW`, ping the reviewer in Zalo
Report a defect found in review	Jira → `BUG` on the same card
Report a standalone defect	**GitHub Issue**, `type:bug` label
Report being blocked	Jira → `NEEDS HELP` + tell the leader why
Close out	Verify Jira → `DONE`, update `specs/` and your own `ignore/` garden
Change the backlog	You can't. Ask the leader.

Team Communication, Daily Reports & Schedule Awareness

> How the team stays synchronised: where each kind of message belongs, the daily report ritual, the merge-announcement obligation, and the automated schedule countdown.

---

1. Channels: What Goes Where

Channel	Use for	Do not use for
Zalo group	Merge announcements, reviewer pings, "I'm blocked", fast coordination	Anything that needs to survive the week. Zalo is not a record.
GitHub Issues (`treklink-docs`)	Daily reports, standalone bugs, blockers needing a written thread	Feature work, that's Jira
Jira	All work tracking: epics, stories, tasks, subtasks, sprint state	Discussion. Cards are state, not conversation.
PR comments	Code review, design debate tied to a specific change	Status updates
`treklink-docs` `_docs/`	Decisions, conventions, anything that must outlive the term	Work-in-progress notes
`ignore/{your_name}/`	Your own notes, session files, scratch	Anything a teammate needs to see

The rule of durability: if losing a message would cost the team, it does not belong in Zalo.

---

2. The Two Mandatory Announcements

2.1 Merging into `dev`

> **Every merge into `dev` must be announced in the Zalo group, immediately.**

Everyone else must stop, pull, and rebase, see `07-github-workflow-git-conventions.md` (`07-github-workflow-git-conventions.md`) §4.2. That obligation cannot be honoured if nobody knows the merge happened. Merging silently and letting four teammates discover it via conflicts three hours later is the most expensive avoidable mistake on this project.

```
Merged PR #42 → dev : feat(TK-45) device registration
Everyone please pull + rebase your branches.
```

2.2 Pinging your reviewer

Opening a PR is not a request for review, GitHub notifications are not reliably read. **Ping the reviewer in Zalo with the PR link.** That ping starts the SLA clock (§5.5 of the Git chapter: same-day if opened 08:00–17:00, one day absolute ceiling).

---

3. Daily Reports

3.1 The ritual

Every working day, an automated GitHub Issue titled `DAILY REPORT DD/MM/YYYY` opens in `treklink-docs` at **08:00 ICT**. Every member comments their report on **that day's issue**. The issue **auto-closes at 12:00 ICT**.

3.2 Format: exactly three lines

```
KhoaDD

Yesterday: compile docs for report1
Today: Plan main flows

Issues: None
```

- **Yesterday**, what you did. Link the PR or the issue if there is one.
- **Today**, what you will do.
- **Issues**, blockers, or `None`. Do not leave this blank.

Keep it to those three fields. The value is in everyone being able to read five reports in thirty seconds.

3.3 The 12:00 cutoff and lateness

> **Reports are due before 12:00 (midday).** CI closes the issue at 12:00 sharp regardless of who has reported.

If you missed the cutoff:

1. **Reopen the issue.**
2. Post your report.
3. Close it again.

The reopen is deliberately visible. The issue timeline records who reported after the cutoff and when, it doubles as the lateness log, without anyone having to police it manually. The closing comment CI posts at 12:00 names anyone who hadn't reported yet.

This is a discipline mechanism, not a punishment one. Consistent lateness is a signal to the leader that something needs addressing, which is exactly what a leader should be able to see.

3.4 Language

English is required and strongly preferred. Vietnamese is tolerated *by exception* in daily reports only, when writing it in English would genuinely cost you clarity about a blocker.

This is the only place in the entire project where non-English text is acceptable. Everything else , code, commits, PRs, specs, Jira cards, documentation, is English without exception. See `07-github-workflow-git-conventions.md` (`07-github-workflow-git-conventions.md`) §6.

3.5 Schedule

Opens	08:00 ICT (UTC+7), **Monday–Friday**
Closes	12:00 ICT, same day
Weekends	No automatic issue
Holidays / no-class days	Skipped automatically where the CI can determine it

Working outside business hours or at a weekend? **Create the daily report issue manually**, same title format, and report into it.

3.6 Notifications

The opening issue **@-mentions all five members**, which triggers GitHub's email notification to everyone. That is intentional, the mail is the nudge. It also posts a checklist so who has and hasn't reported is visible at a glance:

```
### Reported
- [ ] @ruskicoder (KhoaDD)
- [ ] @lamphilong2004 (LongLP)
- [ ] @wangf28 (HoangTK)
- [ ] @NgocLong216 (LongNN)
- [ ] @nguyenbatan21112003 (TanNB)
```

The roll-call carries the handle **and** the team code. The handle is what notifies. The code is what a reader recognises, because the handles do not resemble the codes.

> `@handle` is what sends the notification. A markdown link, `[@handle](https://github.com/handle)`, renders identically and notifies nobody. Never tidy the mentions into links.
>
> A handle must also be an **org member**. GitHub does not notify a mention of someone without access, so a wrong-but-real handle fails silently. `LongLP` pointed at a non-member account until 2026-09-17, and every daily report until then pinged nobody.

3.7 The closing comment

At 12:00 the script posts one comment and closes the issue. It names both groups in the same format as the opening roll-call:

```
**Cutoff reached, 12:00 ICT 17/09/2026.**

Reported (4/5): @ruskicoder (KhoaDD), @lamphilong2004 (LongLP), @wangf28 (HoangTK), @NgocLong216
(LongNN)

**Did not report before the cutoff:** @nguyenbatan21112003

Reopen this issue, post your report, and close it again. The reopen is the record.
```

Everyone is mentioned, including the members who did report. The closing comment is the day's record, and a record that names four people by handle and the fifth by code is not one record.

Attendance is counted from the **human** comments only. The script filters out its own comments before scanning, because the closing comment contains every reported member's code; an unfiltered scan on a reopen-report-reclose cycle would read its own previous roll-call back and credit everybody. See `.github/scripts/daily_report.py`.

---

4. Raising Problems

4.1 General rules (carried over from the team's existing practice)

1. The leader opens a daily report thread each working day, now automated.
2. Members report into that day's thread.
3. **Create additional issues whenever something needs solving together.** Do not wait for permission.
4. When something is blocking you, open an Issue so the leader can help resolve it.
5. A finished feature needing review: ping the reviewer via the PR **and** Zalo.

4.2 Blocked vs broken

Mirrors the Jira distinction in `10-jira-tracking-and-workflow.md` (`10-jira-tracking-and-workflow.md`) §4:

You are…	Do
Blocked (you can't proceed)	Jira → `NEEDS HELP`, tell the leader **why**, specifically
Given broken code (review found a defect)	Jira → `BUG`, fix on the same branch
Hitting a standalone defect	GitHub Issue, `type:bug` label

Raise blockers **early**. A blocker on day 1 costs someone an hour. The same blocker on day 4 costs the sprint.

---

5. Schedule Awareness: Automated Countdown

`treklink-docs` CI maintains a live countdown so nobody discovers a deadline the day it lands.

5.1 Two independent clocks

Clock	Source	Cadence
Team sprint	Jira / the roadmap sprint mapping	Every 2 weeks
Capstone roadmap milestone	`../00-project-context/02-roadmap-and-milestones.md` (`02-roadmap-and-milestones.md`)	Fixed, external, immovable

> **These are not the same clock and one never implies the other.** A sprint ending is internal. A Review 1 dossier deadline is external and graded. When both land in the same week, the CI flags it loudly, those weeks are where things get dropped.

5.2 What the countdown shows

Posted to the repo `README.md` badge area and into the daily report issue:

```
NEXT UP
  Sprint 2 ends ............... 6 days   (Sun 20 Sep)
  Report 2 due ................ tomorrow (Sun 20 Sep)   ⚠ COLLISION
  Review 1 dossier ............ 1 week 3 days (Sun 27 Sep)
  Scope lock (form FROZEN) .... 4 weeks 2 days (Sun 18 Oct)
```

Phrasing degrades naturally as a deadline approaches: `4 weeks 2 days` → `6 days` → `tomorrow` → `TODAY`.

---

6. Weekly Rhythm

When	What
Daily, 08:00–12:00	Daily report in that day's GitHub Issue
Daily, as it happens	Merge announcements + reviewer pings in Zalo
Daily, 08:00–17:00	PRs opened in work hours get reviewed before 17:00
Every 2 weeks	Sprint close + planning; leader creates the next sprint's cards
Weekly	Supervisor (GVHD) meeting, leader prepares, per the roadmap
Per roadmap milestone	Report deliverable; see the roadmap calendar

7. Quick Reference

Situation	Channel	Urgency
Merged to `dev`	Zalo, immediately	Everyone must rebase
Need a review	PR assignment **+** Zalo ping	Same day if before 17:00
Blocked	Jira `NEEDS HELP` + tell the leader	Immediately, do not sit on it
Found a standalone bug	GitHub Issue, `type:bug`	Same day
Daily status	GitHub Issue, that day's thread	Before 12:00
Made a decision that outlives today	`_docs/00-project-context/03-decisions-and-risk-register.md`	Same working pass
Need to change a convention	`docs:` PR to `treklink-docs`	Team consensus

AI-First Development

The AI-First Doctrine, Toolchain & Prompting Standard

> **TrekLink is an AI-first project.** The team's primary development mode is **prompt and code review**, not hand-typing implementations. That is a deliberate choice, and it has a hard consequence: *the quality of what ships is bounded by the quality of the prompting and the rigour of the review.* Guideline adherence is therefore not bureaucracy, it is the difference between an agent that delivers and an agent that burns 200k tokens producing plausible garbage.
>
> **Core philosophy: docs first, code later.**

---

1. The Doctrine in One Page

Principle	What it means in practice
Docs first, code later	The spec is the deliverable that matters. Code is a projection of it. No production code before `requirements.md` → `design.md` → `tasks.md`.
The agent asks; you decide	An agent that starts implementing without asking questions is misconfigured. Demand the clarification gate.
Never auto-approve	Read what the agent produced. Disagree with it. Debate it. You are co-authoring, not accepting deliveries.
Ground everything	No claim without a file reference, a test result, or a cited source. "Seems right" is not evidence.
English out, always	Prompt in any language. Everything written to the repo is English.
Save tokens	End sessions early and cleanly. Never extend a session for any reason except finishing the current task.
Persist findings immediately	Chat context is volatile. If it isn't on disk, it will be re-derived at full price.

---

2. Model Policy (Hard Requirement)

> **Every team member must hold a paid frontier-model plan.** There is no free-tier path to delivering this project. Precision-critical work, embedded/IoT integration, wire-format handling, the idempotency layer, anything that touches firmware behaviour, does not tolerate a weak model, and a wrong answer there costs days.

Model facts verified 2026-09-13. Re-verify before quoting these in any report.

2.1 Approved for critical coding

Vendor	Model	Released	API $/MTok in–out	Notes
Anthropic	**Claude Sonnet 5**	2026-06-30	$2 / $10	The workhorse. Price locked permanently.
Anthropic	**Claude Opus 5**	2026-07-24	$5 / $25	1M context, Fast Mode, adaptive thinking. Default on the Max plan.
Anthropic	**Claude Fable 5.1**	—	frontier tier	*If that member can afford it.* Not expected of anyone.
OpenAI	**GPT-5.6 Sol**	—	—	Acceptable. Superseded by Astra.
OpenAI	**GPT-6 Astra**	2026-09-03	$10 / $50	1.05M context, knowledge cutoff 2026-04-30.

Planning, architecture, critical thinking and design work goes to Claude. That is a project-level preference, not a per-member choice.

2.2 Restricted: never for critical modules

Vendor	Model	Permitted uses only
Google	**Gemini 3.8 Flash** (2026-09-02, 0.75/3.75)	Ingestion, subagents, chores, codebase understanding, knowledge ingestion, explanations
Google	**Gemini 3.1 Pro**	Same

> **Google models are prohibited from coding important modules.** Ingestion, summarising, chores, explaining unfamiliar code, and running as cheap subagents, yes. Authoring the gateway queue, the incident FSM, the auth layer, or anything touching firmware, no. **No exceptions.**
>
> **Any other model, however powerful, falls under the same restriction.** The allow-list above is exhaustive for critical coding. A model not on it is an ingestion/chore tool, full stop.

> `gemini-3.1-pro` **is the current Pro-tier stable, there is no "3.8 Pro".** Also worth knowing: Google's own documentation states 3.8 Flash is built on 3.7 Flash rather than a new base model and recommends staying on 3.7 Flash for efficiency-first workloads.

2.3 If you cannot afford a plan

Raise it in the team chat. There is no silent fallback and no permission to use a weak model instead.

The sanctioned routes, in order:

1. **ChatGPT Plus free trial**, one month, giving GPT-5.6 Sol / GPT-6 Astra. Switch back to Claude when it expires.
2. **The team helps.** Five people, one shared problem. A member blocked on tooling cost is a team problem to solve, not an individual embarrassment to hide.

Given both routes, skipping a paid plan should not realistically occur.

2.4 Declaring your model

The PR template carries a **mandatory** `Model used:` **field.**

This is a **guideline backed by self-declaration, not an enforcement mechanism**, nobody can detect from a diff which model produced it, and pretending otherwise would be theatre. It exists because:

- it makes the policy visible at the moment of delivery, when it actually matters;
- it gives the reviewer a prior, a subtly wrong async pattern reads differently when the PR says "Gemini Flash" than when it says "Opus 5";
- it makes deviation a deliberate written act rather than a drift nobody noticed.

Reviewers should treat a restricted model declared on a critical module as grounds to request changes on process, independent of whether the code looks correct.

---

3. Required Toolchain

3.1 prompt-orchestrator: mandatory for everyone

The team's shared agent ruleset: `github.com/ruskicoder/system-prompts`, `prompt-orchestrator/` folder. It installs plain Markdown only, no binaries, no daemons, nothing that touches your system config.

```
git clone https://github.com/ruskicoder/system-prompts.git
cd system-prompts/prompt-orchestrator

# Linux / macOS
bash install/install-all.sh          # every supported agent
bash install/install-claude.sh       # Claude Code only

# Windows (PowerShell)
.\install\install-all.ps1
.\install\install-claude.ps1
```

Works on Linux, macOS and Windows. Installs ~40 skills into `~/.claude/skills/` and merges an `@AGENTS.md` reference into your personal `~/.claude/CLAUDE.md` without clobbering what's already there. Per-agent installers exist for Cursor, Codex, Gemini, Windsurf, Antigravity, OpenCode and OpenClaw.

Verify:

```
ls ~/.claude/skills | wc -l          # expect ~40
grep -c "prompt-orchestrator" ~/.claude/CLAUDE.md
```

3.2 TrekLink custom skills: mandatory for everyone

Project-specific skills that encode *this* project's workflow, shipped in this repo and installed the same way:

```
cd treklink-docs/skills
bash install/install-all.sh          # Linux / macOS
.\install\install-all.ps1            # Windows
```

See `../../skills/README.md` (`README.md`). The headline skill is `/treklink-session`, which drives the mandatory session workflow in §5 as a single command.

3.3 Marketplace skills & MCP connectors: install from the Claude store

Open **Explore Skills** in Claude and install these yourself. They are not vendored here because they update independently.

Bundle	Why you need it
Development skillsets	Debugging, testing strategy, code review, refactoring
Design skillsets	`design-critique`, `design-system`, `accessibility-review`, `ux-copy`
PR skillsets	`review-pull-request`, diff triage, structured review submission
Jira MCP	Read and move cards without leaving the agent session
Neon MCP	Query and inspect the shared Postgres (see D-010)
Utilities	`search`, `documentation`, `docs-coauthoring`, `system-design`

> Jira and Neon are **MCP connectors**, not Markdown skills. Each member installs and authorises them under their own account, credentials are personal and never committed. If a connector shows as unauthorised, run the OAuth flow from an interactive session; it cannot be done headlessly.

3.4 Per-member skill loadout

Everyone installs prompt-orchestrator, the TrekLink skills, and the PR + utilities bundles. Beyond that, install what matches your lane (derived from the charter skill matrix):

Member	Lane	Install additionally
Đỗ Đăng Khoa (KhoaDD), Leader / PO	Backend architecture, review, all cross-cutting decisions	`architecture`, `system-design`, `review-software-architecture`, `review-pull-request`, `security-audit-codebase`, **Jira MCP**, **Neon MCP**
Lâm Phi Long (LongLP)	Gateway Bridge (Node.js), data-heavy backend	`debug`, `testing-strategy`, `api-integration`, `code-quality-testing`, **Neon MCP**
Trần Khải Hoàng (HoangTK)	Frontend	`design-critique`, `design-system`, `ux-copy`, `accessibility-review`, `design-handoff`
Nguyễn Ngọc Long (LongNN)	Backend CRUD modules (Spring Boot → NestJS transfer)	`codebase-understanding`, `documentation`, `code-quality-testing`, `debug`
Nguyễn Bá Tân (TanNB)	Reporting / DB, monitoring frontend	`data-analysis`, `sql-queries`, `design-critique`, **Neon MCP**

3.5 Personal SSOT scaffold in `/ignore`: mandatory

> Previously "recommended". **Now required**, you will use it every session regardless, and an agent that finds no personal scaffold cannot resume your context.

Scaffold your own garden inside each repo you work in:

```
ignore/{your_name}/
├── docs/
│   ├── 00-index.md
│   ├── current-progress.md          # rolling POINTER to the latest session file
│   ├── sessions/                    # one file per session instance
│   │   └── YYYY-MM-DD-HHMM-topic.md
│   ├── diagrams/                    # .mmd
│   └── flows/                       # .mmd
├── scripts/                         # local tooling, scratch scripts
└── envs/                            # .env templates — never committed
```

Install it:

```
cd treklink-docs/skills
bash install/scaffold-garden.sh {your_name} /path/to/repo
```

The full convention is `09-doc-driven-scaffold-and-ssot-conventions.md` (`09-doc-driven-scaffold-and-ssot-conventions.md`).

Recommended: `git init` inside `ignore/{your_name}/` as its own private repository and push it to a **private** GitHub repo. You get versioned notes without dirtying the project repo. The parent repo git-ignores `ignore/` entirely, so this is safe.

---

4. Agent Auto-Steering

Every repository carries an identical `AGENTS.md` (and `CLAUDE.md` pointing at it). Combined with prompt-orchestrator, **the agent reads the conventions itself, at session start, with no input from you.** You should never have to tell an agent "follow our conventions", if you do, the setup is broken; fix the setup rather than repeating yourself every session.

The `capstone/` parent folder carries one too, so an agent opened on the parent (the recommended setup) sees all three repos plus `Documents/` at once.

Open `capstone/` , not a single repo. Cross-repo work, a spec in `treklink-docs` , a wire format in `treklink-firmware` , the code in `treklink-web` , is the normal case on this project, and an agent scoped to one repo will guess at the other two.

---

5. The Mandatory Session Workflow

> **Every session on any TrekLink repo follows this shape.** Run `/treklink-session` to have the agent drive it, or follow it manually. Skipping the questions step is the single most common cause of wasted tokens on this project.

See **Figure 24**.

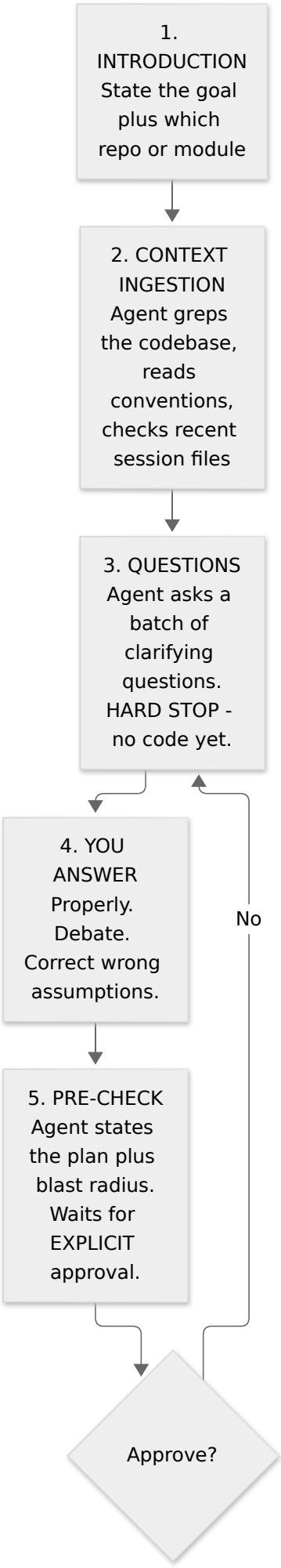

1. INTRODUCTION State the goal plus which repo or module
2. CONTEXT INGESTION Agent greps the codebase, reads conventions, checks recent session files
3. QUESTIONS Agent asks a batch of clarifying questions. HARD STOP - no code yet.
4. YOU ANSWER Properly. Debate. Correct wrong assumptions.
5. PRE-CHECK Agent states the plan plus blast radius. Waits for EXPLICIT approval.
Approve?
No

Figure 24: The AI session workflow, steps 1 to 5, introduction through the pre-check approval gate. The hard stop at step 3 is what prevents an agent guessing at the domain. Placement: inline, 49.6 x 266.0 mm, labels at 9.27 pt.

See **Figure 25**.

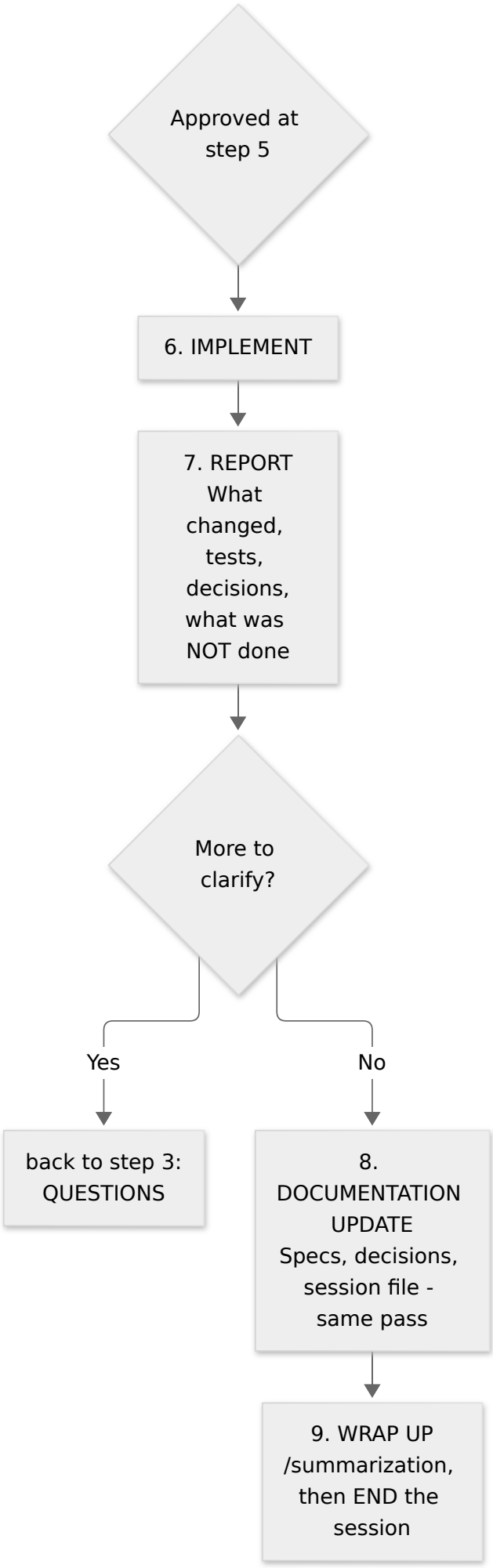

Approved at step 5
6. IMPLEMENT
7. REPORT What changed, tests, decisions, what was NOT done
More to clarify?
Yes
No
back to step 3: QUESTIONS
8. DOCUMENTATION UPDATE Specs, decisions, session file - same pass
9. WRAP UP /summarization, then END the session

Figure 25: The AI session workflow, steps 6 to 9, implement, report, document in the same pass, then end the session cleanly. Placement: inline, 88.5 x 266.0 mm, labels at 10.44 pt.

5.1 Step 1: the read-context prompt

Always open a session by orienting the agent. Never start with the task.

5.2 Step 3: the questions step is not optional

> **Tell the agent to ask you a lot of questions.** An agent that goes straight to implementing is guessing at your domain, and you will pay for those guesses in review cycles.
>
> **You must review what the agent returns.** Do not assume. Do not auto-approve. Do not let it self-advance through the gate. Apply critical thinking, argue back, and co-author the result. The agent is a very fast colleague with no context, not an oracle.

5.3 Step 8: document in the same pass

Deferred documentation is a **banned pattern** on this project. The instant you confirm a fact, resolve a risk, or change something a document governs, edit that document *now*, not at session end. Rationale and the full rule: `08-ai-agent-steering-and-discipline.md` (`08-ai-agent-steering-and-discipline.md`) Stage 2.5.

5.4 Step 9: end sessions early and cleanly

> **Do not extend a session for any reason other than completing the current task.** When the task is done, wrap up and close. "While you're here, can you also…" is how a 40k-token session becomes a 300k-token session with degraded reasoning in its final third.

---

6. Token & Context Discipline

6.1 The hard rule

> **Maximum 80% context window per session.** Crossing it triggers `/summarization` immediately, then a fresh session with the resulting handoff prompt.

Past roughly 70–80% of context, reasoning fidelity measurably drops: citations get looser, caveats get dropped, earlier decisions get paraphrased into something subtly different. Claude and ChatGPT both expose context/usage inspection, use it rather than guessing.

6.2 Session size is the agent's call, not a quota

A session may legitimately run anywhere from **20k to 500k tokens**. There is no per-task budget to hit.

The rule is proportionality, and the **agent determines it**:

- **Heavy work**, a new module, a cross-cutting refactor, a gateway design pass, *should* consume a lot. Do not artificially constrain it; a truncated architecture pass is worse than an expensive one.
- **Light work**, a chore, a cleanup, a doc typo, a one-line config change, should consume as little as possible while still being correct. Cheapness never justifies an unverified answer.

6.3 Practical savings

Habit	Why
Grep for the error; don't dump the whole log	Logs are mostly noise
Read targeted line ranges on large files	A 3000-line file read in full crowds out reasoning
Check `docs/sessions/` and `ignore/*/docs/sessions/` first	A teammate's agent may have already derived what you're about to re-derive. Reading a 20-line session file is orders of magnitude cheaper.
Write findings to disk at the moment of discovery	Re-derivation is the most expensive thing an agent does
Batch independent tool calls	One round trip instead of five
One session, one task	Context pollution degrades everything after it

---

7. How to Prompt Well

The gap between a good and a bad prompt on the same task is routinely 5× in tokens and 2× in review cycles. This is a learnable skill and it is the highest-leverage one on an AI-first team.

7.1 Principles

1. **State the goal, not the steps.** The agent can plan; let it. Prescribing the method throws away its ability to spot a better one.
2. **Give it the map.** Name the repo, module, spec, and any related decision (`D-005`). Context you supply is context it doesn't have to find.
3. **Say what "done" means.** "Tests pass and `specs/devices/requirements.md` AC-03 is satisfied" beats "make it work".
4. **Name the constraints up front.** "Don't touch the gateway package", "Prisma only, no raw SQL", "must not break the response envelope".
5. **Ask for questions explicitly.** "Ask me everything ambiguous before writing code" is the single highest-value sentence you can add.
6. **Demand evidence.** "Cite file:line for every claim" turns confident guesses into checkable statements.
7. **Be specific about scope.** Vague scope is how a one-file fix becomes a twelve-file refactor.

7.2 Anti-patterns

Don't	Why
"Fix the bug"	Which bug? What's the symptom? What have you already ruled out?
"Make it better"	Unbounded. The agent invents a definition and optimises for it.
"Do whatever you think is best"	Abdication. You own the decision; the agent owns the execution.
Approving a plan you skim-read	You will find out what it actually said during review, expensively.
Pasting a stack trace with no context	It will fix the symptom at the throw site, not the cause.
Continuing a session past its task	Degraded reasoning + polluted context, for no benefit.

---

8. Starter Prompts

Copy, paste, append your specifics. These are deliberately generic, the value is in the shape.

8.1 Session opener: read context

```
You are working on the TrekLink capstone (FA26SE159). The workspace root is `capstone/`,
containing treklink-docs (SSOT), treklink-web (the build), treklink-firmware (inherited),
and Documents/ (graded reports).

Before doing anything:
1. Read treklink-docs/_docs/01-conventions/ — especially 07 (git), 10 (Jira), 11 (AI-first).
2. Read treklink-docs/_docs/00-project-context/03-decisions-and-risk-register.md and tell me
   if anything marked OPEN blocks what I'm about to ask.
3. Check docs/sessions/ and ignore/*/docs/sessions/ for anything from the last 24h touching
   the same area — do not re-derive what a recent session already found.
4. Tell me which sprint and roadmap week we're in.

Then stop and confirm you have context. Do not start work yet.
```

8.2 Feature request

```
I want to implement {WHAT} in the {MODULE} module of treklink-web.

Backlog story: US-0nn / Jira: TK-nn
Spec: treklink-web/specs/{module}/

Constraints:
- Spec-before-code: if requirements.md/design.md/tasks.md don't cover this, we write those first.
- Response envelope { result, isSuccess, statusCode, message } is non-negotiable.
- Prisma only. Module isolation: no cross-module repository access.
- Unit tests required. They must pass before we open a PR.

Before writing anything, ask me every clarifying question you have — domain edge cases,
authorization rules, error conditions, anything ambiguous. Batch them. Then STOP and wait.
```

8.3 The questions prompt (use when the agent skips the gate)

```
Stop. Before you implement anything, ask me a comprehensive batch of clarifying questions.

Cover at minimum:
- Domain edge cases and failure modes
- Authorization: which of Admin/Staff/Guide/Customer, and what ownership checks
- Error conditions and what the system should do for each
- Anything in the spec that is ambiguous, contradictory, or missing
- Anything you are about to assume

State your assumptions explicitly so I can correct them. Do not write code until I answer.
```

8.4 Pre-implementation approval gate

```
Before you touch any file:
1. List every file you will create or modify, and why.
2. State the blast radius — what else depends on these, what could break.
3. State what you are deliberately NOT doing.
4. Flag anything you're under 80% confident about.

Then wait for my explicit approval. Do not proceed on silence.
```

8.5 Debugging

```
Symptom: {EXACT ERROR OR OBSERVED BEHAVIOUR}
Expected: {WHAT SHOULD HAPPEN}
Already ruled out: {WHAT YOU'VE CHECKED}
Repro: {STEPS}

Work hypothesis-first: state ONE falsifiable hypothesis, name the exact check that would
confirm or refute it, run that check, state the finding once, move on. Do not guess at causes
without evidence. Cite file:line. When you find it, fix the root cause only — no drive-by
refactors — and add a regression test that fails without the fix.
```

8.6 PR review

```
Review PR #{N} on {REPO}.

gh pr checkout {N}, then run the actual test suites — don't just read the diff.

Check:
- EARS criteria in specs/{module}/requirements.md actually satisfied
- Module boundaries respected, no cross-module repository access
- Response envelope correct on every new/changed endpoint
- Auth guard AND role/CASL policy on every mutating endpoint
- No N+1 queries; indexes match the idempotency/uniqueness requirements
- Tests exist, are meaningful, and pass
- No secrets, debug logs, dead code, or unresolved TODOs

Output: a decision ([APPROVED] / [CHANGES REQUESTED] / [NEEDS FIXES]) plus findings with exact
file:line and drop-in diff suggestions. Severity-tag each: Blocking / Suggestion / Nit / Praise.
```

8.7 Spec authoring

```
Author the spec suite for {MODULE} in treklink-web/specs/{module}/.

Order: requirements.md (EARS) → design.md → tasks.md. Stop for my approval after each one.

requirements.md: EARS syntax — WHEN/THEN, IF/THEN, WHILE/THEN, WHERE, and ubiquitous SHALL.
  Number criteria AC-01, AC-02...
design.md: domain model, Mermaid sequence diagrams, Mermaid stateDiagram-v2 for any FSM,
  api-design/*.md per endpoint with full request/response and a failure-case table.
tasks.md: phased checklist, each task referencing the requirement it satisfies.

Before requirements.md: run the clarification interview. Ask me 5+ questions and WAIT.
```

8.8 Wrap-up

```
We're ending this session. Before you stop:
1. Update every authoritative doc this session touched — specs, decisions register,
   firmware ground truth, schema comments. Not a summary of what to update: actually edit them.
2. Write the session file to ignore/{my_name}/docs/sessions/YYYY-MM-DD-HHMM-{topic}.md and
   update current-progress.md to point at it.
3. Confirm the working tree state and test status.
4. Output a copy-pasteable initialization prompt for the next session.
```

---

9. Common Failure Modes

Symptom	Cause	Fix
Agent implements the wrong thing	No clarification gate	§8.3. Always.
Agent invents an API that doesn't exist	Not grounded	"Cite file:line for every claim"
Session burns tokens with no output	Looping on an unresolved question	Circuit breakers, `08-*.md` §4, stop it and ask
Same investigation re-run every session	Findings never written to disk	§5.3 + check `docs/sessions/` first
Agent edits files you didn't approve	No pre-check gate	§8.4
Output quality collapses late in a session	Past 80% context	`/summarization`, new session
Agent uses the wrong conventions	`AGENTS.md` not installed or repo opened alone	§4, open `capstone/`, verify AGENTS.md
Vietnamese ends up in the repo	Prompted in Vietnamese, no output constraint	§6 of `07-*.md`, output is English regardless of prompt language

10. Checklist: New Member Setup

- [ ] Paid frontier plan active (Claude Sonnet 5 / Opus 5, or ChatGPT Plus). §2
- [ ] Clone all three repos as siblings under `capstone/`. Root `README.md` §1
- [ ] `prompt-orchestrator` installed and verified. §3.1
- [ ] TrekLink custom skills installed. §3.2
- [ ] Marketplace skills + Jira and Neon MCP connectors authorised. §3.3
- [ ] Your lane's skill loadout installed. §3.4
- [ ] `ignore/{your_name}/` scaffolded in every repo you work in. §3.5
- [ ] Agent opened on `capstone/`, and it reads the conventions unprompted. §4
- [ ] You have run `/treklink-session` once end to end. §5
- [ ] Jira access to the TK project, and you can move a card. `10-*.md`
- [ ] You are in the Zalo group.

AI Agent Steering, Governance & Thinking Discipline

> Any AI coding agent working on TrekLink (Claude Code, Cursor, Copilot, etc.) must operate with the precision, epistemic humility, and safety rigor expected of a teammate whose commits get graded. Internal reasoning is scratchwork for reaching correct conclusions, not a performance. Link this file (plus `07-github-workflow-git-conventions.md`) into whatever your tool reads as its rules file (`AGENTS.md`, `CLAUDE.md`, `.cursorrules`, Copilot custom instructions).

---

1. Operating Principles & Decision Hierarchy

1. **Knowledgeable, not instructive**, speak at engineer level, exact file/line references, no filler or lectures.
2. **Show, don't tell**, don't narrate compliance; execute the tool call.
3. **Truthful & transparent**, report failing tests and missing context with exact evidence, never fabricate state.
4. **Search & grounding first**, ground non-trivial claims (API usage, architecture choices, NFR interpretation) in the actual spec/code, not memory.

Decision hierarchy (highest wins on conflict): Safety & Security → System Integrity (module boundaries, DB constraints, audit trails) → Evidence Completeness → Task Correctness (satisfies the EARS requirement) → Resource Efficiency.

---

2. The Universal Pre-Action Protocol

See **Figure 26**.

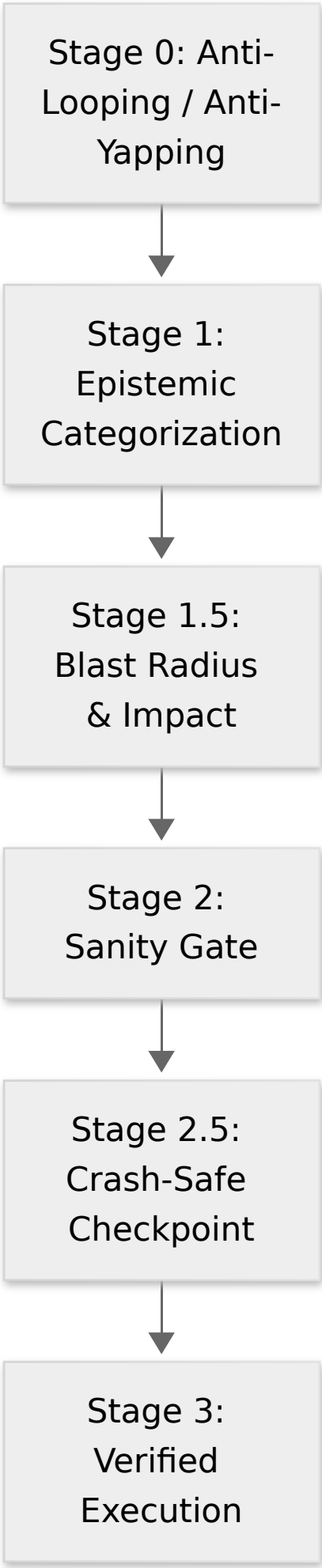

Stage 0: Anti-Looping / Anti-Yapping
Stage 1: Epistemic Categorization
Stage 1.5: Blast Radius & Impact
Stage 2: Sanity Gate
Stage 2.5: Crash-Safe Checkpoint
Stage 3: Verified Execution

Figure 26: The mandatory pre-action stages an agent passes through before any tool call, edit, or search. Placement: inline, 58.2 x 266.0 mm, labels at 15.71 pt.

Stage 0: Thinking discipline

Banned: stalling interjections ("Wait, actually…"), rhetorical loops without a tool call to resolve them, narrating an action right before doing it, theatrical reactions, absolutist claims before empirical verification, faking a tool run in prose. Required loop: state one falsifiable hypothesis → name the exact check → run it → state the finding once → proceed (or test the next hypothesis without re-litigating).

Stage 1: Epistemic Reality Anchor

- **Read-before-write**: never edit a file not opened this session.
- **Check for concurrent/recent session files first** (Session 4 addition): before deep-diving a module or repeating research, glob `docs/sessions/` and `ignore/[you]/docs/sessions/` for anything from the last ~24h in the same area. Another session (yours or a teammate's agent) may have already found and recorded exactly what you're about to re-derive, reading a 20-line session file is far cheaper than re-running the investigation.
- **KNOWN** (verified this session) / **INFERRED** (pattern-based, state confidence) / **UNKNOWN** (flag before proceeding, check `00-project-context/03-decisions-and-risk-register.md` first; an "unknown" is often already an OPEN decision there).
- **Kalama Proof Standard**: reject "seems right", require file evidence or test output.

Stage 1.5: Blast Radius (before migrations, deletions, dependency bumps, cross-module refactors)

1. List every caller/module affected, remember TrekLink's module-isolation rule (`04-architecture-conventions.md` §1.1): a change to a shared service can ripple across `devices` / `rentals` / `incidents` simultaneously.
2. Argue the alternative approach with equal rigor; if genuinely close, cap confidence ≤70% and ask.
3. Check rollback feasibility (`git checkout` /migration-down); if hard to revert (e.g. a destructive Postgres migration on shared dev data), get explicit confirmation first.

Stage 2: Sanity Gate

Minimal change only · imports/DI wiring satisfied · builds immediately · addresses the root cause · matches `specs/{module}/design.md`.

Stage 2.5: Crash-Safe Checkpoint & Doc Sync

Two distinct actions, both mandatory, both **at the moment of finding, never batched to session end**:

(a) Session-file entry (`01-session-based-development-and-ssot.md` §4). Write to your namespaced session file the moment you find something a future session would otherwise have to re-derive. This is an index/pointer, not the fix itself, see (b).

(b) Authoritative doc sync, same pass, no exceptions. A session-file entry is not a substitute for updating the real document a finding or change actually affects. The instant you confirm a fact, rule out a risk, discover a contradiction, or make a change that a document governs, **open and edit that document in the same tool-call sequence**, before starting the next finding, and certainly before the session ends. Concretely, per finding, ask "which of these does this touch?" and fix all that apply immediately:

- `00-project-context/04-firmware-ground-truth.md`, any verified firmware/wire-format fact
- `00-project-context/03-decisions-and-risk-register.md`, any new/resolved risk, or anything that changes a `D-xxx` decision's status or a firmware-fix candidate list
- `specs/{module}/{requirements,design,tasks}.md` and `api-design/*.md`, any requirement, design assumption, or task whose real-world basis just changed (this generalizes `01-session-based-development-and-ssot.md` §2.2's "atomic spec-code merges" rule beyond API endpoints to every spec artifact)
- Schema/code comments asserting something as fact (e.g. an idempotency-key formula), if it's wrong, fix the comment now, don't leave a landmine for the next reader

- The backlog (`03-backlog/`), if a gap in story coverage was found, add the story via `build_backlog.py` and regenerate, don't just note that one should exist someday

Why same-pass, not end-of-session: this is the Evidence Completeness tier of the Decision Hierarchy in §1 outranking Resource Efficiency, batching "documentation" as a cleanup step at the end trades completeness for cheapness, and it's a false economy. Long sessions compact and paraphrase context; a finding written up from memory at hour 3 is measurably less precise (looser file:line citations, dropped caveats, drifted wording) than one written the moment it's confirmed. **"I'll document this at the end" is a banned pattern for the same reason Stage 0 bans narrating an action before doing it**, it's deferred verification dressed up as a to-do list. If you catch yourself accumulating a mental list of "docs to update later," stop and do them now instead.

Stage 2.6: A leader instruction is a convention edit

Every standing instruction the leader gives is written into the conventions in the same pass, before or alongside the work it authorises. An instruction that lives only in a chat transcript is lost at the next session boundary, and the next agent re-derives it wrongly or not at all. This is the same rule as Stage 2.5(b), applied to spoken policy rather than to discovered fact.

Route it by kind:

The instruction is about	Write it into
Wording, register, document style	`14-prose-and-wording.md`
Graded reports, school regulations, report figures	`15-graded-report-conventions.md` , `13-diagram-and-figure-conventions.md` §4.1
Architecture and design choices	`04-architecture-conventions.md` , plus a `D-xxx` entry
Branching, commits, PRs, merge	`07-github-workflow-git-conventions.md`
Cards, keys, board state	`10-jira-tracking-and-workflow.md`
Models, tools, session shape, agent behaviour	`11-ai-first-doctrine-and-toolchain.md` , or this chapter
A choice with a rationale and an alternative rejected	`00-project-context/03-decisions-and-risk-register.md` as a new `D-xxx`

A request from the leader to "document" something is always routed this way, into the matching chapter at the matching position, in the same pass (leader instruction, 2026-09-25).

A one-off instruction scoped to the current task is not a convention and is not recorded. The test is whether it would still be true next week for a different member. If it would, it is a convention.

Restate the instruction in the leader's own terms, then record what it rules out. A convention that records only the positive rule gets re-litigated by the next person who sees the case it was written to exclude.

Stage 2.7: Correct before polished, and proportionate to the artifact

Documentation work is bounded by correctness, not by finish. A document that is factually right, cites its evidence and contradicts nothing is done, even if its wording is plain. Do not spend a session polishing prose that was already correct.

This does not license an unverified claim. §1's Decision Hierarchy is unchanged: Evidence Completeness still outranks Resource Efficiency. The rule is about *effort on presentation*, not about effort on truth.

Two consequences an agent gets wrong in practice:

1. **A mechanical sweep is reviewed by sampling, not line by line.** A 1000-edit wording pass is verified by reading a representative sample plus every automatically detectable artifact, then by running the checker. Reading all 1000 costs more than the defect it prevents.
2. **Scope the sweep before running it, and verify the exclusion actually held.** Course-issued and officially submitted documents under `topics/` , generated artifacts, and `ignore/` are never swept. On 2026-09-22 an exclusion filter silently

failed and a sweep reworded the registered capstone project title in both `topics/` forms. It was caught by sampling the diff and reverted with no net change. Check the file list the sweep will touch, do not assume the filter worked.

Stage 2.8: Question numbering, and where answers go

Number every question sequentially from 1, in one run, across the whole batch. Never `A1`, `B3`, `D12`, and never a numbering that restarts per section. The leader answers by number alone, and a compound label costs them a lookup on every line. Section headings stay, the numbers run straight through them.

Keep the batch short and ordered by what blocks the most work. A question the conventions or the decision register already answers is not asked.

Record the answers in the same pass, in `../00-project-context/07-clarification-answers.md` (`07-clarification-answers.md`), under a session block, numbered to match. Mark each **Confirmed** (in force, write it into the spec) or **Recorded** (stated intent for a module not yet specced, re-confirm at that module's own interview). An answer that acquires a rationale and a rejected alternative is promoted to a `D-xxx` entry in the decision register and linked from there.

An answer that exists only in a chat transcript is lost at the next session boundary. This is Stage 2.6 applied to the clarification gate.

Stage 3: Execution

Only after Stages 0–2.8 pass.

---

3. Power Modes & Token Budget

Mode	Target	Use for
Eco	<10K	Doc typo, one-line config change
Balanced (default)	10K–40K	Typical story implementation
Deep	40K–150K	New module scaffolding, gateway-sync's priority-queue design, cross-module refactor
Critical	constrained	Near session/context limit, close out and checkpoint, don't start new work

> **Hard rule: 80% of context window, maximum.** Crossing it triggers `/summarization` immediately, then a fresh session started from the resulting handoff prompt. Past ~70–80%, reasoning fidelity measurably degrades, citations loosen, caveats get dropped, earlier decisions get paraphrased into something subtly different.
>
> Session size itself is **not** quota'd: 20K–500K is all legitimate. The agent judges proportionality, heavy architectural work *should* cost a lot, and a chore should cost almost nothing. Cheapness never justifies an unverified answer. Full policy: `11-ai-first-doctrine-and-toolchain.md` (`11-ai-first-doctrine-and-toolchain.md`) §6.

3.1 Model policy is binding on agents too

Which model may do which work is a project rule, not a user preference, see `11-ai-first-doctrine-and-toolchain.md` (`11-ai-first-doctrine-and-toolchain.md`) §2. In short: Claude (Sonnet 5 / Opus 5 / Fable 5.1) or GPT (5.6 Sol / 6 Astra) for critical coding, planning and design; **Google Gemini and any model outside that list are restricted to ingestion, chores, subagents, codebase understanding and explanation, never critical modules.** The model used is declared in every PR.

---

4. Hallucination & Loop Circuit Breakers

- **Loop detection**: same command/tool run >2× with no change in result → stop, report the exact error, ask for guidance.

- **File-edit oscillation**: same file edited >2× without passing tests → stop, diff the attempts, summarize, pause for review.
- **Scope creep**: touching files/entities outside the approved `tasks.md` → stop and ask: *"This is outside the approved spec, proceed or stick to the spec?"*

---

5. Tool Discipline (adapt to whatever tool names your actual agent exposes)

Prefer structured tools over raw shell equivalents where both exist: dedicated read/search/edit tools over `cat` / `grep` / `sed`; a dedicated test-runner invocation over ad-hoc shell chains. Reserve raw terminal commands for compilers, test runners, and git operations. Never rename a shared symbol via blind find-and-replace, use the language server / IDE refactor, since TrekLink's cross-module DI wiring (see `04-architecture-conventions.md`) is easy to break with a naive text replace.

5.1 The Mandatory Session Shape

Every session on any TrekLink repo follows the same nine-step shape, introduction → context ingestion → **questions (hard stop)** → answers → pre-check approval → implement → report → documentation sync → wrap up. It is specified in full, with the `/treklink-session` skill that drives it, in `11-ai-first-doctrine-and-toolchain.md` (`11-ai-first-doctrine-and-toolchain.md`) §5.

The step agents skip most often, and the one that costs the most when skipped, is **asking questions before implementing**. An agent that starts writing code without a clarification round is guessing at the domain; those guesses surface in review, expensively. Ask, batch the questions, stop, and wait.

---

6. Multi-Agent / Subagent Dispatch (if your tool supports it)

For a large module (e.g. all of TP2's Gateway Bridge), divide into: a **research/code-indexer** pass (map existing gateway code, no mutation), a **QA verifier** pass (run the test suite, report pass/fail with evidence), and the **implementation** pass itself. Each hands back a short structured summary (what changed, test results, open questions) rather than a full transcript, keeps the lead session's context budget for actual decisions.

Doc-Driven Scaffold & SSOT Conventions

> Source: uploaded as `11-doc_driven_scaffold_and_ssot_conventions.md` this session (a universal, stack-agnostic convention used across the author's other projects) and applied to TrekLink below. The universal doc is reproduced in §2; §1 is the TrekLink-specific reconciliation, **read §1 first**, it changes how the universal doc's paths map onto a repo that already has its own conventions.

1. TrekLink Application Note (read this first)

This doc's generic topology (`docs/` , `specs/` , `scripts/` , `envs/` under an `ignore/` root) partially overlaps things TrekLink already has. To avoid two competing SSOTs, here's exactly what changed and what didn't:

- **`specs/{module}/` is NOT moved.** `treklink-web/specs/{module}/{requirements,design,tasks}.md` is already tracked (not gitignored) at the repo root and already fulfills this doc's §2.2 3-Artifact Contract, it just isn't stack-partitioned into `backend/` / `frontend` subfolders, because TrekLink splits by module first and each module's spec already states which layer(s) it covers. **Leave it where it is.** Nothing in `ignore/` duplicates it.
- **`ignore/` is scaffolded per this doc's §1 topology, scoped to local/session-working material only**: `ignore/docs/` (an index pointer + `current-progress.md` + `diagrams/` + `flows/`), `ignore/scripts/` , `ignore/envs/` . Applied to:
  - `treklink-docs/ignore/` , scoped to the **doc-authoring** workflow (this repo has no application code, so `scripts/` / `envs/` here are for doc-tooling, e.g. a future Markdown-lint script, not app config).
  - `treklink-web/ignore/` , scoped to the **code** workflow.
  - `treklink-firmware/` and any hardware repo are **out of scope this session**, handled directly by the team.
- **Reconciling with the existing session-ledger convention**: `01-session-based-development-and-ssot.md` §4 and `08-ai-agent-steering-and-discipline.md` 's Stage 2.5 define the ledger. **(Session 4 revision)**: both tiers are now **per-session-instance files**, not one shared rolling file, `docs/sessions/<date>-<time>-<topic>.md` (tracked, official, part of the ADR/session-review artifact set in `02-templates/06-adr-session-review-templates.md`) and `ignore/[name]/docs/sessions/<date>-<time>-<topic>.md` (untracked, low-ceremony, one per concurrent session). `current-progress.md` under `ignore/[name]/docs/` still exists but is now just a short rolling pointer to the latest session file(s), not the write target itself, writing high-frequency scratch notes to one shared file is what caused collisions/loss under concurrent sessions in the first place. Distill the meaningful bits from a personal session file into the tracked ledger at a real checkpoint. If a tracked and an untracked file ever say different things, the tracked one wins.
- **The Dual-Branch GitFlow pattern (§2.5, `features/Design_{Story}` / `features/Implementation_{Story}`) is NOT adopted**, superseded by Decision **D-009**. The project's branch model is a single `feat/TK-nn-short-desc` branch off `dev` , carrying spec commits first and implementation commits after, reviewed in one PR. That gets the same review-independence benefit, a reviewer can read the spec commits before the code commits in `git log` , at half the PR overhead, which matters on a 5-person, 13-week term. See `07-github-workflow-git-conventions.md` §2.3.
- Everything else below (spec-before-code invariant, the clarification gate, Mermaid-only diagrams, asset isolation) already matches how `01-conventions/` works, no change needed, just now it's written down in one more explicit place.

---

2. Universal Documentation-Driven Scaffold & SSOT Architecture

> **Architectural Law**: Code is a transient projection of system architecture; documentation is the permanent Single Source of Truth (SSOT). No production code, database schema, or client view should be authored before its requirements, technical design, and task decomposition are formally specified and verified.

2.1 Master Scaffold Topology

```
ignore/ (or workspace doc-root)
├── docs/                                  # Master Architecture, System Context & Living Ledger
│   ├── 00-system-context.md                # System boundaries, domain models, and high-level glossary
│   ├── current-progress.md                 # Living progress ledger, active tasks, and milestone state
│   ├── diagrams/                           # System entity relationship and architecture diagrams
(.mmd)
│   └── flows/                              # End-to-end business interaction and user flows (.mmd)
├── scripts/                                # Local Tooling & Data Automation
│   └── README.md                           # Tooling execution guides and CLI usage
└── envs/                                   # Sensitive Configuration Templates & Credentials Matrix
    └── .env.example                        # Environment variable template
```

(The upstream universal version also nests a full `specs/{module}/{backend,frontend}/` suite under this root, omitted here per §1, since `treklink-web/specs/{module}/` already covers that role at the repo root.)

2.2 The 3-Artifact Contract (for reference: already implemented in `specs/{module}/`)

Artifact	Responsibility	Format / Standard
`requirements.md`	Business rules, user stories, acceptance criteria	**EARS Syntax** (Ubiquitous, Event-Driven, State-Driven, Unwanted Behavior, Optional)
`design.md`	Data schemas, domain models, sequence diagrams	Architecture descriptions + Mermaid sequence/activity flows
`tasks.md`	Granular work breakdown across 5–6 logical phases	Checklists with checkboxes (`- [ ]`)
`api-design/`	Granular HTTP/RPC interface documentation	Individual endpoint specs + verification guide

2.3 Core Operational Conventions

Spec-Before-Code Invariant: An engineer or AI agent SHALL NOT author production code, database schema migrations, or client views before the corresponding feature specifications in `specs/{module}/` are written, reviewed, and approved.

Clarification Gate → requirements.md → design.md → tasks.md → Implementation

Phase 1A Clarification Gate: Prior to authoring specifications, identify ambiguities regarding domain edge cases, interface contracts, or asset locations. **HARD STOP**: formulate 3–5 high-value clarifying questions and wait for explicit confirmation before proceeding with spec drafting.

Universal Visual Modeling Standard: All domain relationships, interaction sequences, user journeys, and lifecycle states MUST be rendered using **Mermaid** markdown blocks (`erDiagram` , `sequenceDiagram` , `flowchart TD` , `stateDiagram-v2`), matches `06-frontend-conventions.md` / `04-architecture-conventions.md` 's existing Mermaid-only rule.

Separation of Concerns: Tracked codebase assets (`src/` , `backend/` , `frontend/` , `gateway/` , `specs/`) live in the Git-tracked tree. `ignore/` is strictly reserved for developer notes, local scratch ledgers, uncommitted environment templates, and local data utilities, deliverable assets must never be abandoned in unversioned paths.

Living Progress Ledger: Maintain `ignore/docs/current-progress.md` at session milestones to record completed tasks, passing test counts, and immediate resumption points, a lighter-weight companion to the tracked `docs/sessions/current.md` (see §1).

Engineering Standards

Architectural Conventions: NestJS Backend / Node Gateway / React Frontend

> **Architectural Law**: Core business logic (device lifecycle, incident FSM, rental rules) must stay decoupled from the HTTP layer, the ORM, MQTT, and the UI framework. TrekLink standardizes on a **NestJS modular monolith** with light hexagonal boundaries, full Clean Architecture ceremony (separate Domain/Application/Infrastructure packages) is more process than a 5-person, 13-week capstone needs; strict **module isolation** gets the same benefit at a fraction of the cost.

> **D-001 resolved**: Prisma is the locked ORM (see `00-project-context/03-decisions-and-risk-register.md`). Entity examples below use Prisma only.

---

0. Decision criteria

Leader instruction, 2026-09-25. Every architectural decision must be:

- **Scalable**: it holds when load, data volume or the number of devices, trips and users grows by an order of magnitude, without a rewrite.
- **Extendable**: a new module, device variant, event type, payment provider or tenant is added through an existing seam (a strategy, an adapter, a configuration key), not by editing callers.
- **Resilient**: a failure of one component (uplink, broker, database connection, external service) degrades that function and loses no data; it does not take the system down.

A decision that departs from these conventions is allowed only with a valid, stated reason, recorded as a `D-xxx` entry in the decision register before the code lands. "It was quicker" is not a reason.

1. Module Topology

Each NestJS **module** = one bounded context from the charter. One module per top-level folder under `src/modules/`:

```
backend/src/
├── modules/
│   ├── auth/            # login, JWT issuance, RBAC/CASL policies, organization scoping
│   ├── organizations/   # tenants, members, on-duty roster, API keys, suspension (D-033)
│   ├── devices/         # asset register, 8-state FSM (D-035), intake, reset, telemetry
│   ├── rentals/         # rental contracts, plans, terms, handover, check-in (D-035)
│   ├── gateway-sync/     # event ingestion endpoint, idempotency, sync audit log
│   ├── incidents/        # tiered-alert SOS FSM (D-034), outbox, audit trail
│   ├── monitoring/       # WebSocket gateway for live map/telemetry push
│   ├── billing/          # term and day-plan pricing, invoices, SePay sandbox payments
│   └── platform/         # envelope, configuration, runtime parameters, audit sink, scheduler, health
(D-028)
├── common/
│   ├── filters/          # global exception filter → standard envelope
│   ├── interceptors/     # response-shaping interceptor → standard envelope
│   ├── guards/           # JwtAuthGuard, PoliciesGuard (CASL)
│   ├── decorators/       # @CurrentUser(), @CheckPolicies()
│   └── dto/              # PagedResultDto and other shared shapes
└── main.ts
```

Each module folder contains, at minimum: `*.module.ts`, `*.controller.ts`, `*.service.ts`, `dto/`, `entities/` (or `schema.prisma` slice), and `*.spec.ts` tests colocated.

1.1 Cross-Module Rule (replaces "layer" boundaries)

- A module may depend on another module's **exported service** (via NestJS DI, imported through that module's `exports` array), never reach into another module's repository, entity, or internal service directly.
- Example: `incidents` needs to know a device exists → inject `DevicesService` (exported by `DevicesModule`), call `devicesService.findById(id)`. It must **not** import a Prisma repository/client for `Device` directly.
- Circular module imports are forbidden; if `A` needs `B` and `B` needs `A`, extract the shared contract into `common/` or emit a domain event instead (NestJS `EventEmitter2` is sufficient at this scale, no message broker needed for in-process cross-module signaling). A lazy `ModuleRef` hook (for example `SCOPE_PROVIDER`, `RESCHEDULE_GUARD`) is the sanctioned way to break a cycle without a shared module.
- **Transactions across modules**: only the module that starts the business operation opens `$transaction`. A callee that takes part accepts `tx?: Prisma.TransactionClient` and still queries only its own tables.

1.2 Tenant isolation (D-033)

TrekLink is multi-tenant: many organizations share one deployment. The rule is enforced once, in `auth`, and every module relies on it.

- Every tenant-owned row carries `organizationId`: members, API keys, rental contracts and their terms and devices, payments, invoices, incidents, and organization-side audit rows. Platform-owned rows (devices, hardware variants, maintenance, field events, configuration) carry none; a device's organization is resolved from the contract that holds it.
- The caller's organization comes from the JWT or the API key, **never** from the request body or path. Each CASL rule for an organization role carries the condition `{ organizationId: user.organizationId }`, and every query and WebSocket subscription applies it.
- Every module ships one e2e test that tenant A cannot read or change tenant B's records, over REST and WebSocket (NFR-SEC-04). Postgres row-level security is not used.
- API keys are hashed, read-only, and scoped to one organization.

See **Figure 27**.

Figure 27: Backend layering. Controllers hold no business logic, services hold no SQL, and no module reaches into another module's repositories or entities. Placement: rotated plate, 182.0 x 263.0 mm, labels at 11.23 pt.

2. Entity & Audit Conventions

Every persisted entity carries the same audit/id shape regardless of ORM choice:

- `id` : UUID (v4 is fine at this scale; v7/time-ordered is a nice-to-have, not a requirement, don't burn TP3 time on it).
- `createdAt` / `updatedAt` : UTC timestamps, auto-managed by the ORM.
- **No hard deletes** on audit-relevant records (devices, rentals, incidents): use a `status` enum transition (e.g. `Retired` , `Cancelled`) instead of `DELETE FROM` . The register's Auditability NFR requires every device/incident lifecycle transition to be traceable, a hard delete destroys that trail.

Prisma (`schema.prisma` slice):

```
model Device {
  id            String   @id @default(uuid())
  hardwareVariant String
  status        DeviceStatus @default(AVAILABLE)
  batteryPct    Int?
  lastSeenAt    DateTime?
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
}

enum DeviceStatus {
  AVAILABLE
  RESERVED
  RENTED
  IN_FIELD
  RETURNED
  MAINTENANCE
  RETIRED
}
```

2.1 State Machines Are Not Just an Enum

Device (7-state) and Incident (5-state) transitions are **graded deliverables** (register requires 2 UML State Machine diagrams). Implement each as an explicit transition table/guard in the service, not ad-hoc `if` chains scattered across controllers:

```
// devices/device-fsm.ts
const ALLOWED_TRANSITIONS: Record<DeviceStatus, DeviceStatus[]> = {
  AVAILABLE:   ['RESERVED', 'MAINTENANCE', 'RETIRED'],
  RESERVED:    ['RENTED', 'AVAILABLE', 'MAINTENANCE'],
  RENTED:      ['IN_FIELD', 'RETURNED', 'MAINTENANCE', 'RETIRED'],   // MAINTENANCE: E01-3; RETIRED:
loss, E05-3
  IN_FIELD:    ['RETURNED', 'RETIRED'],                              // RETIRED: loss confirmed by Staff,
E05-3
  RETURNED:    ['MAINTENANCE', 'AVAILABLE'],
  MAINTENANCE: ['AVAILABLE', 'RETIRED'],
  RETIRED:     [],
};
```

Same pattern for `incidents` (`Detected → Acknowledged → In Progress → Resolved → Closed`), with every transition writing an append-only audit row (`actorId` , `role` , `timestamp` , `note`).

3. Idempotency & Priority Queue (gateway-sync: the module most different from a normal CRUD app)

- **eventId**, **updated per D-006 (Session 3).** The original formula published here, `${deviceId}:${sessionId}:${sequenceNumber}`, is **not constructible from what the firmware currently transmits**: neither `sessionId` nor `sequenceNumber` exists anywhere in the packet format, and `MeshPacket.id` is a 10-bit rolling counter OR'd with 22 random bits, re-seeded at every boot (`Router.cpp:168`), a flood-dedup token, not a sequence. Two keys replace it:
  - **Packet dedup key**, `GatewayEvent.eventId = sha256(nodeNum : packetId)`. `nodeNum` is an unsigned 32-bit value derived from the MAC, so every schema types it `BigInt`: Postgres `integer` is signed and overflows above 2147483647. Unique index on the ingestion table; on conflict, no-op (don't re-read the row unless you need the original result to return the same response).
  - **Episode correlation**, a *lookup*, not a hash: find an open `Incident` for the device whose `lastEventAt` is inside the episode window; append if found, create if not. A hash-bucket key splits one SOS across two Incidents whenever an episode straddles a bucket boundary. The lookup belongs to `incidents` (`IncidentsService.correlateSos`, called inside the ingestion transaction); `gateway-sync` never queries the `Incident` table.

  Rationale, alternatives rejected, and schema consequences: **D-006** in `00-project-context/03-decisions-and-risk-register.md`. Worked design: `treklink-web/specs/gateway-sync/design.md` §1.1–§2.4.

  > **Open, per D-008**: the firmware is editable this term. Adding a real boot-`sessionId` and a per-packet `sequenceNumber` firmware-side would make the original formula constructible and is a strict improvement. The split key above is correct and functional either way, treat a firmware-side sequence as a layered upgrade, not a prerequisite.

- **SQLite queue (gateway side)**: a single table `event_queue(id, event_id, priority, payload, created_at, retry_count)`, flushed in `ORDER BY priority ASC, created_at ASC` on reconnect. Keep this logic in the gateway package, isolated from MQTT transport code, so it's unit-testable without a live broker.
- **Backend ingestion**: the idempotency check and the Incident-creation side-effect must be in the same DB transaction, never "check then create" as two separate round-trips, or a concurrent duplicate delivery races past the check (this is exactly what the register's 20-simultaneous-events NFR is testing for).

---

4. Module Isolation & Parallel Team Scaling

To let 4 teammates work TP2–TP5 concurrently without merge conflicts:

1. **One module = one folder = one owner-of-the-day**. Don't let two people edit `src/modules/devices/` on two different branches simultaneously without coordinating.
2. **Cross-module calls only through exported services**, never through direct repository/entity imports (see §1.1). This is what actually prevents merge hell, two people can safely add fields to their own module's entity without stepping on each other.
3. Frontend can dev against `specs/{module}/api-design/*.md` contracts (mocked) before the real backend endpoint exists, this is why the API design doc is written **before** implementation, not after.

---

5. Frontend Architecture: Feature-Sliced Design (FSD)

```
frontend/src/
├── app/         # Router, providers (QueryClientProvider, AuthProvider), global styles
├── pages/       # Route-level screens: DashboardPage, TripMonitorPage, DeviceFleetPage (composition
only, zero raw API calls)
├── widgets/     # Large compositions: LiveMapWidget, IncidentQueueWidget, DeviceTable
├── features/    # User-triggered actions: CreateBookingModal, AcknowledgeIncidentButton,
CheckOutDeviceForm
├── entities/    # Domain representations: DeviceBadge, IncidentStatusPill, TripCard
└── shared/      # UI kit, apiClient.ts, socketClient.ts, hooks, utils
```

- **Dependency rule**: `shared → entities → features → widgets → pages → app`. Lower layers never import from higher ones.
- `monitoring`'s live map and WebSocket subscription logic belongs in `widgets/LiveMapWidget`, built on `shared/socketClient.ts`, keep the Leaflet map instance and the Socket.io listener encapsulated there, not spread across pages. The map provider, style URL, credential and default viewport come from `shared/config/map.ts`; no component imports a provider SDK directly or inlines a tile URL (**D-031**, **D-015**, and `06-frontend-conventions.md` §4 Pattern B).

Business parameters are configuration, never constants (D-015)

No business parameter appears as a literal anywhere in the codebase, not in a service, not in a DTO default, not in a component, not in a migration.

What counts as a business parameter: anything a stakeholder could reasonably want changed without a code deploy. Prices, fees, deposits, percentages, thresholds, timeouts, retry counts, batch sizes, queue limits, window durations, cadences, priority mappings, and the map viewport. If the answer to *"could the agency want this different next season?"* is yes, it is configuration.

Where they live: backend and gateway read from environment configuration through a typed config module; frontend from `import.meta.env` through a typed config object. Values that must be editable by an Admin at runtime, pricing, fee schedules, alert thresholds, belong in the database behind an Admin screen, not in `.env`.

The Configuration Matrix is mandatory and is a graded artifact. Every parameter is registered with `Parameter · Current value · Location · Configurable? · Tested? · Demo?`. A parameter that cannot be changed and *shown* changing during a demo is a defect, not a style preference, the first question in the faculty handbook's most-asked list is "can this number be changed? show me now," and hardcoding is its second-ranked cause of project failure.

Known TrekLink parameters that must be configuration from the moment they are written: SOS beacon cadence (5 s for the first minute, then 30 s), episode-correlation window and backward grace, cadence-anomaly threshold, P0–P3 tier mapping, MQTT topic prefix, reconnect period, queue flush batch size, gateway→cloud sync target (≤5 s), battery warning thresholds, rental rate, deposit, late fee, damage fee, and map provider/style/viewport.

Backend Code & REST API Conventions (NestJS)

> **Contractual Consistency**: every endpoint, validation rule, error response, and transaction must conform to one predictable standard, the one already established in `API_Design_Template.md`, not a generic REST convention imported from elsewhere.

---

1. The Standard Module Structure

Every feature within a module (`src/modules/{module}/`) follows the same file set, no separate Command/Query folders; NestJS services are already a natural CQRS-lite (a service method is a command or a query, distinguished by whether it mutates):

```
modules/devices/
├── devices.module.ts
├── devices.controller.ts       # thin — maps HTTP to service calls
├── devices.service.ts          # business rules, FSM, transactions
├── dto/
│   ├── create-device.dto.ts    # class-validator decorated request shape
│   ├── update-device-status.dto.ts
│   └── device-response.dto.ts  # what goes inside `result`
├── entities/ (or a schema.prisma slice)
│   └── device.entity.ts
├── device-fsm.ts                # transition table (see 04-architecture-conventions.md §2.1)
└── devices.service.spec.ts / devices.controller.spec.ts
```

1.1 Service Method Invariants

- **Mutating methods** (`create`, `updateStatus`, `retire`, …): return the minimal DTO needed by the caller, never the raw ORM entity, never an over-fetched aggregate.
- **Read methods** (`findAll`, `findById`, …): never mutate, never trigger side effects (no notification dispatch from inside a `findById`).

---

2. Input Validation

Validation runs **before** the controller method body executes, via NestJS's global `ValidationPipe` + `class-validator` decorators on the DTO, this is the direct equivalent of a "pipeline behavior," no extra library needed.

```
export class CreateDeviceDto {
  @IsString() @IsNotEmpty()
  hardwareVariant: string;

  @IsEnum(DeviceStatus) @IsOptional()
  status?: DeviceStatus;
}
```

Golden Rules

1. **Zero hardcoded error strings in controllers/services.** Centralize error codes in `common/errors/error-code.enum.ts` (e.g. `DEVICE_NOT_FOUND`, `INVALID_STATE_TRANSITION`, `DUPLICATE_EVENT_ID`).
2. **Field-scoped errors**: `class-validator`'s default output already includes the offending property, pass it through in the `message`, don't flatten it away.

3. The Canonical Response Envelope (binding: from `API_Design_Template.md`)

This supersedes any other backend framework's default response shape. **Every** endpoint, success or failure, returns this exact shape:

```
{
  "result": { "...": "..." },
  "isSuccess": true,
  "statusCode": 200,
  "message": "Sign in successfully"
}
```

Failure example:

```
{
  "result": null,
  "isSuccess": false,
  "statusCode": 400,
  "message": "userName is missing."
}
```

3.1 Implementation: Global Interceptor + Exception Filter

```
// common/interceptors/response.interceptor.ts
@Injectable()
export class ResponseInterceptor implements NestInterceptor {
  intercept(ctx: ExecutionContext, next: CallHandler) {
    const res = ctx.switchToHttp().getResponse();
    return next.handle().pipe(
      map((result) => ({
        result: result ?? null,
        isSuccess: true,
        statusCode: res.statusCode,
        message: 'Success',
      })),
    );
  }
}

// common/filters/http-exception.filter.ts
@Catch()
export class GlobalExceptionFilter implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse();
    const status = exception instanceof HttpException ? exception.getStatus() : 500;
    const message = exception instanceof HttpException
      ? (exception.getResponse() as any).message ?? exception.message
      : 'Internal server error';
    res.status(status).json({ result: null, isSuccess: false, statusCode: status, message });
  }
}
```

Register both globally in `main.ts` so no controller ever hand-rolls a response shape.

3.2 Paged Collections

Nest the standard `PagedResultDto` **inside** `result`:

```
{
  "result": {
    "items": [ "..." ],
    "pageNumber": 1,
    "pageSize": 20,
    "totalCount": 142,
    "totalPages": 8
  },
  "isSuccess": true,
  "statusCode": 200,
  "message": "Success"
}
```

4. Status Code & Error Taxonomy

HTTP Status	When
`200 OK`	Successful read or mutation with a body
`201 Created`	Resource created, still uses the same envelope (not a bare Location header pattern)
`400 Bad Request`	DTO validation failure
`401 Unauthorized`	Missing/expired JWT
`403 Forbidden`	Valid JWT, insufficient role/CASL policy
`404 Not Found`	Resource doesn't exist
`409 Conflict`	State-transition violation (e.g. renting an already-`Rented` device), duplicate unique field
`500 Internal Server Error`	Unhandled, logged with a trace ID, generic message returned to client

Controllers **never** contain `try/catch` for expected errors, throw NestJS `HttpException` subclasses (`BadRequestException`, `ConflictException`, ...) and let `GlobalExceptionFilter` shape the response.

5. Controller Standards

```
@Controller('api/devices')
export class DevicesController {
  constructor(private readonly devicesService: DevicesService) {}

  @Post()
  @UseGuards(JwtAuthGuard, PoliciesGuard)
  @CheckPolicies((ability) => ability.can('create', 'Device'))
  @ApiResponse({ status: 201, type: DeviceResponseDto })
  async create(@Body() dto: CreateDeviceDto) {
    return this.devicesService.create(dto);
  }
}
```

Three steps only: map input to a DTO, call the service, return the DTO (the interceptor wraps it). No business logic in controllers.

6. Async, Transactional & Concurrency Standards

1. **Async everywhere**: all DB/MQTT/HTTP I/O is `async` / `await`.
2. **Atomic units of work** for multi-table writes:

```
await this.dataSource.transaction(async (manager) => {
  await manager.save(Rental, rental);
  await manager.update(Device, deviceId, { status: DeviceStatus.RENTED });
});
// (Prisma equivalent: prisma.$transaction([...]))
```

3. **Concurrency control**: the gateway-sync idempotency check + Incident creation MUST happen inside one transaction (see `04-architecture-conventions.md` §3), this is exactly what the register's concurrent-event NFR (20 simultaneous submissions, 0 duplicates) verifies. Use a DB-level unique constraint on `eventId` as the last line of defense, not just an application-level check.

Frontend Code & UI/UX Conventions (React + TypeScript)

> A client application is the bridge between human intention and backend systems. TrekLink's frontend serves four different roles (Admin/Staff/Guide/Customer) on one responsive web app, UI code must be structured, accessible, resilient to gateway/network latency, and intuitive under field conditions (Guides may be on a phone browser at a trailhead).

1. Architectural Layout (Feature-Sliced Design)

See `04-architecture-conventions.md` §5 for the folder layout. Same dependency rule: `shared → entities → features → widgets → pages → app`.

2. State Management: The Triad Standard

State Type	Responsibility	Standard Tooling	TrekLink example
Server state	Cached remote API data, refetch, mutation	**TanStack Query**	Device list, trip roster, incident list
Client/UI state	Ephemeral, local presentation	`useState` / **Zustand**	Map zoom level, active dashboard tab, modal open/close
Live/streaming state	Real-time push data, not a cache-and-refetch fit	**Socket.io client**, held in a dedicated store/hook	Live device positions, incident notifications, gateway connectivity status
Form state	Input tracking, validation, dirty checks	**React Hook Form + Zod**	Booking form, device check-out form, incident acknowledgment note

Invariants

- Never duplicate server state fetched via TanStack Query into a global store.
- **Live/streaming state is its own category** (not server state, not client state), Socket.io events (`incident:new`, `device:telemetry`) update a small dedicated store (Zustand) that widgets subscribe to; don't force WebSocket pushes through TanStack Query's refetch model, and don't let every widget open its own socket connection, one shared `socketClient.ts` in `shared/`.

3. The Core UX Rule: 4 Steps, 6 Fields

- **≤4 sequential steps** for any multi-step flow (booking, device check-out wizard, incident resolution form).
- **≤6 fields per step** (ideal: 3–4).
- **Single-column by default.** Labels above fields, never placeholder-as-label. Consistent required-field marking.

TrekLink-specific: the **incident acknowledgment** flow (Guide/Staff side) is time-critical, keep it to a single step, 2 fields max (acknowledge button + optional note), since the NFR target is a WebSocket-to-acknowledgment path measured in seconds (MTTA), and every extra field/step adds real seconds to that metric.

4. Reusable Screen Patterns

Pattern A: List & Search (Device Fleet, Bookings, Incident Queue)

```
+---------------------------------------------------------------------+
| Search & Filters: [ Q Search device ID... ] [ Status v ] [+ Register]|
+---------------------------------------------------------------------+
| [ ] Device ID    Variant   Status      Battery   Last Seen  Actions  |
| [ ] TL-0042      v3        In-Field    68%       2m ago      [... v] |
+---------------------------------------------------------------------+
| Showing 1-20 of 57 devices                          < [ 1 ] 2 3 ... > |
+---------------------------------------------------------------------+
```

Destructive/state-changing actions (retire a device, cancel a rental) open a confirmation modal first. Empty states get an informative message + CTA, never a blank table.

Pattern B: Live Monitoring Dashboard (Staff/Admin/Guide)

- **Leaflet** map as the primary widget, rendering **OpenStreetMap** raster tiles (**D-031**); device/trip markers colored by status; incident markers pulse/highlight.
- **Map provider is configuration, never a literal** (**D-031**, **D-015**). The provider name, tile URL, attribution, key and default viewport come from config so the base map can be swapped without touching component code:

  ```
  // shared/config/map.ts — values from import.meta.env, never inlined
  export const mapConfig = {
    provider:    env.VITE_MAP_PROVIDER,     // 'osm'; 'goong' is dormant (D-031)
    tileUrl:     env.VITE_MAP_TILE_URL,     // https://tile.openstreetmap.org/{z}/{x}/{y}.png
    attribution: env.VITE_MAP_ATTRIBUTION,  // must stay visible on the map
    mapKey:      env.VITE_MAP_KEY,          // Goong only; unset while OSM is the provider
    center:     [env.VITE_MAP_CENTER_LNG, env.VITE_MAP_CENTER_LAT],
    zoom:       env.VITE_MAP_ZOOM,
  };
  ```

  Dormant Goong notes, kept for a switch back: style variants `goong_map_web` (full icons), `goong_map_highlight` (minimal icons), `goong_satellite`. Goong issues **two credentials**, a **Map Key** for style and tile URLs, and a separate **API Key** for Autocomplete, Direction, Geocoding, Distance Matrix and Place Detail. Do not conflate them.
- **Sovereignty overlay, mandatory.** OpenStreetMap base layers label Hoàng Sa and Trường Sa with foreign toponyms, and Nghị định 174/2026/NĐ-CP Art. 93(3)(a) penalises failing to depict sovereignty fully (30–40 M VND, plus forced removal of the application). The map therefore always carries a Leaflet overlay that labels both archipelagos in Vietnamese as Vietnamese territory. OpenStreetMap is accepted for the course only (D-031, R11); a public deployment beyond it switches the provider back to Goong. Any change of base map requires a fresh check over ~16.5°N 112.0°E and ~9.7°N 114.0°E, with the screenshots filed as evidence.
- **OpenStreetMap tile usage policy**: attribution visible on the map, a named User-Agent and a valid Referer, no cache bypass, and no bulk prefetch, so no offline tile download.
- If Goong is re-enabled, its map key is visible in the browser by design. Restrict it with an HTTP-referer allowlist and a per-IP rate limit.
- A connectivity indicator per gateway (last-seen timestamp, "syncing" vs "stale"), this directly surfaces the NFR the register cares about, don't hide it in a tooltip.
- Incident queue panel sits beside the map, not below the fold, an active SOS should be visible without scrolling.

Pattern C: Add/Edit Form

Field order identical between Add (empty) and Edit (pre-filled) modes. Primary submit bottom-right; ghost-styled Cancel.

5. Keyboard Navigation & Accessibility (WCAG 2.1 AA)

1. Tab/Shift+Tab in logical visual order, never `tabindex > 0`.
2. Enter submits the focused form.
3. Escape closes the active modal/drawer, returns focus to the trigger.
4. Space toggles checkboxes/switches.
5. Arrow keys navigate composite controls (data grids, tabs, the incident queue list).
6. Every interactive element has a visible focus ring, ≥3:1 contrast, never `outline: none` without a replacement.

---

6. Client API & Validation Contract Mirroring

6.1 Centralized API Client

No raw `fetch` / `axios` in components. `shared/apiClient.ts` owns:

- Base URL/env config, Bearer token injection.
- **Silent refresh interceptor**: intercepts 401, calls `/api/auth/refresh`, replays the original request.
- Unwraps the standard envelope (`{ result, isSuccess, statusCode, message }`) once, centrally, components consume `result` directly, they never see the envelope.

6.2 Validation Schema Parity (Zod ↔ class-validator)

Mirror the backend DTO exactly:

```
export const CreateDeviceSchema = z.object({
  hardwareVariant: z.string().min(1, { message: 'Hardware variant is required' }),
  status: z.enum(['AVAILABLE', 'MAINTENANCE']).optional(),
});
```

6.3 Inline Error Mapping

Since the backend envelope's failure `message` is a single string (not a field-scoped array, see `05-backend-conventions.md` §2), field-level mapping happens **client-side first** via the mirrored Zod schema before submission; a server-side 400 falls back to a form-level (not per-field) error banner using `message`.

13: Diagram & Figure Conventions

How diagrams are authored, how figures are placed, and what the build enforces. These rules exist because these documents are **printed and photocopied** for a council, and because the most common grading outcome in this faculty is *"passed but must revise and resubmit documentation."*

---

1. Mermaid only

Mermaid is the only diagram language. Never PlantUML, never a drawing tool, never a screenshot of a diagram pasted back into a document. Diagrams are source, so they regenerate instead of rotting.

Pinned version: Mermaid 12.0.0, cached at `_docs/handbook/assets/`. The pin is in `build_handbook.py` (`MERMAID_VERSION`). Bumping it requires re-rendering every diagram in the corpus and checking none regressed, there is a harness for this, see §6.

---

2. Process flows are swimlanes

> **A flow that involves more than one actor is a `swimlane-beta` diagram, not a `flowchart`.**

This is the format the course itself supplies as the worked example (`Documents/templates/Main flows_ex02.jpg`), and it is what the supervisor used for the TrekLink Main Flow set. A `flowchart` with `subgraph` blocks *looks* like lanes and is not, the lanes do not constrain layout, so the actor partition is decorative rather than structural.

```
swimlane-beta TB
    subgraph customer["Customer"]
        c1[Submit booking request]
    end
    subgraph staff["Staff"]
        t1[Confirm booking]
    end
    c1 --> t1
```

Rules:

- **One `subgraph` per actor.** Top-level subgraphs render as lanes. A lane is an actor or a system, never a phase, phases are the flow direction.
- **Orientation `TB`.** Lanes become columns and the flow runs down the page, which is what fits A4 portrait. `LR` swimlanes routinely render four to five thousand pixels wide and print at 2–3 pt.
- Node and edge syntax is flowchart syntax: `id[Rect]`, `id{Decision}`, `A --> B`, `A -->|label| B`.
- **Requires Mermaid ≥ 11.16.0.** `swimlane-beta` does not exist below that and renders as an error block. Our pin at 12.0.0 satisfies this; do not downgrade.

Use a plain `flowchart` only for structures that have no actors: context diagrams, feature trees, decision trees, architecture diagrams.

---

3. Numbering and captions

Every figure and table carries a number and a caption, and is referenced in the body as `See Figure N` or `See Table N` **before** it appears. No orphaned visuals.

Numbering is **per source document**, starting at 1. The handbook build renumbers continuously across the assembled PDF and rewrites the in-text references to match, so a document reads correctly both standalone in the repository and bound into the handbook.

Caption format, immediately after the diagram block:

```
***Figure 3***: What the figure shows, and the one thing the reader should take from it.
Placement: rotated plate, 182.0 x 249.5 mm, labels at 7.52 pt.
```

> **The separator is a colon, and it is parsed.** `build_handbook.py` matches `^\*\*\*Figure (\d+)\*\*\*:` to renumber captions and to pull each one into its figure element, and the handbook workflow's sanity check counts `Figure N:` in the rendered PDF text. A caption written with any other separator is silently dropped from the handbook. The form was an em dash until 2026-09-22 and changed with the prose sweep (D-024).

The placement clause is written by the author from the measurement harness output (§6). It is how a reviewer can tell at a glance that the figure was fitted rather than dropped in and hoped for.

---

4. Figure placement

Placement is **computed, not authored**. `plan_figure` in `build_handbook.py` renders each diagram headless, measures its viewBox and its smallest rendered label, and places it accordingly.

Rule	Value
Page orientation	A4 portrait, always. A wide diagram is turned, never the sheet
Page box	182 × 266 mm (A4 minus the handbook's 15/14/16/14 mm margins)
Legibility floor	7 pt. Above it a figure stays inline in the text column
Below the floor	The figure takes a plate page, turned counter-clockwise if turning gives it more room
Rotation	Inside the SVG coordinate system. Never a CSS transform
Caption	Upright, and inside the `figure` element so a break cannot separate the two
Sizing	Width and height both set, in millimetres. Never `height: auto`

Do not force a page break before a figure. The break is what strands captions on near-empty pages. `break-inside: avoid` moves the figure on its own and lets the text above it keep flowing.

If the build reports a figure below the floor, layout has done what it can. **Re-source the diagram**, change a `flowchart LR` to `TB`, shorten labels, reduce the chain depth, or split it into two or three figures. A figure below the floor is a *note, not a build failure*: fitting the page without clipping matters more than reaching 7 pt.

4.1 Graded documents: upright figures, the page turns instead

Leader instructions, 2026-09-25. These apply to every graded document under `capstone/Documents/reports/` (SRS, SDD, test documents, the final report) and to their slides.

1. **Every diagram is upright. Never rotate a diagram.** A page shown on a projector must not need the viewer to tilt their head.
2. **The page fits the diagram.** A wide diagram gets a landscape page of its own, as a separate landscape section; a tall one gets a portrait page of its own. The surrounding text stays portrait.
3. **Nothing is clipped or overflows.** A figure fits the page box on both axes. This is the hard requirement.

4. **The 7 pt floor is best practice, not a gate.** Aim for it; re-source or split a diagram when that is cheap. A figure that fits its page without clipping but stays below 7 pt is acceptable.
5. **The caption is the figure's title only.** No placement clause, no measurement, no status note, no explanatory second sentence. The handbook keeps its placement clause (§3); graded documents do not.

Rule	Value
Page box	The report template's text area: A4, 1-inch margins; 159 x 246 mm portrait, 246 x 159 mm landscape
Inline	Up to 159 x 225 mm in the text column, leaving room for the caption
Own page	A figure below 7 pt inline gets a page of its own: portrait up to 159 x 235 mm, or landscape up to 246 x 150 mm, whichever gives it the larger scale
Rotation	None, in any graded document
Caption	Below the figure, title only (chapter 15)

The handbook PDF (§4 above) keeps its rotated plates, because the handbook is built for print and is not graded. The report generator lives in `capstone/scripts/srs/` (`measure_figures.py` computes the placement, `build_srs.py` emits the landscape sections).

Figures currently below the floor

Handbook: none. As of 2026-09-17 the corpus is 27 figures and every one clears the floor; the lowest is 7.40 pt.

Report 3 (SRS), measured 2026-09-25. Each fits its page without clipping (rule 3), so each is acceptable under rule 4.

Report figure	Diagram	Source size	Placement	Labels
SRS Figure 17	MF-02 swimlane	816 x 1608 px	portrait page, 119 x 235 mm	6.63 pt
SRS Figure 18	MF-03 swimlane	1322 x 1116 px	landscape page, 178 x 150 mm	6.10 pt
SRS Figure 19	MF-04 swimlane	1276 x 1084 px	landscape page, 177 x 150 mm	6.28 pt
SRS Figure 20	MF-05 swimlane	1068 x 1294 px	portrait page, 159 x 193 mm	6.75 pt
SRS Figure 21	Entity Relationship Diagram	504 x 1648 px	portrait page, 72 x 235 mm	5.66 pt

If a figure ever has to be carried below the floor, record it here with its measurement and the reason the obvious fix was worse. This table is a record of deliberate exceptions, not a place to park failures, anything below 7 pt that is not listed here is a defect.

---

5. Hard rules for every figure

Enforced by the generator. Do not work around them in a source document.

1. A figure fits inside its page box on **both** axes. Nothing is clipped, cropped, or allowed to run past a margin.
2. Width and height are both set, in millimetres. Never `height: auto`, which constrains one axis and lets the other overflow.
3. A wide figure is turned counter-clockwise, **inside the SVG coordinate system**. Never with a CSS transform: a transformed box paints in one place and paginates in another, which prints content on the wrong page, clipped.
4. The caption stays upright and lives inside the `figure` element, so no break can separate it from what it names.
5. No figure forces a page break. `break-inside: avoid` moves it on its own and lets the text above keep flowing into the space.
6. **Monochrome.** A figure must not depend on hue to be understood, because these documents are printed and photocopied. Use hatching, dashes, or line weight. In Mermaid that means `style X stroke-width:3px` to emphasise a node, and

`-.->` versus `-->` to distinguish edge kinds, never `fill:#1e40af`.

Raster figures

A drawn or scanned figure is placed by the same rules, fitted rather than judged, because label size cannot be measured in a bitmap.

Rule	Value
Minimum resolution	200 dpi at final print size. Below that it is not print quality
Format	PNG, tracked in `_docs/assets/`
Rotation	Not supported. A raster has no coordinate system to turn inside
Colour	Subject to the monochrome rule above

---

6. The measurement harness

Before committing a new or re-sourced diagram, measure it:

```
python3 _docs/handbook/build_handbook.py --measure _docs/00-project-context/05-main-flows.md
```

For each diagram it reports source size in pixels, smallest rendered label, the chosen placement, the final millimetre box, and the effective point size, the same computation the build uses. Paste the placement and point size into the caption.

To check the whole corpus, including after a Mermaid version bump:

```
python3 _docs/handbook/build_handbook.py --measure-all
```

A diagram that fails to render at all is a hard error and blocks the build. A diagram that renders below the floor is reported and the build continues.

---

7. Practical guidance

Rules of thumb that come from actually fitting this project's diagrams:

- **Chain depth drives height.** In a `TB` swimlane the page height is the longest path, not the node count. Merging two sequential steps buys about 100 px.
- **Lane count drives width**, at roughly 260–270 px per lane regardless of label length. Five lanes is about the practical maximum for A4 portrait. Shortening node text does *not* narrow a lane.
- **A tree with more than ~16 leaves will not fit.** Split it by branch. The TrekLink feature tree is three figures for this reason, as one diagram it renders 6696 px wide and prints at 1.58 pt.
- `<br/>` **inside a label is free vertically** and cheap horizontally. Prefer two short lines over one long one.
- **Emoji inflate width** and do not survive photocopying. Do not use them in figures.

Appendices

TrekLink: Roadmap & Milestone Calendar

> **Rewritten Session 6 (2026-09-16) against the SEP490 Fall-2026 documents.** The previous version was derived from `SWP490_Lo-trinh-Capstone_v1.0.pdf` and described a different execution model, Review 1 as a Week 4–5 window, councils at Weeks 14 and 15, and delivery organised by TP package and Iteration. The course now issues SEP490 material that supersedes it on **timing and structure**. Do not rely on any copy of the old table. See **D-013**.
>
> **Authoritative sources**, in order:
>
> 1. `capstone/Documents/course-material/SEP490_Student_Project_Execution_Schedule.pdf` (50 pp), the master schedule, weekly Definition of Done, and anti-fail checks.
> 2. `capstone/Documents/course-material/SEP490_Huong_dan_nhanh_cho_sinh_vien.pdf` (6 pp), the student summary.
> 3. `capstone/Documents/course-material/TrekLink-proposed-mainflow-ducndm.png`, **the supervisor's own Main Flow set for this project** (D-016).
> 4. `capstone/Documents/templates/Cam-nang-tranh-loi-Capstone-SE.pdf`, the faculty fault- prevention handbook. Its failure modes are graded criteria, not advice.
> 5. `topics/Phieu_FA26SE159.md` (`Phieu_FA26SE159.md`), governs **content** (D-000).
>
> `SWP490_Lo-trinh-Capstone_v1.0.pdf` is retained deliberately: practices it describes that SEP490 does not contradict remain in force, the phase discipline, the deliverable-ledger shape, and the **Week-6 scope lock** (§6), which SEP490 never mentions but which is real.

Project code: FA26SE159 · **Group code**: GFA26SE55 · **Supervisor**: Đặng Ngọc Minh Đức **Term start**: Monday, Sep 7, 2026 · **Working plan ends**: Sunday, Dec 20, 2026 (Week 15)

> **On the "09/2026 – 03/2027" duration** printed on the registration form and in Report 1 §1.1: that span includes the **retake window**. A group that fails at either council sits a council in the following term, around March 2027. The figure is correct as registered. **Plan for December.**

---

1. The five things to know before reading anything else

1. **Three gates, and they are checkpoints rather than start lines.** Review 1 (W4), Review 2 (W8), Faculty Council (W13), then Final Submission & Defense (W15). The faculty guide opens with *"Review là điểm kiểm tra, không phải điểm bắt đầu"*, arriving at a gate having just started the work it assesses is the standard way to fail it.
2. **Delivery is organised by Main Flow, not by module or phase.** From W3 to W12 the project is built incrementally, one Main Flow at a time, each going Prototype/Figma → detailed spec → implement → test → demo. The five flows are fixed by the supervisor (§4). The **Mainflow Coverage Matrix** is the single instrument used to confirm progress at the weekly Group Meeting.
3. **The registration form still locks at the end of Week 6.** SEP490 does not mention a scope lock; the SWP490 roadmap does, and the team lead confirms it is real. Anything the form says that current decisions contradict must be amended before then, see §7.
4. **Everything produced must have evidence, and documents must match the software.** Commit, document, screenshot, test result, meeting minute. The faculty handbook's most common outcome is *"passed but must revise and resubmit documentation"*, the product ran and the paperwork pulled the mark down. The golden rule is *"có làm mới ghi, không làm đừng ghi"*: write only what was built, and be able to point at where each written claim is implemented or tested.
5. **Everything is in English**, following the SEP490 report templates.

---

2. Gate schedule

Gate	Week	Focus	The questions the panel will ask
Review 1, Gate 1	**W4**	Proposal & Requirement	What is the problem? Who uses it? How far does scope go? Are the requirements clear and verifiable?
Review 2, Gate 2	**W8**	System & Design	How will the system be built? Is the design clear enough to code from?
Faculty Council, Gate 3	**W13**	Final Product Evaluation	What did the group build? How do you prove it runs? Which part did *you* do?
Final Submission & Defense	**W15**	Submission	Is the package complete, consistent, and installable on a clean machine?
Retake council	~Mar 2027	—	only if W13 or W15 fails

The dual-council naming. "Hội đồng 1.1 / 1.2" is the *Hội đồng kín* terminology, conveyed through the supervisor's outline and never published as an official document. It is **not** a separate pair of events, it maps onto the table above: **Hội đồng 1.1 = Faculty Council (W13)**, **Hội đồng 1.2 = Final Submission & Defense (W15)**. Use it in internal planning; cite only the SEP490 names in graded documents and slides.

Preparation loop. Every gate is preceded by the same four-step cycle, run three times in the term: Common Meeting → team self-check → Group Meeting → fix → readiness check → gate.

```
CM1 (W1)  -> self-check -> GM -> fix -> readiness -> (foundation for the term)
CM2 (W3)  -> self-check -> GM -> fix -> readiness -> REVIEW 1  (W4)
CM3 (W7)  -> self-check -> GM -> fix -> readiness -> REVIEW 2  (W8)
CM4 (W12) -> self-check -> Mock Council -> fix -> readiness -> COUNCIL (W13)
```

Meetings. Four **Common Meetings** (all groups together) at **W1, W3, W7, W12**. A **Group Meeting** every week, ours on **Sunday**, time set by the supervisor. In a Common Meeting week the Group Meeting follows the CM, it exists to handle this group's specific problems, not to repeat what was just briefed to everyone.

Feedback is closed-loop and tracked. Every comment from any gate is recorded as `Feedback → Required Action → Owner → Deadline → Evidence → Status`, in the Review Feedback Tracker. Repeating a Review 1 mistake at Review 2 is named as a direct cause of failure, and a comment the group chooses not to follow needs a stated, defensible reason, not silence.

---

3. Week-by-week calendar

Weeks run Mon–Sun from Sep 7, 2026. "Due" means end of that week unless stated.

Wk	Dates 2026	Common Mtg	Phase	Gate / event	Deliverable due
1	Sep 7–13	**CM1** Kick-off & Working Guideline	Initiation	Group kick-off	Project Introduction · Initial Scope · Team Working Plan
2	Sep 14–20	—	Planning	WBS / timeline / risk review	**PMP Draft · WBS · Feature Tree · Mainflow list · Gantt · Responsibility Matrix · Risk List**
3	Sep 21–27	**CM2** Review 1 Clinic	Requirement + Foundation Design	SRS draft / UC / BR review	**SRS Draft · Architecture Diagram · ERD · Package Design · prioritised Mainflow list** · demo MF-01
4	Sep 28–Oct 4	—	Review 1	**REVIEW 1, GATE 1**	**Registration Form · PMP Draft · SRS Draft · Review 1 slides**
5	Oct 5–11	—	System Design	Architecture + ERD approval	Figma/prototype per Main Flow · SDD update
6	Oct 12–18	—	Detailed Design	Sequence/Class/Workflow approval · **SCOPE LOCK**	**Amended registration form, GVHD-confirmed, FROZEN**
7	Oct 19–25	**CM3** Review 2 Clinic	Design Baseline	Baseline SRS/SDD, Test Plan, readiness	**MF-01 + MF-02 complete · MF-03 in progress · MF-04/05 have Figma + spec**
8	Oct 26–Nov 1	—	Review 2	**REVIEW 2, GATE 2**	**PMP Updated · SRS Final · SDD Initial · Test Plan Draft · slides** · demo MF-01,02
9	Nov 2–8	—	Implementation I	Iteration 1 review	Product v0.1 · demo highest-priority Main Flow end-to-end
10	Nov 9–15	—	Implementation II	Iteration 2 review	All Main Flows demoable · Coverage Matrix updated
11	Nov 16–22	—	Implementation III	Iteration 3 review + readiness	**Feature complete · Product v0.9 / Release Candidate · Configuration Matrix updated · Report 5**
12	Nov 23–29	**CM4** Council Readiness	Testing & Council Prep	**Mock Council** + buffer sprint	**STD / Test Results · Deployment Package · Demo Script** · 100 % Coverage Matrix
13	Nov 30–Dec 6	—	Final Evaluation	**FACULTY COUNCIL, GATE 3**	**Final Product · PMP/SRS/SDD/STD · Software Package · Reports 1–5 · final slides**
14	Dec 7–13	—	Finalization	Feedback closure + consistency audit	**User Guide · Installation Guide · Reports 6–7 · Final Project Package**
15	Dec 14–20	—	Submission	**FINAL SUBMISSION & DEFENSE**	**Final Report · final slides · Software Package installable on a clean machine**

> **Week↔date mapping** assumes Week 1 is the calendar week containing Sep 7, 2026, with no holiday gaps. No Vietnamese public holiday falls inside W1–W15. Exact gate dates, rooms and reviewer assignments are notified per group; this table fixes the windows.

3.1 Two contradictions inside the SEP490 source, and how we resolve them

Flagged rather than hidden, so nobody "corrects" this file back later.

Contradiction	Resolution
Review 1 at **W4** (Roadmap table, Master Schedule 5.2, quick guide) vs **W3** (§4 prose item 2)	**W4**, two sources to one
"Two Main Flows complete at **W7**" (§4, Master Schedule 5.1) vs "Sprint 1 (**W8**) → MF-01, MF-02" (Sprint Plan)	Plan to the **stricter W7**; W8 is the confirmation demo

3.2 Continuous obligations

Not one-off deliverables; checked at every gate.

- **Group Meeting every week**, all members attend. A member who misses reviews and contributes little **fails individually even if the group passes.**
- **Progress Log** (`GFA26SE55_Progress_Log.xlsx`), updated **before every Group Meeting**, per-member and per-week, each entry tied to a named deliverable. This is the primary evidence for Individual Contribution and it starts at W1, not at the council. "Continued working on the project" is not an entry.
- **Mainflow Coverage Matrix**, the supervisor's progress instrument. Kept current weekly.
- **Review Feedback Tracker**, every gate comment carried to a closed status with evidence.
- **Evidence habit**, commits, documents, screenshots, test results, meeting minutes.

3.3 The weekly routine

`Plan → Execute → Team meeting → Evidence → Update Progress Log & report to GVHD → Weekly close`, closing each week as DONE / IN PROGRESS / BLOCKED / NEXT WEEK. Anything that misses its Definition of Done carries into the next Group Meeting rather than quietly disappearing.

Our own cadence, inside that frame: asynchronous daily report via GitHub Issues; online team meetings **Monday** and **Thursday**; offline meeting **Sunday**; an offline retrospective and review at each two-week sprint boundary.

---

4. The five Main Flows

Redrawn after Review 1 by **D-033** (superseding D-016 and the supervisor's original image); identifiers and owners are unchanged. These are the units the Coverage Matrix tracks, the units demoed at each Iteration review, and the units the council evaluates. Full flows: `05-main-flows.md`.

MF	Name	Substance	Owner
MF-01	Organization onboarding → Rental contract → Handover	Self-registration → verification at the counter → Admin approval → plan request (monthly or day, variant, quantity ≥ MOQ) → approval reserves devices → first payment (SePay or counter) → provisioning with the organization key → signed handover	**TanNB**
MF-02	Field data → Field Station → Cloud synchronization	Device SOS/GPS/telemetry over LoRa mesh → Field Station SQLite priority queue while offline → MQTT → NestJS; P0–P3 with idempotency; priority-ordered flush on reconnect	**KhoaDD**
MF-03	SOS → Tiered alert → Escalation	SOS → one incident per episode → primary, backup, Manager tiers with timeouts → acknowledgement, status, outcome → escalation to TrekLink and the authority report when nobody answers (D-034)	**HoangTK**
MF-04	Live telemetry: web map and organization API	Device → Field Station → backend → organization-scoped map and REST/WebSocket API with a key per organization; the Field Station's local page offline	**LongNN**
MF-05	Term end → Return → Inspection → Billing → Maintenance	Notice or day-plan end → check-in at the counter → reset and inspection → term balance, late, damage and loss charges → payment → close; device restocked, serviced or retired (D-035)	**LongLP**

E7 (DevOps/CI-CD) and E8 (Research & Experimental Evaluation) are cross-cutting and deliberately are not Main Flows. State this wherever the Coverage Matrix is presented, so their absence does not read as an oversight.

4.1 How a Main Flow is built

Each one goes through the full cycle; the system is **not** designed end-to-end before coding starts. Architecture, the foundational ERD and the technology decisions are made **once** at W3 and serve every flow.

```
Prototype / Figma
  -> detailed functional spec  (input, output, validation, business rule, happy + unhappy path)
  -> sequence + class diagram, state diagram for anything with a lifecycle
  -> API / interface spec      (endpoint, request/response, DB or third-party call)
  -> implement
  -> test
  -> demo
```

A flow whose design finishes first may be implemented first, in parallel with the design of the others. No flow enters implementation (W9–W11) without Figma **and** a detailed spec.

Traceability must hold in both directions: Main Flow → Use Case → Sequence → Class/Service → Database → Code. A reviewer will pick a requirement and walk it to the implementation, or pick a diagram component and ask which code it is.

4.2 Sprint plan

Sprint boundaries align to gates, never to a fixed drumbeat.

Sprint	Weeks	Closes on	Carries
1	1–3	Review 1 readiness (W3)	Requirements, architecture + ERD foundation, MF-01 first cycle
2	4–6	**Scope lock (W6)**	Review 1 feedback closure, MF-01/MF-02 design, **form amendments**
3	7–8	**Review 2 (W8)**	MF-01 + MF-02 complete; MF-03 in progress; MF-04/05 spec'd; Test Plan
4	9–10	Iteration reviews	MF-03, MF-04 implemented, Product v0.1
5	11	Feature complete (W11)	MF-05 + full-system integration, Release Candidate
6	12	**Mock Council (W12)**	Buffer for anything outstanding, system testing, regression, deployment, demo rehearsal
7	13–15	Councils	Council, feedback closure, user/installation guides, final report, defence

The W12 buffer is a **safety net, not the plan**. The target is every Main Flow finished by the end of W11. The council expects all Main Flows working plus at least 80 % of use cases.

---

5. Deliverable ledger and ownership

Deliverable	Planned	Owner	Reviewer	Status
Report 1, Project Introduction	W1 → W4	KhoaDD	TanNB	Draft complete
Report 2, Project Management Plan (PMP)	**W2 draft → W4 → updated W8**	**KhoaDD**	LongLP	In progress
Report 3, Software Requirement Specification (SRS)	**W3 draft → W4 → final W8**	**TanNB**	KhoaDD	Not started
Report 4, Software Design Document (SDD)	W3 draft → W8 initial → W13 final	KhoaDD + HoangTK	LongLP	Not started
Report 5, Test Plan → STD & Test Results	W7 plan → W11 → W12 complete	TanNB	HoangTK	Not started
Report 6, User Guide	W14	LongNN	TanNB	Not started
Report 7, Final Project Report	W15	KhoaDD	all	Not started
Installation Guide	W14	LongLP	KhoaDD	Not started
Software Package (v0.1 / v0.9 / final)	W9 / W11 / W13	whole team	—	Not started
Presentation / slides	W4, W8, W13, W15	KhoaDD	all	Not started
Progress Log	weekly from W1	KhoaDD	all	To create
Mainflow Coverage Matrix	weekly from W3	KhoaDD	supervisor	To create

Report 3 is split across two deadlines. W4 needs the draft, Product Overview, User Requirements, System Functional Overview, Context Diagram, Actors, Use Cases, initial Business Rules, Exception Scenarios. The full EARS SRS is W8. Write the W4 draft so the W8 version grows out of it rather than replacing it.

Two UML State Machine Diagrams are named, graded deliverables (register §f): Device Lifecycle (7 states) and Incident Lifecycle (5 states). They belong in Report 4 and in `specs/devices/design.md` / `specs/incidents/design.md`, and they are visible in MF-03 and MF-05.

5.1 Module lanes

Assigned against the enrolment framework codes. `BIT_SE_NJS_18D` is the Node.js track and is treated as covering NestJS; backend capability is **3 of 5**, not 1 of 5 as previously recorded.

Member	Track	Lane	Main Flow
KhoaDD, leader	IC 19A	Prisma schema, `devices`, firmware, `gateway-sync`, UI layout, architecture	MF-02 owner, MF-05 support
LongLP, delegate	**NJS** 18D	`auth`, `rentals`, `billing`	MF-05 owner, MF-01 support
HoangTK	**NJS** 18D	`incidents`, `monitoring` ingestion, WebSocket	MF-03 owner, MF-04 backend
LongNN	IC 19A	`frontend` (FSD), map and monitoring UI	MF-04 owner
TanNB, delegate	IC 18D	`trips`, booking, QA and test lead; secondary Meshtastic knowledge	MF-01 owner

Lanes run as concurrent branches across decoupled NestJS modules rather than serialising on one branch. Ramp-up for the two IC-track members on `01-conventions/05-backend-conventions.md` (`05-backend-conventions.md`) during W2–W5 is budgeted work, not slack, and is a tracked risk in the register.

---

6. ⚠️ The Week-6 scope lock

SEP490 does not mention it; the SWP490 roadmap does, and it is real (D-013). After the end of W6 the registration form is frozen, and the Faculty Council grades the product against that frozen form. Note there are now **two** baselines and they are not the same: the **requirement** baseline is set at Review 1 (W4), the **form** freeze at W6.

6.1 Supervisor questions: all closed

#	Question	Decision	Outcome
1	Keep Stage B (the basecamp bridge, renamed Stage C by D-018) or drop it?	**D-005**	**Keep, permanently.** It is the substance of MF-02. Offline NFRs and RQ1/RQ2 stay binding
2	Do firmware commits earn graded credit?	**D-008 Q3**	**Yes**, with the caveat that code volume alone is not what is assessed
3	Are targeted firmware fixes read as out-of-scope "firmware redesign"?	**D-008 Q2**	**No**, firmware enhancement and integration are in scope; only a mesh-stack rearchitecture is excluded

The supervisor's position as given on 2026-09-13: the team may change tech stack, flows and scope as it judges best, then pitch and defend the result at the next meeting. His own outline is guidance, not a restriction. **Nothing is blocked on him.** What remains is our own obligation to put the amended form in front of him before W6.

6.2 Register text to amend in the Week-6 update

The submitted form contains statements later work proved wrong. Each must be amended, or the product will diverge from the document it is graded against.

Register says	Reality	Source
"Apply PostgreSQL migrations via **TypeORM**" (§e)	Prisma, locked as team mandate	**D-001**
`eventId = Device ID + Session ID + Sequence Number` (§b, §c)	Neither field exists in firmware; key is `sha256(nodeNum:packetId)` plus open-Incident correlation	**D-006**, `04-firmware-ground-truth.md` (`04-firmware-ground-truth.md`) §3
Firmware "version-locked to Summer 2026 release" (§b)	Editable, credited, and in scope for enhancement	**D-008**
Gateway is dedicated hardware with its own uplink (§b)	Staged: Stage A = node's own MQTT; Stage B = on-device durable queue; Stage C = basecamp bridge. All in scope	**D-005**, **D-016**, **D-018**
Frontend map library	**Leaflet + OpenStreetMap** with a Vietnamese sovereignty overlay; Goong Maps dormant in configuration	**D-031** (supersedes D-012)
—	v1 hardware compiles MQTT out and is **out of the demo set**	**D-005**, ground truth §5
Booking, trek packages, Customer, Guide, trip management (§3.1.2, §3.2, §e)	Enterprise rental platform: organizations rent devices by monthly or day plan; asset management replaces booking	**D-033**
Device lifecycle of seven states with `In-Field`	Eight states with `In Intake` and `Lost`; no `In-Field`	**D-035**
Incident lifecycle of five states	Tiered-alert lifecycle with escalation to TrekLink	**D-034**
Multi-tenancy out of scope	In scope: organizations, members, roster, API keys	**D-033**

The adjusted form, with every change highlighted for the supervisors, is `treklink-docs/topics/Phieu_FA26SE159_adjusted_2026-10.md`.

> The `eventId` amendment matters most. The register makes it a **named scientific contribution** (§3.3d, "a formally defined eventId scheme") and the duplicate-prevention NFR is written in terms of it. Changing the formula silently and presenting a different one at defence is exactly what the council exists to catch. Amend the wording to the constructible form and keep the intent.

---

7. Gate readiness checklists

Before Review 1 (W4)

- [ ] Problem, target users, objective and scope stated clearly
- [ ] **Context Diagram**, Actors, Use Case Diagram complete
- [ ] FR and NFR written at a verifiable level, no "Admin manages everything"
- [ ] **Feature Tree** complete
- [ ] **Business Rule Matrix** first version, `BR ID | Business Rule | Requirement | Implementation | Test Case`
- [ ] At least one **exception scenario per Main Flow**
- [ ] Main Flow list identified and prioritised; the slides show high-level steps only, never an activity diagram (swimlane diagrams stay in the SRS)

- [ ] Deck states system or platform and names the partners (chapter 15 §3)
- [ ] Registration Form, PMP Draft, SRS Draft submitted; slides consistent with the SRS
- [ ] Use cases are functions, not workflows; `include` / `extend` / generalization used correctly
- [ ] Every decision-register item either resolved or explicitly tabled

Before Review 2 (W8)

- [ ] SRS Final, SDD Initial, Test Plan Draft
- [ ] Architecture, ERD with PK / FK / relationships / cardinality, Package Design
- [ ] State / Sequence / Class diagrams for the main data objects
- [ ] Traceable Use Case → Design → Class/Service → Database
- [ ] Working prototype or workflow demo; **MF-01 and MF-02 demoed**
- [ ] Business parameters configurable, not hardcoded (**D-015**)
- [ ] Entity and attribute names consistent between ERD, SRS and code
- [ ] Every architecture and technology choice has a stated reason
- [ ] No component drawn that the team does not intend to implement
- [ ] Review 1 feedback closed with evidence, item by item against the Review 1 minutes (they are tracked)
- [ ] Deck says explicitly that multi-tenancy moved from out of scope (Review 1 minutes) to in scope, and why

Before the Faculty Council (W13)

- [ ] **Every Main Flow runs end-to-end**; ≥ 80 % of use cases working
- [ ] Coverage Matrix 100 % Implemented / Tested / Demo Ready
- [ ] Each important FR has at least one test case with evidence
- [ ] Product deploys outside a developer machine; demo data ready and realistic
- [ ] Demo rehearsed at least twice, with a backup video of each Main Flow
- [ ] Business parameters demonstrably configurable, expect to be asked live
- [ ] Documentation matches the software that will be demoed
- [ ] RQ1 / RQ2 / RQ3 results written up
- [ ] Every member can answer: *"what did you do, and where is it in the code?"*
- [ ] Review 1 and Review 2 feedback closed with evidence, tracker in hand

Before Final Submission (W15)

- [ ] Package complete and internally consistent; no template placeholders left
- [ ] Software Package installs on a clean machine
- [ ] User Guide and Installation Guide complete
- [ ] Every member can answer questions about the **whole** system, not only their own part
- [ ] Submitted build is the build that was demoed

---

8. Known failure modes to design against

From the faculty fault-prevention handbook. These are graded criteria.

The six causes of not passing, in order of frequency: a Main Flow that breaks during the demo or cannot be shown in the allotted time · hardcoded business parameters · documentation that is wrong, incomplete, or not on the official template · business logic detached from reality with no exception handling · Review feedback not acted on · not being approved to defend, or a member who contributed too little.

Specific to this project:

- **Demo failure is the single most common cause of failure.** Rehearse twice, keep a backup video, keep an IDE open on the repository, use short demo passwords, never build a special "demo version", and if something breaks, stop and explain the fault rather than skimming past it.
- **Hardcoding**, see **D-015**. The first question in the handbook's most-asked list is *"can this number be changed? show me now."*
- **The word "Smart" in our registered title.** The handbook devotes a section to projects whose name promises AI and whose demo is CRUD, and says plainly that a rule-based system must be called rule-based. TrekLink has no ML component. The honest answer, rehearsed, and known by every member, is that the intelligence is **device-level autonomy**: IMU fall detection, autonomous SOS broadcast, multi-hop mesh routing, priority-ordered store-and-forward. Never write "AI" for the platform in a graded document or slide.
- **AI-written documentation and fabricated test results** are called out as a fast-growing and serious failure. Every NFR and business rule must point at the code or test that implements it. Graded documents describe the work as **AI-assisted development with mandatory human review**.
- **Data, security and legal**, collect only what is used; no secrets in source; cite third-party data sources; keep a short Legal & Compliance section. The map-provider migration (**D-012**) sits here and is a legal obligation, not a preference.
- **Individual contribution**, commits and weekly minutes are the evidence. A repository where one or two people committed everything cannot demonstrate who did what.

Appendix A: TP1–TP6 (historical, no longer the planning unit)

The register (§g) organises work into six task packages. SEP490 organises delivery by Main Flow, and **the Main Flow model governs** (D-013). This table is retained so register cross-references resolve, and is mapped onto the current plan.

TP	Register scope	Now carried by
TP1	SRS, 2 UML state machines, architecture doc, inherited-component analysis, DB schema, RQ/experiment protocol, PoC parser	W2–W3 foundation work + Report 3/4
TP2	Gateway Bridge: serial parser, SQLite P0–P3 queue, MQTT publish, reconnect/flush, gateway dedup, health API	**MF-02**
TP3	NestJS auth/RBAC, device fleet FSM, rental lifecycle, idempotent ingestion, sync audit log, migrations	**MF-01**, **MF-02**, **MF-05**
TP4	Telemetry ingestion, live map + WebSocket, SOS→Incident pipeline, incident FSM, notifications	**MF-03**, **MF-04**
TP5	Trip/booking, reservation/assignment, check-out/in, agreements, billing, role-based views	**MF-01**, **MF-05**
TP6	LoRa experiments (RQ1/RQ2), RQ3 drills, dedup/concurrency results, evaluation report, Docker Compose, CI/CD, final docs	**E7 + E8**, cross-cutting; W11–W13

The one TP-era constraint that survives unchanged: **the message-schema freeze is a hard dependency for everything downstream.** Do not start Gateway or ingestion implementation before the schema is committed to `specs/gateway-sync/design.md`.

Appendix B: Superseded framings

Do not reintroduce these; they appear in older documents and in the SWP490 roadmap.

Superseded	Replaced by
Review 1 as a "Weeks 4–5 window with a Week-3 dossier"	Review 1 = **W4**
Review 2 as "Weeks 8–9 with a Week-7 dossier"	Review 2 = **W8**
Hội đồng 1.1 at W14 and 1.2 at W15 as separate extra events	**W13 Faculty Council** and **W15 Final Submission & Defense**
Iteration 1 / 2 / 3 completing at W7 / W11 / W13	Iteration reviews at **W9 / W10 / W11**, feature complete W11
Seven sprints on a fixed two-week drumbeat	The gate-aligned sprint plan in §4.2
"NestJS is a skill only for Khoa"	**3 of 5**, see §5.1
Leaflet.js as the map library	Kept, over OpenStreetMap with a sovereignty overlay (D-031)

TrekLink: Decisions Log & Risk Register

Use this file like a lightweight ADR index. Anything marked **OPEN** blocks the dependent work listed under it, resolve before starting that work, not during.

Decisions

D-000: Which register document is authoritative

- **Status**: ✅ Resolved
- **Context**: `Phieu_FA26SE159.docx` (individual registration form, detailed RQs/NFRs/TP1–TP6) and `Phieu_dang_ky_de_tai_TrekLink_FA26_FINAL.docx` (team draft, M1–M5 module framing) describe the same project at different fidelity, with minor differences (supervisor email domain `fe.edu.vn` vs `fpt.edu.vn`; module count 5 vs 5 core capabilities; TypeORM vs Prisma, see D-001).
- **Decision**: `Phieu_FA26SE159.docx` is authoritative for anything it specifies explicitly (RQs, NFRs, FSMs, task packages, experiment design). The FINAL draft is used only to fill gaps it doesn't cover.
- **Owner**: Team lead, confirm with supervisor at Review 1 kickoff that FA26SE159 is the version on file.

D-001: ORM: Prisma vs TypeORM

- **Status**: ✅ **Resolved** (Prisma ORM)
- **Decision**: Prisma ORM is locked as the team mandate for maximum productivity and type safety across backend services. PostgreSQL schema models and migrations are maintained in `backend/prisma/schema.prisma`.

D-002: Response envelope standard

- **Status**: ✅ Resolved
- **Decision**: All backend responses use the shape already defined in `API_Design_Template.md`: `{ "result": ... "isSuccess": bool, "statusCode": int, "message": string }`. This supersedes the raw-DTO / `errorDetails[]` pattern from the previously-used `dev-flow.zip` (a different, .NET-project convention), see `02-templates/04-api-endpoint-template.md` and `01-conventions/05-backend-conventions.md`.

D-003: GitLab → GitHub adaptation is cosmetic, not philosophical

- **Status**: ⛔ **SUPERSEDED by D-009** (2026-09-13). Kept verbatim below for history, do not follow it.
- **Decision** *(superseded)*: Branch topology (`main` / `develop` / `features/Implementation_*` / `features/Design_*` / `hotfix/*` / `release/sprint_x`), commit discipline, and the dual Design+Implementation branch pattern from `Git_Lab_Guide.pdf` carry over unchanged. Only the tool-specific mechanics change: GitLab Issues/Labels/Milestones → GitHub Issues/Labels/Milestones+Projects; GitLab MRs → GitHub PRs (multi-template via `.github/PULL_REQUEST_TEMPLATE/`); GitLab child tasks → GitHub sub-issues/tasklists. See `01-conventions/07-github-workflow-git-conventions.md`.

D-004: Monorepo vs. multi-repo

- **Status**: ✅ **Resolved** (updated Sep 7, Session 2, repos exist and this is what's actually cloned)
- **Context**: TrekLink has 4 codebases: firmware (inherited, read-only), gateway bridge (Node/TS), backend (NestJS), frontend (React). The org is `github.com/TrekLink-Team`. The original sketch (below, kept for history) considered 4 separate app repos.
- **Original options considered**: (a) 4 separate repos under the org (`treklink-firmware`, `treklink-gateway`, `treklink-backend`, `treklink-web`) with independent CI, or (b) one monorepo with workspaces (`apps/gateway`, `apps/backend`, `apps/web`, `firmware/` as a git submodule).
- **Decision actually taken**: a **hybrid**, 3 repos total under `TrekLink-Team`:

  - `treklink-docs`, this repo (SSOT, docs-only).
  - `treklink-firmware`, inherited SU26 firmware, frozen/read-only this term.
  - `treklink-web`, **one** active application repo containing the Gateway Bridge, NestJS backend, and React frontend as workspace packages (`gateway/`, `backend/`, `frontend/`), rather than 3 separate repos.

- **Rationale**: The firmware is a frozen, inherited dependency, keeping it fully separate avoids accidental edits being graded as "new" work, matching the register's note that firmware effort must be excluded from statistics. Splitting the *active* code further into 3 repos (original option a) was reconsidered: for a 5-person/13-week term, one repo with workspace packages keeps shared TypeScript types (e.g. the `eventId`/DTO shapes gateway and backend both touch), a single CI pipeline, and a single `npm install`, at the cost of coarser per-package branch protection, which `module:*` labels and path-scoped PR reviews (see `01-conventions/07-github-workflow-git-conventions.md`) substitute for.
- **Consequence**: `treklink-web/specs/{module}/` is the one spec root for gateway, backend, and frontend modules alike, see `01-conventions/02-spec-driven-development-workflow.md`. Update that doc's example paths if this ever splits back into separate repos.

D-005: Field-to-cloud bridge topology

- **Status**: ✅ **Resolved and closed** (2026-09-16, Session 6), staged rollout, **both stages in permanent scope**. Supersedes the earlier "Stage B is tradeable" framing.
- **Context**: The register describes the Gateway/Bridge as **dedicated TrekLink hardware with its own Wi-Fi/cellular uplink** (charter §2, FA26SE159 §b). Session 3 established the physical constraint that closes this: **the Gateway Bridge machine is not carried during the trek.** It sits at basecamp. The guide carries a node, not a laptop.
- **Options considered**:
  1. **Basecamp-fixed bridge**, a designated node USB-tethered to the basecamp machine running `gateway/src`; the SQLite P0–P3 queue buffers the machine's intermittent Wi-Fi uplink. Requires no firmware work.
  2. **Node's built-in MQTT uplink, no bridge**, the guide's node publishes to Mosquitto over its own Wi-Fi using the stock Meshtastic MQTT module already compiled into the frozen firmware. Simplest possible path; **has no buffer anywhere.**
  3. **Fork the Meshtastic mobile apps**, rejected outright. iOS requires Xcode on macOS, which the team does not have; an Android-only fork would split Guide/Customer behavior by platform, and mobile apps are explicitly out of charter scope.
- **Decision**: Build **Option 2 first (Stage A, TP1)**, then **add Option 1 on top (Stage B, TP2)** as an additional ingress adapter plus a hard-wired gateway node. Option 3 stays rejected.
- **Rationale**: Option 2 is the shortest path to an end-to-end demo and needs no serial parser, no hardware tether, and no protobuf decoding. It is only non-throwaway because of the ingress-adapter seam defined in **D-007**, Stage B reuses the entire normalizer and ingestion pipeline and contributes only a new adapter in front of it.
- ⚠️ **Known gap, deliberately accepted for Stage A**: Option 2 has **zero offline buffering**. Stock Meshtastic MQTT drops packets when the uplink is down (verified Session 2), and Store & Forward is a mesh-internal, PSRAM-only, ~30-packet chat replay, not a cloud-sync buffer. Therefore, **while only Stage A exists**, the following charter §5 NFRs are unmeasurable and RQ1/RQ2 are unanswerable:
  - Offline recovery delivery rate ≥99% after reconnection (30s–30min loss)
  - Priority-ordering compliance ≥99%, all P0 flushed before P2/P3 on reconnect
  - Gateway→cloud sync ≤5s *when uplink returns* (the ≤5s steady-state figure is still measurable)

  These are restored by Stage B, where the SQLite queue sits in the uplink path and loss is induced by killing the bridge's Wi-Fi.
- **Scoping note that makes Stage B sufficient**: every offline NFR in charter §5 is scoped to the **gateway→cloud** leg, not the **node→gateway** mesh leg. Charter §1 gap #1 reads *"no store-and-forward path to the cloud… there is no Gateway between the mesh and a backend."* Mesh-range loss (a node out of LoRa range of any relay) is an RF-coverage problem,

already carried as an RF-reliability caveat in the risk table below, and is **not** claimed as in scope. A basecamp-fixed bridge therefore exercises exactly the leg the research questions measure.

- **Hardware scope** (settled Session 3): **Stage A targets v2/v3/v4 only. v1 is out of the demo set.** v1 compiles MQTT out entirely (`-D MESHTASTIC_EXCLUDE_MQTT=1` , `variants/esp32/treklink_v1_0/platformio.ini:10`), the module is removed at build time, so no runtime configuration can enable it. v2/v3/v4 omit that flag and all carry Wi-Fi silicon, so any of them can serve as the uplink node. Were v1 ever needed, deleting the build flag is a one-line change now that D-008 permits firmware edits.
- ~~**Stage B is tradeable scope**~~, **struck (Session 6).** Stage B is permanent scope. Two independent reasons, either of which alone is sufficient:
  1. **The supervisor put it in the mainflow set.** `Documents/course-material/TrekLink-proposed-mainflow-ducndm.png` defines **MF-02, Field Data → Offline Gateway → Cloud Synchronization**, and draws the Gateway Bridge (Node.js) with its SQLite offline queue and priority-ordered reconnection flush as the substance of that flow. Dropping Stage B would delete a supervisor-specified mainflow. See **D-016**.
  2. **The scope constraint that motivated the trade is gone.** At the 2026-09-13 meeting the supervisor granted the team latitude to set its own scope and tech stack, subject to pitching and defending it. There is no longer a scope-reduction pressure to spend Stage B on.
- **Consequence**: charter §5's offline-recovery and priority-ordering NFRs and **RQ1/RQ2 stay binding and must be delivered.** Stage A remains the first increment, not a substitute, never present Stage A as satisfying the offline-recovery NFR.
- **Blocks**: nothing, `specs/gateway-sync/{requirements,design,tasks}.md` are written against this decision. Stage B's isolation in Phase 9 is retained as sequencing, no longer as a removal seam.

D-006: `eventId` redefinition: the charter's scheme is not constructible

- **Status**: ✅ **Resolved** (Session 3), supersedes charter §2 and `01-conventions/04-architecture-conventions.md` §3
- **Context**: Charter §2 and the architecture conventions both specify `eventId = DeviceID + SessionID + SequenceNumber` as the backend idempotency key. Direct inspection of the frozen firmware shows **neither `SessionID` nor `SequenceNumber` exists anywhere**, not in the TrekLink modules, not in the Meshtastic packet format. The only per-packet identifier transmitted is `MeshPacket.id` , and `Router.cpp:168` (`generatePacketId()`) shows it is a 10-bit rolling counter OR'd with 22 random high bits, re-seeded randomly at every boot. It is a flood-dedup token, **not** a monotonic sequence. The charter's key cannot be built from what the hardware actually sends.
- **Options considered**:
  1. **Split the key** into a packet-level dedup key and an episode-level correlation key.
  2. **Single content-addressed key**, `hash(from : rx_time : portnum : payload)` , rejected: the SOS beacon retransmits every 5s (then 30s) with a fresh `rx_time` , so one SOS episode would mint a new key per tick and create N Incidents from one fall.
  3. **Bridge synthesizes session + sequence**, rejected: the key would then describe the *bridge*, not the device. Restart the bridge, replay the same packets, and every key is new; deduplication becomes a silent no-op, which is precisely the failure the 10× replay NFR exists to catch.
- **Decision** (Option 1):
  - **Packet dedup key**, `GatewayEvent.eventId = sha256(nodeNum : packetId)` . Satisfies charter §5's *"0 duplicate Incidents for repeated eventId delivery (10× replay test)."* Enforced by a unique index; conflict is a no-op insert inside the ingestion transaction.
  - **Episode correlation**, *not* a hash. On SOS ingest, look up an open `Incident` for the same device whose `lastEventAt` falls inside the episode window; append if found, create if not. An earlier draft used `hash(nodeNum : timeBucket)` , which was discarded: a bucket boundary falling mid-episode splits one SOS into two Incidents, the exact failure the key exists to prevent.
- **Consequences**:

  - `backend/prisma/schema.prisma`: `Incident.eventId @unique` currently serves as *both* keys and must be split, `Incident` gains `lastEventAt` and an index on `(deviceId, status, lastEventAt)`, and references its originating `GatewayEvent` by FK rather than carrying a duplicate unique string.
  - **`01-conventions/04-architecture-conventions.md` §3 still states the old formula and is now contradicted by this decision. It must be updated before any `gateway-sync` implementation branch opens.**
  - Charter §2's wording stands as the *intent* (a natural, device-derived idempotency key); this decision records the constructible form of it.

- **Reopened by D-008** (firmware is editable): adding a boot-`sessionId` and a per-packet `sequenceNumber` firmware-side would make the charter's original formula constructible and would give a genuinely monotonic sequence, strictly better than hashing a randomized packet id, and it would let the platform *detect gaps* (a missing sequence number proves loss) rather than merely deduplicating what arrives. **The split key above remains correct and functional with or without it**; treat a firmware sequence as a layered upgrade to evaluate next session, not a prerequisite for TP1.

D-007: Canonical event envelope + Strategy-based normalizer (the ingress seam)

- **Status**: ✅ **Resolved** (Session 3)
- **Context**: Stage A (node MQTT) and Stage B (serial bridge) do **not** speak the same wire format. The node's MQTT module publishes to a hardcoded topic shape, `<root>/2/e/<channelId>/<nodeId>` carrying a `ServiceEnvelope` protobuf, or `<root>/2/json/<channelId>/<nodeId>` carrying JSON when `json_enabled` (`src/mqtt/MQTT.cpp:423–430`, `:798`). `<root>` is configurable; `/2/e/` and the payload type are not. Meanwhile `gateway/src/mqtt/mqtt-client.ts` publishes `treklink/events/priority/{0..3}` with a TrekLink-shaped payload. If the backend parses Stage A's format directly, Stage B's arrival forces a rewrite of the ingestion layer, and D-005's staged plan quietly becomes throwaway work.
- **Decision**: The backend ingests exactly one internal type, **`TrekLinkEvent`**. Between the wire and the domain sit two seams:
  1. **Ingress adapters** (`MeshIngressAdapter`), one per topology, responsible only for getting bytes off a transport and producing a `RawMeshPacket`. Stage A ships `MqttJsonIngressAdapter`; Stage B adds `SerialBridgeIngressAdapter`. Adapters hold no domain logic.
  2. **Normalizer** (`MeshNormalizerService`), the Strategy-pattern context. It resolves a `PacketNormalizationStrategy` by PortNum from an injected registry and delegates. Strategies are pure (no I/O, no Prisma) and each owns one PortNum's mapping to a domain event kind and priority tier. Adding a PortNum means adding a strategy class and registering it; the normalizer itself never changes (Open/Closed).
- **Rationale**: the normalizer holds ~all the genuinely hard logic (SOS prefix parsing, priority assignment, `eventId` derivation, episode correlation) and is 100% shared between stages. Purity makes it unit-testable without a broker, a device, or a database, which is what makes the charter's dedup/ordering/concurrency NFRs cheap to test rather than integration-only.
- **Stage A transport config, frozen**: `json_enabled = true`, `encryption_enabled = false`, `root = "treklink"`. JSON removes any protobuf dependency from NestJS, and the JSON serializer covers the three PortNums TrekLink actually emits, `TEXT_MESSAGE_APP`, `POSITION_APP`, `TELEMETRY_APP` (`src/serialization/MeshPacketSerializer.cpp:28`, `:208`, `:56`). `encryption_enabled = false` is required so the broker receives decrypted packets; the field's own comment in `module_config.pb.h` notes decrypted output exists for exactly this purpose. The alternative is reimplementing Meshtastic's AES-CTR channel crypto in NestJS.
- **Known limitation of the JSON path**: the JSON envelope (`MeshPacketSerializer.cpp:410–424`) carries `id`, `timestamp`, `to`, `from`, `channel`, `type`, `sender`, `payload`, `rssi`, `snr`, `hops_away`, but **not** **`MeshPacket.priority`**. The SOS position packet is sent with `priority = MAX` (`TrekLinkSOSHelper.cpp:119`) and is therefore indistinguishable from a routine position report on this path. Handled by backend episode correlation with a backward grace window (see `specs/gateway-sync/design.md` §2.4). Switching to the `/2/e/` protobuf topic would recover `priority` directly and is logged as the deferred precision upgrade.

D-008: The firmware is editable this term (supersedes the "frozen / read-only" framing)

- **Status**: ✅ **Resolved and closed** (2026-09-16, Session 6). All three sub-questions answered; no part of this decision remains open.

- **Context**: `README.md` §1, charter §2, and D-004's rationale all describe `treklink-firmware` as *"Frozen / read-only this term. Branch-protected. Not a deliverable."* Sessions 1–3 reasoned under that constraint and, on its strength, classified several firmware-rooted defects as unfixable-by-design. **The team lead has confirmed this is wrong**: the team owns the repo, can modify and build it, and can reflash the physical units.
- **Decision**: firmware modification is **technically available** and may be used to fix defects that cannot be worked around from the platform side. All prior "unfixable without touching frozen firmware" conclusions are void and must be re-derived.
- **The distinction that still matters**, three separate claims were bundled under one word:
  1. *Can we edit it?*, **Yes.** Settled.
  2. *Is firmware work in scope?*, **Yes** (answered 2026-09-13, supervisor). Firmware **enhancement and integration** is an accepted development path, not the out-of-scope "firmware redesign." Extending the firmware so the platform integrates with it more deeply, a custom PortNum, an explicit SOS discriminator, a boot `sessionId` + per-packet `sequenceNumber`, beacon priority, is treated as building the product, not rebuilding the inherited component. The charter's exclusion is read narrowly, as prohibiting a rearchitecture of the mesh stack.
  3. *Does firmware effort count toward graded deliverables?*, **Yes** (answered 2026-09-13, supervisor), with the caveat that assessment is not code volume alone. This supersedes D-004's rationale citing "the register's note that firmware effort must be excluded from statistics"; that note is treated as an accounting convention for source statistics, not as a bar on credit.
- **Working rule (Session 6)**: firmware changes are a **first-class option**, evaluated on merit against the platform-side alternative rather than treated as a last resort. Prefer whichever is more robust for the same cost; where the firmware fix is strictly better, as with the SOS discriminator and beacon priority, take it. Keep each change individually justified and logged here. Still excluded: rearchitecting the Meshtastic mesh stack.
- **Reopens**:
  - **D-006**, a firmware-side boot-`sessionId` + per-packet `sequenceNumber` would make the charter's original `eventId` formula constructible. The split key stays valid regardless; this would be a layered upgrade.
  - **The Critical SOS risk below**, the single-unacknowledged-text-frame failure is now firmware-fixable (custom PortNum, `want_ack`, or retransmitting the discriminator with each beacon) rather than mitigable only by heuristic.
  - **D-007's SOS discrimination**, a real custom PortNum would retire the `printf`-format string parser entirely.
  - **v1 MQTT exclusion**, a one-line build-flag removal, though v1 is now out of the demo set anyway (see D-005).
  - **New (Session 4), SOS beacon priority.** `TrekLinkSOSHelper::broadcastPosition()` retransmits via `PositionModule::sendOurPosition()`, which sets `priority = BACKGROUND` for a handheld role (`PositionModule.cpp:377–380`), only the very first position packet at trigger time is `MAX`. Every beacon for the rest of an episode is the *lowest*-priority traffic on the mesh. Candidate fix: have the beacon path set `priority = MAX` (or at least `RELIABLE`) explicitly instead of delegating to the generic broadcast method. See `04-firmware-ground-truth.md` §2.
- **Action** (owner: team lead): none outstanding, both questions answered. Continue to log every firmware change in this register, and carry the firmware work in the same backlog and Progress Log as platform work so the contribution evidence is uniform.

D-009: Git, tracking & review model v2 (supersedes D-003)

- **Status**: ✅ **Resolved** (2026-09-13, Session 5)
- **Context**: D-003 ratified the school's `Git_Lab_Guide.pdf` model wholesale. Auditing it against reality found it was never actually implemented and could not be: **no repository has ever had a `develop` branch**, yet `develop` appeared in 34 documentation files, making every documented git command broken. Three documents disagreed with each other on the branch model (`07-*.md` §3.2 and D-003 mandated dual Design/Implementation branches; `09-*.md` §1 said they were "not adopted"). The GitHub-Issues-as-tracker assumption conflicted with the team's actual use of Jira.
- **Options considered**:
  1. **Fix reality to match the docs**, create `develop`, adopt dual branches, drop Jira. Rejected: dual branches double PR overhead for a 5-person/13-week term, and the team already runs Jira.
  2. **Fix the docs to match reality, keeping GitLab's philosophy**, chosen.

3. Leave the contradiction and rely on tribal knowledge. Rejected: agents read the docs literally and will follow the broken commands.

- **Decision**:
  - Integration branch is **`dev`**. Branches: `main`, `dev`, `feat/*`, `fix/*`, `hotfix/*`, `docs/*`, `chore/*`.
  - **`release/*` dropped**, `main` is the release.
  - **Dual Design/Implementation branches killed.** One branch per unit of work; spec commits land before implementation commits on the same branch, which preserves review independence at half the overhead.
  - Branch naming: `{type}/{JIRA-KEY}-{short-kebab-desc}`, e.g. `feat/TK-45-device-registration`. The Jira key is what makes the org-level Jira↔GitHub integration auto-populate each card's Development panel.
  - Commits: **Conventional Commits**, all standard types, Jira key as scope, `feat(TK-45): ...`. Scope omitted when genuinely general. **Not CI-enforced**: commit hygiene must never block a delivery.
  - Merge: **Rebase & merge** default, **Squash & merge** when multi-commit, **merge commits prohibited**. Delete source branch except `dev` → `main`.
  - **No direct pushes to any branch, by anyone, including the leader.**
  - **Jira is the single work tracker.** GitHub Issues hold daily reports and standalone bugs only. GitHub Milestones retired; `points:*` labels retired in favour of Jira story points on the **Fibonacci** scale (1/2/3/5/8/13/21), corrected 2026-09-17; an earlier revision published a non-standard base-5 scale that the backlog was never estimated against.
  - Bugs route three ways: Jira `BUG` **status** (defect found reviewing an active story) / GitHub Issue `type:bug` (standalone, never enters Jira) / Jira `Bug` **work item** (only if >0.5 day or it changes a spec).
  - CI is an **indicator, not a gate**, red CI does not block merge, but the leader must approve any red merge and the reason goes in the PR.
  - PR self-approval permitted **only** for <50-line, non-behavioural housekeeping. The PO is not exempt and must run an independent AI review pass on their own non-trivial PRs.
  - **English only** in every repository artifact. Vietnamese permitted when prompting an AI agent, and by exception in daily reports.
- **Consequence**: `01-conventions/` rewritten as Conventions v2; `07-*.md` fully rewritten; new chapters `10` (Jira), `11` (AI-first), `12` (communication). Two PR templates collapsed into one. `treklink-web/docs/conventions/` deleted (D-011). The generated Developer Handbook PDF is built from these files.
- **Owner**: Team lead.

D-010: Neon as the shared managed Postgres

- **Status**: ✅ **Resolved** (2026-09-13)
- **Context**: `docker-compose.yml` provisions a local Postgres per developer, which is correct for isolated unit/integration work but gives the team no common database to co-develop against. Integration bugs that only appear with shared data (FK collisions, migration ordering, seed drift) surface late, typically during demo prep.
- **Decision**: Adopt **Neon** as the managed Postgres for shared **dev** and **prod/demo** environments. Local Docker Postgres remains the default for unit tests and CI (fast, hermetic, no network).
- **Consequence**: Each member installs the **Neon MCP connector** from the Claude skills store and authorises it under their own account. Connection strings live in each developer's `ignore/envs/` and are **never committed**. `DATABASE_URL` in CI continues to point at the ephemeral service container, not Neon.
- **Open**: branch-per-developer Neon databases vs. one shared dev database, decide before the first cross-module integration sprint.
  - **Update 2026-10-04**: the Neon project has still never been provisioned, so no shared database holds applied migrations. The migration baseline is therefore rewritten in place for the rescope rather than migrated forward (D-036).
  - **Update 2026-09-29**: settled by `07-clarification-answers.md` §6 question 40, one shared dev database with migrations applied only from `dev`, by CI or the leader. The Neon project is **not provisioned yet**; until it is, every member develops against local Docker Postgres from `.env.example`. The leader distributes the team `.env` privately (Zalo), never through a repository.

- **Owner**: Team lead + LongLP (data lane).

D-011: Single canonical convention set; vendored copies deleted

- **Status**: ✅ **Resolved** (2026-09-13)
- **Context**: `treklink-web/docs/conventions/` held a verbatim, unreconciled copy of the author's generic `dev-flow` template, 12 files with zero TrekLink-specific content, still teaching `develop`, bracket-tag commits, and the dual-branch model. `treklink-web/AGENTS.md` explicitly directed agents to **fall back to it**, so an agent opened on `treklink-web` alone would have been actively taught the wrong rules.
- **Decision**: **Delete it.** `treklink-docs/_docs/01-conventions/` is the single canonical convention set for all three repositories. No repo vendors a second copy. A single `AGENTS.md`, identical in content, is placed in `capstone/`, `treklink-docs/`, `treklink-web/`, and `treklink-firmware/`, and points at the canonical folder.
- **Rationale**: A duplicated convention set does not merely drift, it drifts *silently*, and the copy is what gets read when the sibling layout isn't present. The previous fallback was worse than having no fallback, because a wrong answer delivered confidently costs more than a missing one.
- **Consequence**: Agents must be opened on the `capstone/` parent folder. An agent opened on a single repo will find `AGENTS.md` pointing at a path it cannot resolve, and must say so rather than guessing, which is the intended failure mode.
- **Owner**: Team lead.

D-012: Map provider: Goong Maps over MapLibre GL; OpenStreetMap tiles prohibited

- **Status**: ⛔ **Superseded by D-031** (2026-09-29). Resolved 2026-09-16, Session 6
- **Context**: `frontend/src/widgets/LiveMapWidget/LiveMapWidget.tsx:18-19` renders the standard OpenStreetMap tile server (`https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`). The team lead inspected every OSM base layer and confirmed each renders the Hoàng Sa and Trường Sa archipelagos under Chinese toponyms. Under Vietnamese law that makes the rendered product unlawful to publish.
- **Legal basis**: **Nghị định 174/2026/NĐ-CP, Điều 93 khoản 3 điểm a**, in force **1 July 2026**, fine **30–40 million VND** for *"Đăng, phát, sử dụng hình ảnh bản đồ Việt Nam không thể hiện đầy đủ hoặc thể hiện sai chủ quyền quốc gia."* It raises the 20–30M VND of Decree 15/2020/NĐ-CP Art. 99(3)(b). Ancillary sanctions include confiscation of the equipment used, forced takedown of the content and its links, and **forced removal of the application**. Note the wording penalises *failing to represent fully* as well as *misrepresenting*, omission is sufficient; a nine-dash line is not required for liability. Enforcement against software and devices is established practice (imported-vehicle GPS confiscations, Hải Phòng 2019; 25M VND fine to the Oceanman 2023 organiser under the predecessor article).
- **Options considered**:
  1. **Goong Maps via MapLibre GL**, chosen. Vietnamese provider (IMAP JSC), servers in VN and SG, free tier of $100 standing credit with 30,000 requests/month, 180 req/min and 1,000 map loads.
  2. **VietMap**, rejected on cost. Technically the closest fit (it publishes a documented `@vietmap/vietmap-gl-leaflet` wrapper that would have preserved the Leaflet API and a raster XYZ satellite endpoint), but it is not free.
  3. **Viettel Maps**, rejected on capability. Its public documentation exposes only a Static Map image endpoint, an `<iframe>` embed, Place and Routing. No XYZ tile layer and no GL style are documented, so it cannot back an interactive operations map.
  4. **Self-hosted sovereignty-corrected tiles**, rejected. Tile pipeline and storage cost far exceed a 13-week budget for no graded benefit.
- **Decision**: **Goong Maps rendered through MapLibre GL JS.** This replaces Leaflet as the map rendering library, Goong publishes GL styles, not an XYZ raster layer, so Leaflet is no longer the right host. Style URL form: `https://tiles.goong.io/assets/goong_map_web.json?api_key={MAP_KEY}`; variants `goong_map_web` (full icons), `goong_map_highlight` (minimal icons), `goong_satellite`. Integration is via the `maplibre-gl` CDN bundle, Goong documents no npm package of its own.
- **Two distinct credentials**: a **Map Key** authenticates style/tile URLs; a separate **API Key** authenticates Autocomplete, Direction, Geocoding, Distance Matrix and Place Detail. Do not conflate them.
- **Provider-agnostic by construction** (team lead, Session 6): the map layer must not hardcode Goong. Provider identity, style URL, key and default viewport are configuration, so a future swap to another GL-style provider is a config change and

not a code change. See **D-015**.

- **Consequences**:
  - Leaflet is removed from the stack. Ten documents name it, `01-project-charter.md` (×2), `01-conventions/00-index.md`, `04-architecture-conventions.md`, `06-frontend-conventions.md`, `02-roadmap-and-milestones.md`, `AGENTS.md` (×4 copies), backlog story **US-055** via `build_backlog.py`, and `Documents/reports/Report1_Project_Introduction_DRAFT.md` §6.1, all must be updated in the same pass.
  - The API key is visible in the browser by design. Mitigate with an HTTP-referer allowlist and a per-IP rate limit on the key, and record that mitigation, it is the direct answer to the "API key exposed in source" failure mode in the faculty fault handbook §10.
  - **Acceptance test, mandatory before Review 1**: load the operations map over Hoàng Sa (~16.5°N, 112.0°E) and Trường Sa (~9.7°N, 114.0°E), confirm Vietnamese toponyms and correct sovereignty depiction, and file the screenshots as evidence. This is a team-performed check; it has **not** been carried out yet.
  - **Update 2026-09-25**: `07-clarification-answers.md` question 28 (Session 8, Confirmed) records that the team lead has performed the sovereignty check and found Goong compliant. Whether the screenshots are filed as evidence is not recorded (needs check).
  - **Update 2026-09-25, later**: the team lead confirms no screenshots are filed yet (`07-clarification-answers.md` §5 question 23). The acceptance test's evidence is outstanding.
  - **Update 2026-09-25, evidence filed**: the leader captured Goong Maps over Hoàng Sa and Trường Sa, both labelled in Vietnamese, and OpenStreetMap over the same area, labelled with foreign names and boundaries. Filed in `capstone/Documents/evidence/map-sovereignty/`. TC-24 passes for Goong; OpenStreetMap stays prohibited.
- **Owner**: Team lead (schema/UI layout lane) + LongNN (frontend lane).

D-013: Execution model: SEP490 2026 mainflow-incremental, retaining SWP490 practice

- **Status**: ✅ **Resolved** (2026-09-16, Session 6)
- **Context**: `02-roadmap-and-milestones.md` was derived from `SWP490_Lo-trinh-Capstone_v1.0.pdf`. The Fall 2026 course issues `SEP490_Student_Project_Execution_Schedule.pdf` (50pp) and `SEP490_Huong_dan_nhanh_cho_sinh_vien.pdf`, which describe a materially different model: gates at W4/W8/W13 plus final submission at W15, four Common Meetings (W1, W3, W7, W12), a weekly Group Meeting, and **incremental delivery per Mainflow** rather than by TP package or Iteration.
- **Decision**: the **SEP490 schedule is the spine**. TP1–TP6 and the Iteration 1/2/3 framing are demoted to a historical appendix and mapped onto mainflow ownership; they no longer drive planning. Practices from SWP490 that the SEP490 documents do not contradict are **retained deliberately** because they are good practice, the phase discipline, the deliverable ledger shape, and the Week-6 scope lock (which SEP490 does not mention but which remains real).
- **The dual council is real but informal** (team lead, Session 6): "Hội đồng 1.1 / 1.2" is the *Hội đồng kín* naming, never published as an official document and conveyed only through the supervisor's outline. It maps cleanly onto the SEP490 calendar and is **not** a separate pair of events: **Hội đồng 1.1 = Faculty Council, W13** (Gate 3, defence eligibility); **Hội đồng 1.2 = Final Submission & Defense, W15**. Record it in internal planning; cite only the SEP490 names in graded documents.
- **Retake path**: failure at either council moves the group to a council in the following term, ~March 2027. This is why the registration form's stated duration reads 09/2026 – 03/2027 while the working plan ends in December 2026. **Plan for December.**
- **Known contradictions inside the SEP490 source**, resolved as follows and flagged wherever cited:
  - Review 1 at **W4** (Roadmap table, Master Schedule 5.2, quick guide) vs W3 (§4 prose item 2) → **W4** adopted, two sources to one.
  - "Two mainflows complete at W7" (§4, Master Schedule 5.1) vs "Sprint 1 (W8) → MF-01, MF-02" (Sprint Plan) → the **stricter W7** reading adopted; W8 is the confirmation demo.
- **Owner**: Team lead.

D-014: Tracking artifacts: the SEP490 workbook and Progress Log

- **Status**: ✅ **Resolved** (2026-09-16, Session 6)
- **Context**: SEP490 §§27–37 name ten tracking artifacts and recommend consolidating them into one 8-sheet workbook. `Documents/tracking/Report3_Project Tracking.xlsx` (WBS / Issues / Defects / Q&A) is the 2021 artifact and covers a strict subset. The course also ships `templates/Mã nhóm_Progress_Log.xlsx`, named as **the** primary evidence for Individual Contribution, a dimension on which an individual member can fail even when the group passes.
- **Decision**: adopt both.
  1. **`GFA26SE55_Progress_Log.xlsx`**, one file for the whole term, from the course template. Updated **before every Group Meeting**. Per-member, per-week, tied to a named deliverable; "continued working on the project" is not an entry.
  2. **An 8-sheet tracking workbook**, `01_MASTER_SCHEDULE`, `02_WEEKLY_TASK_TRACKER`, `03_DELIVERABLE_TRACKER`, `04_TRACEABILITY_MATRIX`, `05_REVIEW_ACTION_TRACKER`, `06_RISK_ISSUE_CONFIG`, `07_FINAL_READINESS_CHECKLIST`, `08_INDIVIDUAL_CONTRIBUTION_LOG`.
  3. The **Mainflow Coverage Matrix** is the single sprint-tracking instrument the supervisor uses to confirm progress in the weekly Group Meeting. It is authoritative over any other view.
- **Risk register interface**: the Risk & Issue Log sheet adopts **the school's columns** (`ID · Risk/Issue · Impact · Probability · Mitigation/Action · Owner · Deadline · Status`) rather than a composite exposure score. The register below stays the engineering source of truth; the workbook sheet is its reporting projection and must be regenerated from it, never edited apart.
- **Consequence**: `Project Weekly Report_GroupName.xlsx` is retired. The old Project Tracking sheets fold into `02` and `06`.
- **Owner**: Team lead.

D-015: Business parameters are configuration, never constants

- **Status**: ✅ **Resolved** (2026-09-16, Session 6)
- **Context**: the faculty fault handbook ranks hardcoded business parameters the **#2 cause of failure**, and its list of most-asked council questions opens with *"can this number be changed? Demo it for me now."* SEP490's W11 Definition of Done requires "tham số nghiệp vụ cấu hình được" and a maintained **Configuration Matrix**.
- **Values already hardcoded** (found by inspection, Session 6): `gateway/src/queue/priority-queue.ts:50` peek limit `10` · `gateway/src/mqtt/mqtt-client.ts:20` `reconnectPeriod: 3000` · `:41` flush batch `20` · `:49` topic prefix `treklink/events/priority/` · `frontend/src/widgets/LiveMapWidget/LiveMapWidget.tsx:6` `DEFAULT_CENTER`.
- **Values that must be born configurable** (not yet written): SOS beacon cadence (5 s for the first minute, then 30 s), episode-correlation window and backward grace, cadence-anomaly threshold (N positions in window W), P0–P3 tier mapping, gateway→cloud sync target (≤5 s), battery warning thresholds, rental rate, deposit, late fee, damage fee, and the map provider/style/viewport (D-012).
- **Decision**: no business parameter appears as a literal in source. Each is registered in the Configuration Matrix with parameter, current value, location, configurable (Y/N), tested, and a demo path. A parameter that cannot be changed and shown changing during a demo is treated as a defect, not as a style preference.
- **Owner**: Team lead (schema) + each module owner for their own parameters.

D-016: The five Main Flows are supervisor-specified and binding

- **Status**: ✅ **Resolved** (2026-09-16, Session 6). ⛔ **Superseded by D-033 (2026-10-03)**: the five Main Flows are now the D-033 set. The flow identifiers and owners are kept; MF-01, MF-03, MF-04 and MF-05 change substance.
- **Context**: SEP490 organises all delivery from W3 to W12 around Mainflows, and the supervisor supplied the set for this project directly as `Documents/course-material/TrekLink-proposed-mainflow-ducndm.png`. They are not a format suggestion; they are the units the Mainflow Coverage Matrix tracks, the units demoed at each Iteration review, and the units the council evaluates.
- **Decision**: adopt exactly these five, with these identifiers.

MF	Name	Substance
MF-01	Booking → Rental → Trip Preparation	Customer browses package, submits booking, reserves device → Staff reviews, confirms, allocates device, assigns guide, generates rental agreement, checks out device → Guide receives device and prepares
MF-02	Field Data → Offline Gateway → Cloud Sync	Device (SOS/GPS/telemetry) over LoRa mesh → Gateway Bridge with SQLite priority queue while offline → MQTT → NestJS; P0–P3 processing with idempotency; priority-ordered flush on reconnection
MF-03	SOS → Incident → Emergency Response	SOS broadcast → gateway → idempotency check on `eventId` → Incident created if absent → FSM Detected → Acknowledged → In Progress → Resolved → Closed, each transition recording actor, timestamp and note → WebSocket notification to Staff and Guide
MF-04	Real-Time Trip Monitoring	Device → gateway → MQTT → backend → WebSocket → dashboard showing active trips, device positions, battery status, incident alerts, last-seen time, for Admin / Staff / Guide
MF-05	Return → Inspection → Billing → Maintenance	Check-in, return inspection, charge calculation, payment, rental close; device FSM Available → Reserved → Rented → In-Field → Returned → Maintenance → Retired

- **Backlog mapping**: MF-01 ← E3 + E2 + E1 · MF-02 ← E4 · MF-03 ← E5 · MF-04 ← E5 (frontend) · MF-05 ← E6 + E2. **E7 (DevOps) and E8 (Research & Evaluation) are cross-cutting and are deliberately not mainflows**, state this explicitly wherever the Coverage Matrix is presented, so the omission does not read as an oversight.
- **Consequences**:
  - MF-02 makes the offline buffer non-negotiable, see D-005.
  - MF-03 prints "idempotency check (using eventId)" on the supervisor's own diagram, which turns D-006's split key from an internal design note into a **demo obligation**: a replay must be shown producing exactly one Incident.
  - MF-05 and MF-01 together make the 7-state device lifecycle and the rental lifecycle demonstrable end-to-end, which is what the W13 council grades.
- **Owner**: Team lead.

D-017: Diagram and figure conventions; Mermaid pinned at 12.0.0

- **Status**: ✅ **Resolved** (2026-09-17, Session 6)
- **Context**: the Main Flow diagrams were authored as `flowchart` blocks with `subgraph` groupings. That *looks* like a swimlane and is not, the groups do not constrain layout, so the actor partition is decorative. The course supplies actor-partitioned swimlanes as its worked example (`Documents/templates/Main flows_ex02.jpg`) and the supervisor's own TrekLink diagram uses that form. Separately, nothing in the pipeline checked whether a diagram was *legible once printed*: the handbook carried figures that render at 1.58 pt on A4.
- **Decision**:
  1. **Multi-actor process flows are Mermaid `swimlane-beta`**, one `subgraph` per actor lane, orientation `TB`. Plain `flowchart` is reserved for structures with no actors, context diagrams, feature trees, architecture diagrams.
  2. **Mermaid is pinned at 12.0.0.** `swimlane-beta` requires ≥ 11.16.0 and did not exist in the previous pin of 10.9.1. All 12 pre-existing diagrams were re-rendered at 12.0.0 and none regressed.
  3. **Figure placement is computed, not authored.** `plan_figure` in `build_handbook.py` renders each diagram headless, measures its viewBox and smallest rendered label, and chooses inline or a counter-clockwise rotated plate, whichever affords the larger scale. Both axes are always constrained, so nothing clips.
  4. **A 7 pt legibility floor**, with the escape hatch that a figure below it is a note rather than a build failure; the fix is to re-source the diagram, and any carried exception is recorded in `01-conventions/13-diagram-and-figure-conventions.md`.
  5. **Monochrome.** No figure may depend on hue, these documents are printed and photocopied.

6. Every figure carries a number and a caption and is referenced before it appears. Numbering is per source document; the build renumbers continuously across the handbook and rewrites the in-text references.

- **Consequences**:
  - Diagrams are **pre-rendered to inline SVG** before pagination. Previously mermaid rendered in the browser during `--print-to-pdf`, which meant layout could not know a figure's size until after pagination had been decided, computed placement is impossible that way.
  - Rotation happens **inside the SVG coordinate system**, never as a CSS transform: a transformed box paints in one place and paginates in another, which prints content on the wrong page.
  - Three pre-existing figures were below the floor and were re-sourced by splitting: the session lifecycle sequence diagram, the Jira delivery loop, and the AI session workflow. The corpus is now **27 figures, all above the floor**, lowest 7.40 pt.
  - The CI "raw mermaid source" text check was removed. Its markers (`commit id:` , `[*] -->` , `-->|`) now all appear in the conventions prose that documents them, so they false-positive. CI validates figure numbering instead; an unrenderable diagram is already a hard build error.
- **Owner**: Team lead.

D-018: Three-tier buffering topology; the on-device durable queue is inserted as Stage B

- **Status**: ✅ **Resolved** (2026-09-17, Session 7), **refines D-005, does not supersede it.** D-005's staged-rollout principle and its permanent-scope conclusion both stand.
- **Context**: D-005 described two stages, node MQTT with no buffer (A), then a basecamp bridge holding the SQLite queue (B). Session 7 inspection of `src/mqtt/MQTT.{h,cpp}` found that **the node already has a queue**, and that it is actively harmful: 16 entries, RAM-only, strict FIFO, **discard-oldest** on overflow (`MQTT.cpp:821–823`). Because eviction takes the oldest entry, the single `"SOS - …"` text frame that identifies an episode is **the first thing dropped** during an outage, the exact inverse of MF-02's binding priority rule. Full evidence: `04-firmware-ground-truth.md` §4.1.
- **Decision**, three tiers, delivered in this order:

Stage	Where	Buffer	Status
A	Guide's node publishes direct to MQTT over its own Wi-Fi	the stock 16-entry RAM FIFO, i.e. effectively none	**already exists, pure configuration, nothing to build**
B	**On the device (firmware)**, RAM tier, then a flash tier	priority-ordered, reboot-proof, bounded, shed-aware	**built now; this term's focus**
C	Basecamp gateway, node tethered to a laptop or phone	the large central queue, aggregating the whole local mesh	built after B; form settled by **D-020**

- **Naming note, to stop a rename from creating drift**: what documents written before Session 7 call *"Stage B, the basecamp bridge with the SQLite queue"* is **now Stage C**. Stage B is the new on-device work. Every `[B]` -tagged requirement in `gateway-sync/requirements.md` predating this decision refers to Stage C and is retagged in the same pass.
- **Rationale**: Stage B is chosen ahead of Stage C on three independent grounds.
  1. **It fixes a defect rather than adding a layer.** Stage C buffers the gateway→cloud leg; it does nothing about a node that evicts its own SOS 80 seconds into an outage. Those are different failures and only Stage B addresses the second.
  2. **It needs no hardware the team does not already have.** Stage C needs a basecamp machine plus a tether; Stage B needs a node and a broker.
  3. **It is the strongest available answer to the council's individual-contribution question.** A cited before/after on inherited firmware is a materially better claim than new application code.

- **What Stage B does not do, stated so it is never overclaimed**: an on-device queue buffers only *that node's own* events. It cannot buffer for peer nodes that never reach Wi-Fi. Multi-node aggregation is Stage C's job and Stage B is not a substitute for it, the same relationship D-005 established between A and B.
- **Consequences**:
  - RQ1/RQ2 become answerable **this term, against a real baseline**, the unmodified firmware at its current commit (see D-019's preservation requirement, which is what makes the baseline reproducible).
  - The risk row *"Stage A has no offline buffer, so RQ1/RQ2 cannot be evaluated until Stage B lands"* is downgraded: Stage B lands first and is the smaller of the two.
  - `specs/gateway-sync/*` gains Stage-B requirements; `treklink-firmware/specs/onboard-queue/*` is created as the firmware-side spec suite, following the convention that specs live in `specs/{module}/` inside the repo that holds the code (`01-conventions/02-spec-driven-development-workflow.md` §2).
- **Owner**: KhoaDD (MF-02 owner).

D-019: Firmware changes are additive and non-destructive; stock behaviour is preserved and must keep working

- **Status**: ✅ **Resolved** (2026-09-17, Session 7)
- **Context**: D-008 established that the firmware is editable. It did not say *how*. The queue work in D-018 touches `src/mqtt/MQTT.{h,cpp}` , stock Meshtastic files on the critical path of the app-pairing and MQTT-uplink features the product depends on. Overwriting them in place would destroy the measurement baseline RQ1/RQ2 need, and would risk the two integrations the demo cannot lose.
- **Decision**, three binding constraints on every firmware change this term:
  1. **Never delete or overwrite stock logic. Preserve it and branch around it.** The replaced stock path is kept intact and reachable, either behind a compile-time flag defaulting to the TrekLink path or extracted verbatim into a clearly-named function that the new code can call. A reviewer must be able to read what upstream did without consulting git history.
  2. **Stock Meshtastic mobile-app compatibility is a hard requirement.** The phone app pairs over BLE and, with `proxy_to_client_enabled` , carries the MQTT uplink itself (`MQTT.cpp:800` , `:607`). That path must keep working unmodified, it is how a guide with a phone and no laptop reaches the backend at all, and it is Stage C's mobile ingress.
  3. **The stock MQTT endpoint stays the publish target.** TrekLink does not invent a wire format here. Events still leave as `ServiceEnvelope` on `<root>/2/e/<channelId>/<nodeId>` and JSON on `<root>/2/json/...` (`MQTT.cpp:423–430`). The queue changes *which event leaves next and whether it survives a reboot*, never what it looks like on the wire. This keeps `gateway-sync` 's normalizer (D-007) unchanged and keeps the units legible to any stock Meshtastic tooling.
- **Rationale**: constraint 1 is what makes the D-018 baseline reproducible, "unmodified firmware" must be a build the team can still produce on demand, not a git tag nobody can flash. Constraints 2 and 3 protect the two integrations whose loss would cost the demo, for no benefit: nothing about a durable queue requires a new wire format or a forked app protocol.
- **Consequence**: a firmware PR that deletes stock code, changes a published topic shape, or breaks app pairing is rejected on this decision regardless of how well the queue works.
- **Owner**: KhoaDD.

D-020: Basecamp gateway (Stage C) delivery form: browser-first, native desktop as fallback, no iOS

- **Status**: 🟡 **Decided in principle, build deferred** (2026-09-17, Session 7). Stage C is scheduled after Stage B; this records the form so Stage B's interfaces do not foreclose it. **Form superseded by D-033 (2026-10-03)**: Stage C ships as the Field Station, a single bundled executable on the renter's own laptop with a local browser UI.
- **Context**: Stage C needs a machine at basecamp with more storage than a node and an intermittent internet uplink. Three delivery forms were considered, against a guide who is not a technician.
- **Options considered**:
  1. **Purely web-based**, the guide opens a tab. The page reaches the node over **Web Serial** (tethered laptop) or **Web Bluetooth** (phone or laptop), holds the queue in browser-local storage (IndexedDB), and forwards upstream when the

network returns. No install, cross-platform by construction, one artifact to maintain.

2. **Native cross-platform desktop app** (Windows/Linux/macOS), or the `server.sh` / `server.bat` one-click equivalent over the existing `gateway/src`. Full serial access, no browser-permission friction, real filesystem storage. Three build targets and a signing/distribution problem.
3. **A second Android app running alongside Meshtastic**, rejected. iOS is unviable without macOS/Xcode (already established in D-005), so this splits guide behaviour by platform, and it duplicates the pairing the stock app already does.

- **Decision**: **Option 1 is the target form**, with Option 2 retained as the documented fallback for any venue where browser device APIs are unavailable. Option 3 stays rejected, consistent with D-005.
- **Rationale**: Option 1's install cost is zero, which is the only property a non-technical guide actually experiences. `gateway/src` is not wasted under it, the queue policy, priority tiers and flush ordering are the substance and port directly; only the transport and storage adapters differ, which is the same ingress-adapter seam D-007 already defines.
- ⚠️ **Known risks, accepted and carried** (see the risk table):
  - **Web Serial and Web Bluetooth are Chromium-only**, no Safari, no Firefox. On iOS this means *no browser at all* can reach the node, so an iPhone guide is served only by the stock Meshtastic app path (D-019 constraint 2), not by Stage C.
  - **A tab is a fragile host for a durable queue.** Backgrounded tabs get throttled, and a closed tab stops forwarding. IndexedDB survives the close, so nothing is lost, but delivery stalls until the tab is reopened. This must be surfaced in the UI and stated honestly at the council, not presented as an always-on service.
  - **WebRTC is not the mechanism.** It was raised as a candidate; it addresses peer-to-peer transport, not device access or storage, and is not required by Option 1. Recorded here so it is not reintroduced as an assumption.
- **Owner**: KhoaDD; revisit at Stage C kick-off.

D-021: Encryption: provision a custom channel PSK; keep the MQTT hop plaintext behind TLS

- **Status**: ✅ **Resolved** (2026-09-17, Session 7)
- **Context**: "turn on end-to-end encryption" was raised as a requirement. Inspection shows the premise needs splitting, because two independent switches were being treated as one.
  - **LoRa channel PSK.** `Channels::initDefaultChannel()` (`Channels.cpp:128–134`) already sets `psk.bytes[0] = 1`, which `:238` documents as *"no change vs defaultPSK"*. So RF traffic **is** AES-encrypted out of the box, with **Meshtastic's published default key**. Any stock client in range decrypts it, SOS positions included. Encryption is present; confidentiality is not.
  - **`moduleConfig.mqtt.encryption_enabled`.** Controls only whether the packet handed to the broker stays encrypted. D-007 froze it `false` so the backend receives decoded payloads.
- **Decision**:
  1. **Provision a custom per-fleet PSK on the primary channel**, replacing the default key, as part of device provisioning in MF-01. This is the change that delivers actual confidentiality on the RF hop, and it is configuration, not code.
  2. **`encryption_enabled` stays `false`** (D-007 unchanged). The gateway→cloud hop is secured by **TLS plus a per-gateway API key** instead.
- **Rationale**: setting `encryption_enabled = true` would force reimplementing Meshtastic's AES-CTR channel crypto inside NestJS *and* would eliminate the JSON topic entirely (`MeshPacketSerializer` needs a decoded payload), taking out the whole Stage A ingress path, to protect a hop that TLS already protects. The real exposure was never the broker hop; it was the well-known RF key, which item 1 closes for the cost of a provisioning step.
- **Consequences**:
  - Device provisioning acquires a new mandatory step, and the PSK becomes a managed secret, never committed, never printed in a graded document, distributed per D-015 as configuration.
  - A custom PSK means a **stock Meshtastic app must be given the channel** to pair usefully. This does not violate D-019 constraint 2, the app path still works, it just needs the channel QR like any Meshtastic channel.

    - Losing the PSK bricks the fleet's interoperability until every unit is re-provisioned. Treat it as a recovery-critical secret with an escrowed copy.

- **Owner**: KhoaDD; provisioning step to be written into `specs/devices/`.

D-022: `capstone` is a live auto-sync repository; the no-direct-push rule is scoped to the code repos

- **Status**: ✅ **Resolved** (2026-09-17, Session 7)
- **Context**: `capstone/` was already a local git repository with no remote, tracking `AGENTS.md`, `CLAUDE.md` and `Documents/`, the three sibling repos are gitignored. The team needs graded paperwork (progress log, reports, diagrams, meeting notes) to move between members continuously, edited from Obsidian. That is incompatible with `01-conventions/07-github-workflow-git-conventions.md` §3, which requires every change to arrive by PR and forbids direct pushes to any branch.
- **Options considered**:
    1. Fold `Documents/` into `treklink-docs`, one repo, but every progress-log edit becomes a PR, which defeats the purpose and would make the workbook a merge-conflict generator.
    2. `capstone` with the three siblings as submodules, rejected: detached HEADs and stale checkouts are a known failure mode for a team under deadline, and it solves nothing here.
    3. Give the existing `capstone` repo a remote and let it auto-sync.
- **Decision** (Option 3): `capstone` is published as a **private** repository and runs **auto-commit / auto-push to `main`**. The no-direct-push rule is explicitly **scoped to `treklink-docs`, `treklink-web` and `treklink-firmware`**, which keep full PR discipline.
- **Rationale**: PR review exists to gate code that can break a build or a contract. Meeting notes and a progress log cannot. Applying the same ceremony to both makes the ceremony get skipped for the thing that actually needs it. Separating them keeps the gate credible where it matters.
- **Scope limits, all three are load-bearing**:
    1. **`capstone` only.** A change to a sibling repo is still a PR, always.
    2. **`_docs/` stays read-only in the vault.** `treklink-docs` is a nested repository that `capstone/.gitignore` excludes, so Obsidian can read and link conventions while edits to them still go through a PR. This is a feature, not a limitation.
    3. **History stays linear.** `pull.rebase = true` is set in the repo config; merge commits remain prohibited everywhere (§3).
- **Consequences**: the repo is private, because `Documents/` holds supervisor meeting notes and course-issued PDFs. Auto-commit intervals must exceed a typical agent session, or in-progress edits get committed mid-work.
- **Owner**: KhoaDD.

D-023: A Main Flow owner is not the assignee of every story in that flow

- **Status**: ✅ **Resolved** (2026-09-17, Session 7)
- **Context**: raised by LongLP, the charter (`01-project-charter.md:14`) and roadmap (`02-roadmap-and-milestones.md:164`, `:249`) name **LongNN** as the frontend/FSD, map-and-monitoring-UI member and **MF-04 owner**, while the generated backlog assigned every `module:monitoring` story to TanNB. Verified by inspection, and the report was correct on MF-04 and over-generalised beyond it.
- **What the audit actually found**:
    - LongNN held **zero** monitoring and **zero** frontend stories, his nine were devices ×4, trips ×3, auth ×1, billing ×1. He did not own `TK-63`, the MapLibre/Goong live map, despite being the designated map member.
    - Load was skewed: TanNB 28 stories across nine modules, LongNN 9.
    - Cross-tabulating all 75 MF-tagged stories showed 66 "mismatches" *if* one assumes the flow owner owns every story in it. **That assumption is wrong**, MF-01's stories are spread across all five members by design.
    - **MF-04 was the only flow whose named owner held none of its stories.** That is what made it a real defect rather than a naming artifact.

    - It was **not a generator bug**: `build_backlog.py` documented a deliberate Session-2 rearrangement that was never reflected back into the charter.

- **Decision**, two parts:
    1. **The roles are distinct and both are kept.** A **Main Flow owner** (D-016) is accountable for that flow reaching Demo Ready on the Mainflow Coverage Matrix. A **story owner** implements one story. A flow legitimately spans several implementers. Do not reassign a whole flow to its owner.
    2. **MF-04 is corrected by layer, not by flow.** UI stories → **LongNN** (`TK-63` live map, `TK-64` telemetry display, `TK-73` gateway connectivity indicator, exactly the "map and monitoring UI" the charter names). Backend infrastructure stays with **TanNB** (`TK-61` Socket.io gateway, `TK-46` monitoring read API). Session 2's reasoning for placing infrastructure there is unchanged.
- **Rationale**: this satisfies the charter's *skill* basis and the Session-2 risk weighting simultaneously, the frontend member gets the frontend work, the riskiest backend infrastructure stays where it was deliberately put. No charter or roadmap edit is needed, because the charter was already right.
- **Consequence, individual contribution**: the register already carries the risk that a member with thin visible contribution fails individually even when the group passes. LongNN listed as MF-04 owner while another member demos MF-04 is exactly the *"which part did you do?"* failure the faculty handbook describes. This correction is as much about defensible individual evidence as about tidy assignment. New balance: TanNB 25, LongLP 20, Khoa 18, HoangTK 13, LongNN 12.
- **Also corrected in the same pass**: `build_backlog.py` carried an assignment rationale naming a teammate as unreliable. That file is tracked and forms part of the graded documentation set. The rationale is rewritten in terms of layer and track record; person-level judgements belong in `ignore/`, not in a graded repository.
- **Follow-up**: Jira must be re-synced for `TK-63`, `TK-64`, `TK-73` before implementation starts.
- **Owner**: KhoaDD.

D-024: Prose conventions are binding, and Jira keys are scoped to Epics and User Stories

- **Status**: ✅ **Resolved** (2026-09-22, Session 8). Two rulings recorded together because both were issued in the same instruction and both touch `01-conventions/07`.
- **Context**: two separate wording and tracking defects were raised by the leader.
    1. Every tracked document, and every AI chat reply, was using the em dash as a general-purpose clause separator. At volume it reads as machine-generated, it is inconsistent with the graded reports' register, and it hides the sentence boundary a reader under time pressure needs.
    2. `07-github-workflow-git-conventions.md` §2.2 said a branch without a Jira key was *"rare, prefer creating the card."* Followed literally, that mints a Jira card for every CI fix, tooling change and conventions edit, which pollutes the backlog, the Mainflow Coverage Matrix and the burndown. Those three are exactly what the supervisor reads at the weekly Group Meeting.
- **Decision**, two parts:
    1. **Prose conventions are binding**, published as `01-conventions/14-prose-and-wording.md` (`14-prose-and-wording.md`). No em dash in prose anywhere, no en dash as a clause separator, no banned openers, no aphorism formulas, no two-beat antithesis, no unverified claims, no fabricated precision. Code fences, inline code spans, "not applicable" table cells and verbatim quotations are exempt as data. Every AI assistant loads the `prose-and-wording` skill and `caveman` level `full` at session start, without being asked; chat is compressed, anything persisted to a repository stays normal prose.
    2. **A Jira key belongs to an Epic or a User Story and to nothing else.** A branch and a commit scope carry a key only when the unit of work is one of those cards. Housekeeping, CI, automation, tooling, conventions and non-deliverable documentation carry no key and use a plain descriptive branch. Task and Subtask breakdown is tracked inside the parent story card, not as its own keyed branch. Do not create a card so that a branch can have a key.
- **Rationale**: part 1 is a register decision, and the register of a graded artifact is not cosmetic, because the Faculty Council reads the reports. Part 2 restores the backlog's meaning: a card should represent delivered product scope, so that the Coverage Matrix stays an honest instrument.
- **Consequences**:

    - `07-github-workflow-git-conventions.md` §2.2 and §3.1 are amended in the same pass.
    - `00-index.md` gains row 14.
    - The four canonical `AGENTS.md` copies gain §2.7 and a line in §6.
    - A **mechanical sweep ran on 2026-09-22** across `treklink-docs`, replacing 1230 em dashes in 54 files. It was verified by diff sampling, `git diff --check` for the trailing whitespace a blind join leaves behind, and the new checker, per chapter 14 §6.2. `treklink-web` and `treklink-firmware` are **not** swept; chapter 14 §6 corrects a file when it is next edited for another reason.
    - **`.github/scripts/check_prose.py` and `.github/workflows/prose-check.yml` are built and green.** They enforce the two regex-detectable rules, the em dash in prose and a banned opener at the start of a sentence, on every pull request and push to `dev` and `main`. The rest of chapter 14 stays review-enforced, because a checker that guesses at aphorism formulas or unverified claims produces false positives.
    - **Never swept, and skipped by the checker**: `topics/`, `ignore/`, and the two generated backlog files. Chapter 14 §6.1 carries the list.

- **Incident recorded in the same pass**: the first run of the sweep had an exclusion filter for `topics/` that silently failed. It reworded the **registered capstone project title**, in English and in Vietnamese, in both `topics/Phieu_FA26SE159.md` and `topics/Phieu_dang_ky_de_tai_TrekLink_FA26_FINAL.md`. Caught by sampling the diff, reverted with `git checkout` before any commit, net change zero. Those forms are the version the school holds and the title is frozen until the Week 6 scope lock. The rule that came out of it is `08-ai-agent-steering-and-discipline.md` Stage 2.7: print the file list a sweep will touch and confirm the exclusion held, do not assume the filter worked.
- **Two further standing rules were issued in the same instruction** and are recorded as `08-ai-agent-steering-and-discipline.md` Stages 2.6 and 2.7:
    - **Stage 2.6**: a standing instruction from the leader is written into the conventions in the same pass, routed by kind. An instruction that lives only in a chat transcript is lost at the next session boundary.
    - **Stage 2.7**: correct before polished. A document that is factually right, cites its evidence and contradicts nothing is done, even if its wording is plain. This does not license an unverified claim; Evidence Completeness still outranks Resource Efficiency.
- **Owner**: KhoaDD.

D-025: `dev` reaches `main` by fast-forward push, and the handbook PDF is committed on the PR branch

- **Status**: ✅ **Resolved** (2026-09-25). Leader-approved.
- **Context**: `main` and `dev` diverged in `TrekLink-Team.github.io` and in this repository, although neither held content the other lacked. Three mechanisms caused it. (1) GitHub's "Rebase and merge" re-creates each commit with a new SHA, even when a fast-forward is possible, so after landing landing-site PR #12 `main` and `dev` each held 9 commits the other lacked, with byte-identical trees. (2) "Squash and merge" of a `dev` → `main` PR creates one new commit on `main`. (3) `.github/workflows/handbook.yml` tried to commit the rebuilt PDF to whichever of `dev` or `main` it ran on, so every push to `main` could add a bot commit that existed on `main` alone. Since branch protection was applied on 2026-09-22 that push has been rejected on both branches (the workflow token cannot bypass protection), so every handbook run on `dev` failed and the committed PDF stopped at `eb033a7`. In this repository `main` carried 6 such commits: resync copies of older `dev` states, a PDF rebuild, and the retired `mermaid-10.9.1.min.js` cache. Every file they changed has a newer version on `dev`.
- **Decision**:
    1. The `dev` → `main` release PR is opened and approved as before, then landed by the leader with `git push origin origin/dev:main` after `git merge-base --is-ancestor origin/main origin/dev` succeeds. Never with the merge button. `07-github-workflow-git-conventions.md` §5.7 carries the command.
    2. `handbook.yml` commits the rebuilt PDF to the head branch of a same-repository pull request, so the PDF reaches `dev` inside the reviewed PR and `main` through the release fast-forward. On pushes to `dev` and `main` it only validates. `_docs/handbook/README.md` describes the flow.
    3. Existing divergence is repaired once per repository. Where `dev` already contains all of `main`'s content, `main` is moved to `dev`'s tip. Where `main` holds content `dev` lacks, `dev`'s own commits are replayed onto `main` and `dev`

is pointed at the result. Both need a temporary `allow_force_pushes` on the target branch, restored immediately after a push leased on the old SHA.

- **Rejected**: (c) commit the PDF on pushes to `dev` only. It fails for the same protection reason. (d) Give the workflow a token that bypasses protection. It reopens the direct-push path that protection exists to close. (a) keep releasing with "Rebase and merge" and realign after each release. It needs a force-push on a protected branch every time. (b) Allow merge commits for `dev` → `main` only. It breaks the linear-history rule in §5.7 for the one PR where traceability matters most.
- **Consequences**: a hotfix landed on `main` (§2.4) is the only way `main` gains a commit `dev` lacks, and §2.4 already requires syncing it into `dev`. Everyone with a local `dev` or `main` in a realigned repository runs `git reset --hard origin/<branch>`, announced in Zalo.
- **Owner**: KhoaDD.

D-026: A failure response carries its error code in `result`

- **Status**: ✅ **Resolved** (2026-09-25). Leader-approved (treklink-web C-003 item 2).
- **Decision**: on failure the D-002 envelope keeps its four keys and sets `result` to `{ "errorCode": "..." }`, a stable machine-readable code such as `DEVICE_NOT_AVAILABLE`; `message` stays human text.
- **Rejected**: `result: null` with the code prefixed in `message` (parsing text is fragile); a fifth key (breaks D-002).
- **Consequence (2026-09-29, TK-90, `07-clarification-answers.md` §7 questions 1 and 7)**: from TK-90 on, the global exception filter sets `result.errorCode` on every failure. A `DomainException` carries its own code; a Nest `HttpException` maps by status to the catalogue code for 400, 401, 403, 404, 413 and 503, and any other 4xx keeps its status with the generic `CLIENT_ERROR`; an unhandled exception is a 500 with `INTERNAL_ERROR`, which is never used for a client error.
- **Owner**: KhoaDD.

D-027: One-endpoint operation dispatch is limited to `gateway-sync`

- **Status**: ✅ **Resolved** (2026-09-25). Leader-approved (C-003 item 3).
- **Decision**: `gateway-sync` keeps its single `POST /api/gateway-sync` route with an `op` discriminator (Session 7). Every other module uses REST routes as in `05-backend-conventions.md` §5.
- **Rejected**: reverting `gateway-sync` to REST (its design already relies on the dispatch); extending dispatch to all modules (loses per-route policy and HTTP caching).
- **Owner**: KhoaDD.

D-028: A cross-cutting `platform` module

- **Status**: ✅ **Resolved** (2026-09-25). Leader-approved (C-003 item 1).
- **Decision**: `backend/src/modules/platform/` and `specs/platform/` hold the response envelope, configuration, runtime parameters (UC-19), the audit sink and viewer (UC-20), the scheduler and health.
- **Rejected**: folding them into `auth`, which would make every module import `auth` for reasons unrelated to identity.
- **Owner**: KhoaDD.

D-029: Backlog Jira keys follow the keys Jira issued, offset by five

- **Status**: ✅ **Resolved** (2026-09-25).
- **Context**: the first `jira_sync.py --apply` run found `TK-1` to `TK-5` already in the project. The epics were issued `TK-6` to `TK-13` and the stories `TK-14` to `TK-103`, five above every key the backlog had predicted.
- **Decision**: `build_backlog.py` sets `JIRA_EPIC_BASE = 6` and `JIRA_STORY_BASE = 14`, and the backlog is regenerated. Every `TK-nn` cited before this entry, for example `TK-63`, `TK-64` and `TK-73` in the MF-04 correction above, refers to the old prediction; add five to find the live card. `TK-45` in branch and commit examples is illustrative and unchanged.
- **Rejected**: deleting and recreating the cards to reach `TK-1`, because Jira never reuses an issue number.

- **Owner**: KhoaDD.

D-030: The module specs govern the device, incident and rental state machines

- **Status**: ✅ **Resolved** (2026-09-26). **Superseded by D-034 and D-035 (2026-10-03)**: the booking, trip and rental machines it reconciled no longer exist after the D-033 rescope.
- **Context**: the Report 4 SDD draft (capstone PR 1) follows `treklink-web` specs and `schema.prisma`, which disagree with the SRS state machines drawn from `07-clarification-answers.md` §5. Incident: §5 question 6 reopens `Resolved` to `In Progress` and dismisses only from `Detected`; `specs/incidents/design.md` §2 reopens to `DETECTED` and dismisses from `DETECTED` or `ACKNOWLEDGED`. Rental: §5 question 27 has `Created`, `Active`, `Returned`, `Closed`, `Escalated`; `specs/rentals/design.md` has `DRAFT`, `READY`, `CHECKED_OUT`, `OVERDUE`, `RETURNED`, `CLOSED`, `CANCELLED`. Device: SRS Figure 22 failed the handover check from `Reserved`, while §6 question 12 already made `specs/devices/design.md` §2.1 authoritative (`RENTED` to `MAINTENANCE`). Trip: the same eight states under different names (§5 question 26 `On Prepare`, `On Booking`, `On Start`, `Ongoing`; the schema `PREPARING`, `BOOKING_OPEN`, `READY`, `IN_PROGRESS`).
- **Decision**: the specs and the schema govern the device, incident and rental machines. The SRS is amended to match (Record of Changes 2026-09-26): rental `Overdue` replaces `Escalated` and is entered at the return time plus the late-fee grace; a device still out after `rentals.nonReturnGraceDays` is flagged loss-suspected (§6 question 14). §5 questions 6 and 27 are superseded. Trip names stay as they are on both sides: the SRS keeps the display names of §5 question 26, and the SDD maps each to its stored value. Booking names are unchanged by this entry; §6 question 23 still governs them.
- **Rejected**: renaming the specs and the schema to the §5 answers, because the seven-state rental covers cancellation before check-out and the late state that merged code already uses, and it would need a forward migration of three enums. Renaming the trip enum, because a mapping costs no code.
- **Owner**: KhoaDD.

D-031: The map reverts to Leaflet over OpenStreetMap, with a Vietnamese sovereignty overlay (supersedes D-012)

- **Status**: ✅ **Resolved** (2026-09-29).
- **Context**: D-012 moved the map to Goong Maps over MapLibre GL because every OpenStreetMap base layer labels Hoàng Sa and Trường Sa with foreign toponyms; the leader's screenshots in `capstone/Documents/evidence/map-sovereignty/` confirm it. The leader asked the supervisor, who accepts OpenStreetMap for this university project as an educational use.
- **Decision**: Leaflet renders OpenStreetMap raster tiles. Goong stays dormant: its configuration keys and the D-012 integration notes are kept, no code path calls Goong, and no Goong key is required. A small Leaflet overlay labels Hoàng Sa and Trường Sa in Vietnamese as Vietnamese territory, because Nghị định 174/2026/NĐ-CP Art. 93(3)(a) penalises failing to depict sovereignty fully as well as depicting it wrongly. Provider, tile URL, attribution and default viewport stay configuration (**D-015**), so a return to Goong is a configuration and renderer change.
- **Residual risk**: OpenStreetMap's own foreign labels still render beneath the overlay. The supervisor's acceptance covers the course; it does not change the decree. R11 is reopened as an accepted risk within educational scope: nothing built on OpenStreetMap tiles is deployed publicly beyond the course, and any such deployment first switches the provider back to Goong and re-runs TC-24.
- **OpenStreetMap tile usage policy** (https://operations.osmfoundation.org/policies/tiles/): attribution visible on the map, a named User-Agent and a valid Referer, no cache bypass, and no bulk prefetch. The last rule means OpenStreetMap tiles cannot be cached for offline field use by the Basecamp gateway (D-020).
- **Rejected**: keeping Goong as the renderer (leader decision); OpenStreetMap without the overlay, because it leaves the omission clause open for the cost of one layer; a viewport that excludes the archipelagos, because the user can still pan to them.
- **Consequences**: D-012 is superseded. The conventions, charter, roadmap, Main Flows, requirements foundation and the canonical `AGENTS.md` are updated in this pass. Queued: the graded reports and their generators in `capstone`, and the `treklink-web` specs (`frontend`, `monitoring`, `platform`) and code (`shared/config/map.ts`,

`widgets/LiveMapWidget`, the `maplibre-gl` dependency), which go to the frontend lane as a spec change first. TC-24 now tests the overlay.

- **Owner**: KhoaDD; LongNN (frontend lane).

D-032: The health endpoint reads MQTT state through an optional probe that `gateway-sync` registers

- **Status**: ✅ **Resolved** (2026-09-29). Leader-approved in TrekLink-Team/treklink-web#15, recorded in `07-clarification-answers.md` §7 questions 2 and 9.
- **Context**: `specs/platform/api-design/01-get-health.md` reports MQTT broker reachability by calling `MqttIngressAdapter.isConnected()` (REQ-EVT-04, US-077). `specs/gateway-sync/design.md` §2.1 defines `MeshIngressAdapter` with only `path`, `start()` and `stop()`, and the adapter (gateway-sync task 5.1) is not built. `platform` depends on no other module (`treklink-web` `specs/platform/requirements.md:28`, "Depends on: nothing"; the module itself is D-028), so the health service cannot import `gateway-sync`.
- **Decision**: `platform` defines an optional `MqttHealthProbe` interface, `isConnected(): boolean`, and the injection token `MQTT_HEALTH_PROBE`. The health service injects it as optional. While no implementation is registered, `components.mqtt` is `unknown` and does not change `status`. A registered probe reporting a lost connection makes `status` `degraded`. `MqttJsonIngressAdapter` implements the probe and registers the token when `gateway-sync` builds it.
- **Rejected**: omitting `mqtt` from the response until `gateway-sync` Phase 5, because the response shape would change later and clients would code against both shapes; a broker connection owned by `platform`, because it duplicates the ingress connection and reports the reachability of a second client, not of the path events actually arrive on.
- **Consequences**: TK-90 amends `api-design/01-get-health.md` (the `mqtt` value set and the `status` rule), `specs/platform/requirements.md` REQ-EVT-04 and `specs/gateway-sync/design.md` §2.1.
- **Owner**: KhoaDD; LongLP (TK-90).

D-033: TrekLink is an enterprise device-rental platform built on asset management (rescope after Review 1)

- **Status**: ✅ **Resolved by the leader** (2026-10-03). Supervisor sign-off of the adjusted registration form is pending (co-supervisor Bùi Văn Phúc, 2026-10-04; then Đặng Ngọc Minh Đức). Supersedes the booking scope of **D-016**, the Stage C form of **D-020** and the state machines of **D-030**.
- **Context**: at Review 1 (2026-10-01) the reviewers held that the registered title, "Smart Device Rental Management **Platform for Trekking Agencies**", contradicts the presented scope, which worked like one tour agency taking customer bookings. Report 3 SRS §1 called the product "a single-tenant operations system for one trekking agency". The registered form body (`treklink-docs/topics/Phieu_FA26SE159.md:111-153`, `:285`, `:305`) and the supervisor's flow image (`Documents/course-material/TrekLink-proposed-mainflow-ducndm.png`, MF-1 "Booking → Rental → Trip Preparation") both carry the booking flow, so the contradiction sits inside the registration, between its title and its body. Reviewer Lâm Hữu Khánh Phương asked for an IoT solution provider renting equipment with a management platform by package and monthly payment; the Review 1 minutes listed multi-tenancy as out of scope. At the 2026-10-02 meeting the supervisor ruled that the title cannot change, that the registration form may be adjusted one last time with changes highlighted, and that the product must be a platform, renting to a single company being a system (`capstone/Documents/meetings/2026-10-02_Rescope_Meeting_and_Review_Guidance_Summary.md`). The course defines a platform as one whose services come partly from the operator and partly from partners.
- **Decision**:
  1. **TrekLink is the sole provider** of the devices, the firmware, the Field Station software and the cloud platform, and it is the **asset manager** of every device. There are no third-party device providers.
  2. **Renters are corporate organizations only.** An organization self-registers, is verified in person or by phone at TrekLink, signs a master contract and is approved by a TrekLink Admin. Many organizations use one platform and each manages its own members, on-duty roster, rented devices, alerts and API keys: that is the platform. **The partners' part stays outside the system**: each organization's trips, customers and emergency response are its own business and are not modelled. Inside the system it has only what alert routing needs.
  3. **No individual renters, no bookings, no trips, no trek packages.** The website sells rental plans, nothing else.

4. **Plans**, priced per device by hardware variant, with a minimum order quantity per contract that TrekLink sets:
    - **Monthly plan**: the term is one month and renews until the organization gives notice during a running term. Each term opens with a holding fee of half the term fee, which is part of the fee and not refundable, and closes with the other half. Mid-contract, a month boundary therefore carries a full term fee; the final term ends with only the second half.
    - **Day plan**: a fixed length from a configurable list (3, 4, 7 days); the per-day rate is the monthly rate divided by 30 times a premium of at least 1, so a day plan always costs more per day. Paid in full at handover. Never renewed or extended.
    - **No change once a contract is running**: no added or removed devices, no variant change. An organization that needs more opens an additional contract. A device never passes from one contract to the next without being returned and inspected.
    - **Charges outside the plan**: late return per device per day after a grace period; damage from inspection, with charges above a threshold approved by a TrekLink Admin; loss at the device's remaining value.
5. **Handover and return happen in person at the TrekLink counter**, with identity, signature and payment checked there. No shipping.
6. **Monitoring is always on** for every rented device; it is what the organization rents. Alerts go first to the organization's on-duty member, then its backup member, then its Manager; if nobody acknowledges, TrekLink logs it and reports to the authorities (**D-034**). TrekLink never coordinates a rescue.
7. **Live telemetry reaches the organization** on the TrekLink web platform and through a REST plus WebSocket API with one key per organization, on every plan.
8. **Field side**: device holders use the stock Meshtastic app only and need no TrekLink account. The organization runs the **Field Station**, a single bundled executable on its own laptop, with one of its rented nodes on a USB serial port and a local browser UI (for example `localhost:4321`). It shows everything locally while offline, queues events durably, and uploads when the laptop has internet. Organization operators are trained from the TrekLink manuals and handbook before deployment. The Field Station is Stage C of **D-018**.
9. **Payments** are recorded in the platform. SePay (VietQR with a confirming webhook, sandbox environment) is the third-party payment gateway; cash and bank transfer at the counter are recorded by Staff.
10. **The five Main Flows**, numbered so that each owner keeps their flow:

MF	Name	Owner
MF-01	Organization onboarding → Rental contract → Handover	TanNB
MF-02	Field data → Field Station → Cloud synchronization	KhoaDD
MF-03	SOS → Tiered alert → Escalation	HoangTK
MF-04	Live telemetry: web map and organization API	LongNN
MF-05	Term end → Return → Inspection → Billing → Maintenance	LongLP

    Device intake and provisioning are supporting use cases that feed MF-01 and close MF-05's loop.

- **Rejected**: (a) keep the single-agency booking system and argue the title; the supervisor ruled the title fixed and the reviewers rejected the scope. (b) Third-party device providers listing fleets on the platform, the supervisor's reading of "platform"; it would reduce TrekLink's own hardware, firmware and gateway work to one listing among many and invite the reviewer's "what is unique here" objection. (c) Selling device packages with a maintenance fee; it drops "Rental Management" from a fixed title. (d) Individual renters; they force TrekLink to coordinate rescues and add a second payment model, and the title names trekking agencies. (e) Organization-created groups or trips inside the platform (create a group, add people and devices, pair them, export), raised by the supervisor at Review 1 (`ignore/khoa/docs/rescope/review-1-panel-transcript.md`); the leader ruled on 2026-10-04 that it contradicts the

supervisor's own "device management platform, not a trekking management platform" in the same review. A device carries a holder label for alert routing; trips stay in the organization's own systems.

- **Consequences**: Report 1 background and vision, Report 2 scope, Report 3 SRS, Report 4 SDD, `05-main-flows.md`, `06-requirements-foundation.md`, the adjusted registration form (`treklink-docs/topics/`), the backlog, and the `treklink-web` specs and schema for `rentals`, `trips`, `billing`, `auth`, `incidents`, `monitoring` and `frontend`. Multi-tenancy moves from out of scope to in scope; Review 2 must say so against the Review 1 minutes.
- **Owner**: KhoaDD.

D-034: Incident lifecycle with tiered notification and escalation

- **Status**: ✅ **Resolved by the leader** (2026-10-03; the leader delegated the design to the orchestrator with "multiple states, transient and atomic, a fallback for every edge case"). Replaces the five-state incident machine of the registration form and of `specs/incidents/design.md` §2.
- **States**:

State	Kind	Meaning
`DETECTED`	transient	Created from a confirmed SOS, a fall SOS or a suspected cadence episode; routed inside the same transaction, never observed at rest
`NOTIFY_PRIMARY`	timed	The on-duty member is alerted
`NOTIFY_BACKUP`	timed	The primary did not acknowledge in time; the backup member is alerted
`NOTIFY_MANAGER`	timed	The backup did not acknowledge in time; the organization Manager is alerted
`UNROUTED`	stable	No organization to route to (device not on a running or overdue contract, or the organization has no reachable member); TrekLink Staff own it
`ESCALATED`	stable	Nobody acknowledged, or an acknowledged response went silent too long; TrekLink Staff own it
`REPORTED`	stable	TrekLink Staff recorded a report to the authorities (agency, reference, time)
`ACKNOWLEDGED`	timed	An organization member owns the response
`RESPONDING`	timed	The organization reported a response under way; status updates are due on a cadence
`RESOLVED`	timed	The organization reported the outcome
`FALSE_ALARM`	timed	The organization, or Staff for an unrouted incident, declared it false with a reason
`CLOSED`	terminal	Nothing further is accepted

- **Transitions**:

From	Event	To
(none)	SOS, fall or suspected episode with no open incident for the device	`DETECTED`
`DETECTED`	routed to a running or overdue contract with a reachable primary	`NOTIFY_PRIMARY`
`DETECTED`	no contract, or no reachable member	`UNROUTED`
`NOTIFY_PRIMARY`	acknowledge timeout; or no primary on the roster	`NOTIFY_BACKUP`
`NOTIFY_BACKUP`	acknowledge timeout; or no backup	`NOTIFY_MANAGER`
`NOTIFY_MANAGER`	acknowledge timeout	`ESCALATED`
`NOTIFY_*`, `ESCALATED`, `REPORTED`	a member acknowledges	`ACKNOWLEDGED`
`ACKNOWLEDGED`	the owner reports a response under way	`RESPONDING`
`ACKNOWLEDGED`, `RESPONDING`	no status update within the stale limit	`ESCALATED` (owner kept, Manager notified)
`ACKNOWLEDGED`, `RESPONDING`	the owner is deactivated or leaves the organization	`NOTIFY_BACKUP`
`ACKNOWLEDGED`, `RESPONDING`	the owner reports the outcome	`RESOLVED`
`NOTIFY_*`, `ACKNOWLEDGED`, `RESPONDING`	a member declares a false alarm with a reason	`FALSE_ALARM`
`UNROUTED`, `ESCALATED`	Staff record an authority report	`REPORTED`
`UNROUTED`	Staff declare a false alarm with a reason	`FALSE_ALARM`
`REPORTED`	Staff record that the authority case is closed	`CLOSED`
`RESOLVED`, `FALSE_ALARM`	a new SOS episode from the same device within the reopen window	`NOTIFY_PRIMARY` (reopen count + 1)
`RESOLVED`, `FALSE_ALARM`	the reopen window elapses	`CLOSED`

- **Rules**:

  1. **Atomic.** Each transition is one database transaction: a compare-and-set on the incident's state and version, plus one append-only audit row (actor or `SYSTEM`, from, to, reason, UTC time). A losing concurrent writer reads the current state and the actor who won.
  2. **Idempotent timers.** Timeout jobs re-read state and deadline inside the transaction and do nothing when the incident has moved on, so a retried or duplicated job cannot double-escalate.
  3. **Notification never gates state.** Alerts are written to an outbox in the same transaction and delivered after commit (web, WebSocket, email, API event); a failed delivery is retried and logged, and the state machine continues on its timers.
  4. **Safety first.** An incident is created and routed even when the contract is overdue or the organization is suspended. An episode is one incident (correlation window, `gateway-sync`); ~~a device's cancel gesture is recorded on the incident and prompts the owner to confirm a false alarm~~ (withdrawn by D-038: the device never changes an incident).
  5. **Device silent during an incident**: the incident keeps its state and is flagged stale with the last position; the owner is notified.
  6. **A contract cannot be settled while one of its devices has an incident that is not `CLOSED`.**
  7. Timeouts, the stale limit and the reopen window are business parameters (D-015).

- **Owner**: HoangTK; KhoaDD.

D-035: Device, organization and rental-contract lifecycles

- **Status**: ✅ **Resolved by the leader** (2026-10-03). Replaces the seven-state device machine of the registration form and the booking, trip and rental machines of D-030.
- **Device** (asset): `IN_INTAKE → AVAILABLE` (quality check passed) `→ RESERVED` (allocated to an approved contract) `→ RENTED` (handover signed) `→ RETURNED` (checked in) `→ AVAILABLE` (reset and inspection passed) or `MAINTENANCE` (inspection failed). Also `RESERVED → AVAILABLE` (contract cancelled before handover), `RESERVED → MAINTENANCE` (fails its check at the counter and is swapped, E01-5; edge added 2026-10-04 when the devices spec found it missing), `RENTED → LOST` (reported lost on a `RETURN_DUE`, `OVERDUE` or `DEFAULTED` contract, or unrecovered after default; amended by D-038), `LOST → RETURNED` (recovered), `LOST → RETIRED` (written off), `AVAILABLE → MAINTENANCE` (scheduled service), `MAINTENANCE → AVAILABLE` or `RETIRED`. `IN_FIELD` is dropped: telemetry gives last seen, not trail versus storage. Every device carries two identities registered at intake, a printed asset tag and the radio `nodeNum`, never reused. A returned device is reset (channel key, node database, owner name) before it can be `AVAILABLE` again, and the reset is logged.
- **Organization**: `PENDING` (self-registered) `→ ACTIVE` (verified at the counter, master contract signed, Admin approved) or `REJECTED`; `ACTIVE → SUSPENDED` (a contract defaulted, or Admin action) `→ ACTIVE` (settled); `ACTIVE → CLOSED` (no open contract). A `SUSPENDED` organization cannot open a contract; its running devices stay monitored.
- **Rental contract**: `REQUESTED` (the Manager submits plan, variant, quantity, start date) `→ APPROVED` (Staff confirm stock; devices `RESERVED`) or `REJECTED`; `APPROVED → ACTIVE` (first payment recorded and handover signed at the counter) or `CANCELLED` (before handover; devices released); `ACTIVE → ENDING` (monthly: notice given, ends at the current term end) `→ RETURN_DUE`; a day plan goes `ACTIVE → RETURN_DUE` at its end date; `RETURN_DUE → RETURNED` (every device checked in or recorded lost) `→ CLOSED` (every charge paid, no open incident); `RETURN_DUE → OVERDUE` (grace not yet elapsed, late fees accrue) `→ DEFAULTED` (grace elapsed: the organization is suspended, devices are tracked, Staff report to the authorities with the audit log as evidence) `→ RETURNED` once devices are recovered or recorded lost.
- **Monthly terms** are child records of a monthly contract: each term holds its holding-fee and balance payments and its due dates; a term's balance is due at its end, the next term's holding fee at the same boundary.
- **Owner**: TanNB (contract), LongLP (device, billing), KhoaDD (organization).

D-036: Stale artefacts move to `.deprecated/`; the specs and the schema are rewritten from the SRS

- **Status**: ✅ **Resolved by the leader** (2026-10-04).
- **Context**: after D-033 to D-035 every `treklink-web` spec suite and `schema.prisma` describe the booking scope. The specs index marked them stale (treklink-web PR 26), but stale files left in place are what an agent or a member builds against by mistake, and deleting them loses the reference that the rewrite reuses (gateway-sync, platform). The Neon database was never provisioned (D-010), and only `platform` holds code.
- **Decision**:
  1. **A stale artefact moves, whole, into a `.deprecated/` folder at the root of its own repository, keeping its original path** (`treklink-web/specs/rentals/` becomes `treklink-web/.deprecated/specs/rentals/`). Nothing under `.deprecated/` is built, linked from a live document, or cited as a requirement. Each `.deprecated/README.md` states what moved, when, and why.
  2. **Every `treklink-web` spec suite is rewritten** from Report 3 SRS (2026-10-04) and D-033 to D-035, in the full form: `requirements.md` in EARS traced to SRS FR, UC and BR identifiers, `design.md`, `tasks.md`, and one `api-design/` file per endpoint. `trips` is not rewritten; `organizations` is new.
  3. **The state machines are D-034 (incident) and D-035 (device, organization, rental contract).** Where an old spec or the old schema disagrees, the decision wins.
  4. **The migration baseline is rewritten**, not migrated forward: a new initial migration and seed replace the 2026-09-25 pair, which move to `.deprecated/`. Every member drops and recreates the local database once.
- **Rejected**: deleting the stale suites (loses the reuse source; history is a poor reading surface); a forward migration with drops (protects data that does not exist).

- **Consequences**: unmerged branches built on the old schema (the booking-state rename, `feat/TK-22-admin-manage-user-account`) rebase onto the new baseline. Spec approval per module still goes through its owner's review before implementation (AGENTS.md §2.1).
- **Owner**: KhoaDD.

D-037: Field Station credentials are self-service; one invoice per term; handover notes are signed on screen

- **Status**: ✅ **Resolved by the leader** (2026-10-04), during the spec rewrite of D-036.
- **Decision**:
  1. **Field Station credentials are self-service.** An Org Manager creates, names and revokes the organization's Field Station credentials in the portal, the way API keys work (UC-06); each credential is shown once and stored only as a hash. TrekLink Staff and Admin may revoke one for support but never see it. TrekLink provides the service; how the organization deploys its stations is its own business.
  2. **One invoice per monthly term with two dated lines**: the holding fee due at the term start and the balance due at the term end (FR-BILL-01, FR-BILL-06). Payments apply line by line.
  3. **The handover note is signed on screen** at the counter; the drawn signature is stamped into the generated PDF and the PDF's hash is stored (FR-CON-06).
- **Rejected**: Staff-issued Field Station credentials only (puts routine operations on TrekLink); two invoices per term (contradicts FR-BILL-06); a scanned paper signature (needs document storage and is weaker evidence of what was signed).
- **Owner**: KhoaDD.

D-038: SePay webhook answers SePay's own body; no on-device SOS cancel; a device may be recorded lost from RETURN_DUE

- **Status**: ✅ **Resolved by the leader** (2026-10-04), during the spec rewrite of D-036.
- **Decision**:
  1. **`POST /api/payments/sepay/webhook` is the one exception to D-002.** It answers HTTP 200 with the bare body `{"success": true}`, because SePay treats any other body as a failure and retries up to 7 times over 5 hours (https://docs.sepay.vn/tich-hop-webhooks.html, read 2026-10-04). Every other endpoint keeps the envelope.
  2. **An incident is never ended or changed by the device.** The firmware transmits nothing on cancel (`04-firmware-ground-truth.md` §2, `TrekLinkSOSHelper.cpp:49-53`), and even if it did, a cancel pressed on the device cannot be told apart from one pressed under duress. FR-INC-09 and exception E03-8 are withdrawn from the SRS; D-034 rule 4's sentence on the cancel gesture is withdrawn with them. A false alarm is declared only by a person (FR-INC-08).
  3. **D-035 amended**: Staff may record a device `LOST` on a contract in `RETURN_DUE`, `OVERDUE` or `DEFAULTED` when the organization reports the loss or the device is unrecovered after default. The loss is charged at the device's remaining value (FR-BILL-05) and counts as returned for closing the contract.
- **Rejected**: the envelope on the webhook (relies on undocumented SePay behaviour); a firmware cancel text that only logs (no consumer, and it still invites trusting the device over a person); inferring cancel from beacons stopping (indistinguishable from a dead battery); keeping `LOST` after default only (an honest report would suspend the organization and trigger an authority report).
- **Owner**: KhoaDD.

D-039: Where the platform runs, and how software on the organization's side is updated

- **Status**: 🔴 **OPEN** (raised by the leader, 2026-10-11). Blocks the Report 4 SDD deployment view and any update or release design for the Field Station.
- **Context**: D-033 rule 1 makes TrekLink the sole provider of the devices, the firmware, the Field Station software and the cloud platform; rule 2 has many organizations on one platform; rule 8 puts the Field Station on the organization's own laptop. D-010 hosts the database on Neon. Nothing records whether the backend could also run on an organization's own infrastructure (the self-hosted GitLab model), and nothing records how the Field Station, which runs at the customer end,

receives new versions. That answer decides whether TrekLink has to ship over-the-air updates to customer infrastructure at all.

- **Options**:
  1. **One TrekLink-hosted, multi-organization backend; organizations integrate only through the web platform and their API key.** The only software on the organization's side is the Field Station, which checks a TrekLink release endpoint and updates itself (or is re-downloaded from the portal). Device firmware is updated at the counter during intake, reset or inspection, because every device passes through the counter between contracts (D-033 rules 4 and 5).
  2. **Self-hosted backend option**: an organization may run the TrekLink backend on its own infrastructure, with TrekLink shipping versioned releases and updates to it, alongside the Field Station.
  3. **Hybrid**: hosted by default, with a self-hosted tier for organizations that require data residency.
- **Recommendation (not a decision)**: option 1. Tier 3 of D-034 escalates unacknowledged alerts to TrekLink Staff, which needs every organization's incidents in TrekLink's own backend; the platform definition the supervisor accepted (many organizations on one platform) is a single deployment; options 2 and 3 add release management, version skew between installations and remote support of customer infrastructure inside a capstone schedule. Option 1 keeps the update problem to one artefact, the Field Station executable.
- **Rejected**: none until the leader rules.
- **Owner**: KhoaDD.

Risk register (carried from FA26SE159, kept live)

Risk	Likelihood	Impact	Mitigation	Status
Insufficient physical TrekLink devices	Medium	High	Simulate Gateway→Cloud load via MQTT scripts; simulation never substitutes for LoRa RF reliability claims	Open
Gateway sync latency exceeds 5s NFR	Medium	Medium	Tune MQTT QoS, reduce payload size, backpressure on queue flush	Open
RQ3 baseline not objectively measurable	High	Medium	Randomized simulation drills; independent observer triggers SOS, records timestamp; participants unaware of exact trigger time	Open
NestJS + MQTT + WebSocket integration underestimated	Medium	High	PoC gateway→backend integration in TP1 Week 2, not deferred to TP3	Open
Firmware message schema incompatible with new gateway	Low	High	Schema frozen & documented in TP1 before any gateway implementation	Open, tracked as TP1 exit gate
Scope creep	High	Medium	Feature freeze after TP5 Week 10; anything else goes to post-capstone backlog	Open
~~ORM indecision stalls TP3 start~~	Medium	Medium	D-001 resolved (Prisma) ahead of the Week 4 target	**Mitigated**
[Added] Field connectivity may need phone-based bridging instead of a dedicated Gateway node (D-005)	Medium	High	Validate dedicated Gateway (Wi-Fi/cellular node, native Meshtastic MQTT module) in TP1 PoC before committing to it; mobile-app forking treated as out of scope given no macOS/Xcode access	**Mitigated**, D-005 resolved as a staged rollout (Stage A node-MQTT, Stage B basecamp bridge); app forking formally rejected
[Added] ~~NestJS is a skill only for Khoa~~, **corrected Session 6.** The enrolment framework codes show **LongLP and HoangTK are both `BIT_SE_NJS_18D` (Node.js)**; NestJS is a framework over Node and is treated as covered by that training. Backend capability is therefore **3 of 5**, not 1 of 5. Residual exposure is the two IC-track members (LongNN, TanNB) picking up backend stories solo	Low	Medium	Ramp LongNN and TanNB on `01-conventions/05-backend-conventions.md` module conventions during W2–W5, mentored by Khoa/LongLP. Lane assignment already routes core backend to the two NJS-track members. This is the substance of the Report 2 §2.3 Training Plan	**Downgraded** (Session 6)
[Added S3] SOS is announced by exactly one unacknowledged text packet. `TrekLinkSOSHelper::triggerSOS()` sends the `"SOS - …"` text once with `want_ack = false` (`TrekLinkSOSHelper.cpp:157–160`); `tickBeacon` then retransmits **position only**. If that single text frame is lost over RF, the episode is invisible to the backend,	Medium	**Critical**	**Two layers, per D-008.** *Platform side (built now)*: cadence-anomaly detector, the beacon runs 5s for the first minute then 30s (`TrekLinkSOSHelper.cpp:181`), far denser than routine reporting, so ≥N positions in window W raises a *suspected* episode at lower confidence (REQ-EVT-06, `design.md` §2.4). *Firmware side (now available, preferred)*: retransmit	Open, **firmware fix candidate #1**

Risk	Likelihood	Impact	Mitigation	Status
the beacons that follow look like routine position reports.			the SOS discriminator with each beacon, and/or set `want_ack`, and/or introduce a real custom PortNum so SOS stops depending on a `printf` string. The platform mitigation is defence-in-depth and should be kept even after a firmware fix.	
[Added S3] Stage A (node MQTT, D-005) has no offline buffer, so charter §5's offline-recovery and priority-ordering NFRs and RQ1/RQ2 cannot be evaluated until Stage B lands	High	Medium	**Stage B is permanent scope** (D-005 closed, Session 6), it is the substance of supervisor-specified **MF-02** (D-016), so it cannot be traded away. Residual risk is now schedule, not scope: Stage B must land early enough for RQ1/RQ2 evaluation in W9–W12. Do not present Stage A as satisfying the offline NFR at any point. **Further reduced (Session 7, D-018)**: the buffer work is now split, and the *smaller* half, the on-device queue, is sequenced first and needs no basecamp hardware, so a usable offline buffer exists well before the basecamp gateway (now Stage C) is built.	Open, **schedule risk only**, scope question closed, exposure reduced by D-018
~~**[Added S3]** `treklink_v1_0` compiles MQTT out, so v1 units cannot act as the Stage A uplink node~~	Medium	High	v1 is **out of the demo set** (team lead, Session 3), Stage A targets v2/v3/v4, all of which leave MQTT compiled in and carry Wi-Fi silicon. Were v1 ever needed, removing `-D MESHTASTIC_EXCLUDE_MQTT=1` is a one-line build-flag change now that D-008 permits it.	**Closed**
[Added S3] Stage A's JSON MQTT path omits `MeshPacket.priority`, making the SOS position packet indistinguishable from a routine one (D-007)	High	Medium	Backend episode correlation with a backward grace window (`design.md` §2.4). Two upgrades now available: switch to the `/2/e/` protobuf topic to recover `priority`, or emit an explicit SOS marker firmware-side (D-008).	Open, accepted for Stage A
~~**[Added S3]** `04-architecture-conventions.md` §3 still publishes the old `deviceId:sessionId:sequenceNumber` formula~~	High	Medium	§3 rewritten to the D-006 split key, with the D-008 firmware-sequence option noted as a layered upgrade.	**Closed** (Session 3)
[Added S3] Sessions 1–3 reasoned under a false "firmware is frozen" premise (D-008); several conclusions were shaped by a constraint that does not exist	High	Medium	D-008 logs the correction and lists every decision it reopens. Re-derive the affected conclusions next session before they harden into implementation.	Open, **re-derivation pending**

Risk	Likelihood	Impact	Mitigation	Status
~~**[Added S3]** Firmware changes may earn no graded credit (D-008 question 3)~~	Medium	Medium	Supervisor confirmed 2026-09-13 that **firmware commits do earn credit**, and that firmware enhancement is in scope rather than the excluded "redesign". D-008 closed. Firmware work is planned, assigned and evidenced on the same footing as platform work.	**Closed** (Session 6)
[Added S4] SOS beacon retransmits (all but the very first position packet of an episode) are sent at `BACKGROUND` mesh priority, not `MAX`, `PositionModule::sendOurPosition()` sets priority by device role, and a handheld unit isn't `TRACKER` / `TAK_TRACKER`. Under mesh congestion, an ongoing emergency's beacon trail is currently the least-protected traffic class on the network after its opening packet.	Medium	High	Firmware-fix candidate under D-008: set `priority = MAX` (or `RELIABLE`) explicitly on the beacon send path rather than delegating to the generic broadcast method. Platform-side, the episode-correlation logic in `gateway-sync` already doesn't rely on `MeshPacket.priority` at all, so this doesn't block TP1/TP2, it's a firmware-side robustness gap, not a backend defect.	Open, **firmware fix candidate**
[Added S6] Map tiles rendering foreign toponyms over Hoàng Sa / Trường Sa are unlawful to publish in Vietnam. `LiveMapWidget.tsx:18-19` ships the default OpenStreetMap tile server; the team lead verified every OSM base layer labels the archipelagos with Chinese names. Nghị định 174/2026/NĐ-CP Art. 93(3)(a), in force 1 Jul 2026, fines 30–40M VND and permits forced removal of the application	High	**Critical**	Migrate to **Goong Maps over MapLibre GL** (D-012), provider and style held in configuration so the layer is swappable. Referer-allowlist and rate-limit the browser-visible key. **Acceptance test before Review 1**: load the map over ~16.5°N 112.0°E and ~9.7°N 114.0°E, confirm Vietnamese toponyms, file screenshots as evidence	**Accepted, educational scope** (2026-09-29, D-031): the map reverts to OpenStreetMap under the supervisor's acceptance, with a Vietnamese overlay over both archipelagos. No public deployment on OpenStreetMap tiles beyond the course. Previously mitigated 2026-09-25 by Goong Maps, evidence in `capstone/Documents/evidence/map-sovereignty/`
[Added S6] The registered English title contains "Smart Device Rental Management" but the system has no AI/ML component. The faculty fault handbook devotes a whole section to projects whose name promises "AI / smart / thông minh" and whose demo reduces to CRUD, and states plainly that rule-based systems must be called rule-based	High	Medium	Prepare and rehearse the honest answer: the "smart" is **device-level autonomy**, IMU fall detection, autonomous SOS broadcast, multi-hop mesh routing, priority-ordered store-and-forward, not machine learning. Never use "AI" for the platform in any graded document or slide. Every member must be able to give this answer; it is a near-certain council question	Open, **answer to be drafted and rehearsed before W12 Mock Council**
[Added S6] Business parameters hardcoded as literals. Ranked #2 cause of failure in the faculty fault handbook, and the first entry in its most-asked-questions list is "can	High	High	D-015: every business parameter in configuration, registered in the Configuration Matrix with a demo path. Fix the five existing instances now, before the pattern is copied into	Open, **task queued this session**

Risk	Likelihood	Impact	Mitigation	Status
this number be changed? demo it now." Five instances already exist in a codebase with no business logic written yet			the modules that carry the real parameters (fees, deposits, thresholds, episode windows)	
[Added S6] Individual-contribution evidence. A member with few commits and thin meeting presence fails individually even when the group passes; Git organisation and the Progress Log are the named evidence. Implementation currently stands at zero across all modules, so no member has contribution evidence yet	Medium	High	D-014: Progress Log updated before every Group Meeting, per-member and tied to a named deliverable; `08_INDIVIDUAL_CONTRIBUTION_LOG` maintained from W1. Lane assignment gives every member an owned mainflow so commits distribute by construction	Open
[Added S7] The node's own MQTT queue discards an SOS before it discards telemetry. `mqttQueue` is 16 entries, RAM-only, strict FIFO, and evicts the **oldest** entry on overflow (`MQTT.h:27` , `MQTT.cpp:821–823`). An SOS beacon trail plus routine telemetry fills it in ~80 s of outage, and the single `"SOS -…"` text frame that identifies the episode is the oldest entry, so it goes first. A reboot loses the queue entirely. The eviction is a serial-console `LOG_WARN` , invisible to the platform, so the loss is also unmeasurable.	High	**Critical**	**D-018 Stage B**, an on-device priority-ordered, flash-backed, shed-aware queue, built additively per D-019 and following the `MessageStore` precedent (`04-firmware-ground-truth.md` §4.1). Publish per-tier drop counters so the loss becomes observable and RQ1 gets a device-side denominator.	Open, **this is the term's primary firmware deliverable**
[Added S7] v3 has no PSRAM, and a flash-sized queue bound must not be derived from PSRAM. `treklink-v3-tbeam` is ESP32 LX6 with ~4 MB flash shared with two OTA app slots; v2 (S3 N8R8) and v4 (Supreme S3) both carry 8 MB PSRAM. The existing `StoreForwardModule` already branches `ps_calloc` / `calloc` on PSRAM presence (`StoreForwardModule.cpp:80–85`), so a queue sized the same way would silently collapse to near-zero capacity on v3 while tests pass on v2.	Medium	High	Size the durable tier off **remaining LittleFS space**, per variant, with the bound as a `build_flags` override exactly as `MESSAGE_HISTORY_LIMIT` already is (`MessageStore.h:22–24`). Capacity is a Configuration Matrix entry per D-015. Test the bound on v3 hardware specifically, not only on v2.	Open, **v3 is in the demo set** (D-018)
[Added 2026-09-25] Correction to the S7 row "v3 has no PSRAM". The build contradicts it: `treklink-v3-tbeam` inherits `-DBOARD_HAS_PSRAM -mfix-`	Medium	High	The S7 mitigation stands unchanged: size the durable tier off remaining LittleFS space, never PSRAM. The per-variant PSRAM column in	Open

Risk	Likelihood	Impact	Mitigation	Status
`esp32-psram-cache-issue` from `env:tbeam` (`variants/esp32/tbeam/platformio.ini:19-20`), while `treklink-v2` enables no PSRAM (`treklink_v2_0/platformio.ini:19-24` , board `esp32-s3-devkitc-1` declares none). (unverified) physical PSRAM population on the v2 and v3 units. LittleFS is 1 MiB on v3 and 1.5 MiB on v2/v4 (`04-firmware-ground-truth.md` §5).			`onboard-queue/design.md` §4 is corrected in the firmware repo.	
[Added 2026-09-25] Correction to the S7 row "The node's own MQTT queue discards an SOS". Two facts in it were incomplete. (1) In direct Wi-Fi mode stock firmware drains **one** queued entry per successful reconnect and nothing while the link stays up (`MQTT.cpp:604-636` , `:699-709`), so the baseline is worse than drop-oldest alone. (2) The queue also holds **peer** packets the node hears (`Router.cpp:767-769`), so it fills faster than the "~80 s" estimate, which was never measured.	High	**Critical**	Same mitigation, D-018 Stage B. `onboard-queue` must add a drain on the connected branch of `MQTT::runOnce()` (`MQTT.cpp:624-632`), a third stock call site the design did not list.	Open
[Added 2026-09-25] A fall auto-SOS never beacons. `FallDetectionModule::triggerAutoSOS()` sends one `BACKGROUND` position and one text frame (`FallDetectionModule.cpp:144-145`) and never calls `tickBeacon()` , which only the button and gesture modules call (`TrekLinkButtonModule.cpp:312` , `TrekLinkSOSGesture.cpp:116`). A fall episode is two packets and then silence, so the cadence-anomaly detector (REQ-EVT-06) cannot fire for it, and losing the single text frame makes a fall invisible.	Medium	**Critical**	Firmware fix under D-008: have the fall path enter the same beacon loop as a button SOS and send its opening position at `MAX` . Sequence with `onboard-queue` Phase 9 so each change stays separately measurable.	Open, **firmware fix candidate**
[Added 2026-09-25] An SOS raised before the first GPS fix produces no beacons. `PositionModule::sendOurPosition()` returns without sending when there is no fixed position and	Medium	High	Firmware fix under D-008: the beacon path builds its own position packet (as `sendPositionPacket()` already does) instead of delegating to `sendOurPosition()` . The same	Open, **firmware fix candidate**

Risk	Likelihood	Impact	Mitigation	Status
no local position since boot (`PositionModule.cpp:352-355`). Only the trigger-time packets leave the node.			change can carry `priority = MAX` (REQ-OPT-02).	
[Added 2026-09-25] The on-device queue-health report cannot be read on the JSON topic. `PRIVATE_APP` (256) has no case in `MeshPacketSerializer` , so it falls to `default` (`MeshPacketSerializer.cpp:403-404`) and the JSON message has no `payload` . `gateway-sync` Stage A ingests only `<root>/2/json/...` (REQ-EVT-01), so its REQ-EVT-12/13 are unreachable as specified.	High	Medium	Decide the transport before `onboard-queue` Phase 6: an additive `PRIVATE_APP` case in the serializer, or a `/2/e/` subscription for port 256 in the backend. Define the payload schema in both specs.	Open, **question raised by the firmware session**
[Added S7] The default LoRa channel PSK is Meshtastic's published key, so RF traffic is encrypted but not confidential, any stock client in radio range decrypts SOS positions (`Channels.cpp:128–134` , `:238`).	High	High	**D-021**, provision a custom per-fleet PSK during device provisioning in MF-01; treat it as a recovery-critical escrowed secret. Distinct from `mqtt.encryption_enabled` , which stays `false` per D-007 with the broker hop secured by TLS + per-gateway API key.	Open, **provisioning step not yet written into `specs/devices/`**
[Added S7] Stage C's browser form (D-020) relies on **Web Serial / Web Bluetooth, which are Chromium-only**, and on iOS no browser can reach the node at all, so an iPhone guide is served only by the stock Meshtastic app path. A backgrounded or closed tab also stalls forwarding, though IndexedDB means nothing is lost.	Medium	Medium	Document the supported-host matrix before Stage C kick-off; surface forwarding state in the UI; retain the native-desktop fallback (D-020 Option 2) for unsupported venues. State the tab dependency honestly at the council rather than presenting Stage C as an always-on service.	Open, **Stage C deferred; decision recorded so Stage B does not foreclose it**
[Added S4] No backlog user story exists for the cadence-inferred `SUSPECTED` SOS episode path (`gateway-sync/design.md` §1.3/§2.4, REQ-EVT-06), the primary mitigation for the Critical single-unacknowledged-text-frame risk above has no corresponding Staff-facing workflow story yet	Medium	Medium	US-088 added (`03-backlog/02-user-stories.md`), assigned Khoa, Staff sees a Suspected episode visually distinct from Confirmed and can dismiss it with a lighter action than the full Resolved→Closed flow.	**Mitigated** (Session 4)
[Added 2026-09-25, web session] `Device.nodeNum Int?` overflows above 2147483647, and node numbers are unsigned 32-bit values from the MAC. Ingestion would fail for about half of all units.	High	High	Type `BigInt` (04-architecture-conventions §3, platform design §3.3); test with `4294967295` .	Mitigated in spec
[Added 2026-09-25, web session] `gateway-sync` queried the `Incident` table directly (a	High	Medium	Lookup moved behind `IncidentsService.correlateSos`	Mitigated in spec

Risk	Likelihood	Impact	Mitigation	Status
module-isolation breach) and ignored `RESOLVED` incidents in the episode lookup, which would create a second Incident where E03-6 requires a reopen.			; a `RESOLVED` incident in the window reopens.	
[Added 2026-09-25, web session] Stock firmware drains a queued backlog one entry per reconnect, so an arrival-time episode window could split one SOS into several Incidents.	Medium	High	Windows and ordering use the event timestamp when valid (`gateway-sync` REQ-UBI-02).	Mitigated in spec
[Added 2026-09-25, web session] A fall auto-SOS, or an SOS before the first GPS fix, sends no beacons, so losing its single text frame loses the episode and cadence detection cannot fire.	Medium	**Critical**	Firmware `onboard-queue` Phase 9 adds beacons.	Open
[Added 2026-09-25] New backend dependencies: `@nestjs/schedule`, `@nestjs/event-emitter`, `@nestjs/throttler`, `@casl/prisma`, `pdfkit`.	Medium	Low	Approved by the leader for Phase B (C-003 item 4); frontend test dependencies are approved when the frontend work starts.	Resolved
[Added 2026-09-25, leader field report] Fall detection does not work on v1 (MPU6050) or v2 (ICM-20948), and some v2 units stay stuck in SOS. Root cause not established: IMU driver or HAL wiring, value scaling, thresholds, or a race between the fall, button and gesture SOS owners.	High	**Critical**	Reproduce on hardware with serial logs of raw IMU readings and SOS state transitions before designing a fix (`04-firmware-ground-truth.md` §2).	Open
[Added 2026-09-25, code-verified] On v3 and v4 the 3-second SOS hold shares `BUTTON_PIN` with the stock user button, which fires SELECT at 500 ms and SHUTDOWN on a longer hold (`InputBroker.cpp:290-306`). Button SOS on T-Beam boards is unreliable and can shut the unit down.	High	**Critical**	Move SOS to a dedicated input or disable the stock long press on TrekLink variants; decide before the demo.	Open
[Added 2026-10-02, code-verified plus regulation] Transmit settings likely exceed Vietnamese licence-exempt limits. Every variant defaults to `MY_433` (433.0 to 435.0 MHz, 20 dBm conducted, `RadioInterface.cpp:158`, `:847-848`); Circular 08/2021/TT-BTTTT Article 5 licenses 433 MHz LPWAN above 25 mW ERP, so the exempt ceiling is 25 mW ERP in	High	High	Confirm the appendix limit; add a TrekLink Vietnam region or a per-variant power cap so conducted power plus antenna gain stays within 25 mW ERP inside 433.05 to 434.79 MHz; re-measure range at the compliant power, because it will shorten. Firmware ground truth §5.1	Open

Risk	Likelihood	Impact	Mitigation	Status
433.05 to 434.79 MHz (inferred, appendix not read). With the 6 dBi antenna the radiated power is well above that. A reviewer asking "is this legal?" has no clean answer yet.				
[Added 2026-10-02, leader field test] No link across a ridge without a relay on the crest, and range collapses in rain (below 100 m) and inside thick concrete. Typical trekking range is 1 to 3 km.	High	High	Accepted physical limit. Disclose to renters and users: the device is not a sole means of communication; move to high ground or place a repeater. Plan a repeater node as optional equipment. Second field test pending. Firmware ground truth §5.1	Accepted

Contributing to TrekLink

The 2-minute version. Full detail lives in `01-conventions/`, which is also the source of the Developer Handbook PDF.

1. Orient yourself

- What are we building: `00-project-context/01-project-charter.md` (`01-project-charter.md`)
- What week/sprint is it: `00-project-context/02-roadmap-and-milestones.md` (`02-roadmap-and-milestones.md`)
- Anything blocking you: `00-project-context/03-decisions-and-risk-register.md` (`03-decisions-and-risk-register.md`)

2. Where work lives

Jira is the single tracker (TK project). **GitHub Issues** hold daily reports and standalone bugs only. The **backlog** (`03-backlog/`) is read-only to members, it holds the EARS criteria you implement against. See `01-conventions/10-jira-tracking-and-workflow.md` (`10-jira-tracking-and-workflow.md`).

3. Before writing code

Every module goes through `specs/{module}/requirements.md` → `design.md` → `tasks.md` **before** implementation, see `01-conventions/02-spec-driven-development-workflow.md` (`02-spec-driven-development-workflow.md`). Blank templates in `02-templates/`.

4. Branch, commit, PR

```
git fetch origin && git checkout dev && git pull origin dev
git checkout -b feat/TK-45-device-registration        # Jira key in the branch name

# ... work, per 01-conventions/03-operational-workflows.md ...
git commit -m "feat(TK-45): add device FSM transition guard"

npm test                                              # must be green BEFORE the PR
git fetch origin && git rebase origin/dev
git push -u origin feat/TK-45-device-registration --force-with-lease
gh pr create --base dev --title "feat(TK-45): device registration" --label "module:devices"
```

Then **move the Jira card to `IN REVIEW`** and **ping your reviewer in Zalo**. Full convention: `01-conventions/07-github-workflow-git-conventions.md` (`07-github-workflow-git-conventions.md`).

No direct pushes to any branch, ever, including the leader's.

5. Merging

Rebase & merge (single commit) or Squash & merge (multiple). **Merge commits are prohibited.** Delete the source branch, except `dev` → `main`. **Announce every merge to `dev` in Zalo** so everyone pulls and rebases.

6. Backend response contract (non-negotiable)

Every endpoint returns `{ result, isSuccess, statusCode, message }`, see `01-conventions/05-backend-conventions.md` (`05-backend-conventions.md`) §3.

7. Language

English only, code, commits, PRs, specs, docs, Jira cards. You may prompt your AI agent in Vietnamese; what it writes to the repo is English regardless.

8. Using an AI coding agent

Read `01-conventions/11-ai-first-doctrine-and-toolchain.md` (`11-ai-first-doctrine-and-toolchain.md`) once, fully. Install `prompt-orchestrator` and the TrekLink skills, open the `capstone/` parent folder (not a single repo), and run `/treklink-session`. The agent reads the conventions itself, you should never have to tell it to.

9. Definition of Done, every PR

- [ ] EARS criteria in `requirements.md` satisfied
- [ ] **Unit tests written and passing**, before the PR is opened
- [ ] Module boundaries respected (no cross-module repository access)
- [ ] Lint/typecheck clean
- [ ] `api-design/*.md` updated in the same PR if any endpoint changed
- [ ] Rebased on `dev` ; Jira card linked and moved to `IN REVIEW`
- [ ] `Model used:` declared in the PR